



EDB Postgres AI Database Essentials v17

Course Agenda

- Introduction and Architectural Overview
- System Architecture
- Enterprise Postgres Installation
- User Tools - Command Line Interfaces
- Database Clusters
- Database Configuration
- Data Dictionary
- Creating and Managing Database Objects
- Database Security
- Monitoring and Admin Tools Overview
- SQL Primer
- Backup and Recovery
- Routine Maintenance Tasks
- Data Loading
- Data Replication and High Availability



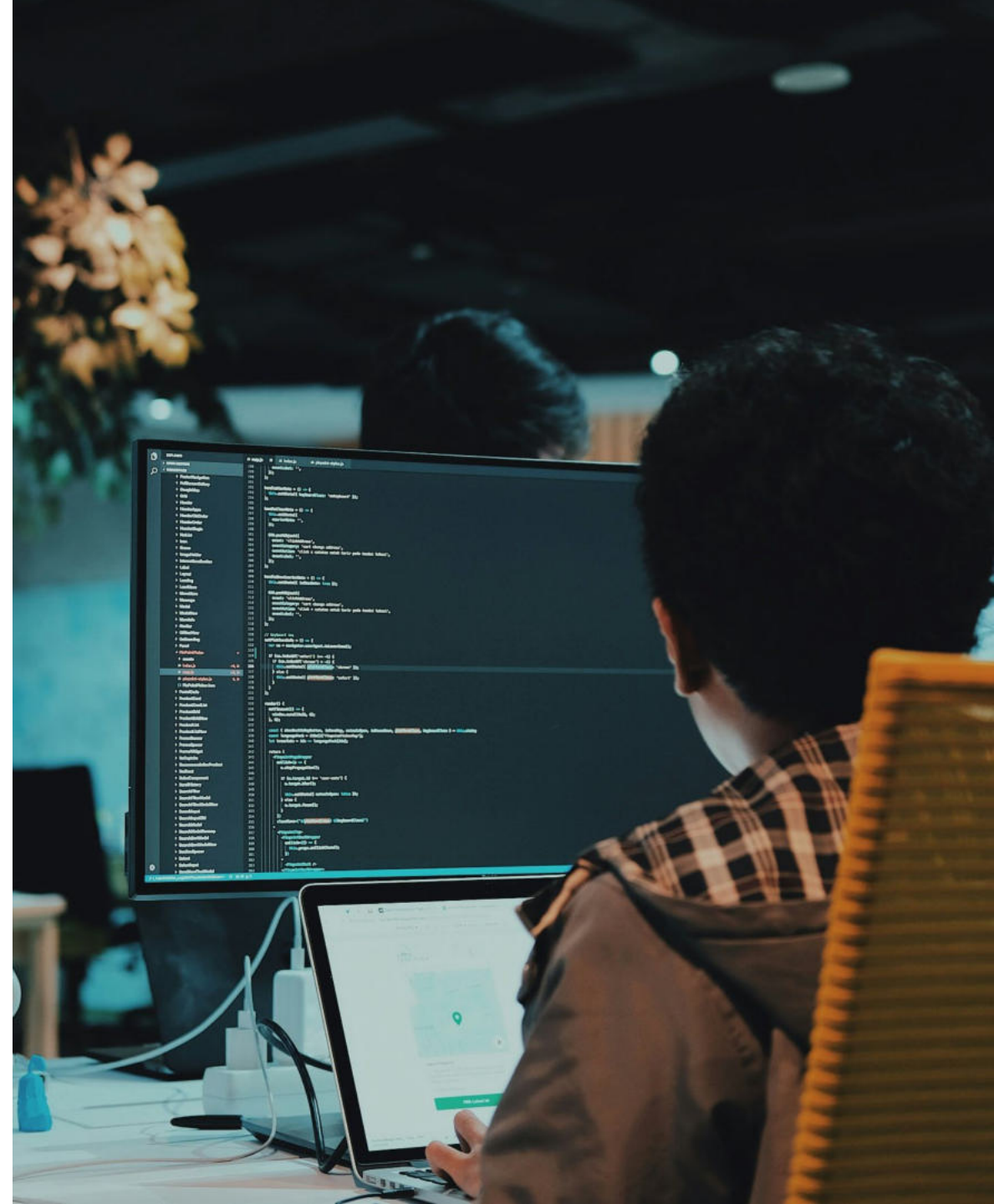


Introduction



Module Objectives

- EDB Postgres AI Platform
- Facts about PostgreSQL and Enterprise Postgres (Oracle Compatible)
- Major Features
- General Database Limits
- Common Database Object Names



Operate Efficiently

- Cloud agility in any environment
- Unified management boosts productivity 30%
- 90% better value than cloud databases

Innovate Faster

- Accelerated time-to-market
- Secure open source flexibility
- AI-driven assistance and automation

Lead Your Market

- Fastest path to launching AI applications
- Competitive edge
- Ethical AI practices

Grow Revenue

- Always-on applications with up to 99.999% HA
- Business growth
- Customer experience

EDB Postgres AI



EDB POSTGRES® AI

YOUR SOVEREIGN DATA AND AI PLATFORM



BUILD WITHOUT BOUNDARIES

A single, sovereign data platform for any workload



ACCELERATE AI INITIATIVES AND GROW REVENUE

Secure, rapid, low-code, fully integrated AI Factory



GET REAL-TIME INSIGHTS WITHOUT COMPLEXITY

Rapid analytics and open ecosystem integrations



DRIVE OPERATIONAL EFFICIENCY WITH HYBRID MANAGEMENT

Cloud-native automation and deep observability across deployments from a single pane of glass

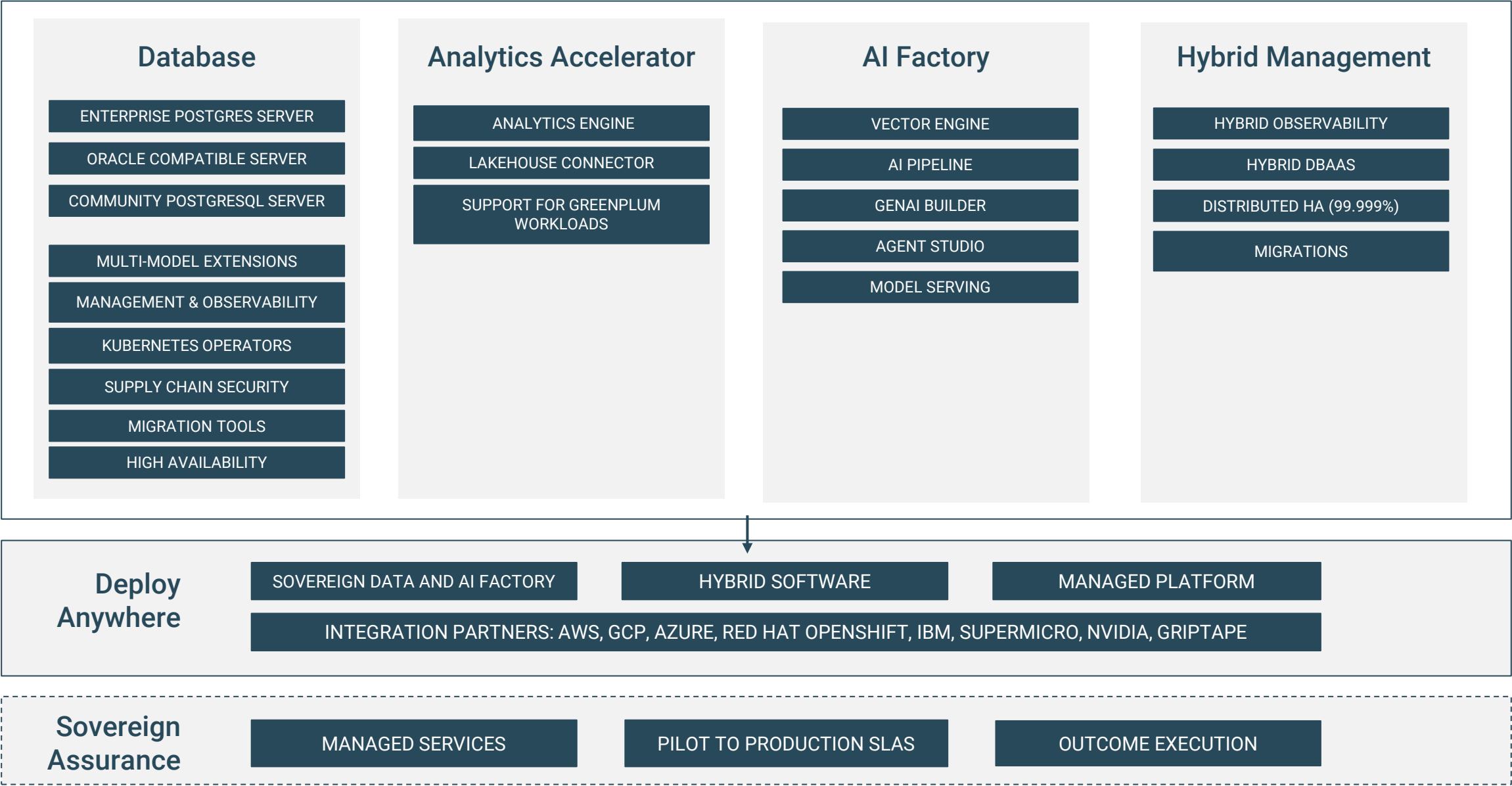


DELIVER CONTROLLED OUTCOMES WITH COMPLETE SOVEREIGNTY

Flexible deployment across hybrid software, sovereign data and AI factory, and managed platform



EDB Postgres® AI



PostgreSQL

The open-source database of choice



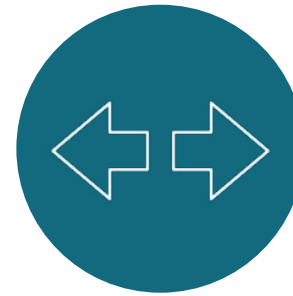
Performance

Handles enterprise workloads with 50% improvement in the last 4 years



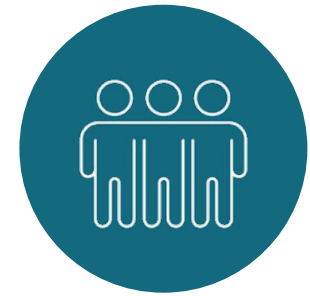
Scalability

Multiple technical options for operating Postgres at scale



Extensibility

Supported by a wide array of extensions plus multiple SQL and NoSQL data models



Community-driven

Multiple companies and individuals contribute to the project and drive innovation



Facts about PostgreSQL

The world's most advanced open-source database

Designed for extensibility and customization

ANSI/ISO compliant SQL support

Actively developed for more almost 40 years

- University Postgres (1986-1993)
- Postgres95 (1994-1995)
- PostgreSQL (1996-current)



Enterprise Postgres (Oracle Compatible)



Enterprise Postgres

- **Oracle Compatibility** - Compatibility for schemas, data types, indexes, users, roles, partitioning, packages, views, PL/SQL triggers, stored procedures, functions, and utilities
- **Additional Security** - Password policy management, session tag auditing, data redaction, SQL injection protection, and procedural language code obfuscation
- **Developer Productivity** - Over 200 pre-packaged utility functions, user-defined object types, autonomous transactions, nested tables, synonyms, advanced queueing
- **DBA Productivity** - Throttle CPU and I/O at the process level, over 55 extended catalog views to profile all the objects and processing that occurs in the database
- **Performance** - Query optimizer hints, SQL session/system wait diagnostics
- **Replication Enhancements** - Enables High Availability functionality such as Group Commit, Commit at Most Once and Eager all-node synchronous replication, timestamp-based snapshots, estimates for replication catch-up times, selective backup of a single database, hold back freezing to assist resolution of UPDATE/DELETE conflicts, multi-node PITR



Major Features

Portable:

- Written in ANSI C
- Supports Windows, Linux, Mac OS/X and major UNIX platforms

Reliable:

- ACID Compliant
- Supports Transactions and Savepoints
- Uses Write Ahead Logging (WAL)

Scalable:

- Uses Multi-version Concurrency Control
- Table Partitioning and Tablespaces
- Parallel Sequential Scans, DDL(Table and Index Creation)



Major Features (continued)

Secure:

- Employs Host-Based Access Control
- Provides Object-Level Permissions and Row Level Security
- Supports SSL Connections and Logging
- Transparent Data Encryption - TDE

Recovery and Availability:

- Streaming Replication, Logical Replication and Replication Slots
- Replication Slots, Sync or Async Options
- Supports Hot-Backup, pg_basebackup, incremental backup and Point-in-Time Recovery

Advanced:

- Supports Triggers, Functions and Procedures
- Supports Custom Procedural Languages
- Upgrade using pg_upgrade
- Unlogged Tables and Materialized Views
- Just-in-Time (JIT) Compilation



General Database Limits

Limit	Value
Maximum Database Size	Unlimited
Maximum Table Size	32 TB
Maximum Row Size	1.6 TB
Maximum Field Size	1 GB
Maximum Rows per Table	Unlimited
Maximum Columns per Table	250-1600 (Depending on Column types)
Maximum Indexes per Table	Unlimited



Common Database Object Names

Industry Term	Postgres Term
Table or Index	Relation
Row	Tuple
Column	Attribute
Data Block	Page (when block is on disk)
Page	Buffer (when block is in memory)



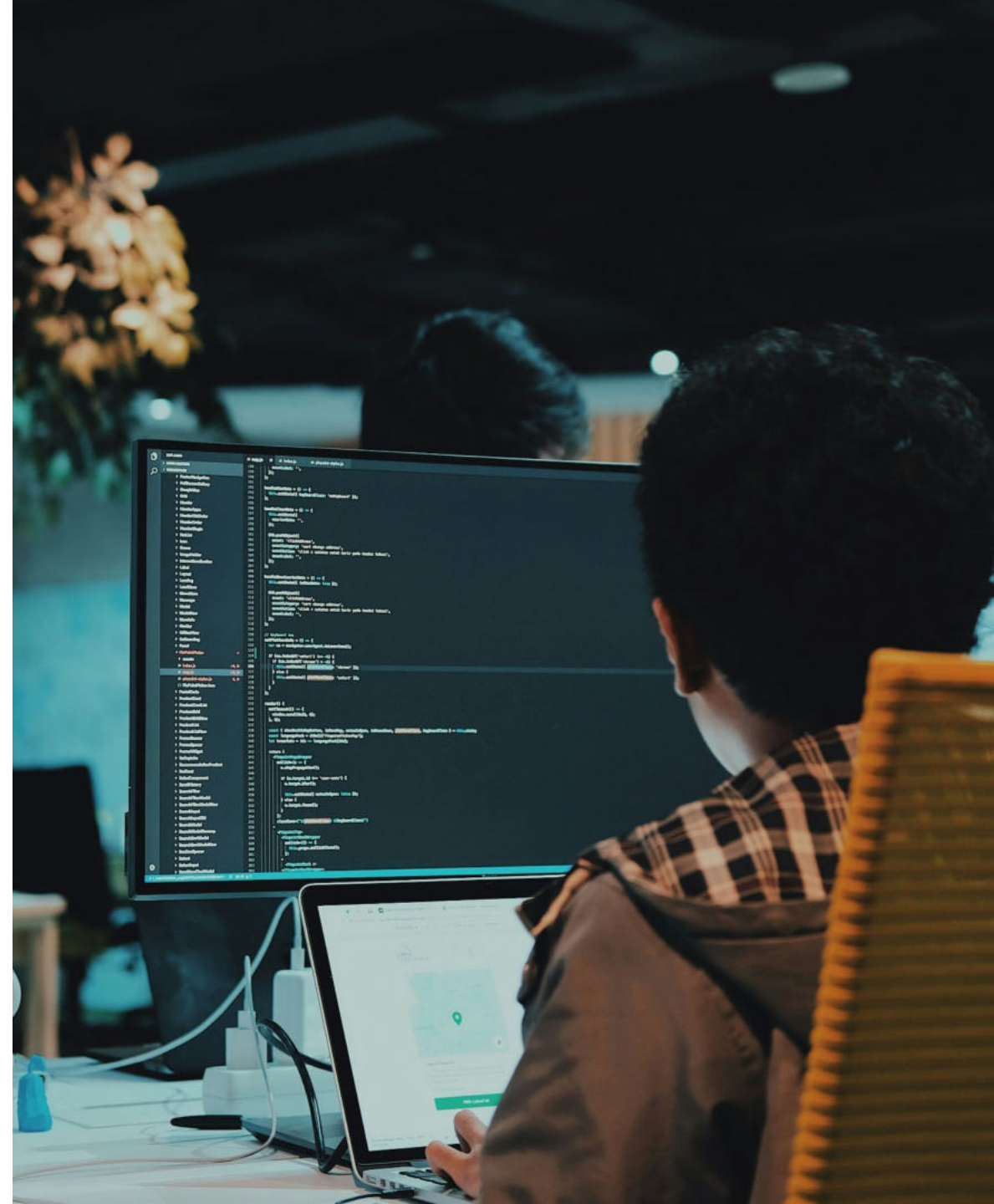
Lab Setup Guidelines

- All the instructor demos and labs are based on Linux
- Rocky 9 machine or virtual machine with at least 1 GB RAM and 20 GB storage space is recommended
- Participants using Linux must follow instructor during the installation module and install Enterprise Postgres



Module Summary

- EDB Postgres AI Platform
- Facts about PostgreSQL and Enterprise Postgres (Oracle Compatible)
- Major Features
- General Database Limits
- Common Database Object Names





System Architecture



Module Objectives

- Architectural Summary
- Process and Memory Architecture
- Utility Processes
- Connection Request-Response
- Disk Read Buffering
- Disk Write Buffering
- Background Writer Cleaning Scan
- Commit and Checkpoint
- Statement Processing
- Physical Database Architecture
- Data Directory Layout
- Installation Directory Layout
- Page Layout



Architectural Summary

Postgres uses **processes**, not threads

The “Postmaster” process acts as a **supervisor**

Several utility processes perform **background** work

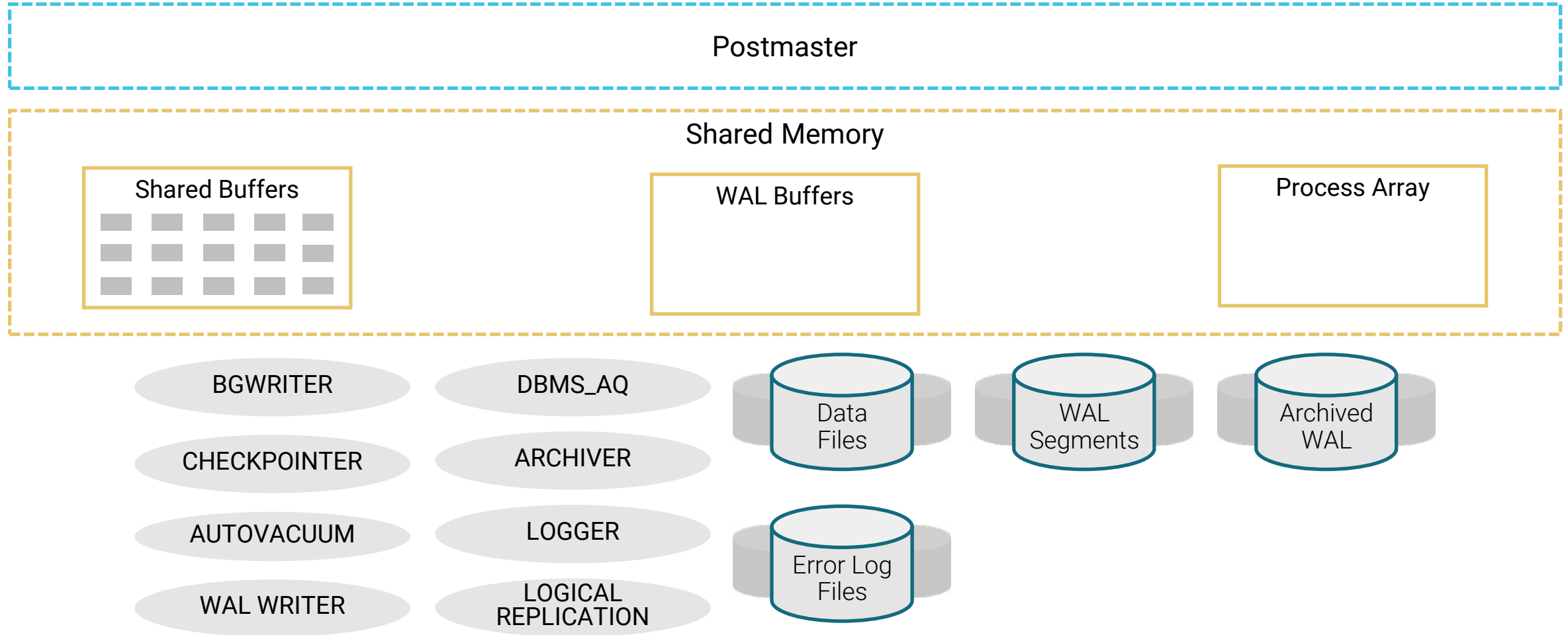
- postmaster starts them and restarts them if they die

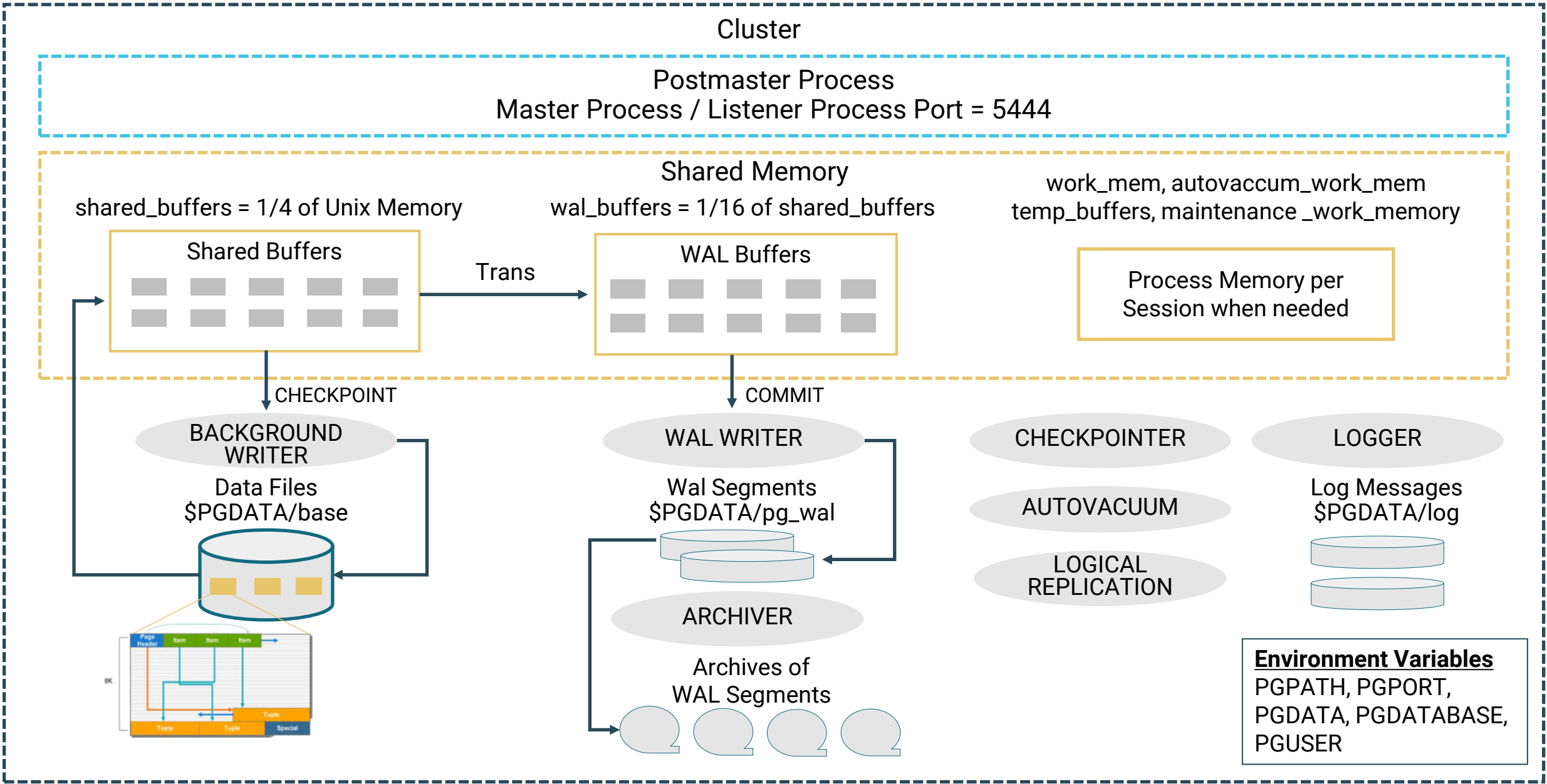
One backend process is created **per user session**

- postmaster listens for new connections

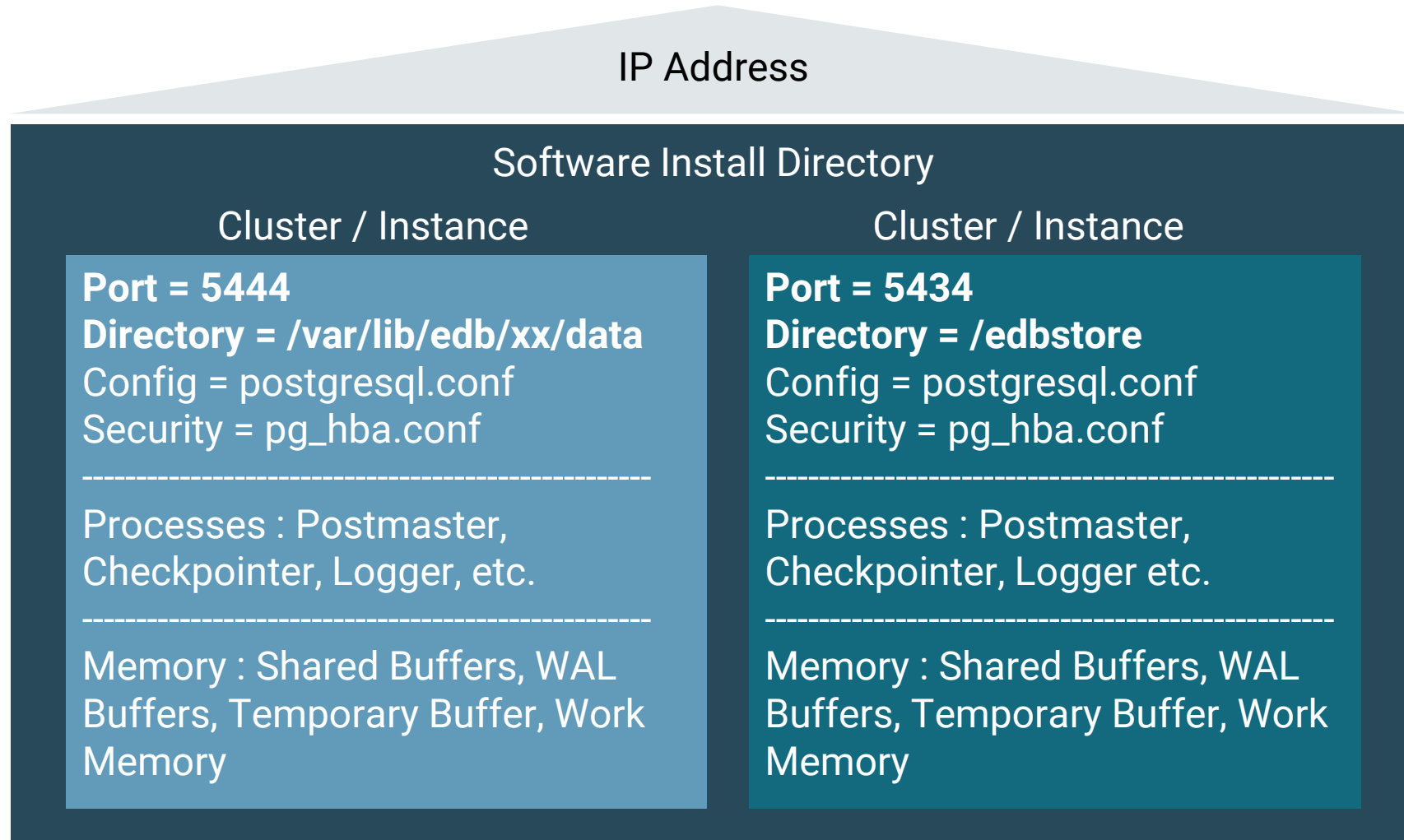


Process and Memory Architecture



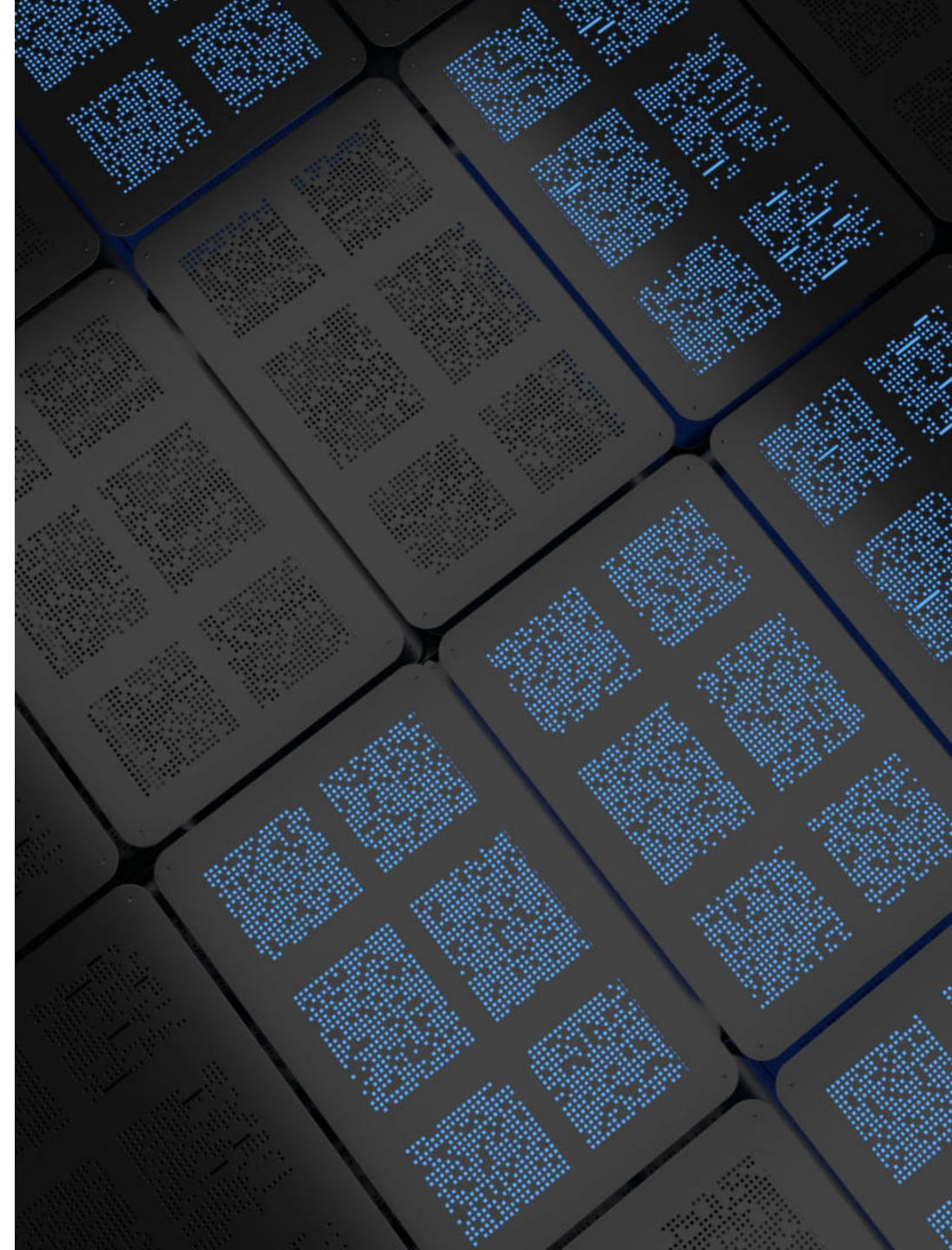


Multiple Instances On One Server



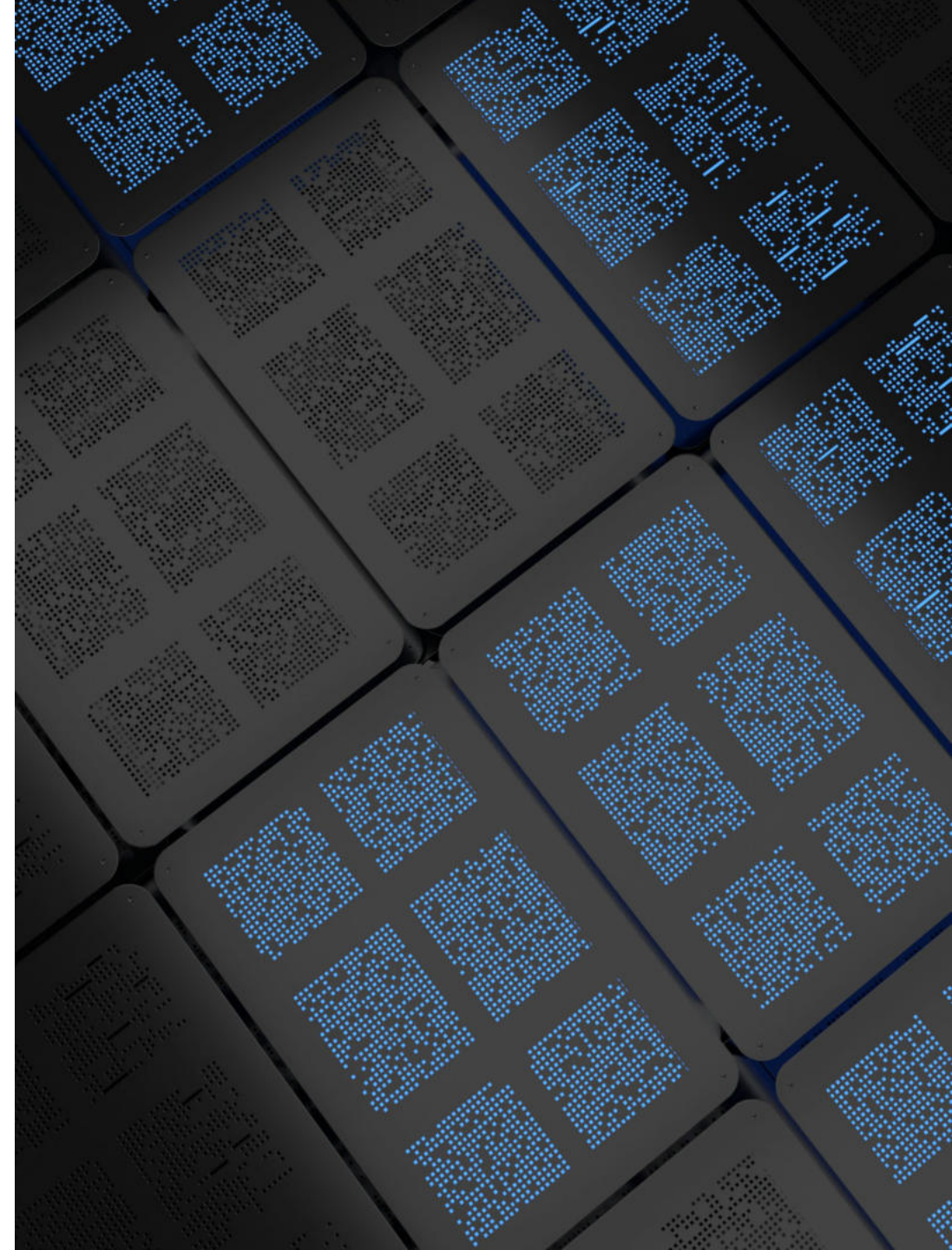
Utility Processes

- Background writer
 - Writes dirty data blocks to disk
- WAL writer
 - Flushes write-ahead log to disk
- Checkpointer
 - Automatically performs a checkpoint based on config parameters
- Autovacuum launcher
 - Starts Autovacuum workers as needed
- Autovacuum workers
 - Recover free space for reuse

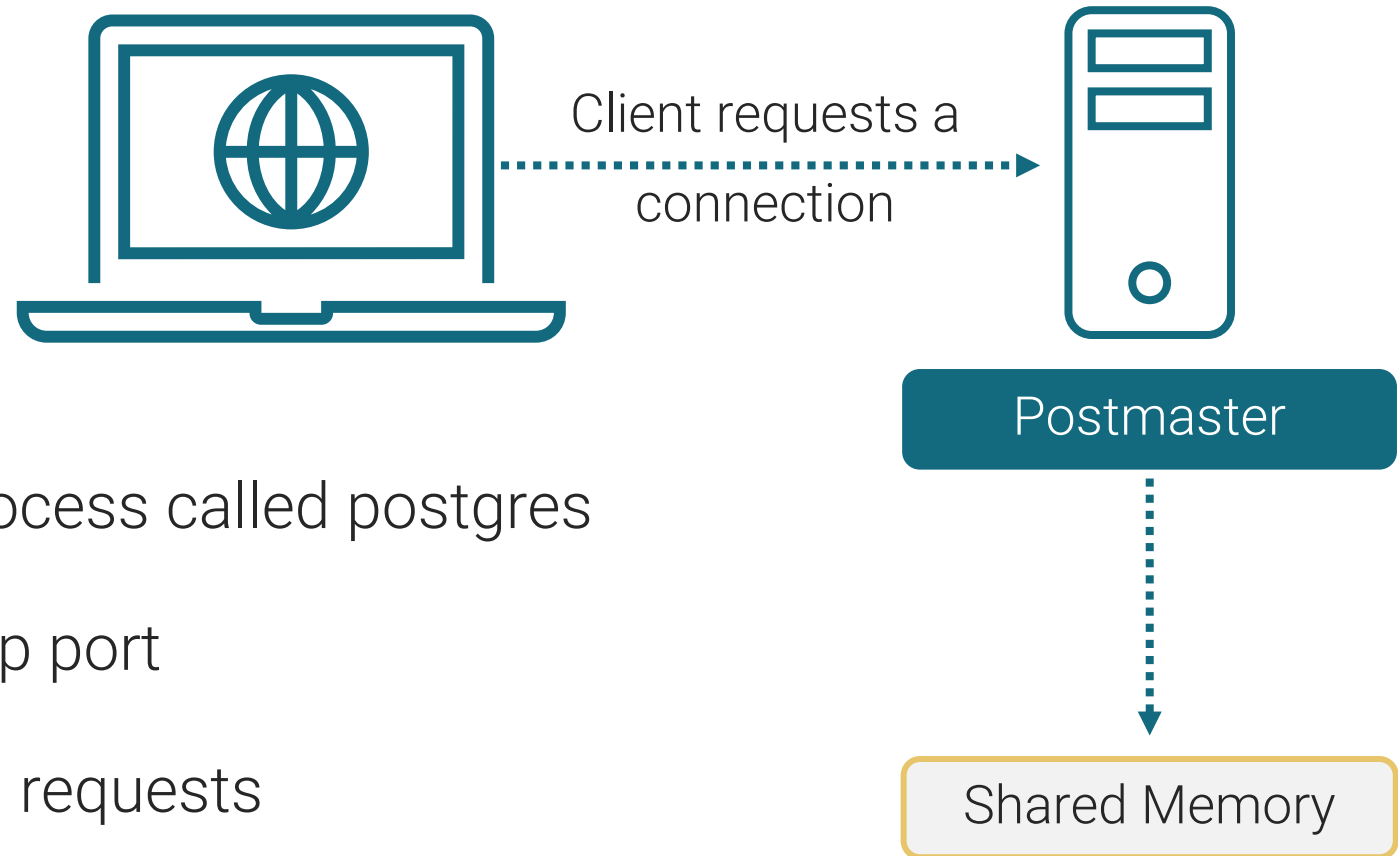


More Utility Process

- Logger - Logging collector
 - Routes log messages to syslog, eventlog, or log files
- Archiver
 - Archives write-ahead log files
- Logical replication launcher
 - Starts logical replication apply process for logical replication
- Dbms_aq launcher
 - Collects information for queueing functionality of advanced server



Postmaster as Listener

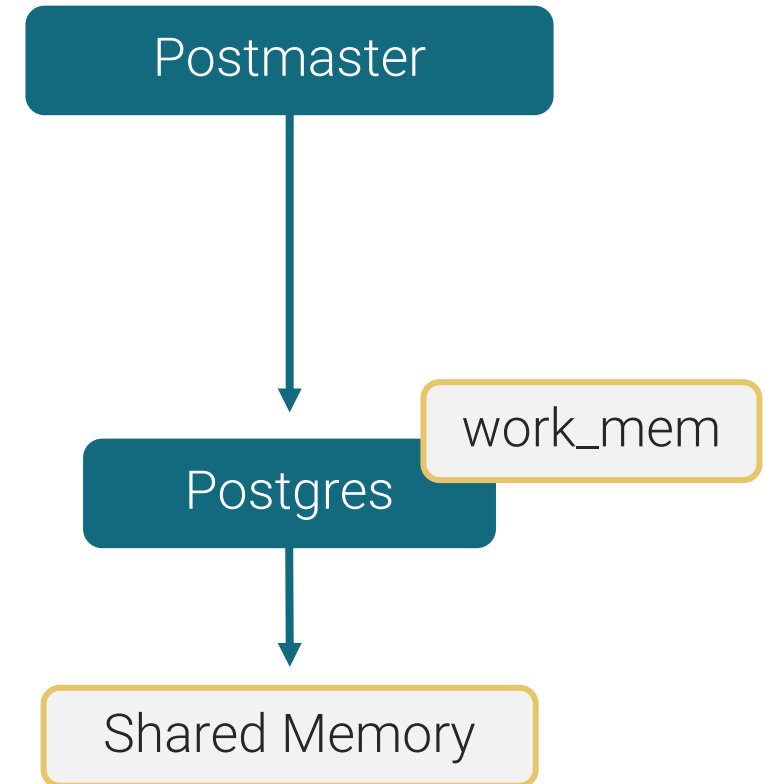


- Postmaster is the main process called postgres
- Listens on 1, and only 1, tcp port
- Receives client connection requests

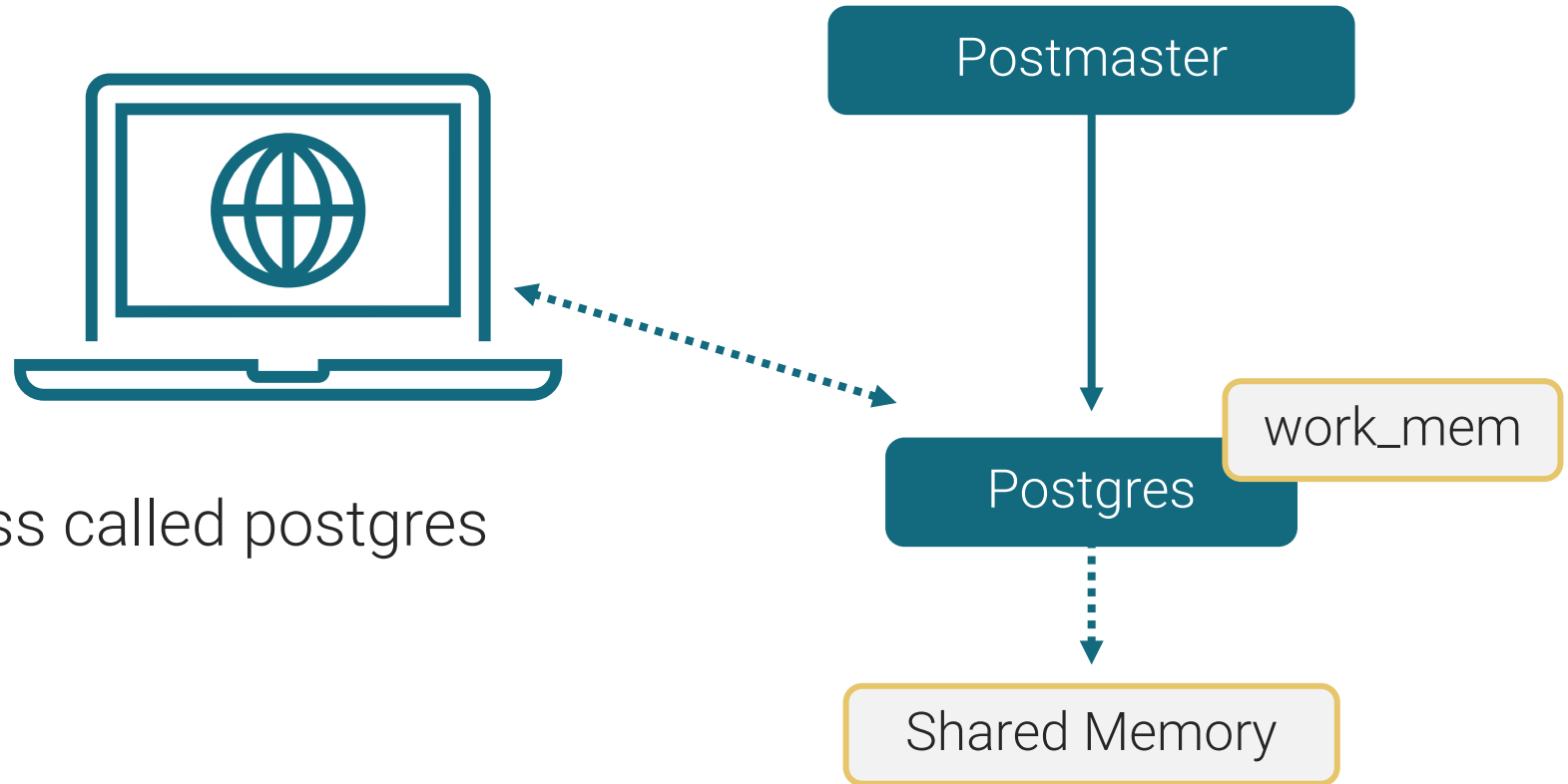


User Backend Process

- Postmaster process spawns a new server process for each connection request detected
- Communication is done using semaphores and shared memory
- Authentication - IP, user and password
- Authorization - Verify permissions



Respond to Client

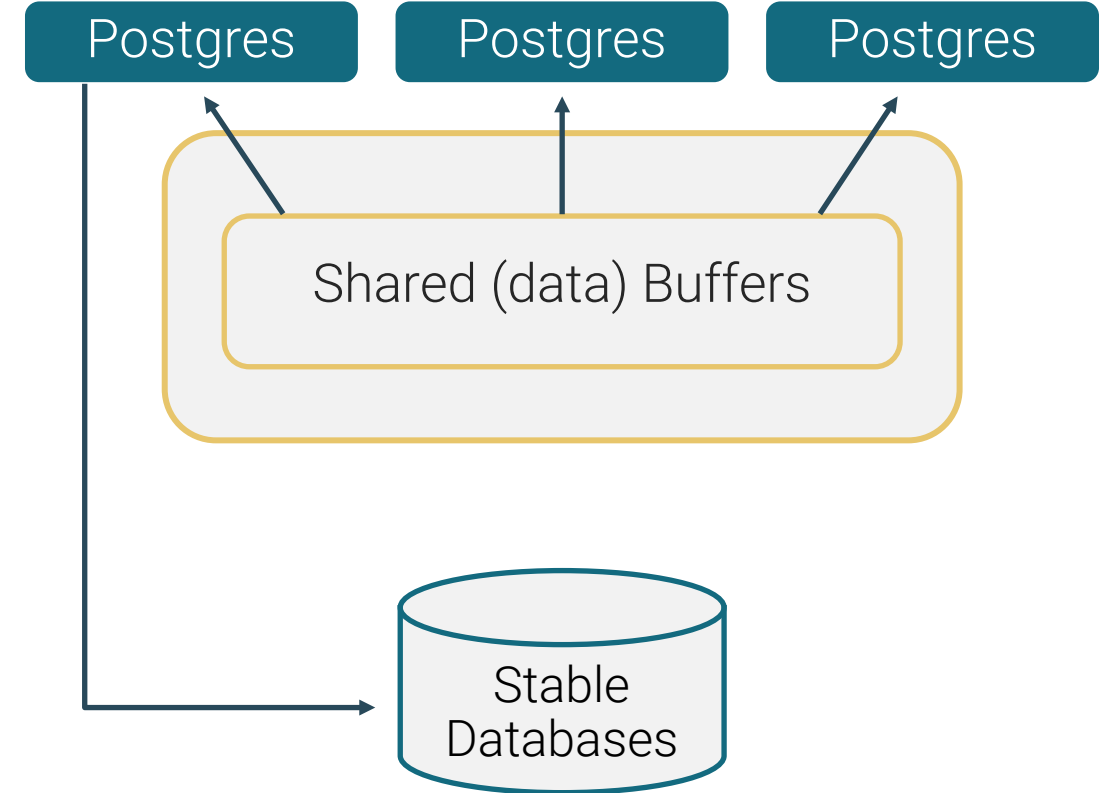


- User backend process called postgres
- Callback to client
- Waits for SQL
- Query is transmitted using plain text



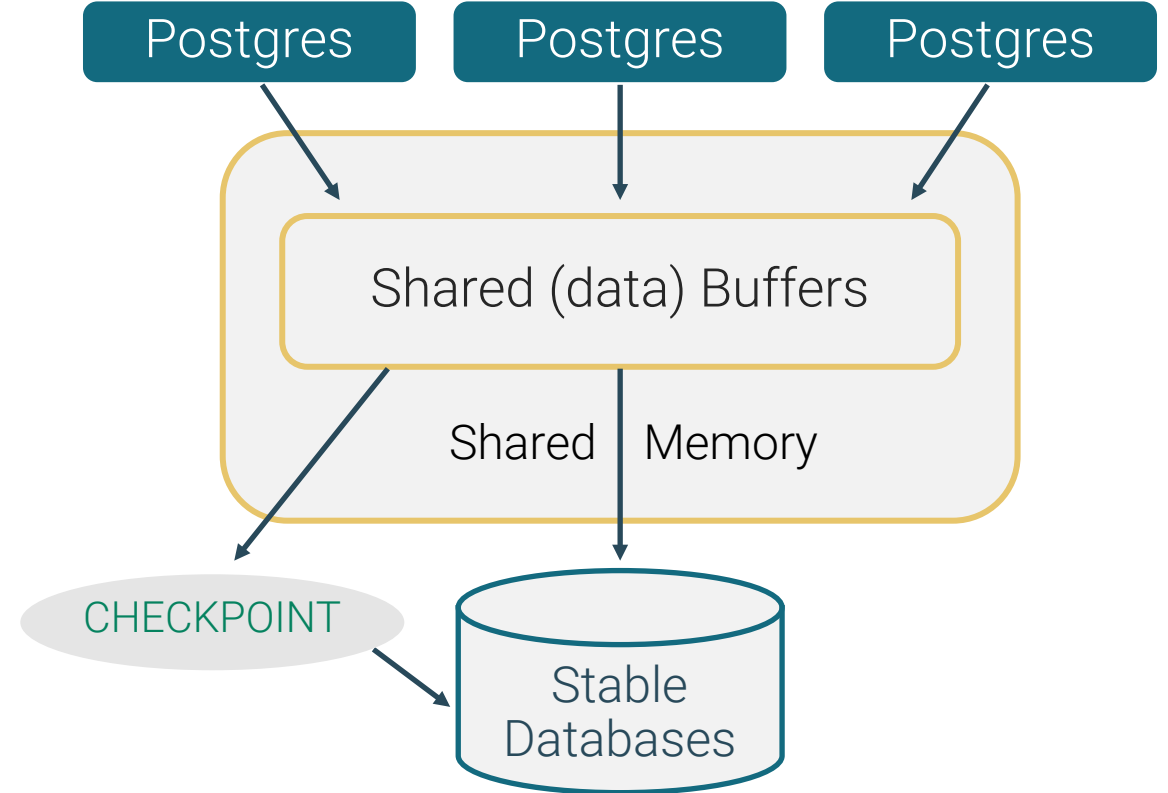
Disk Read Buffering

- EDB Postgres buffer cache (shared_buffers) reduces OS reads
- Read the block once, then examine it many times in cache



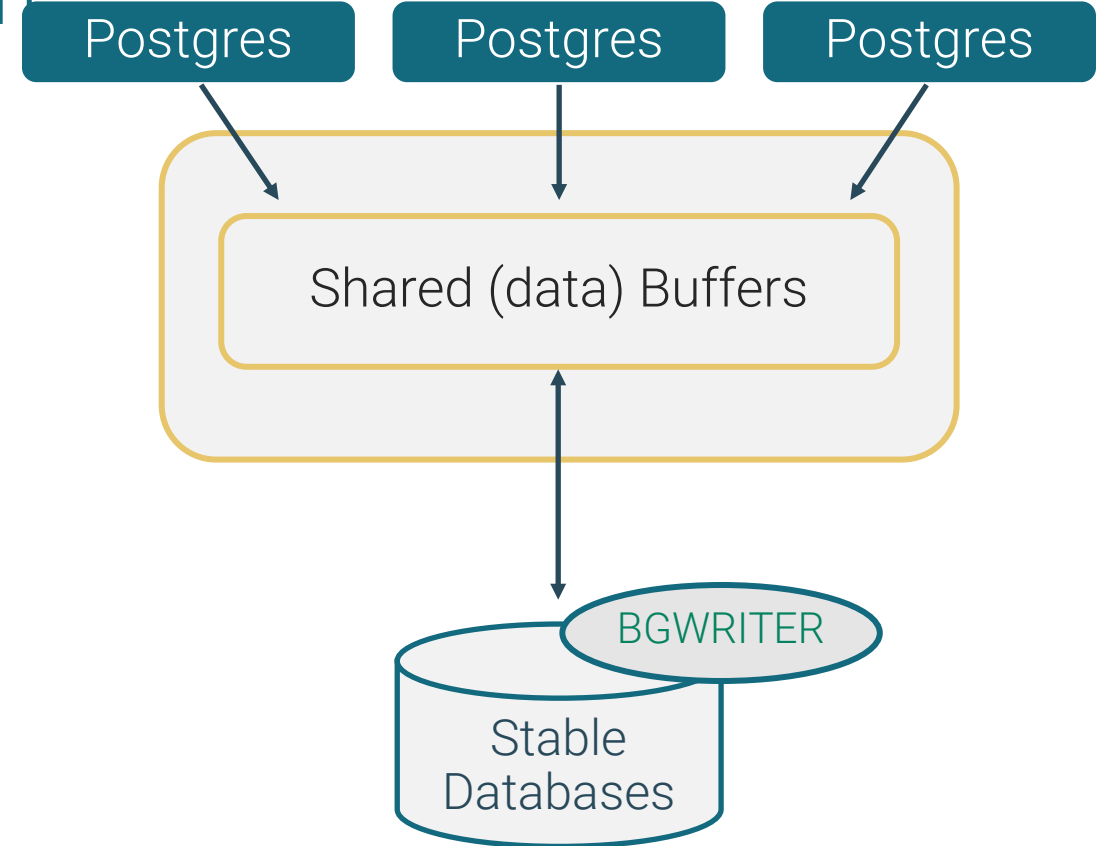
Disk Write Buffering

- Blocks are written to disk only when needed:
 - To make room for new blocks
 - At checkpoint time



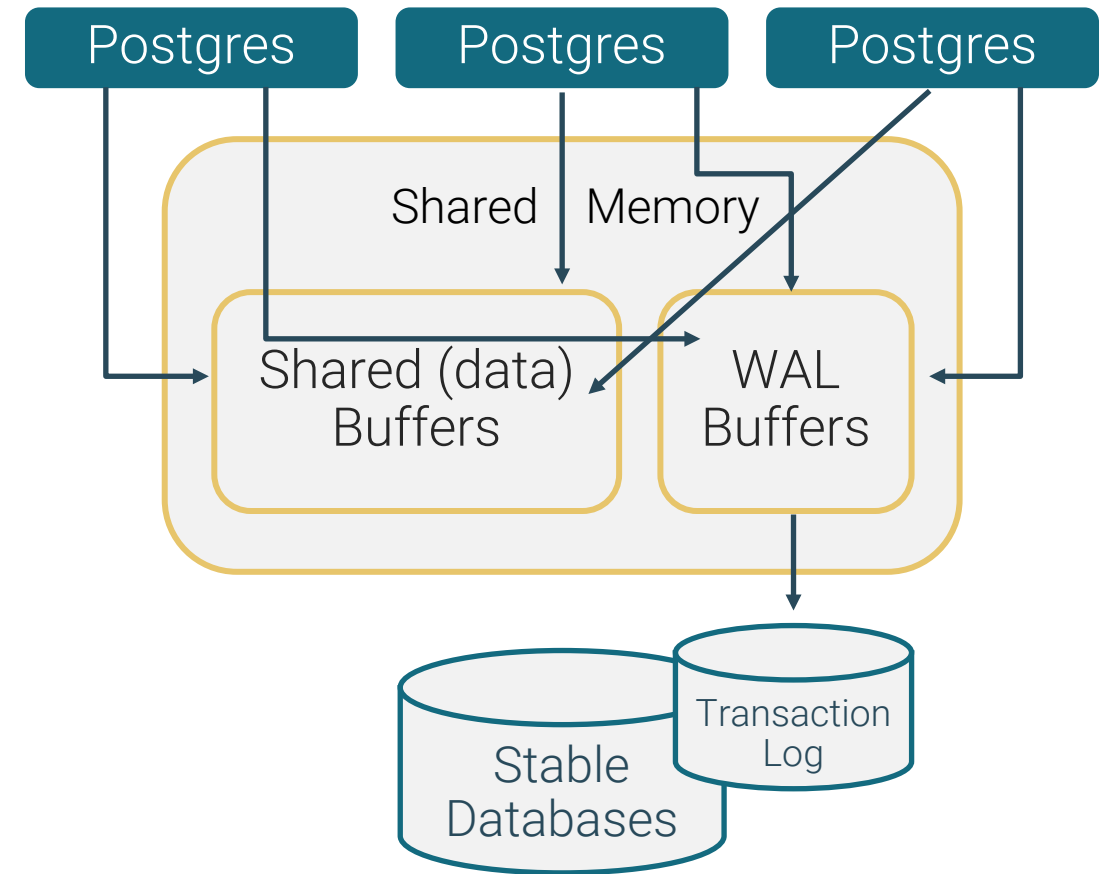
Background Writer Cleaning Scan

- Background writer scan attempts to ensure an adequate supply of clean buffers
- Back end write dirty buffers as need



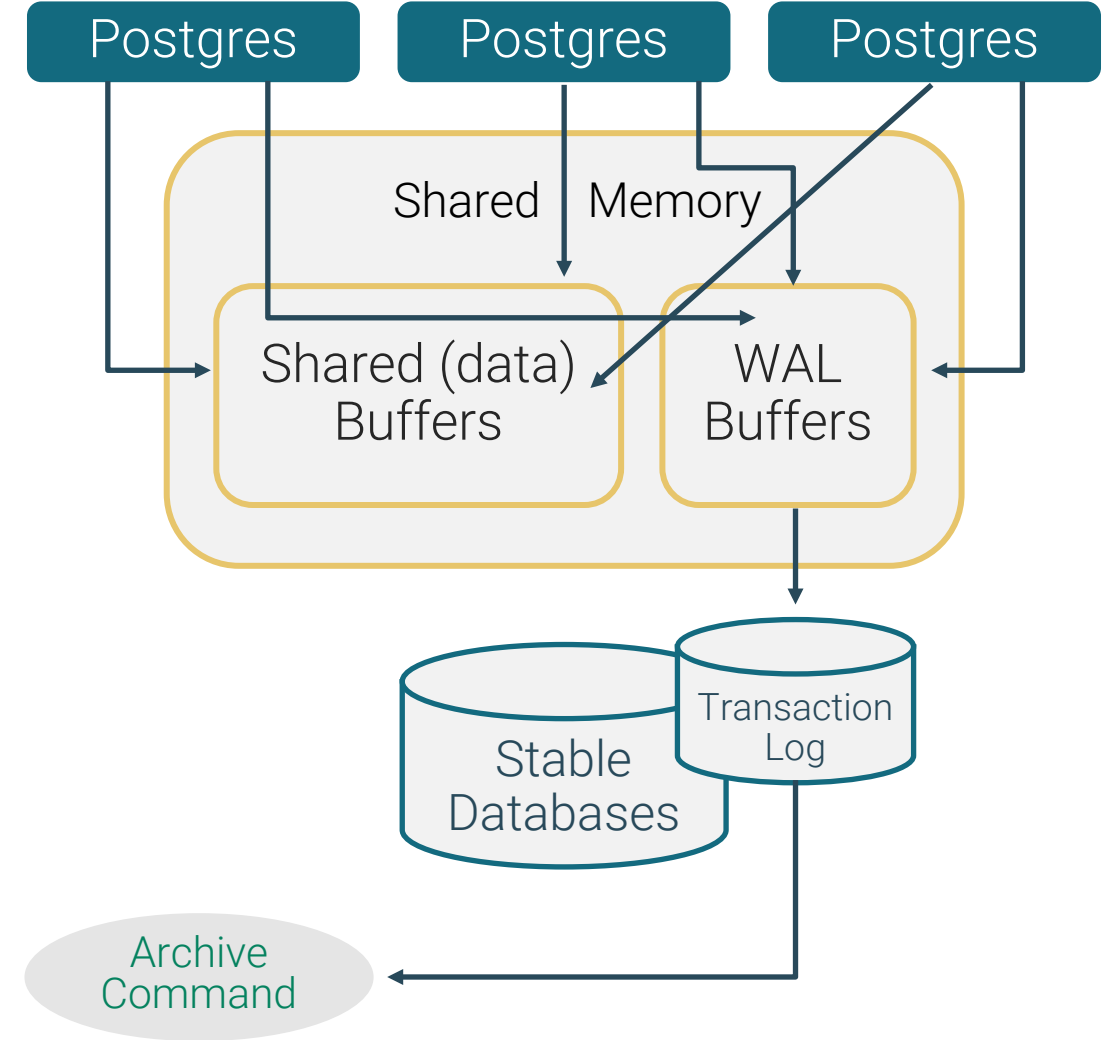
Write Ahead Logging (WAL)

- Back end write data to WAL buffers
- Flush WAL buffers periodically (WAL writer), on commit, or when buffers are full
- Group commit



Transaction Log Archiving

- Archiver spawns a task to copy away **pg_wal** log files when full



Commit and Checkpoint



Before commit

Uncommitted updates are in memory



After commit

WAL buffers are written to the disk (write-ahead log file) and shared buffers are marked as committed



After checkpoint

Modified data pages are written from shared memory to the data files



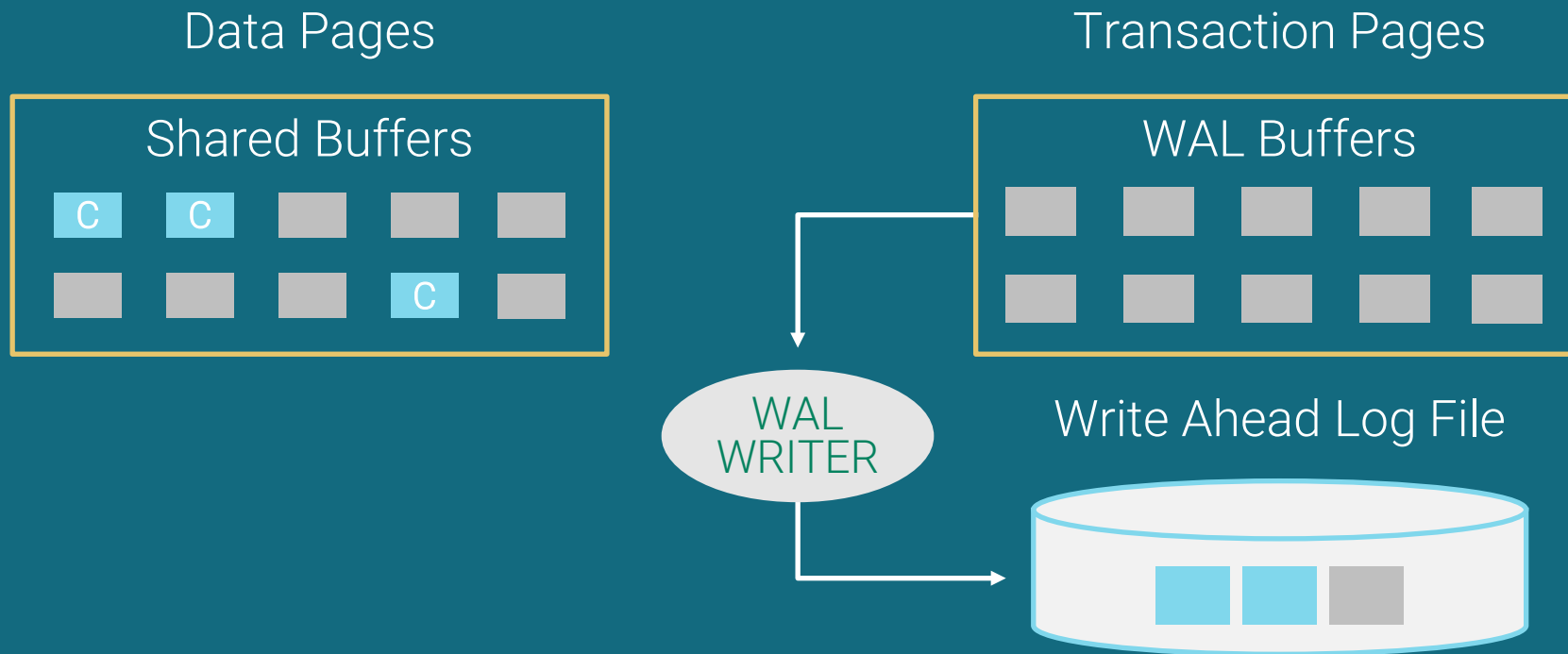
Before Commit

- Updated pages are in Shared Buffers
- Transaction is logged in the WAL Buffers



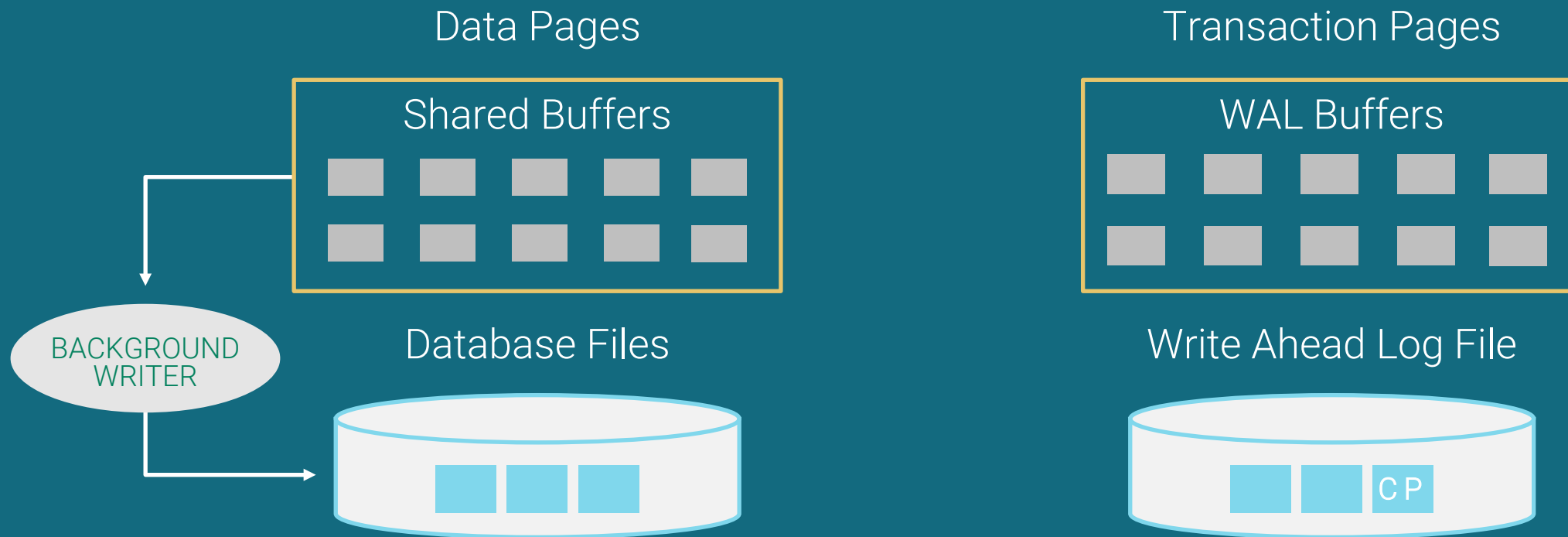
After Commit

- Shared buffers are marked as committed
- WAL buffers are written to disk (write-ahead log file)

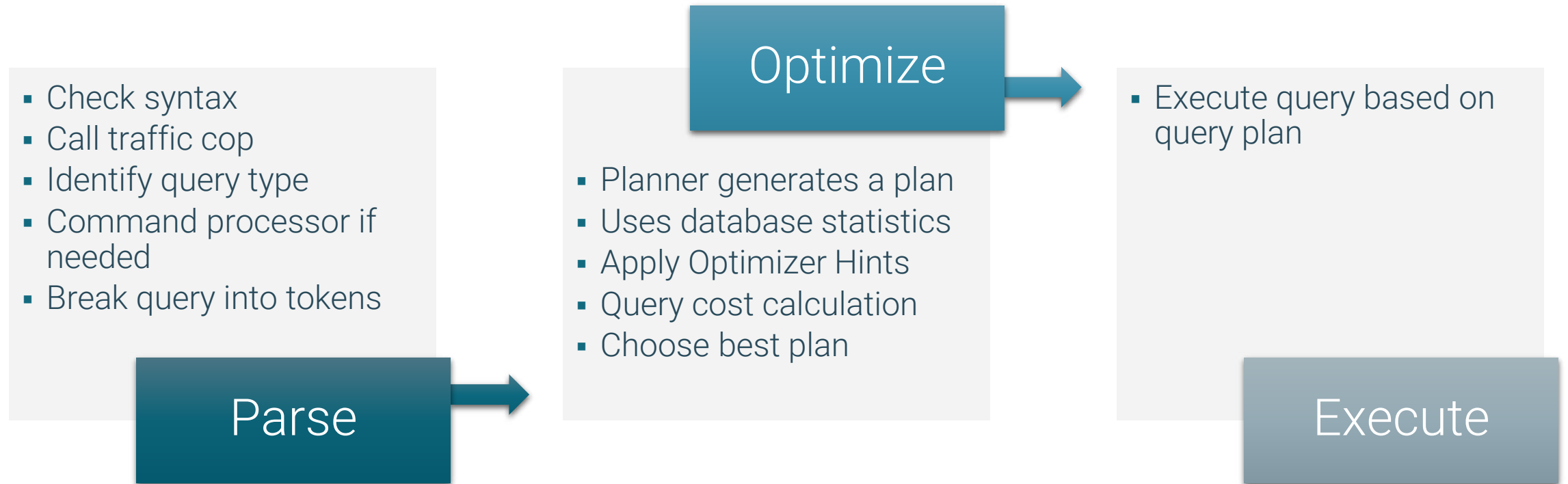


After Checkpoint

- Modified data pages are written from shared memory to the data files
- A Checkpoint record is written to the WAL File used for recovery

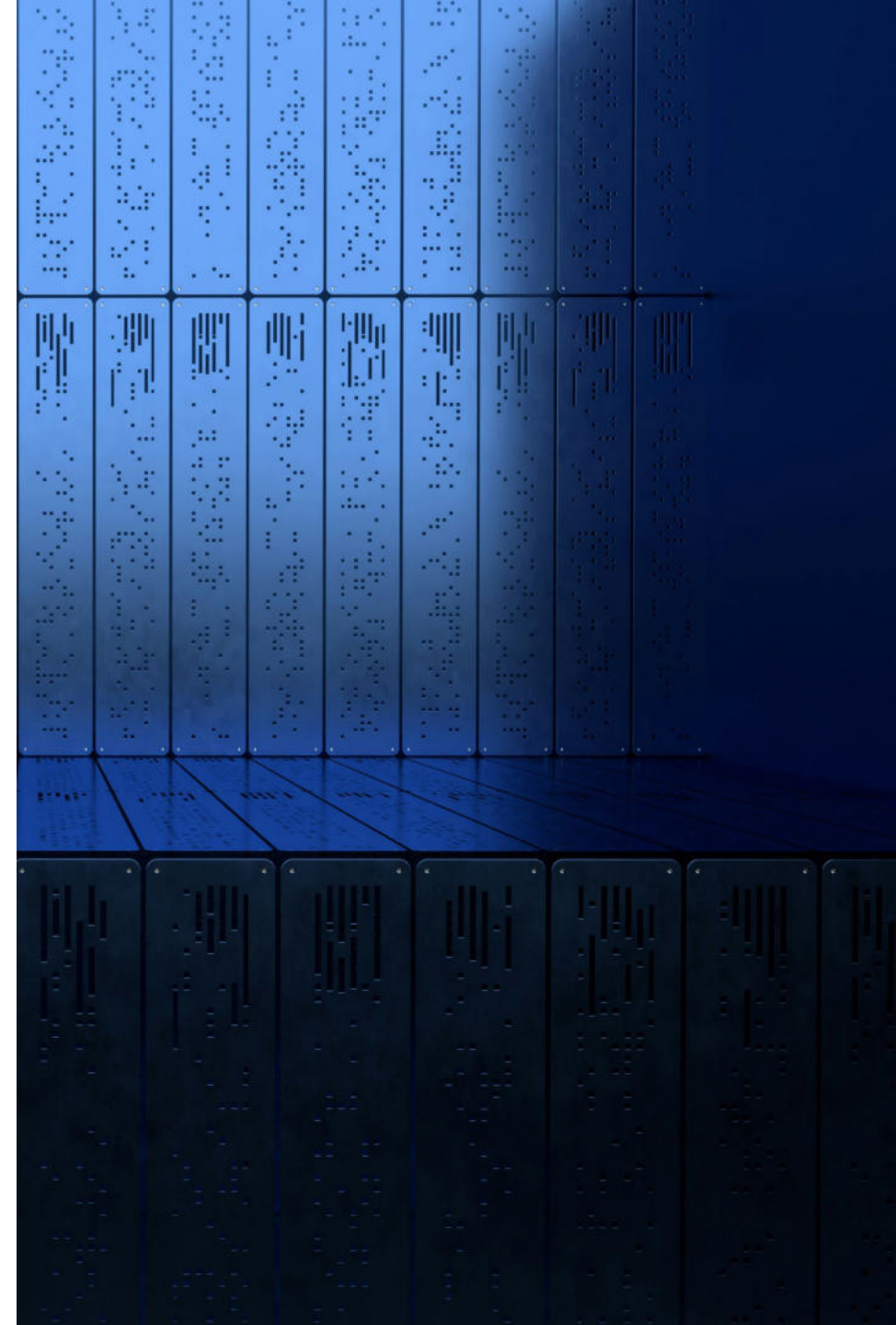


Statement Processing



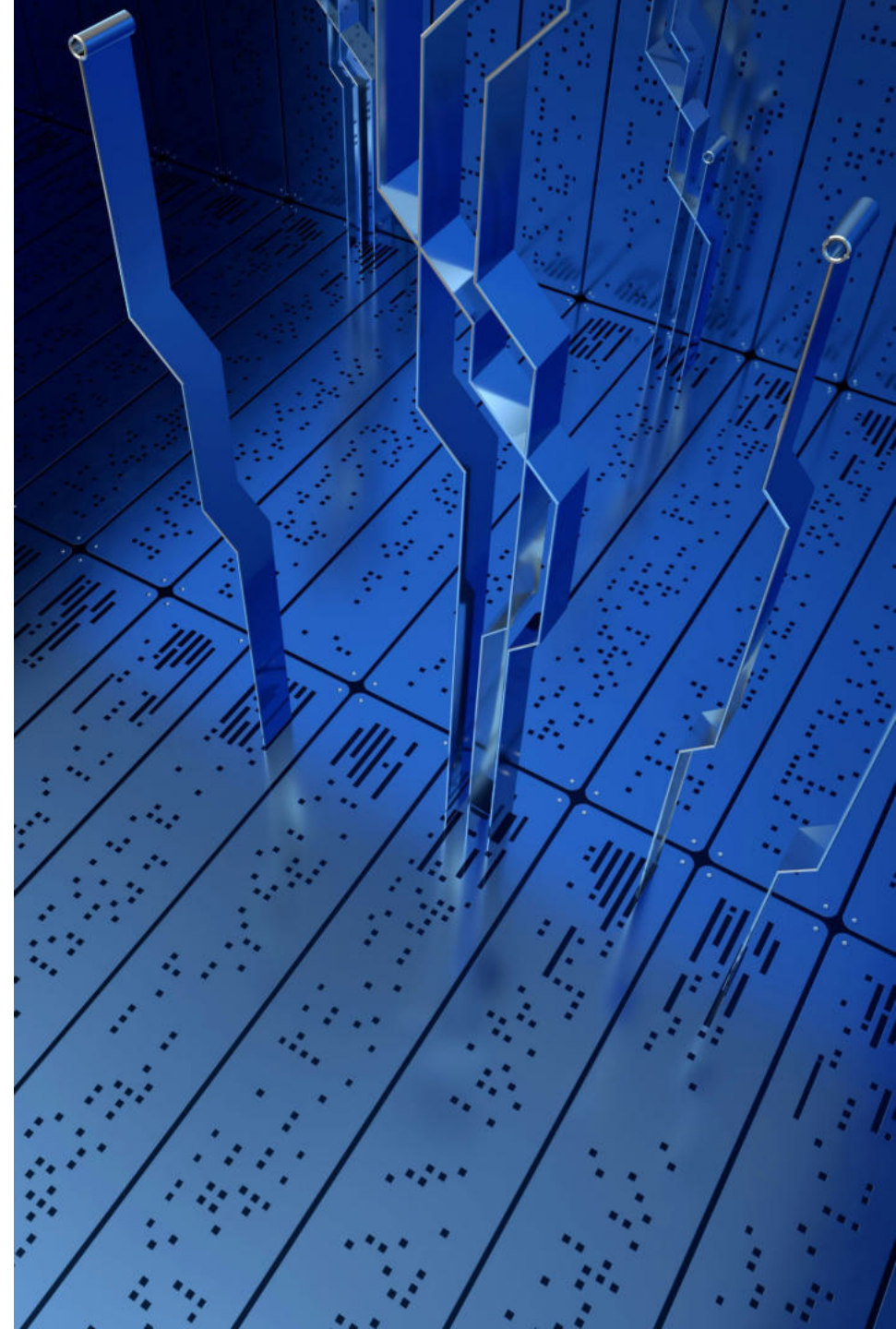
Physical Database Architecture

- Database cluster is a collection of databases managed by single server instance
- Each cluster has a separate
 - Data directory
 - TCP port
 - Set of processes
- A cluster can contain multiple databases

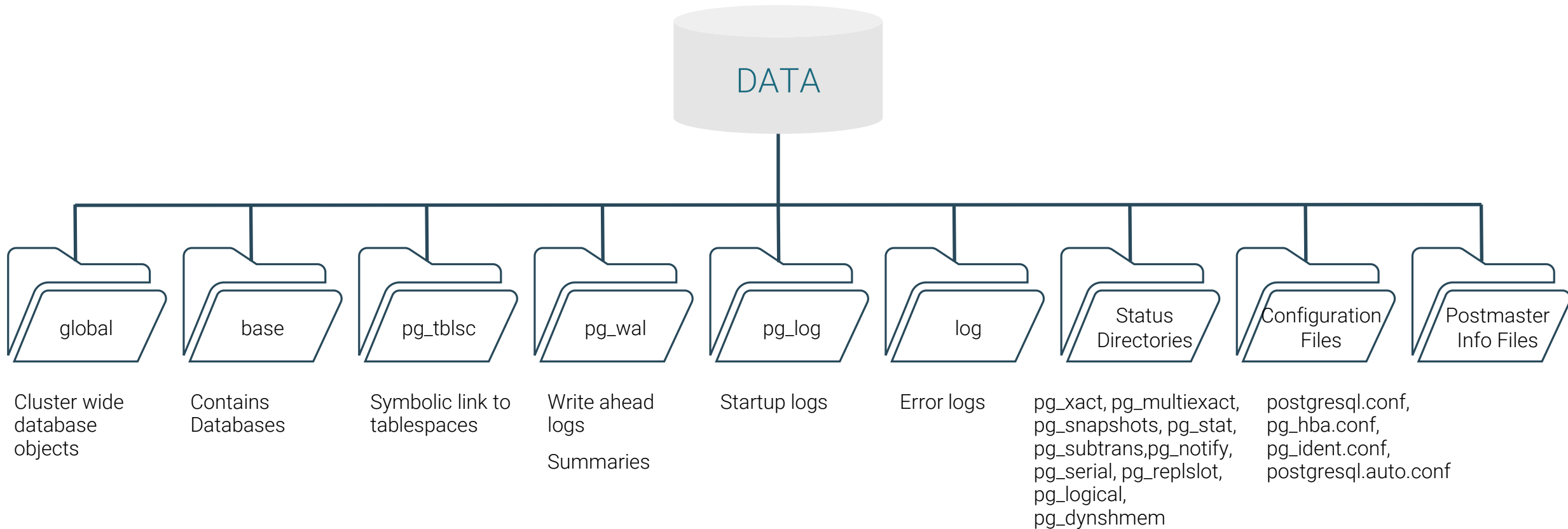


Installation Directory Layout

- Default Installation Directory Location:
 - Linux - `/usr/edb/as17`
 - Windows - `C:\Program Files\edb\as17`
- bin – Programs
- share – Shared data
- include – Header files
- lib or lib64 – Libraries



Database Cluster Data Directory Layout



Physical Database Architecture



File-per-table, file-per-index



A table-space is a directory



Each database that uses that table-space gets a subdirectory



Each relation using that table-space/database combination gets one or more files, in 1GB chunks



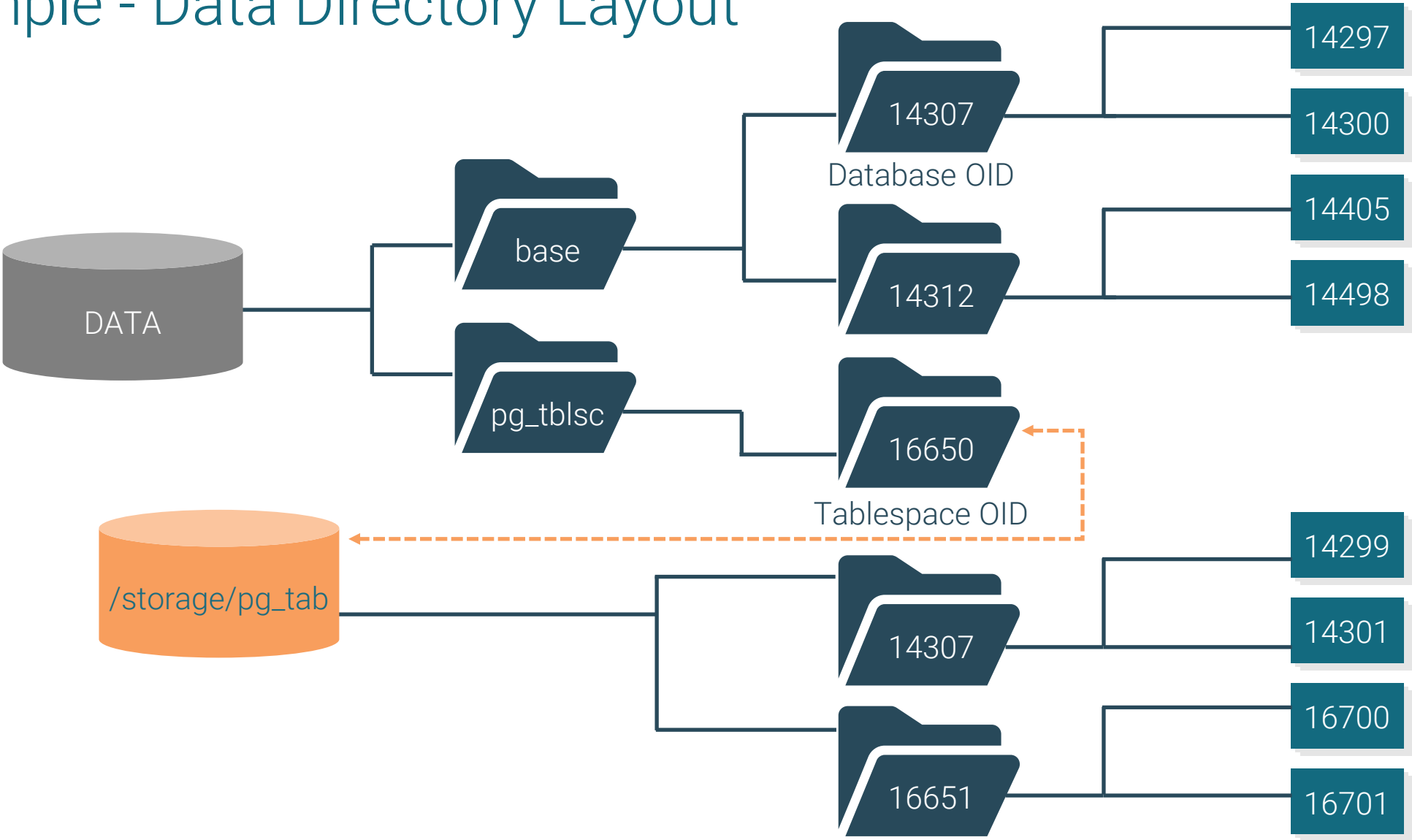
Additional files are used to hold auxiliary information (free space map, visibility map)



Each file name is a number (see `pg_class.relfilenode`)



Sample - Data Directory Layout

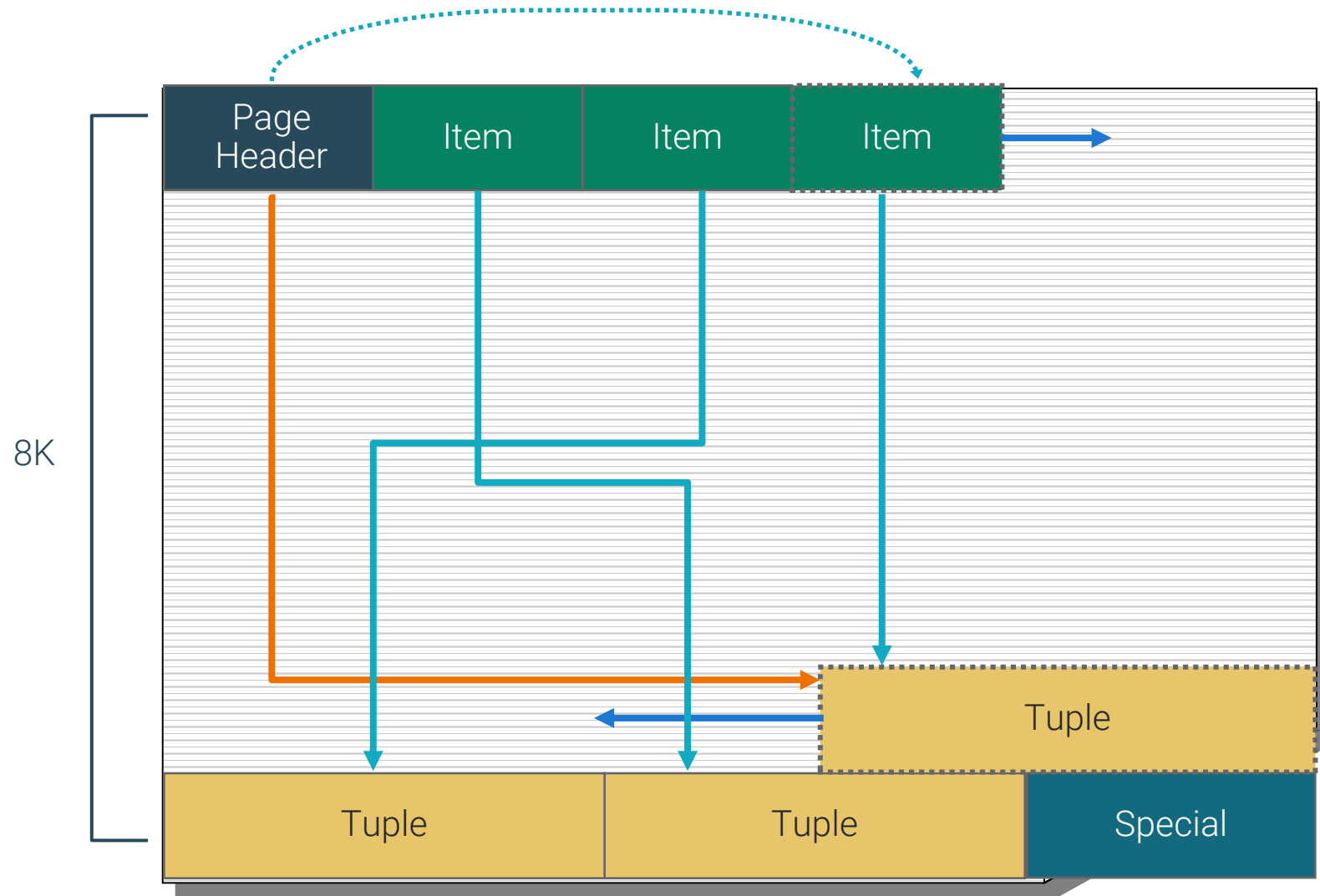


Page Layout

- Page header
 - General information about the page
 - Pointers to free space
 - 24 bytes long
- Row/index pointers
 - Array of offset/length pairs pointing to the actual rows/index entries
 - 4 bytes per item
- Free space
 - Unallocated space
 - New pointers allocated from the front, new rows/index entries from the rear
- Row/index entry
 - The actual row or index entry data
- Special
 - Index access method specific data
 - Empty in ordinary tables



Page Structure



Module Summary

- Architectural Summary
- Shared Memory
- Inter-processes Communication
- Statement Processing
- Utility Processes
- Disk Read Buffering
- Disk Write Buffering
- Background Writer Cleaning Scan
- Commit and Checkpoint
- Physical Database Architecture
- Data Directory Layout
- Installation Directory Layout
- Page Layout





Enterprise Postgres Installation



Module Objectives

- Deployment Options
- OS User and Permissions
- Package Installation
- Installation of Enterprise Postgres
- Setting Environmental Variables



Deployment Options

Deployment methods for Enterprise Postgres and supported Tools



EDB Postgres AI for CloudNativePG Cluster

Operator for managing
and deploying Enterprise
Postgres



Native packages or installers

EDB Repository can be
used for YUM and RPM
based installation.



OS User and Permissions

- Enterprise Postgres runs as a daemon (Unix / Linux) or service (Windows)
- The Enterprise Postgres Installation requires superuser/admin access
- All processes and data files must be owned by an OS user
- During installation an `enterprisedb` locked user will be created on Linux
- On Windows a password is required
- SELinux must be set to permissive mode on systems with SELinux



The enterprisedb User Account

- It is advised to run Enterprise Postgres under a separate user account
- This user account should only own the data directory that is managed by the server
- The `useradd` or `adduser` Unix command can be used to add a user
- The user account named `enterprisedb` is used throughout this training

```
[root@Base ~]# useradd enterprisedb
[root@Base ~]# passwd enterprisedb
Changing password for user enterprisedb.
New password:
Retype new password:
passwd:
all authentication tokens updated successfully.
```



Package Installation Options

Wizard Installer

- Interactive Method
- Graphical or Command Line Mode, available for Windows
- Easy Download from www.enterprisedb.com

RPM Installer

- Preferred Installation Method on Linux
- Access to EnterpriseDB's rpm Repository is required
- Dependencies are resolved manually

DNF Installer

- Attempt to Install required package dependencies
- Can be used to install EDB Postgres in Isolated Environments



Configure EDB Repositories

- An EDB account is required to access our software repositories and downloads
- Setup EDB Repositories: <https://www.enterprisedb.com/repos-downloads>
- Ensure EPEL repositories are setup if running Rocky Linux v9:

```
sudo dnf -y install https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
```

- Check Instructions for various operating systems:
<https://www.enterprisedb.com/docs/epas/latest/installing/>



YUM Installation Overview

- YUM command can be used to install Enterprise Postgres:

```
# dnf install epel-release  
# dnf config-manager --set-enabled crb  
# dnf install edb-as17-server
```

- Configure a Package Installation using service configuration file

```
# /usr/lib/systemd/system/edb-as-17.service
```

- Create a database cluster and start the cluster using services:

```
# /usr/edb/as17/bin/edb-as-17-setup initdb  
# systemctl start edb-as-17
```



Example – EDB Postgres Server Installation

- Step 1 – Login as root user and add user `enterprisedb` using `adduser` or `useradd` command and set its password:

```
# useradd enterprisedb
```

```
# passwd enterprisedb
```

- Step 2 – Install EPEL using yum:

```
# dnf install epel-release
```

- Step 3 – Configure using your secure token:

```
# curl -sLf 'https://downloads.enterprisedb.com/<secure  
token>/enterprise/setup.rpm.sh' | sudo -E bash
```



Example – Enterprise Postgres Installation (cont.)

- Step 4 – Install Enterprise Postgres using yum command

```
# dnf install edb-as17-server
```

- Step 5 - Create a database cluster and start the cluster using services:

```
# /usr/edb/as15/bin/edb-as-17-setup initdb
```

```
# systemctl start edb-as-17
```

```
# systemctl enable edb-as-17
```



Example – Post-Install Environmental Variables setup

```
[enterprisedb@pgsrv1 ~]$ vi .bash_profile
```

Edit User Profile

```
PATH=/usr/edb/as17/bin/:$PATH:$HOME/.local/bin:$HOME/bin
```

```
export PATH
```

```
export PGDATA=/var/lib/edb/as17/data/
```

```
export PGUSER=enterprisedb
```

```
export PGPORT=5444
```

```
export PGDATABASE=enterprisedb
```

Logoff and Login

```
[enterprisedb@pgsrv1 ~]$ exit
```

```
logout
```

```
[root@pgsrv1 ~]# su - enterprisedb
```

Verify
Environmental
Settings

```
[enterprisedb@pgsrv1 ~]$ which psql
```

```
/usr/edb/as17/bin/psql
```

```
[enterprisedb@pgsrv1 ~]$ pg_ctl status
```

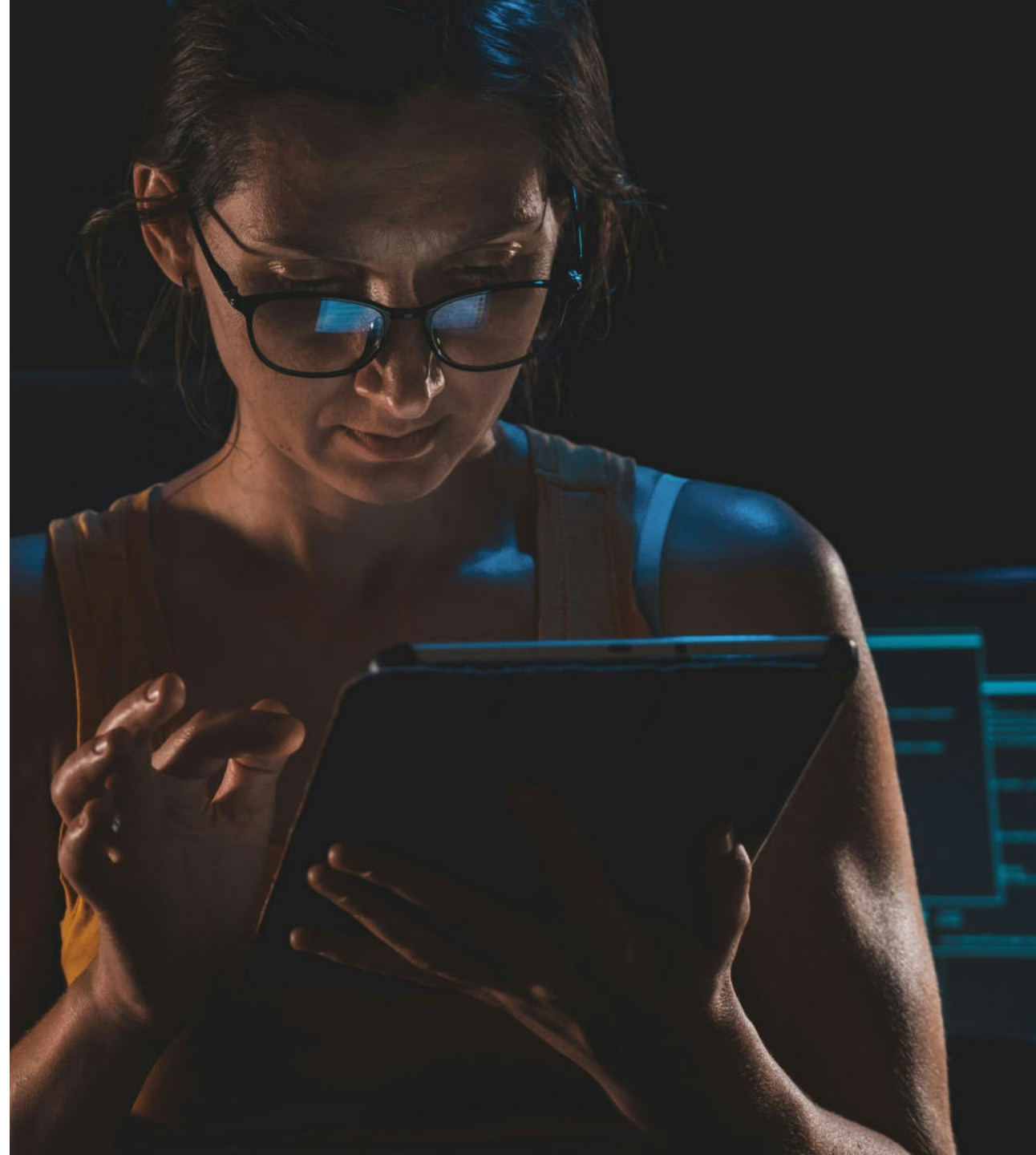
```
pg_ctl: server is running (PID: 66406)
```

```
/usr/edb/as17/bin/edb-postgres
```



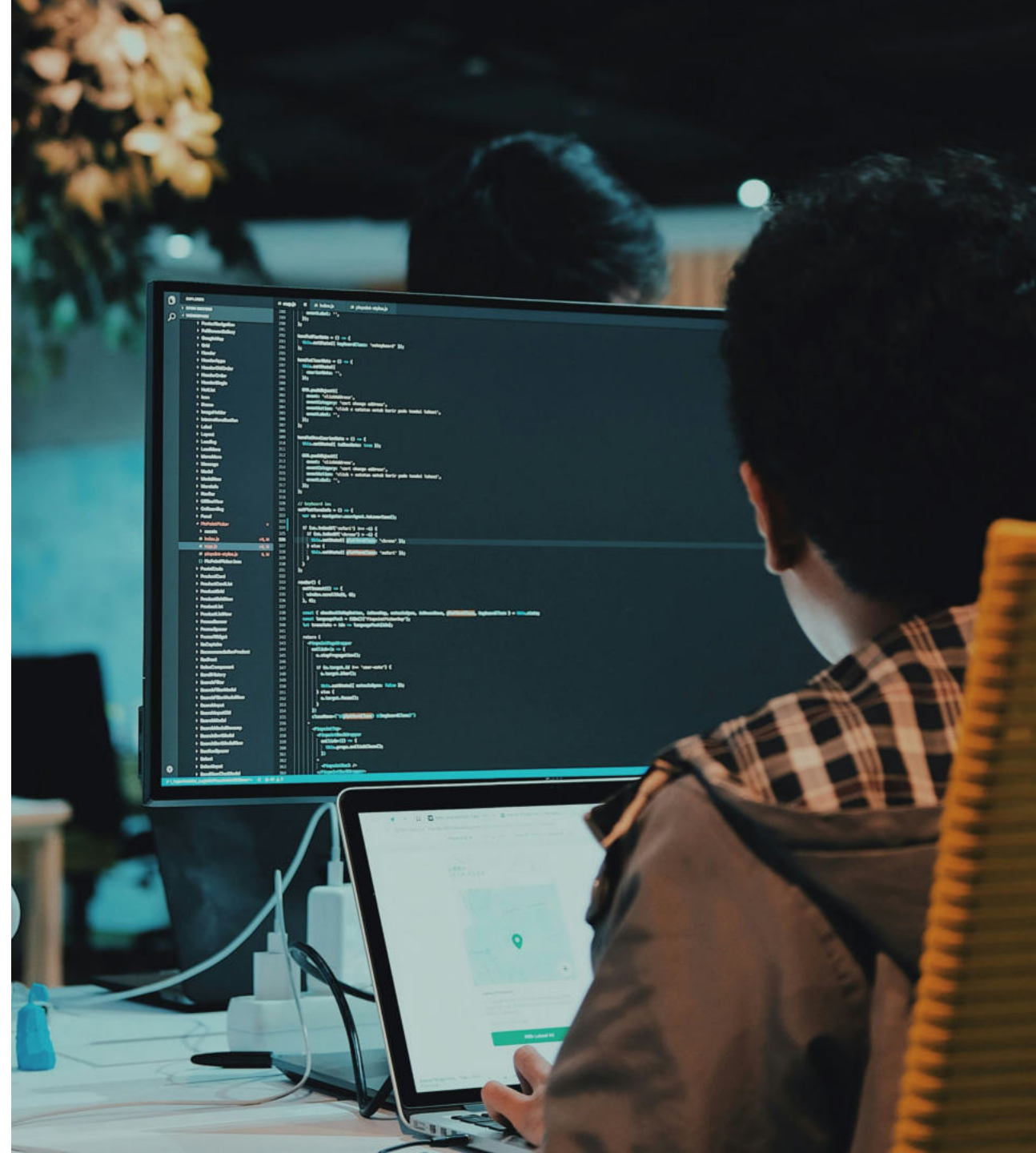
Module Summary

- OS User and Permissions
- Installation Options
- Installation of Enterprise Postgres
- Setting Environmental Variables



Lab Exercise - 1

- Choose the platform on which you want to install Enterprise Postgres (formerly known as EDB Postgres Advanced Server)
- Download the Enterprise Postgres installer from the EnterpriseDB website for the chosen platform
- Prepare the platform for installation
- Install Enterprise Postgres
- Connect to Enterprise Postgres using psql





User Tools - Command Line Interfaces



Module Objectives

- Introduction to psql
- Connecting to Database
- psql Command Line Parameters
- psql Meta-Commands
- Conditional and Information Commands
- EDB*Plus
- Installing and Starting EDB*Plus
- EDB*Plus Commands



Introduction to psql

- **psql** is a command line interface (CLI) to Postgres
- Can be used to execute SQL queries and **psql** meta commands

```
[enterprisedb@Base ~]$ psql -h /tmp -p 5444 -U enterprisedb -d edb  
Type "help" for help.  
edb=# \q
```



Connecting to a Database

psql Connection Options:

- -d <Database Name>
- -h <Hostname>
- -p <Database Port>
- -U <Database Username>

Environmental Variables

- PGDATABASE, PGHOST, PGPORT and PGUSER



Conventions

- **psql** has its own set of commands, all of which start with a backslash (\).
- Some commands accept a pattern. This pattern is a modified regex. Key points:
 - * and ? are wildcards
 - Double-quotes are used to specify an exact name, ignoring all special characters and preserving case



On Startup...

- **psql** will execute commands from `$HOME/.psqlrc`, unless option `-X` is specified
- `-f FILENAME` will execute the commands in `FILENAME`, then exit
- `-c COMMAND` will execute `COMMAND` (SQL or internal) and then exit
- `--help` will display all the startup options, then exit
- `--version` will display version info and then exit



Entering Commands

- psql uses the command line editing capabilities that are available in the native OS. Generally, this means:
 - Up and Down arrows cycle through command history
 - On UNIX, there is tab completion for various things, such as SQL commands



History and Query Buffer

- `\s` will show the command history
- `\s FILENAME` will save the command history
- `\e` will edit the query buffer and then execute it
- `\e FILENAME` will edit FILENAME and then execute it
- `\w FILENAME` will save the query buffer to FILENAME



Controlling Output

- `psql -o FILENAME` or meta command `\o FILENAME` will send query output (excluding STDERR) to FILENAME
- `\g FILENAME` executes the query buffer sending output to FILENAME
- `\watch <seconds>` can be used to run previous query repeatedly



Advanced Features - Variables

- **psql** provides variable substitution
- Variables are simply name/value pairs
- Use `\set` meta command to set a variable

```
=> \set city Edmonton
```

```
=> \echo :city
```

```
Edmonton
```

- Use `\unset` to delete a variable

```
=> \unset city
```



Advanced Features - Special Variables

- Settings can be changed at runtime by altering special variables
- Some important special variables include:
 - AUTOCOMMIT, ENCODING, HISTFILE, ON_ERROR_ROLLBACK, ON_ERROR_STOP, PROMPT1 and VERBOSITY
- Example:

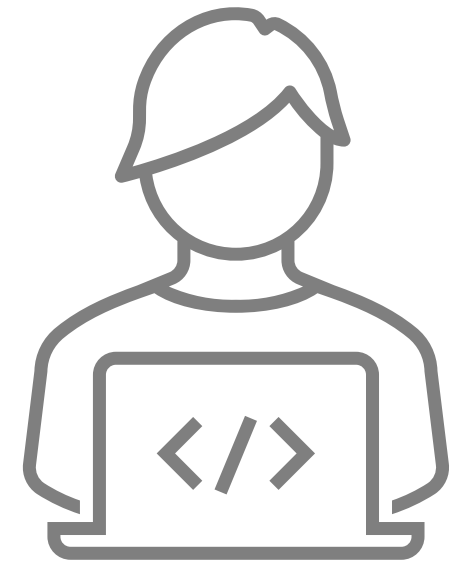
```
=# \set AUTOCOMMIT off
```

- Once AUTOCOMMIT is set to off use COMMIT/ROLLBACK to complete the running transaction



Conditional Commands

- Conditional commands primarily helpful for scripting
 - `\if` `EXPR` begin conditional block
 - `\elif` `EXPR` alternative within current conditional block
 - `\else` final alternative within current conditional block
 - `\endif` end conditional block



Information Commands

- `\d[(i|s|t|v|b|S)]+[pattern]`
 - List of objects (indexes, sequences, tables, views, tablespaces and dictionaries)
- `\d+[pattern]`
 - Describe structure details of an object
- `\l[ist] +`
 - Lists of databases in a database cluster



Information Commands (continued)

- `\dn+ [pattern]`
 - Lists schemas (namespaces)
 - + adds permissions and description to output
- `\df[+] [pattern]`
 - Lists functions
 - + adds owner, language, source code and description to output



Common psql Meta Commands

- `\q` or `^d` or `quit` or `exit`
 - Quits the psql program
- `\cd [ directory ]`
 - Change current working directory
 - Tip - To print your current working directory, use `\! pwd`
- `\! [ command ]`
 - Executes the specified command
 - If no command is specified, escapes to a separate Unix shell (CMD.EXE in Windows)



Help

- `\conninfo`
 - Current connection information
- `\?`
 - Shows help information about psql commands
- `\h [command]`
 - Shows information about SQL commands
 - If command isn't specified, lists all SQL commands
- `psql --help`
 - Lists command line options for psql



EDB*Plus



EDB*Plus

- EDB*Plus is a command line user interface to the Enterprise Postgres
- EDB*Plus accepts SQL commands, SPL anonymous blocks, and EDB*Plus commands
- EDB*Plus commands are compatible with Oracle SQL*Plus commands
- edb-edbplus package is available to install EDB*Plus using yum command and requires java runtime



EDB*Plus Features

- EDB*Plus can be used for:
 - Querying certain database objects
 - Executing stored procedures
 - Formatting output from SQL commands
 - Executing batch scripts
 - Executing OS commands
 - Recording output



Module Summary

- Introduction to psql
- Connecting to Database
- psql Command Line Parameters
- psql Meta-Commands
- Conditional and Information Commands
- EDB*Plus
- Installing and Starting EDB*Plus
- EDB*Plus Commands



Prepare Lab Environment

- In the training materials provided by EDB there is a script file [edbstore.sql](#) that can be executed using `edb-psql` to create a sample `edbstore` database. Here are the steps:
 - Download the [edbstore.sql](#) file and place in a directory which is accessible to the `enterprisedb` user
 - Login as `enterprisedb` OS user
 - Run the `edb-psql` command with the `-f` option to execute the [edbstore.sql](#) file and install all the sample objects required for this training
 - `$ edb-psql -p 5444 -f edbstore.sql -d edb -U enterprisedb`
 - Note - The above command will prompt for `edbuser` password. The password is `edbuser`



Lab Exercise - 1

- In this lab exercise you will have a chance to practice what you have learned through using command line interfaces:

1. Connect to a database using psql
2. Switch databases
3. Describe the customer's table
4. Describe the customers table including description
5. List all databases
6. List all schemas
7. List all tablespaces
8. Execute a sql statement, saving the output to a file
9. Do the same thing, just saving data, not the column headers
10. Create a script via another method, and execute from psql
11. Turn on the expanded table formatting mode
12. Lists tables, views and sequences with their associated access privileges
13. Which meta command displays the SQL text for a function?
14. View the current working directory





Database Clusters



Module Objectives

- Database Clusters
- Creating a Database Cluster
- Starting and Stopping the Server (pg_ctl)
- Connecting to the Server Using psql



Database Clusters

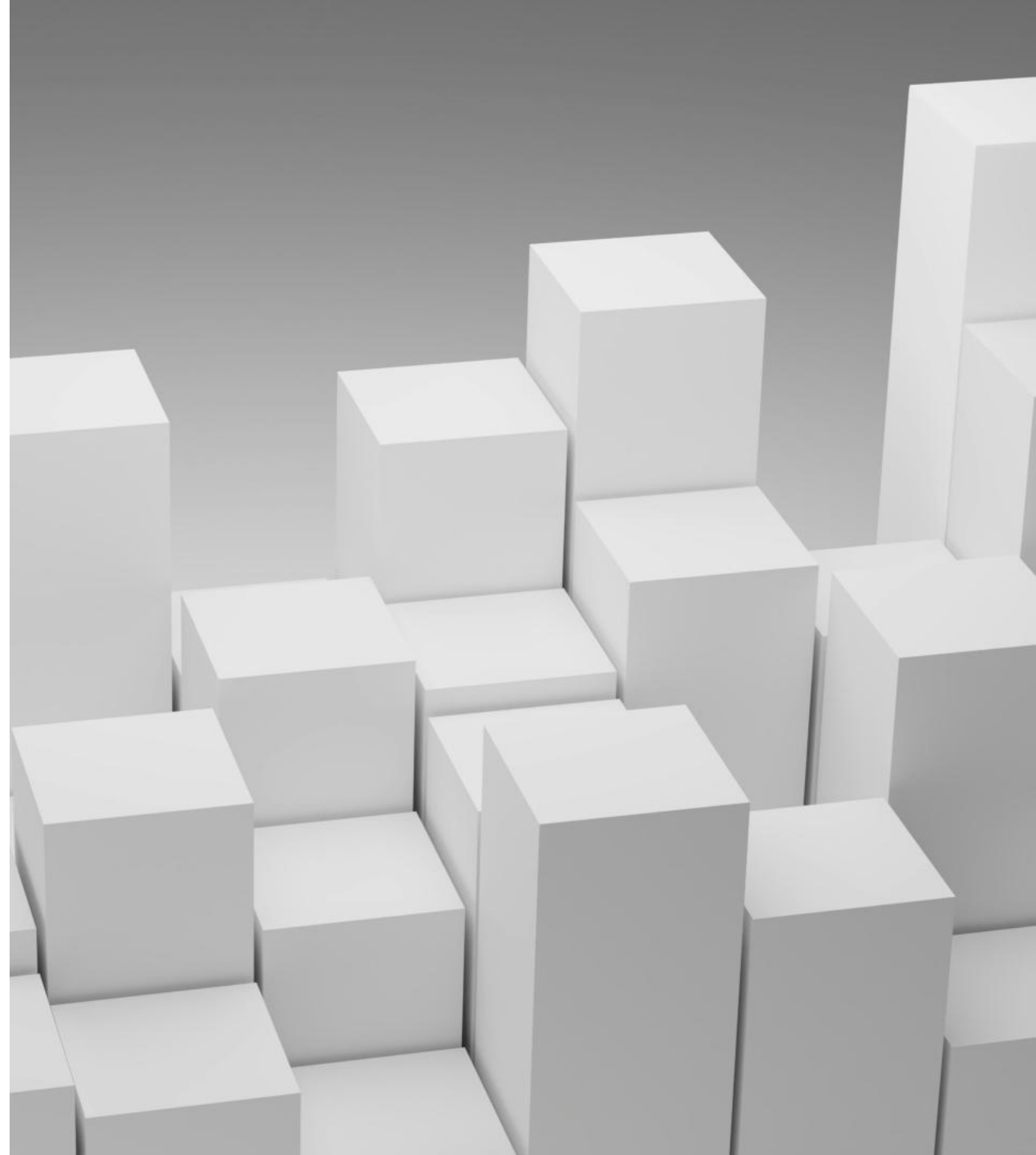
A Database Cluster is a collection of databases managed by a single server instance

Database Clusters are comprised of:

- Data directory
- Port

Default databases are created named:

- template0
- template1
- postgres
- edb



Creating a Database Cluster

- Choose the data directory location for new cluster
- Initialize the database cluster storage area (data directory) using the `initdb` utility
- `initdb` will create the data directory if it doesn't exist
- You must have permissions on the parent directory so that `initdb` can create the data directory
- The data directory can be created manually by superuser and the ownership can be given to **enterprisedb** user



initdb Utility

```
$ initdb [OPTION] ... [DATADIR]
```

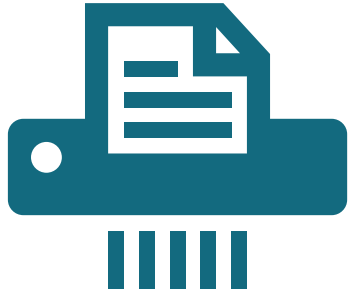
Options:

- `-D, --pgdata` location for this database cluster
- `-E, --encoding` set default encoding for new databases
- `-U, --username` database superuser name
- `-W, --pwprompt` prompt for a password for the new superuser
- `-X, --waldir` location for the write-ahead log directory
- `--wal-segsize` size of wal segments , in megabytes
- `-k, --data-checksums` use data page checksums
- `--no-redwood-compat` Do not install Oracle compatibility features
- `--redwood-like` Use Oracle compatible behavior
- `-y, --data-encryption` enable transparent data encryption
- `-?, --help` show this help, then exit

If the data directory is not specified, the environment variable `PGDATA` is used



Enabling Transparent Data Encryption



Transparent Data Encryption (TDE) can be used to encrypt data files, WAL and temporary files

Can only be done at cluster creation



Following initdb command options can be used to enable TDE:

```
-y, --data-encryption  
--copy-key-from=<file>  
--key-wrap-command=<command>  
--key-unwrap-command=<command>  
--no-key-wrap
```



Example - initdb

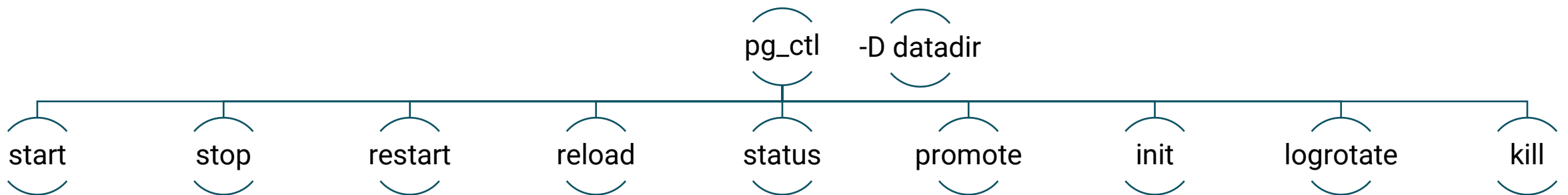
```
[root@Base ~]# mkdir /edbstore  
[root@Base ~]# chown enterprisedb:enterprisedb /edbstore  
[root@Base ~]# su - enterprisedb  
  
[enterprisedb@Base ~]$ initdb -D /edbstore --wal-segsize 1024 -W
```

- In the above example the database system will be owned by user `enterprisedb`
- The `enterprisedb` user is the database superuser
- The default server config file will be created in `/edbstore` named `postgresql.conf`
- `--wal-segsize 1024 MB` specifies the write-ahead log file segment size
- `-W` is used to force **initdb** to prompt for the superuser password



pg_ctl Utility

- **pg_ctl** is a command line utility provided by Postgres to initialize, start, stop and control a Postgres instance
- It provide options for redirecting start log, controlled startup and shutdown
- `-D` option or environmental variable `PGDATA` can be used to specify cluster data directory



Starting a Database Cluster

- After initializing a database cluster, a unique port must be assigned
- Choose a unique port for postmaster in **postgresql.conf**
- Start the database cluster using **pg_ctl** utility
- Example:

```
[enterprisedb@Base ~]$ vi /edbstore/postgresql.conf
port = 5434

[enterprisedb@Base ~]$ pg_ctl -D /edbstore/ -l /edbstore/startlog start
waiting for server to start.... done
server started

[enterprisedb@Base ~]$ pg_ctl -D /edbstore/ status
pg_ctl: server is running (PID: 62239)
```



Connecting To a Database Cluster

- The edb-psql and PEM clients can be used for connections

```
[enterprisedb@Base ~]$ psql -p 5434 -d edb -U
enterprisedb
Type "help" for help.

edb=# show port;
 port
-----
 5434
(1 row)

edb=# show data_directory;
 data_directory
-----
 /edbstore
(1 row)

edb=# \q
[enterprisedb@Base ~]$
```



Reload a Database Cluster

- Some configuration parameter changes do not require a restart
- Changes can be reloaded using the `pg_ctl` utility
- Changes can also be reloaded using `pg_reload_conf()`
- Syntax:

```
$ pg_ctl reload [options]
```

- D location of the database cluster's data directory
- s only print errors, no informational messages



Stopping a Database Cluster

- `pg_ctl` supports three modes of shutdown
 - Smart quit after all clients have disconnected
 - Fast quit directly, with proper shutdown (default)
 - Immediate quit without complete shutdown; will lead to recovery
- Syntax:
`$ pg_ctl stop [-W] [-t SECS] [-D DATADIR] [-s] [-m SHUTDOWN-MODE]`
- Example:

```
[enterprisedb@Base ~]$ pg_ctl -D /edbstore/ stop
waiting for server to shut down.... done
server stopped

[enterprisedb@Base ~]$ pg_ctl -D /edbstore/ status
pg_ctl: no server running
```



View Cluster Control Information

- `pg_controldata` can be used to view the control information for a database cluster
- It can be run with `data directory` as an option

```
[enterprisedb@Base ~]$ pg_controldata /edbstore/
.....
Database system identifier:      6724770293870218226
Database cluster state:         shut down
Latest checkpoint location:     0/41A3AA40
Latest checkpoint's REDO WAL file: 00000001000000000000000001
Latest checkpoint's TimeLineID: 1
Backup start location:          0/0
Backup end location:            0/0
wal_level setting:              replica
Database block size:            8192
WAL block size:                 8192
Data page checksum version:     0
```



Module Summary

- Database Clusters
- Creating a Database Cluster
- Starting and Stopping the Server (pg_ctl)
- Connecting to the Server Using psql



Lab Exercise - 1

- A new website is to be developed for an online music store.
 - Create a new cluster `edbdata` with ownership of `enterprisedb` user
 - Start your `edbdata` cluster
 - Reload your cluster with `pg_ctl` utility and using `pg_reload_conf()` function
 - Stop your `edbdata` cluster with `fast` mode





Configuration



Module Objectives

- Server Parameter File - postgresql.conf
- Viewing and Changing Server Parameters
- Configuration Parameters - Security, Resources and WAL
- Configuration Parameters - Error Logging, Planner and Maintenance
- Viewing Compilation Settings
- Using File Includes



Setting Server Parameters

- There are many configuration parameters that effect the behavior of the database system
- All parameter names are case-insensitive
- Every parameter takes a value of one of five types:
 - boolean
 - integer
 - floating point
 - string
 - enum
- One way to set these parameters is to edit the file postgresql.conf, which is normally kept in the data directory



The Server Parameter File - postgresql.conf

- Holds parameters used by a cluster
- Parameters are case-insensitive
- Normally stored in data directory
- `initdb` installs default copy
- Some parameters only take effect on server restart (`pg_ctl restart`)
- `#` used for comments
- One parameter per line
- Use include directive to read and process another file
- Can also be set using the command-line option



Viewing and Changing Server Parameters

Configuration parameters can be viewed using:

- SHOW command
- pg_settings
- pg_file_settings

Configuration parameters can be modified for:

- Single session using the SET command
- Database user using ALTER USER
- Single database using ALTER DATABASE



Changing Configuration Parameter at Cluster Level

```
[enterprisedb@pgsrv1 ~] psql edb
enterprisedb

edb=# ALTER SYSTEM SET work_mem=20480;
ALTER SYSTEM
edb=# SELECT pg_reload_conf();

edb=# ALTER SYSTEM RESET work_mem;
ALTER SYSTEM
edb=# SELECT pg_reload_conf();
```

- Use `ALTER SYSTEM` command to edit cluster level settings without editing `postgresql.conf`
- `ALTER SYSTEM` writes new setting to `postgresql.auto.conf` file which is read at last during server reload/restarts
- Parameters can be modified using `ALTER SYSTEM` when required



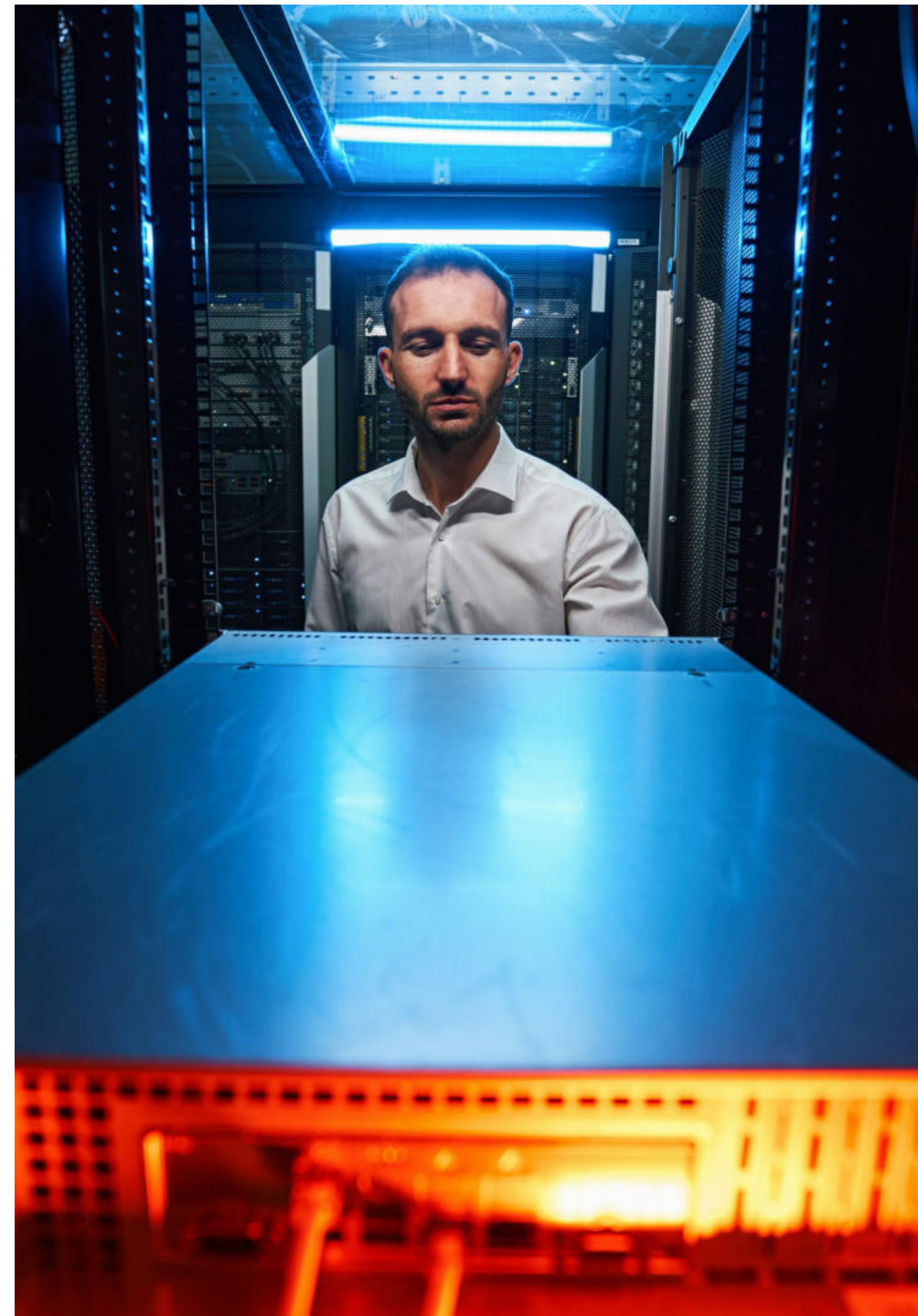
Connection Settings

- **listen_addresses** (default *) - Specifies the addresses on which the server is to listen for connections. Use * for all
- **port** (default 5444) - The port the server listens on
- **max_connections** (default 100) - Maximum number of concurrent connections the server can support
- **superuser_reserved_connections** (default 3) - Number of connection slots reserved for superusers
- **unix_socket_directory** (default /tmp) - Directory to be used for UNIX socket connections to the server
- **unix_socket_permissions** (default 0777) - access permissions of the Unix-domain socket



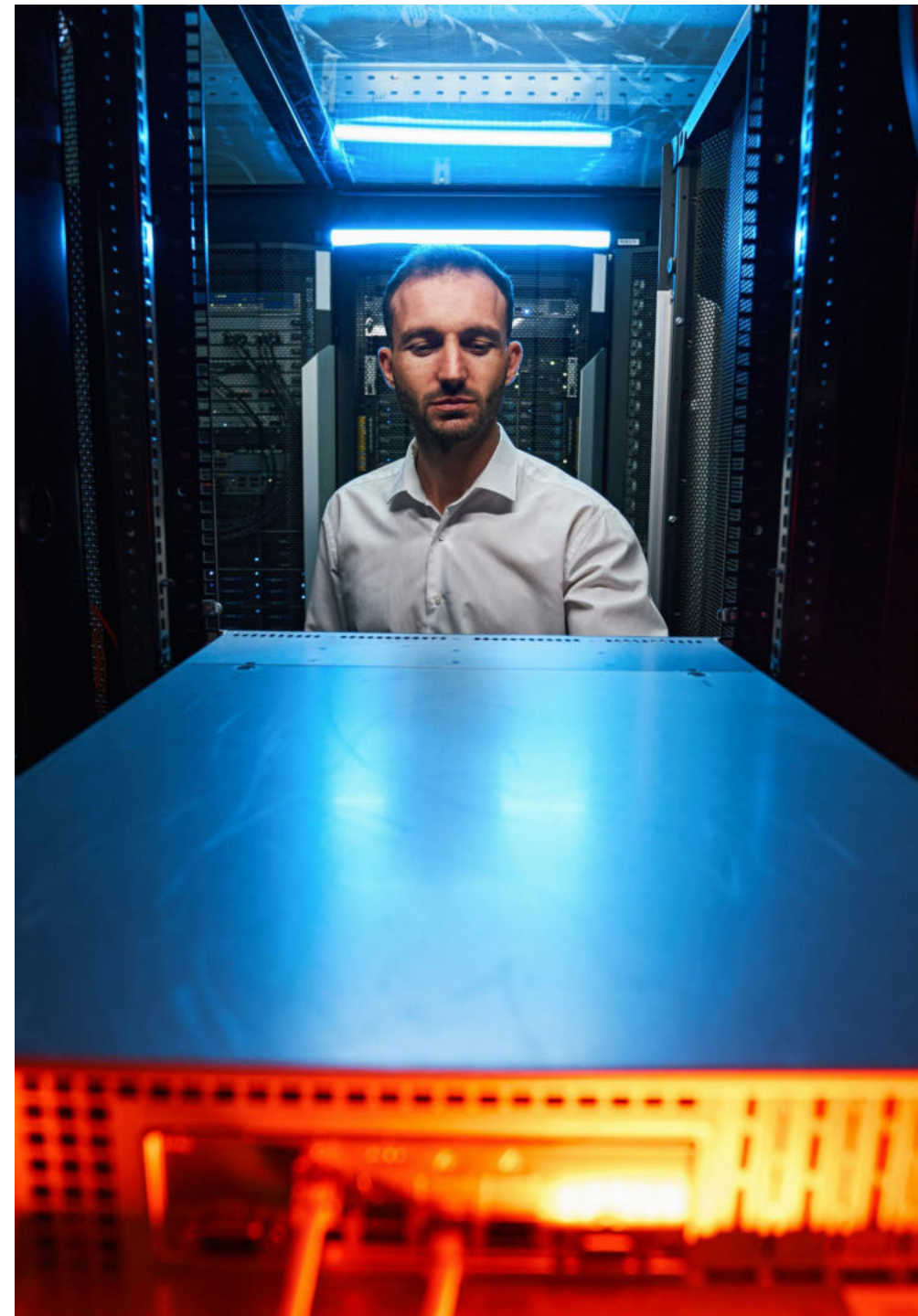
Security and Authentication Settings

- **authentication_timeout** (default is 1 minute) – Maximum time to complete client authentication, in seconds
- **row_security** (default is on) – Controls row security policy behavior
- **password_encryption** (default scram-sha-256) – Determines the algorithm to use to encrypt password
- **ssl** (default: off) - Enables SSL connections

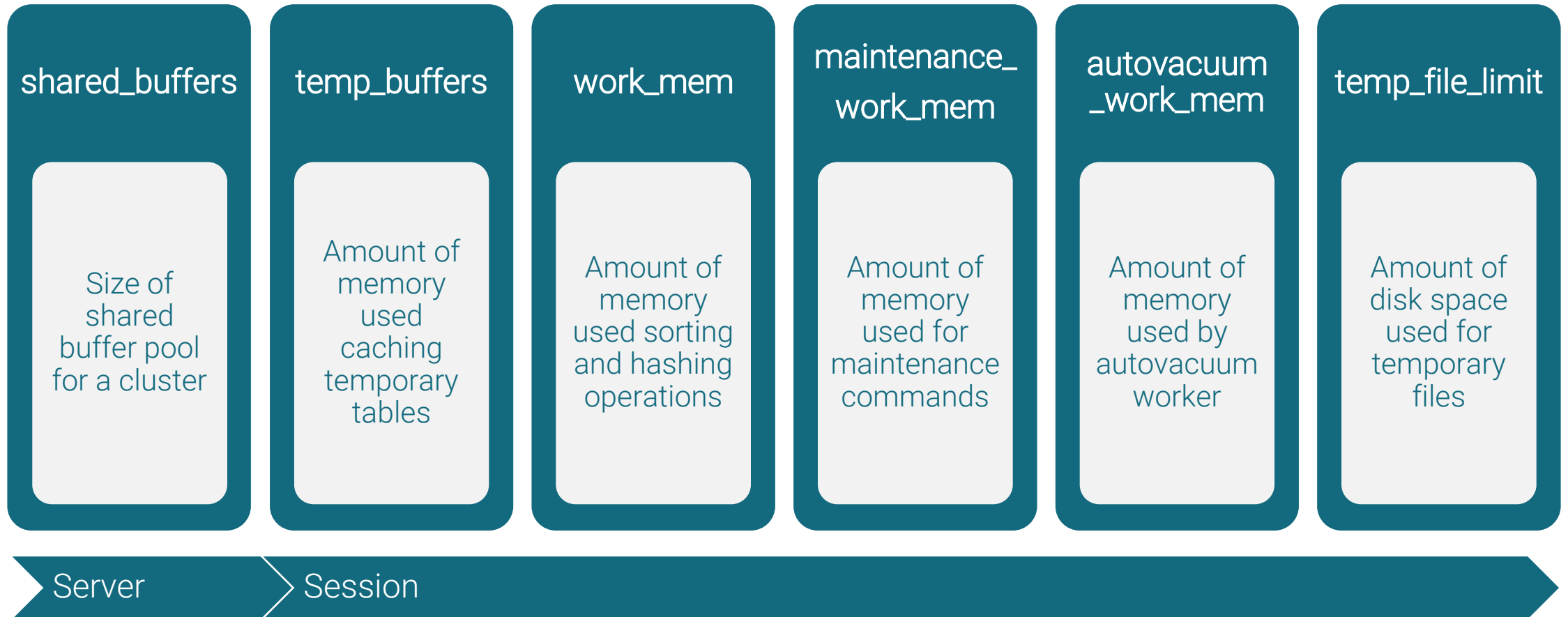


SSL Settings

- **ssl_ca_file** - Specifies the name of the file containing the SSL server certificate authority (CA)
- **ssl_cert_file** - Specifies the name of the file containing the SSL server certificate
- **ssl_key_file** - Specifies the name of the file containing the SSL server private key
- **ssl_ciphers** - List of SSL ciphers that may be used for secure connections
- **ssl_dh_params_file** – Specifies file name for custom OpenSSL DH parameters



Memory Settings



Dynatune Dynamic Tuning

- Dynatune functionality allows Advanced Server to make optimal usage of the system resources
- Dynatune automatically configures various resource parameters at the time of startup

`edb_dynatune (0-100)`

This parameter determines how Advanced Server allocates system resources

`edb_dynatune_profile (oltp | reporting | mixed)`

This parameter controls performance tuning aspects based on the type of work that the server performs



Resource Manager

- Prevents any single process from monopolizing resources to the detriment of other processes
- A resource group can limit the % of cpu or rate of dirty buffer IO
- Helps in managing multiple kinds of workload on your server

edb_max_resource_groups

This parameter controls the maximum number of active resource groups

edb_resource_group

Set per session to the name of the resource group in Resource Manager



Query Planner Settings

`random_page_cost` (default 4.0) - Estimated cost of a random page fetch.

May need to be reduced to account for caching effects

`seq_page_cost` (default 1.0) - Estimated cost of a sequential page fetch.

`effective_cache_size` (default 4GB) - Used to estimate the cost of an index scan.

`enable_hints` (default true) - Controls whether Optimizer hints embedded in SQL commands are utilized or not

`plan_cache_mode` (default auto) – Controls custom or generic plan execution for prepared statements. Can be set to auto, force_custom_plan and force_generic_plan



Write Ahead Log Settings

`wal_level` (default replica) – Determines how much information is written to the WAL. Other values are minimal and logical

`fsync` (default on) – Force WAL buffer flush at each commit, Turning this off can cause lead to arbitrary corruption in case of a system crash

`wal_buffers` (default -1, autotune) – The amount of memory used in shared memory for WAL data. The default setting of -1 selects a size equal to 1/32nd (about 3%) of shared_buffers

`min_wal_size` (default 80 MB) – The WAL size to start recycling the WAL files

`max_wal_size` (default 1GB) – The WAL size to start checkpoint. Controls the number of WAL Segments(16MB each) after which checkpoint is forced

`checkpoint_timeout` (default 5 minutes) – Maximum time between checkpoints

`wal_compression` (default off) – The WAL of Full Page write will be compressed and written

`summarize_wal` (default off) – Must be turned on for incremental backups to work



Where To Log

log_destination

Controls logging type for a database cluster.

Can be set to stderr, csvlog, jsonlog, syslog, and eventlog

logging_collector

Enables logger process to capture stderr and csv logging messages

These messages can be redirected based on configuration settings

Log File and Directory Settings

log_directory - Directory where log files are written

log_filename - Format of log file name (e.g. postgresql-%Y-%m-%d_%H%M%S.log)

log_file_mode - permissions for log files

log_rotation_age - Used for file age-based log rotation

log_rotation_size - Used for file size-based log rotation



When To Log

Duration and sampling

`log_min_messages`

Messages of this severity level or above are sent to the server log

`log_min_error_statement`

When a message of this severity or higher is written to the server log, the statement that caused it is logged along with it

`log_min_duration_statement`

When a statement runs for at least this long, it is written to the server log

`log_autovacuum_min_duration`

Logs any Autovacuum activity running for at least this long

`log_statement_sample_rate`

Percentage of queries(above `log_autovacuum_min_duration`) to be logged

`log_transaction_sample_rate`

Sample a percentage of transactions by logging statements



What To Log

`log_connections`

Log successful connections to the server log

`log_disconnections`

Log some information each time a session disconnects, including the duration of the session

`log_temp_files`

Log temporary files of this size or larger, in kilobytes

`log_checkpoints`

Causes checkpoints and restart points to be logged in the server log

`log_lock_waits`

Log information if a session is waits longer than `deadlock_timeout` to acquire a lock

`log_error_verbosity`

How detailed the logged message is. Can be set to default, terse or verbose

`log_line_prefix`

Additional details to log with each line. Default is `'%m [%p] '` which logs a timestamp and the process ID

`log_statement`

Legal values are none, ddl, mod (DDL and all other data-modifying statements), or all



Auditing

- Enterprise Postgres allows you to track and analyze database activities using the EDB Audit Logging functionality
- The audit logs can be configured to record information such as:
 - When a role establishes a connection to the database
 - What database objects are created, modified or deleted by a role
 - When any failed authentication attempts have occurred
- The parameters can be specified in the [postgresql.conf](#) or [postgresql.auto.conf](#) files



Where To Audit

- `edb_audit` (default `none`) – enables or disables database auditing. Possible values are `none` or `xml` or `csv`
- `edb_audit_directory` (default `PGDATA/edb_audit`) – specifies where the audit log files will be created
- `edb_audit_destination` (default `file`) - Specifies whether the audit log information is to be recorded in the directory as given by the `edb_audit_directory` parameter or to the directory and file managed by the `syslog` process. Set to `syslog` to use the `syslog` process and its location as configured in the `/etc/syslog.conf` file
- `edb_audit_filename` (default `audit-%Y%m%d_%H%M%S`) – file name of the audit file where the auditing information will be stored



What To Audit

- `edb_audit_connect` (default `failed`) – enables auditing of database connection attempts by users. Possible values are `none`, `failed` or `all`
- `edb_audit_disconnect` (default `none`) – enables auditing of disconnections by connected users. Possible values are `all` or `none`
- `edb_audit_statement` (default `ddl,error`) – specifies auditing of different categories of SQL statements. Various combinations of these values may be specified: `none`, `dml`, `insert`, `update`, `delete`, `truncate`, `select`, `error`, `rollback`, `ddl`, `create`, `drop`, `alter`, `grant`, `revoke` or `all`
- `edb_audit_tag` (default `none`) – specifies a string value that will be included in the audit log when the `edb_audit` parameter is set to `csv` or `xml`



Controlling Audit Trail

- `edb_audit_rotation_day` (default `every`) – specifies the day of the week on which to rotate the audit files. Possible values are `sun, mon, tue, wed, thu, fri, sat, sun, every` or `none`
- `edb_audit_rotation_size` (default `0MB`) – specifies a file size threshold in MB when file rotation will be forced to occur
- `edb_audit_rotation_seconds` (default `0`) – specifies the rotation time in seconds when a new logfile should be created



Background Writer Settings

- `bgwriter_delay` (default 200 ms) - Specifies time between activity rounds for the background writer
- `bgwriter_lru_maxpages` (default 100) - Maximum number of pages that the background writer may clean per activity round
- `bgwriter_lru_multiplier` (default 2.0) - Multiplier on buffers scanned per round. By default, if system thinks 10 pages will be needed, it cleans $10 * \text{bgwriter_lru_multiplier}$ of 2.0 = 20
- Primary tuning technique is to lower `bgwriter_delay`



Statement Behavior

- `search_path` - This parameter specifies the order in which schemas are searched. The default value for this parameter is `"$user", public`
- `default_tablespace` - Name of the tablespace in which objects are created by default
- `temp_tablespaces` - Tablespaces name(s) in which temporary objects are created
- `statement_timeout` - Postgres will abort any statement that takes over the specified number of milliseconds A value of zero (the default) turns this off
- `idle_in_transaction_session_timeout` – Terminates any session with an open transaction that has been idle for longer than the specified duration in milliseconds



Parallel Query Scan Settings

- Advanced Server supports parallel execution of read-only queries
- Can be enabled and configured by using configuration parameters
 - `max_parallel_workers_per_gather` (default 2): Enables parallel query scan
 - `parallel_tuple_cost` (default 0.1): Estimated cost of transferring one tuple from a parallel worker process to another process
 - `parallel_setup_cost` (default 1000): Estimates cost of launching parallel worker processes
 - `min_parallel_table_scan_size` (default 8MB): Sets minimum amount of table data that must be scanned in order for a parallel scan
 - `min_parallel_index_scan_size` (default 512 KB): Sets the minimum amount of index data that must be scanned in order for a parallel scan
 - `force_parallel_mode` (default off): Useful when testing parallel query scan even when there is no performance benefit



Parallel Maintenance Settings

- PostgreSQL supports parallel processes for creating an index
- Currently this feature is only available for btree index type
 - `max_parallel_maintenance_workers` (default 2): Enables parallel index creation

```
edb=# set max_parallel_maintenance_workers=0;  
SET  
Time: 0.399 ms  
edb=# create index idx_test_i on test(i);  
CREATE INDEX  
Time: 6297.539 ms (00:06.298)
```

```
edb=# set max_parallel_maintenance_workers=4;  
SET  
Time: 0.151 ms  
edb=# create index idx_test_i on test(i);  
CREATE INDEX  
Time: 4807.879 ms (00:04.808)
```



Vacuum Cost Settings

- `vacuum_cost_delay` (default 0 ms) - The length of time, in milliseconds, that the process will wait when the cost limit is exceeded
- `vacuum_cost_page_hit` (default 1) - The estimated cost of vacuuming a buffer found in the buffer pool
- `vacuum_cost_page_miss` (default 10) - The estimated cost of vacuuming a buffer that must be read into the buffer pool
- `vacuum_cost_page_dirty` (default 20) - The estimated cost charged when vacuum modifies a buffer that was previously clean
- `vacuum_cost_limit` (default 200) - The accumulated cost that will cause the vacuuming process to sleep



Autovacuum Settings

- `autovacuum` (default on) - Controls whether the autovacuum launcher runs, and starts worker processes to vacuum and analyze tables
- `log_autovacuum_min_duration` (default -1) - Autovacuum tasks running longer than this duration are logged
- `autovacuum_max_workers` (default 3) - Maximum number of autovacuum worker processes which may be running at one time
- `autovacuum_work_mem` (default -1, to use `maintenance_work_mem`) - Maximum amount of memory used by each autovacuum worker



Just-in-Time Compilation

JIT configuration parameters

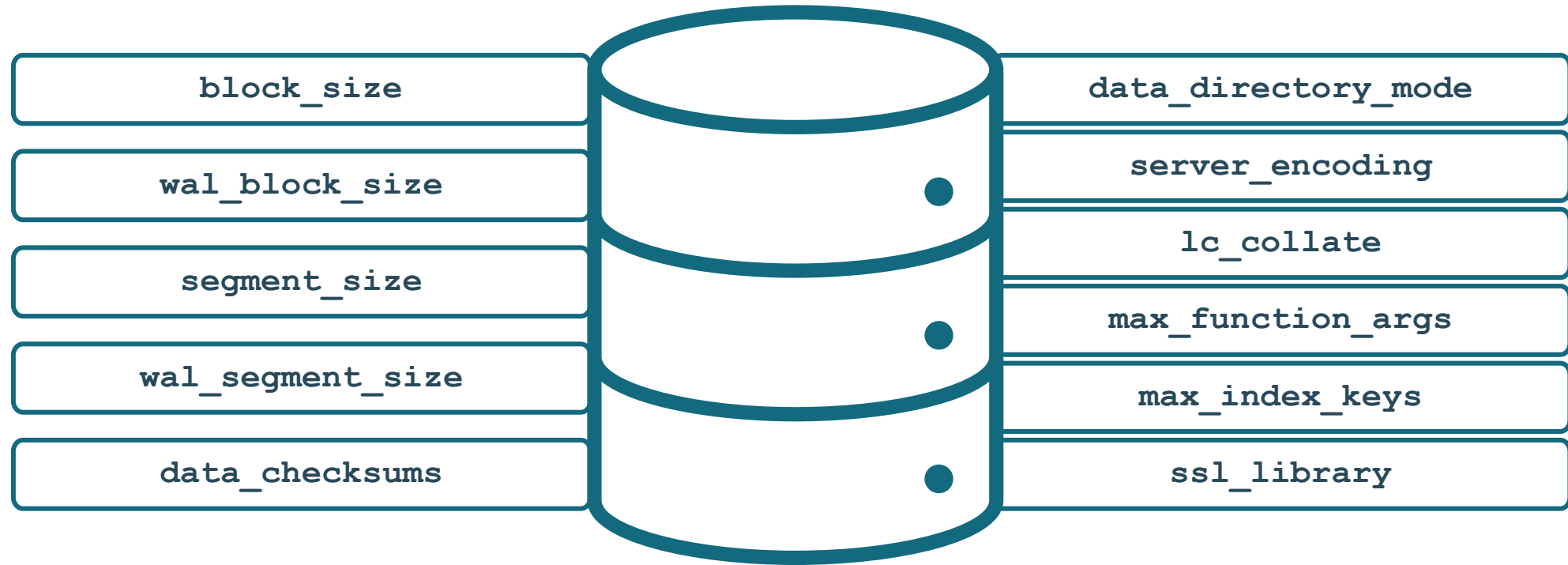
- Just-in-Time(JIT) is a core feature of EDB Postgres for accomplishing high performance
- `edb-as17-server-llvmjit` package must be installed
- JIT in EDB Postgres supports accelerating expression evaluation and tuple deforming

NAME	DEFAULT VALUE
jit	on
jit_above_cost	100000
jit_debugging_support	off
jit_dump_bitcode	off
jit_expressions	on
jit_inline_above_cost	500000
jit_optimize_above_cost	500000
jit_profiling_support	off
jit_provider	llvmjit
jit_tuple_deforming	on
(10 rows)	



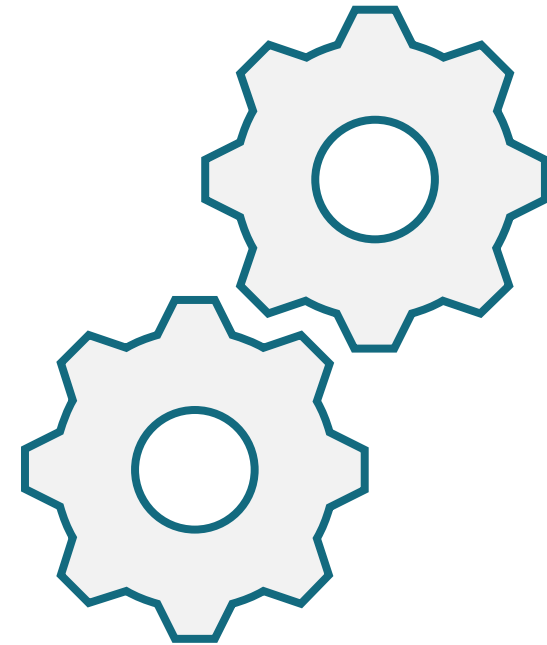
Read-only Parameters

- Enterprise Postgres sources are compiled using various settings
- Various read-only configuration parameters can be used to view build settings



Configuration File Includes

- The `postgresql.conf` file can now contain include directives
- Allows configuration file to be divided in separate files
- Usage in `postgresql.conf` file:
 - `include 'filename'`
 - `include_dir 'directory name'`



Module Summary

- Server Parameter File - postgresql.conf
- Viewing and Changing Server Parameters
- Configuration Parameters - Security, Resources and WAL
- Configuration Parameters - Error Logging, Planner and Maintenance
- Viewing Compilation Settings
- Using File Includes



Lab Exercise - 1

1. You are working as a DBA. It is recommended to keep a backup copy of the [postgresql.conf](#) file before making any changes. Make the necessary changes in the server parameter file for the following settings:
 - Allow up to 200 connected users on the server
 - Reserve 10 connection slots for DBA users on the server
 - Maximum time to complete client authentication will be 10 seconds



Lab Exercise - 2

1. Working as a DBA is a challenging job and to track down certain activities on the database server, logging has to be implemented. Go through the server parameters that control logging and implement the following:
 - Save all the error message in a file inside the log folder in your cluster data directory (e.g. `c:\edbdata` or `/edbdata`)
 - Log all queries which are taking more than 5 seconds to execute, and their time
 - Log the users who are connecting to the database cluster
 - Make the above changes and verify them



Lab Exercise - 3

1. Perform the following changes recommended by a senior DBA and verify them. Set:
 - Shared buffer to 256MB
 - Effective cache for indexes to 512MB
 - Maintenance memory to 64MB
 - Temporary memory to 8MB



Lab Exercise - 4

- Vacuuming is an important maintenance activity and needs to be properly configured. Change the following **autovacuum** parameters on the production server. Set:
 - Autovacuum workers to 6
 - Autovacuum threshold to 100
 - Autovacuum scale factor to 0.3
 - Auto analyze threshold to 100
 - Autovacuum cost limit to 100





Data Dictionary



Module Objectives

- The System Catalog Schema
- System Information Tables and Views
- System Information and Administration Functions
- Oracle-like Dictionaries
- Oracle Compatible built-in Packages



The System Catalog Schema

- Stores information about table and other objects
- Created and maintained automatically in `pg_catalog` schema
- `pg_catalog` is always effectively part of the `search_path`
- Contains:
 - System Tables like `pg_class` etc.
 - System Function like `pg_database_size()` etc.
 - System Views like `pg_stat_activity` etc.



System Information Tables

- `\ds` in psql prompt will give you the list of `pg_*` tables and views
- This list is from `pg_catalog` schema

<code>pg_tables</code>	list of tables
<code>pg_constraints</code>	list of constraints
<code>pg_indexes</code>	list of indexes
<code>pg_trigger</code>	list of triggers
<code>pg_views</code>	list of views



More System Information Tables

pg_file_settings

Provides summary of the contents of the server configuration file

pg_policy

Stores row level security for tables

pg_policies

Provides access to useful information about each row-level security policy in the database



System Information Functions

`current_database()`

`current_schema[]`

`pg_postmaster_start_time()`

`version()`

`current_user`

`current_schemas(boolean)`

`pg_current_logfile()`

`txid_status()`

`pg_conf_load_time()`

`pg_jit_available()`



System Administration Functions

<code>current_setting, set_config</code>	Return or modify configuration variables
<code>pg_cancel_backend</code>	Cancel a backend's current query
<code>pg_terminate_backend</code>	Terminates backend process
<code>pg_reload_conf</code>	Reload configuration files
<code>pg_rotate_logfile</code>	Rotate the server's log file
<code>pg_start_backup, pg_stop_backup</code>	Used with point-in time recovery
<code>pg_ls_logdir()</code>	Returns the name, size, and last modified time of each file in the log directory
<code>pg_ls_waldir()</code>	Returns the name, size, and last modified time of each file in the WAL directory



More System Administration Functions

`pg*_size`

Disk space used by a tablespace, database, relation or total_relation (includes indexes and toasted data)

`pg_column_size`

Bytes used to store a particular value

`pg_size_pretty`

Convert a raw size to something more human-readable

`pg_ls_dir, pg_read_file`

File operation functions. Restricted to superuser use and only on files in the data or log directories

`pg_blocking_pids()`

Function to reliably identify which sessions block others



System Information Views

pg_stat_activity	Details of open connections and running transactions
pg_locks	List of current locks being held
pg_stat_database	Details of databases
pg_stat_user_*	Details of tables, indexes and functions
pg_stat_archiver	Status of the archiver process
pg_stat_progress_basebackup	View pg_basebackup progress
pg_stat_progress_vacuum	Provides progress reporting for VACUUM operations
pg_stat_progress_analyze	Provides progress details for ANALYZE operations
pg_hba_file_rules	Provides a summary of the contents of the client authentication configuration file, pg_hba.conf
pg_stat_io	Provides I/O information



Built-in Packages

- Enterprise Postgres (Oracle Compatible) provides built-in packages compatible with Oracle
- Over 24 of most commonly used Oracle built-in packages are available in Enterprise Postgres
- These built-in packages provide administration and maintenance utilities

DBMS_ALERT	DBMS_AQ	DBMS_AQADM	DBMS_CRYPTO	DBMS_JOB	DBMS_LOB
DBMS_LOCK	DBMS_MVIEW	DBMS_OUTPUT	DBMS_PIPE	DBMS_PROFILER	DBMS_RANDOM
DBMS_RLS	DBMS_SESSION	DBMS_SCHEDULER	DBMS_SQL	DBMS_UTILITY	DBMS_REDACT
UTL_ENCODE	UTL_FILE	UTL_HTTP	UTL_MAIL	UTL_SMTP	UTL_URL
UTL_RAW	DBMS_ASSERT	DBMS_PRIVILEGE_CAPTURE	DBMS_XMLDOM	HTP AND HTF	



System Catalog Views

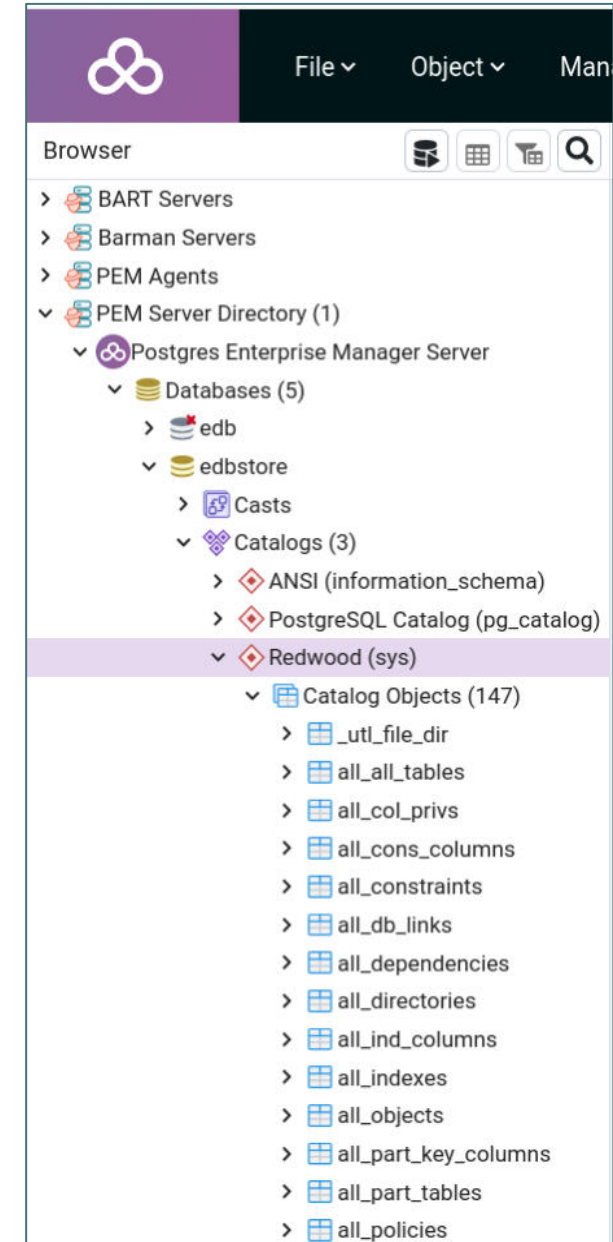
- Enterprise Postgres database stores user data in user tables
- Enterprise Postgres database stores internal information about all the objects and events in the catalog tables
- System catalog tables are kept in separate internal schemas
- Enterprise Postgres provides Oracle compatible catalog view (data dictionaries)
- These views can be queried to find:
 - Definition of schema objects
 - User's information
 - Privileges
 - Other general database information



sys Schema

- sys schema contains Oracle compatible Catalog Views

View	Description
user_*	User's view (what is in your schema; what you own)
all_*	Expanded user's view (what you can access)
dba_*	Database administrator's view (what is in everyone's schemas)
edb\$	Performance-related data



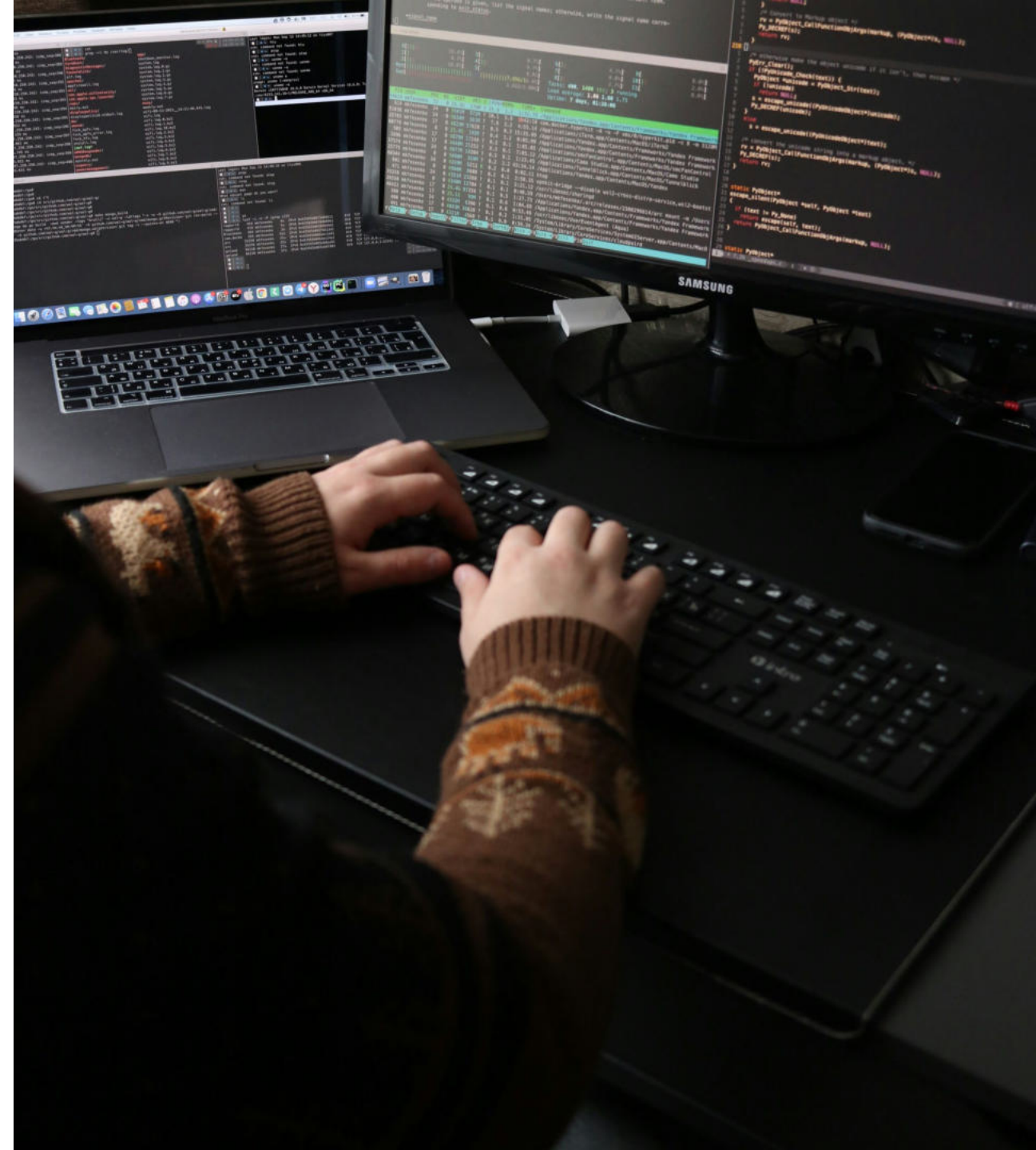
Module Summary

- The System Catalog Schema
- System Information Tables and Views
- System Information and Administration Functions
- Oracle-like Dictionaries
- Oracle Compatible built-in Packages



Lab Exercise - 1

1. You are working with different schemas in a database. After a while you need to determine all the schemas in your search path. Write a query to find the list of schemas currently in your search path.



Lab Exercise - 2

1. You need to determine the names and definitions of all of the views in your schema. Create a report that retrieves view information - the view name and definition text.



Lab Exercise - 3

1. Create a report of all the users who are currently connected. The report must display total session time of all connected users.
2. You found that a user has connected to the server for a very long time and have decided to gracefully kill its connection. Write a statement to perform this task.



Lab Exercise - 4

1. Write a query to display the name and size of all the databases in your cluster. Size must be displayed using a meaningful unit.





Creating and Managing Databases

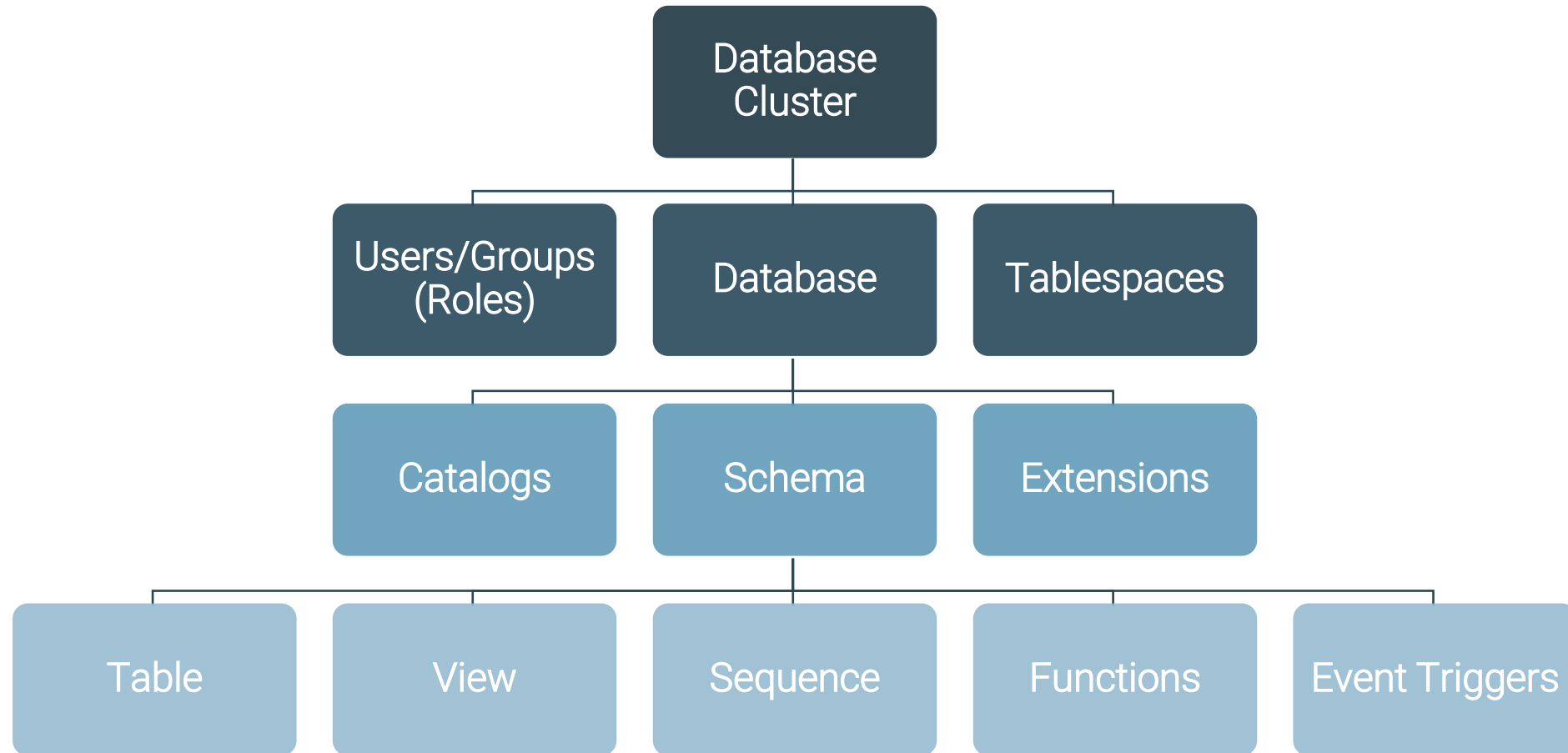


Module Objectives

- Object Hierarchy
- Users and Roles
- Tablespaces
- Databases
- Access Control
- Creating Schemas
- Schema Search Path

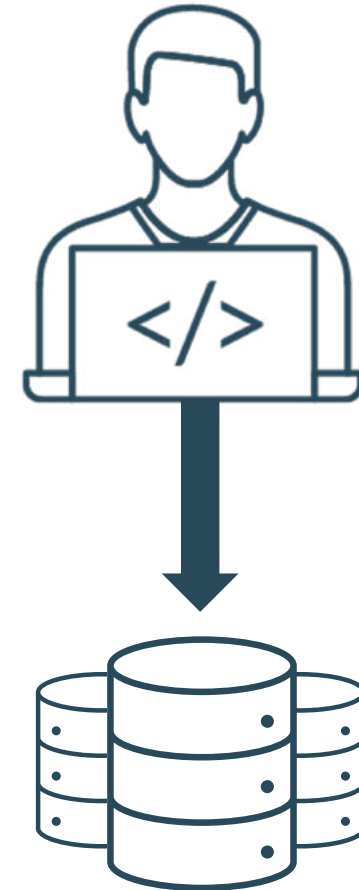


Object Hierarchy



Database Users

- Are global across a database cluster
- Are not the operating system users
- Are used for connecting to a database
- Have a unique name not starting with `pg_`
- `enterprisedb` is a predefined superuser



Creating Users Using psql

- How to create? `CREATE USER` sql command
- How to delete? `DROP USER` sql command
- `superuser` or `createrole` privilege is required for creating a database user

```
CREATE USER name [ [ WITH ] option [ ... ] ]  
where option can be:  
SUPERUSER | CREATEDB | CREATEROLE | LOGIN |  
REPLICATION | BYPASSRLS CONNECTION LIMIT  
connlimit  
| PASSWORD 'password'| VALID UNTIL 'timestamp'  
| PROFILE profile_name  
| ACCOUNT { LOCK | UNLOCK }  
| LOCK TIME 'timestamp'  
| PASSWORD EXPIRE [ AT 'timestamp' ]
```

```
postgres=# CREATE USER rupi  
postgres-# PASSWORD 'rupi123' SUPERUSER;  
CREATE ROLE  
postgres=#
```



Creating Users Using createuser

- The `createuser` utility can also be used to create a user
- Syntax:

```
$ createuser [OPTION]... [ROLENAME]
```

- Use `--help` option to view the full list of options available
- Example:

```
[enterprisedb@localhost ~]$ createuser -P pguser1  
Enter password for new role:  
Enter it again:
```



Alter Users Using alteruser

- The `alteruser` utility can also be used to create a user
- Syntax:
 `$ alteruser [OPTION]... [ROLENAME]`
- Use `--help` option to view the full list of options available
- Example:

```
[enterprisedb@localhost ~]$ psql -c "SELECT r.rolname, r.rolsuper
                                FROM pg_catalog.pg_roles r
                                WHERE r.rolname = 'pguser1' "
```

rolname	rolsuper
pguser1	f

```
(1 row)
```

```
[enterprisedb@localhost ~]$ alteruser -s pguser1
[enterprisedb@localhost ~]$ psql -c "SELECT r.rolname, r.rolsuper
                                FROM pg_catalog.pg_roles r
                                WHERE r.rolname = 'pguser1' "
```

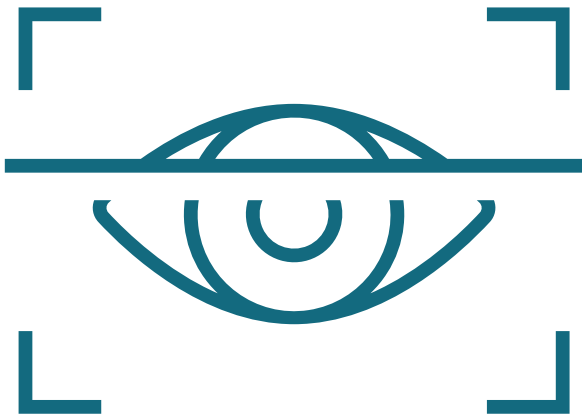
rolname	rolsuper
pguser1	t

```
(1 row)
```



User Profile Management

- A profile is a named set of password attributes
- User profiles can be used to manage account status and password expiration
- The default profile is assigned to all users



- Password attributes:
 - `FAILED_LOGIN_ATTEMPTS`
 - `PASSWORD_LOCK_TIME`
 - `PASSWORD_LIFE_TIME`
 - `PASSWORD_GRACE_TIME`
 - `PASSWORD_REUSE_TIME`
 - `PASSWORD_REUSE_MAX`
 - `PASSWORD_VERIFY_FUNCTION`
 - `PASSWORD_ALLOW_HASHED`



Creating A User Profile

- How to create? `CREATE PROFILE` command
- How to delete? `DROP PROFILE` command
- Example:

```
edb=# CREATE PROFILE dba_top LIMIT FAILED_LOGIN_ATTEMPTS 1;  
CREATE PROFILE  
edb=# CREATE USER user2 PASSWORD 'edb' SUPERUSER PROFILE dba_top;  
CREATE ROLE  
edb=#
```



Roles

- Role is a collection of cluster and object level privileges
- Role makes it easier to manage multiple privileges
- How to create? `CREATE ROLE` statement
- How to assign? `GRANT` statement
- Who it can be assigned to? User or a group



Default Roles

```
aq_administrator_role  
pg_checkpoint  
pg_create_subscription  
pg_database_owner  
pg_execute_server_program  
pg_maintain  
pg_monitor  
pg_read_all_data  
pg_read_all_settings  
pg_read_all_stats  
pg_read_server_files  
pg_signal_backend  
pg_stat_scan_tables  
pg_use_reserved_connections  
pg_write_all_data  
pg_write_server_files
```

- Provide certain administrative capabilities using these default roles
- For Example, create a new user with read only access or a new user with access to view monitoring data only



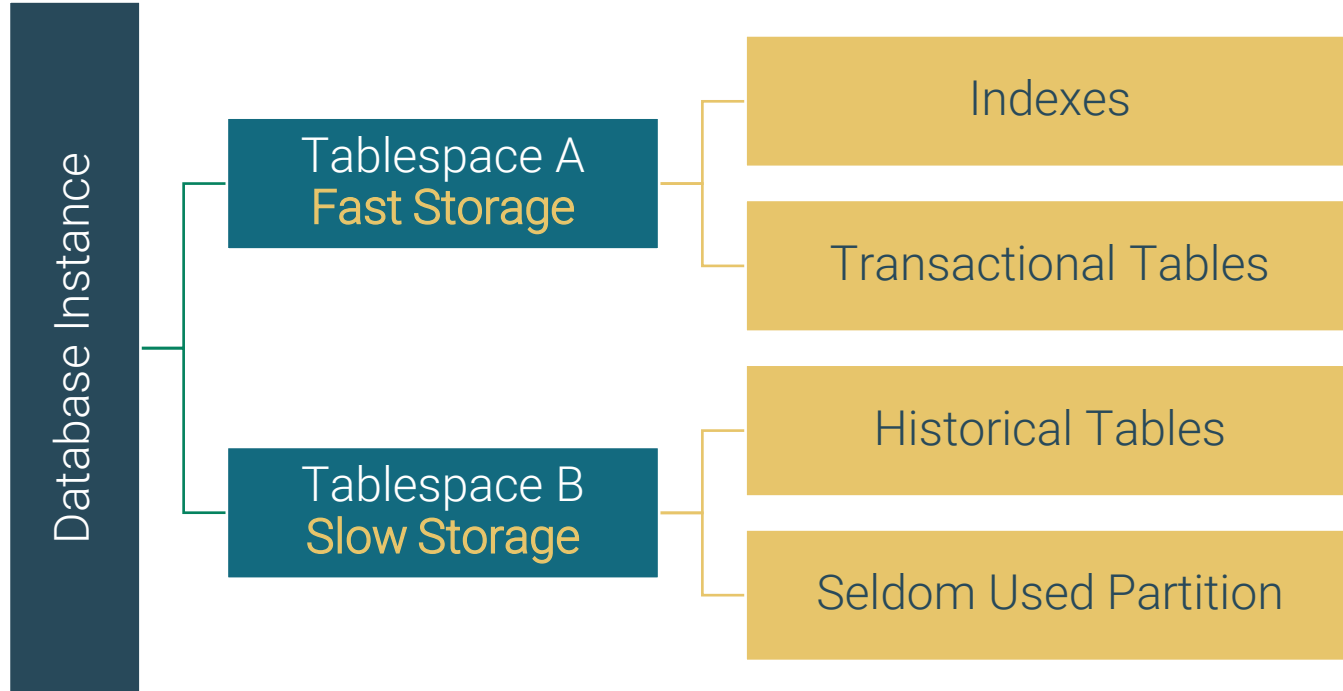
Tablespaces and Data Files

- Data is stored logically in tablespaces and physically in data files
- Tablespaces:
 - Can belong to only one database cluster
 - Consist of multiple data files
 - Can be used by multiple databases
- Data Files:
 - Can belong to only one tablespace
 - Are used to store database objects
 - Cannot be shared by multiple tables (one or more per table)

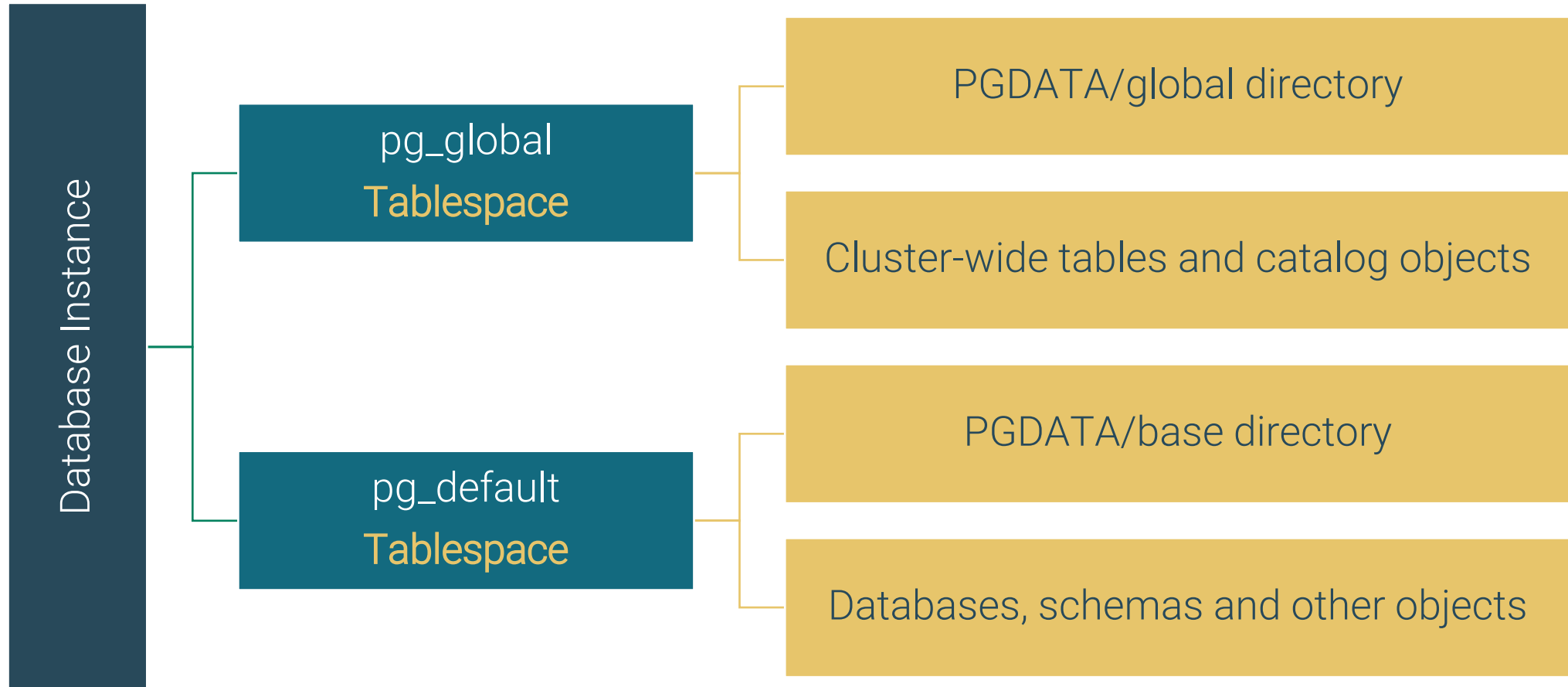


Advantages of Tablespaces

- Control the disk layout for a database cluster
- Store indexes and data physically separated for performance



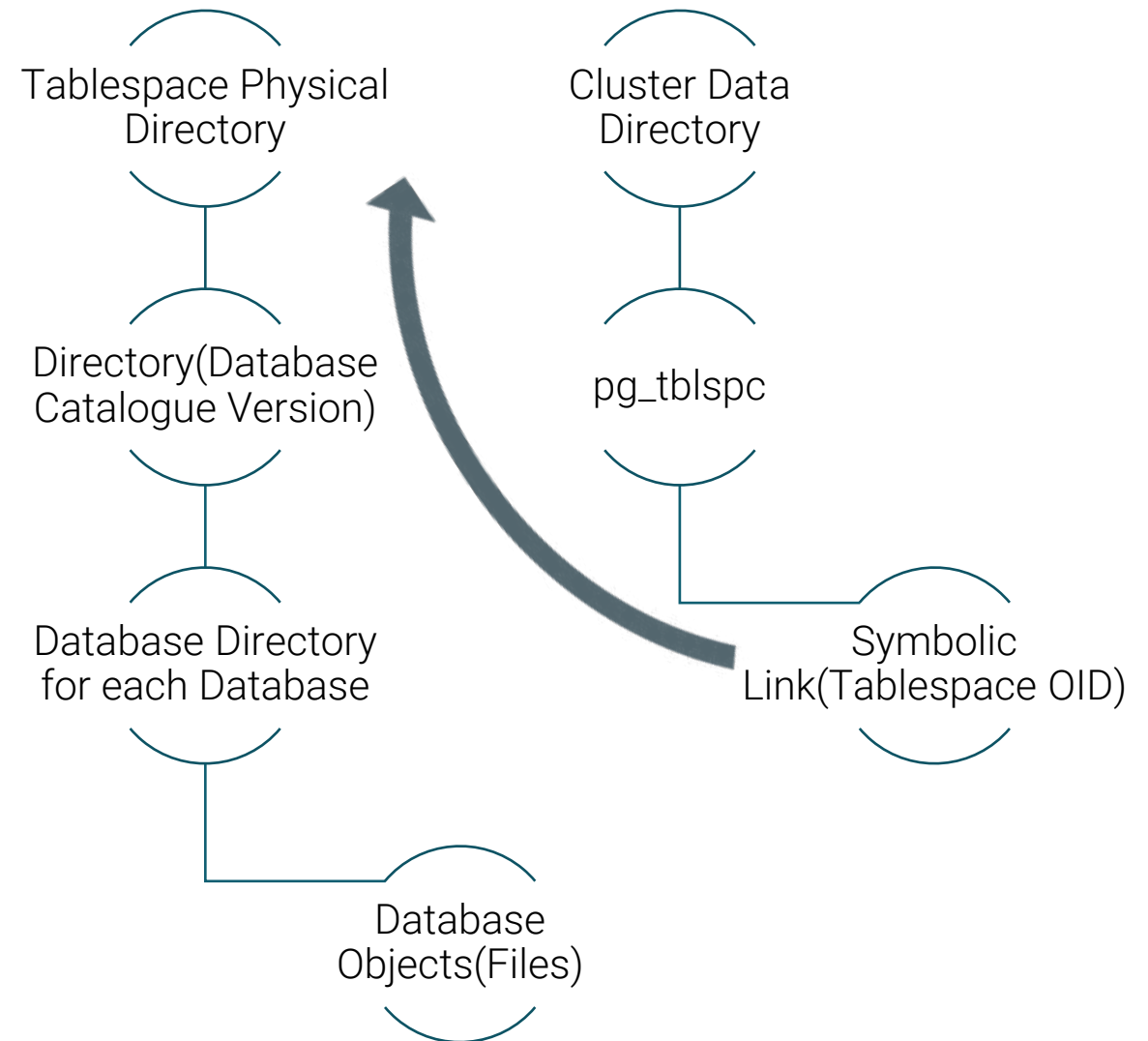
Pre-Configured Tablespaces



Creating Tablespaces

- How to create? CREATE TABLESPACE command
- The tablespace directory must be existing with permissions
- Syntax:

```
CREATE TABLESPACE  
tablespace_name [ OWNER  
user_name ]  
LOCATION 'directory';
```



Example - CREATE TABLESPACE

```
[training@Base ~]$ sudo mkdir /newtab1
[training@Base ~]$ sudo chown enterprisedb:enterprisedb /newtab1
[training@Base ~]$ su - enterprisedb
[enterprisedb@Base ~]$ psql -p 5444 edb enterprisedb
```

```
edb=# CREATE TABLESPACE fast_tab LOCATION '/newtab1';
CREATE TABLESPACE
edb=# \db
```

List of tablespaces

Name	Owner	Location
fast_tab	enterprisedb	/newtab1
pg_default	enterprisedb	
pg_global	enterprisedb	

(3 rows)



Using Tablespaces

- Use the TABLESPACE keyword while creating databases, tables and indexes

```
edb=# CREATE TABLE account(acno NUMBER PRIMARY KEY,  
ac_hldr_fname VARCHAR2(20)) TABLESPACE fast_tab;  
CREATE TABLE
```



Default and Temp Tablespace

- `default_tablespace` server parameter sets default tablespace
- `default_tablespace` parameter can also be set using the `SET` command at the session level
- `temp_tablespaces` parameter determines the placement of temporary tables and indexes and temporary files
- `temp_tablespaces` can be a list of tablespace names

```
edb=# show default_tablespace;  
default_tablespace
```

```
-----
```

```
(1 row)
```

```
edb=# show temp_tablespaces;  
temp_tablespaces
```

```
-----
```

```
(1 row)
```

```
edb=# set temp_tablespaces to 'tblspace1';  
SET
```

```
edb=# show temp_tablespaces;  
temp_tablespaces
```

```
-----
```

```
tblspace1
```

```
(1 row)
```



Altering Tablespaces

- `ALTER TABLESPACE` can be used to rename a tablespace, change ownership and set a custom value for a configuration parameter
- Only the owner or superuser can alter a tablespace
- The `seq_page_cost`, `random_page_cost`, `effective_io_concurrency` and `maintenance_io_concurrency` parameters can be altered for a tablespace



Example - Alter Tablespace

Syntax:

```
ALTER TABLESPACE name RENAME TO new_name
```

```
ALTER TABLESPACE name OWNER TO { new_owner | CURRENT_USER | SESSION_USER }
```

```
ALTER TABLESPACE name SET ( tablespace_option = value [, ... ] )
```

```
ALTER TABLESPACE name RESET ( tablespace_option [, ... ] )
```

```
edb=# ALTER TABLESPACE fast_tab RENAME TO new_tab;
```

```
ALTER TABLESPACE
```

```
edb=# \db
```

List of tablespaces

Name	Owner	Location
new_tab	enterprisedb	/newtab
pg_default	enterprisedb	
pg_global	enterprisedb	



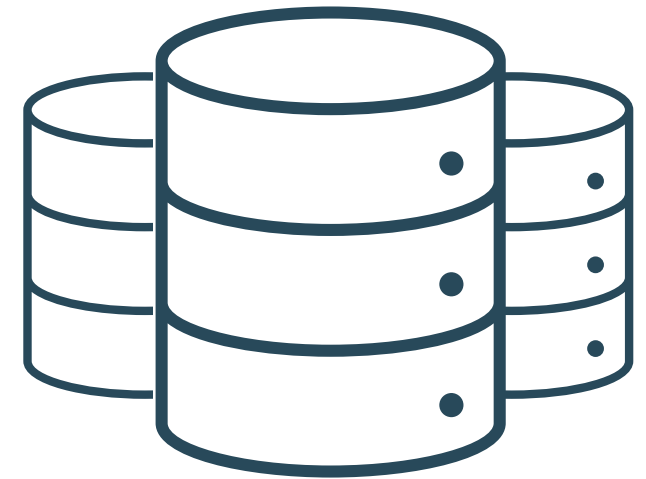
Dropping a Tablespace

- `DROP TABLESPACE` removes a tablespace from the system
- Only the owner or superuser can drop a tablespace
- The tablespace must be empty
- If a tablespace is listed in the `temp_tablespaces` parameter, make sure current sessions are not using the tablespace
- `DROP TABLESPACE` cannot be executed inside a transaction



What Is a Database?

- A database is a named collection of SQL objects
- A running Postgres instance can manage multiple databases
- How to create? `CREATE DATABASE` command
- How to delete? `DROP DATABASE` command
- To determine the set of existing databases:
 - SQL - `SELECT datname FROM pg_database;`
 - psql META COMMAND - `\l` (backslash lowercase L)



Creating Databases

- Database can be created using:
 1. `createdb` utility program
 2. `CREATE DATABASE` SQL command
- SQL Command syntax:

```
CREATE DATABASE name [ [ WITH ] [ OWNER [=] user_name ]  
[ TEMPLATE [=] template ]  
[ ENCODING [=] encoding ]  
[ TABLESPACE [=] tablespace_name ]  
[ ALLOW_CONNECTIONS [=] allowconn ]  
[ CONNECTION LIMIT [=] connlimit ]
```



Example - Creating Databases

```
edb=# CREATE DATABASE prod OWNER enterprisedb;  
CREATE DATABASE  
edb=# REVOKE CONNECT ON DATABASE prod FROM PUBLIC;  
REVOKE  
edb=# \connect prod  
You are now connected to database "prod" as user "enterprisedb".  
prod=# SELECT datname FROM pg_database;  
  datname  
-----  
postgres  
edb  
template1  
template0  
prod  
(5 rows)
```

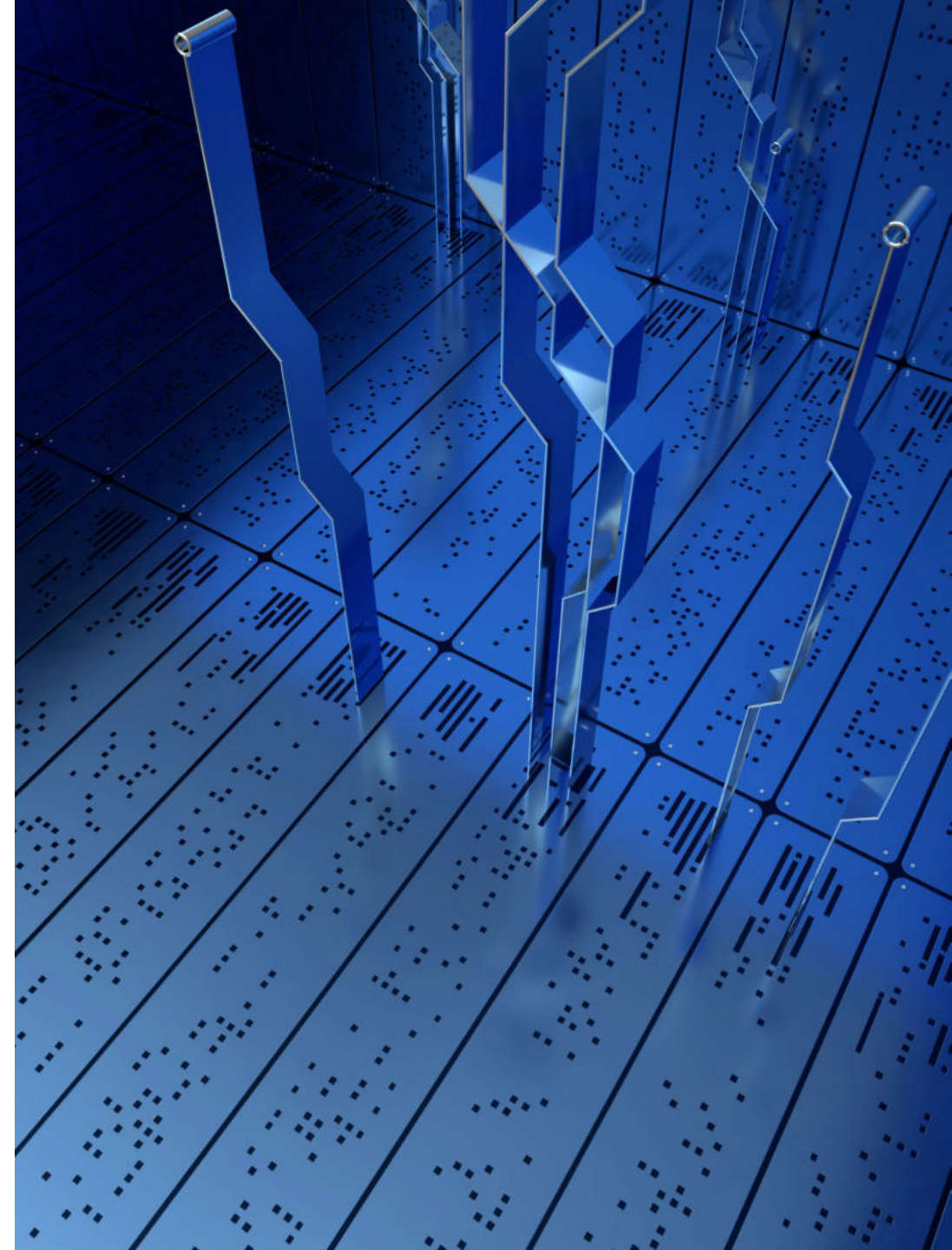


Accessing a Database

- PEM Web Client or psql can be used to access a database
- To use psql, open a terminal and execute:

```
$ psql -U postgres -d prod
```

- Note: If PATH is not set you can execute psql command from the bin directory of postgres installation



Privileges

- Cluster level
 - Granted to a user during `CREATE` or later using `ALTER USER`
 - These privileges are granted by superuser
- Object Level
 - Granted to user using `GRANT` command
 - These privileges allow a user to perform particular actions on a database object, such as tables, views, or sequence
 - Can be granted by owner, superuser or someone who has been given permission to grant privileges (`WITH GRANT OPTION`)



GRANT Statement

- Grants object level privileges to database users, groups or roles
- GRANT can also be used to grant a role to a user
- How to view syntax and available privileges?
 - Type `\h GRANT` in psql



Example – GRANT Statement

```
edbstore=# GRANT CONNECT ON DATABASE edbstore TO user1;
```

```
GRANT
```

```
edbstore=# GRANT USAGE ON SCHEMA edbuser TO user1;
```

```
GRANT
```

```
edbstore=# GRANT SELECT, INSERT ON edbuser.emp TO user1;
```

```
GRANT
```

```
edbstore=# \c edbstore user1
```

```
Password for user user1:
```

```
You are now connected to database "edbstore" as user "user1".
```

```
edbstore=> SELECT * FROM edbuser.emp LIMIT 2;
```

empno	ename	job	mgr	hiredate	sal	comm	deptno
7499	ALLEN	SALESMAN	7698	20-FEB-81 00:00:00	1600.00	300.00	30
7521	WARD	SALESMAN	7698	22-FEB-81 00:00:00	1250.00	500.00	30

(2 rows)



REVOKE Statement

- Revokes object level privileges from database users, groups or roles
- `REVOKE [ GRANT OPTION FOR ]` can be used to revoke only the grant option without revoking the actual privilege
- How to view syntax and available privileges?
 - Type `\h REVOKE` in psql

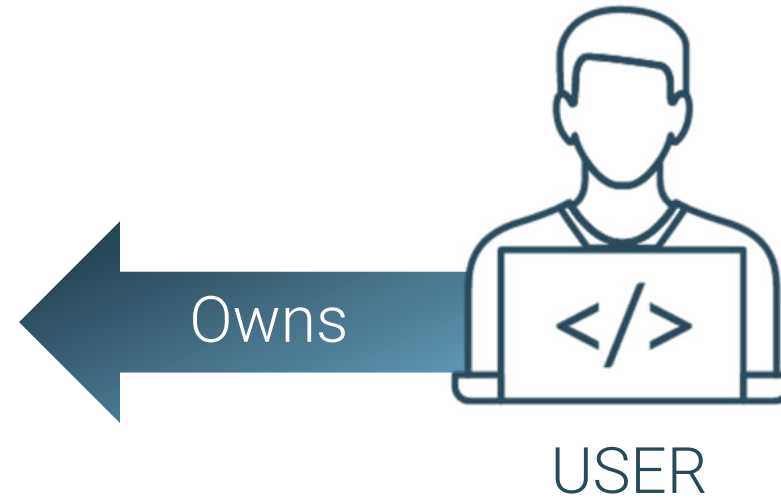
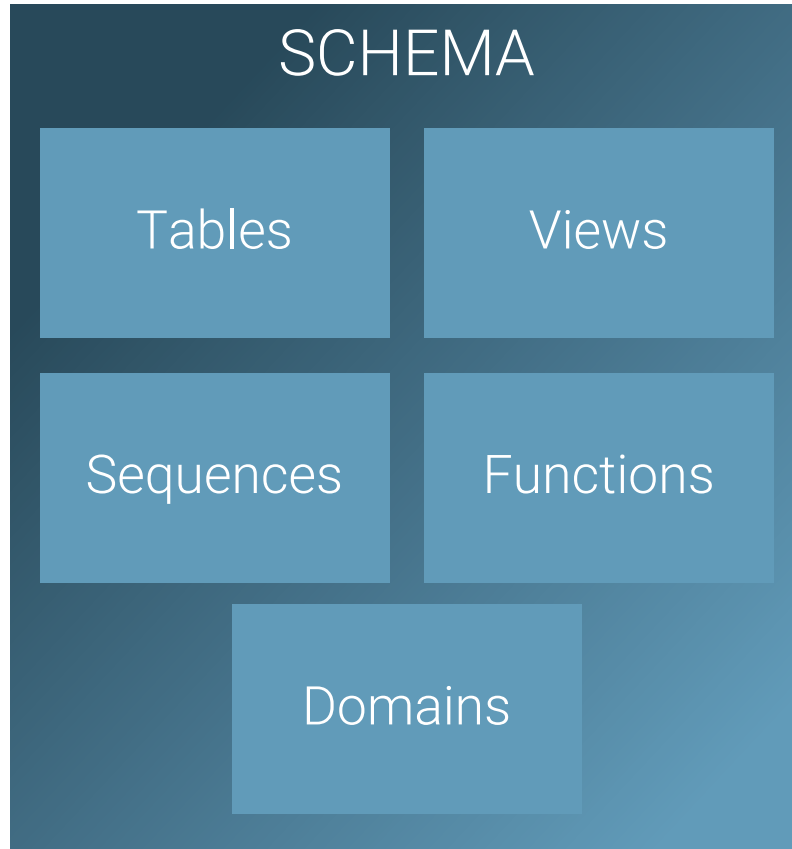


Example - REVOKE Statement

```
edbstore=# REVOKE SELECT, INSERT ON edbuser.emp FROM user1;
REVOKE
edbstore=# \c edbstore user1
Password for user user1:
You are now connected to database "edbstore" as user "user1".
edbstore=> SELECT * FROM edbuser.emp LIMIT 2;
ERROR:  permission denied for relation emp
edbstore=> \c edbstore enterprisedb
Password for user enterprisedb:
You are now connected to database "edbstore" as user "enterprisedb".
edbstore=# REVOKE USAGE ON SCHEMA edbuser FROM user1;
REVOKE
edbstore=# REVOKE CONNECT ON DATABASE edbstore FROM user1;
REVOKE
edbstore=# REVOKE CONNECT ON DATABASE edbstore FROM public;
REVOKE
edbstore=# \c edbstore user1
Password for user user1:
FATAL:  permission denied for database "edbstore"
DETAIL:  User does not have CONNECT privilege.
Previous connection kept
edbstore=# █
```



What is a Schema



Benefits of Schemas

- A database can contain one or more named schemas
- By default, all databases contain a public schema
- There are several reasons why one might want to use schemas:
 - To allow many users to use one database without interfering with each other
 - To organize database objects into logical groups to make them more manageable
 - Third-party applications can be put into separate schemas so they cannot collide with the names of other objects



Creating Schemas

- Schemas can be added using the CREATE SCHEMA SQL command
- Syntax:

```
CREATE SCHEMA IF NOT EXISTS schema_name [ AUTHORIZATION  
role_specification ]
```

- Example:

```
prod=# CREATE SCHEMA rupi AUTHORIZATION rupi;  
CREATE SCHEMA  
prod=# GRANT CONNECT ON DATABASE prod to rupi;  
GRANT  
prod=# \c prod rupi  
Password for user rupi:  
You are now connected to database "prod" as user "rupi".  
prod=> CREATE TABLE city(cityname VARCHAR2);  
CREATE TABLE  
prod=> \dt  
List of relations  
Schema | Name | Type | Owner  
-----+-----+-----+-----  
rupi   | city | table | rupi  
(1 row)  
prod=> 
```



What is a Schema Search Path

- The schema search path determines which schemas are searched for matching table names
- Search path is used when fully qualified object names are not used in a query
- Example:

```
SELECT * FROM employee;
```

- This statement will find the first employee table from the schemas listed in the search path

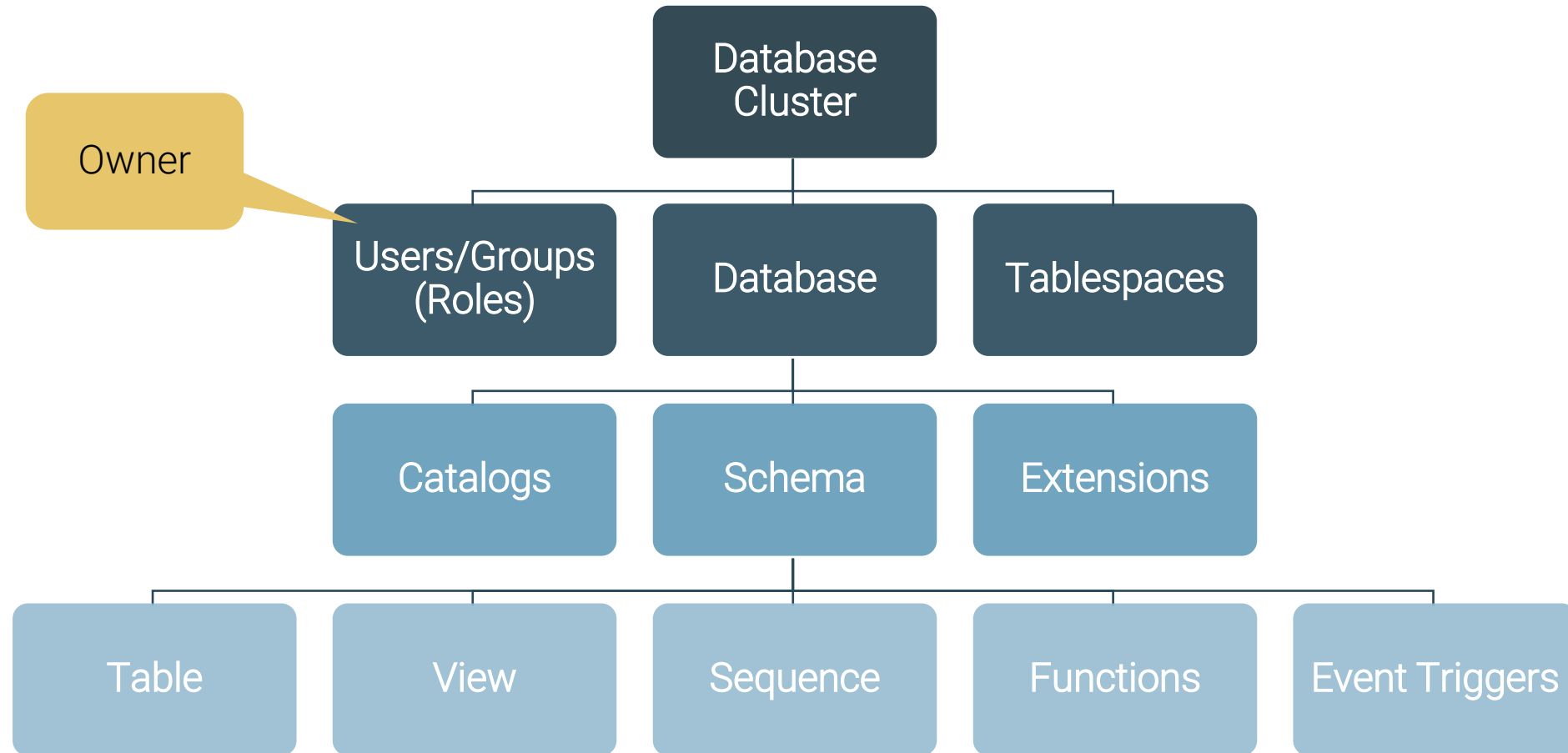


Determine the Schema Search Path

- To show the current search path, use the following command:
 - => `SHOW search_path;`
- Default `search_path` is `"$user", public`
- Search path can be changed using SET command:
 - => `SET search_path TO myschema, public;`



Object Ownership



Module Summary

- Object Hierarchy
- Users and Roles
- Tablespaces
- Databases
- Access Control
- Creating Schemas
- Schema Search Path



Lab Exercise - 1

- An e-music online store website application developer wants to add an online buy/sell facility and has asked you to separate all tables used in online transactions. Here you have suggested to use schemas. Implement the following suggested options:
 - Create an `ebuy` user with password '`lion`'
 - Create an `ebuy` schema which can be used by user `ebuy`
 - Login as the `ebuy` user, create a table `sample1` and check whether that table belongs to the `ebuy` schema or not



Lab Exercise - 2

- Retrieve a list of databases using a SQL query
- Retrieve a list of databases using the psql meta command
- Retrieve a list of tables in the edbstore database and check which schema and owner they have





Database Security



Module Objectives

- Database Security Requirements and Protection Plan
- Levels of Security in Postgres
- Access Control using pg_hba.conf
- Introduction to Row Level Security
- Data Encryption
- Advanced Features - Data Redaction, SQL/Protect and EDB*Wrap
- General Security Recommendations



Why Database Security

- Databases are a core component of many computing systems
- Confidential data like SIN(Social Insurance Number), Healthcare, Banking details is stored and shared using databases
- It is very critical to protect stored information from hackers, insiders, and other groups who intend to steal valuable data
- Database Security is a mechanism to protect the data against threats



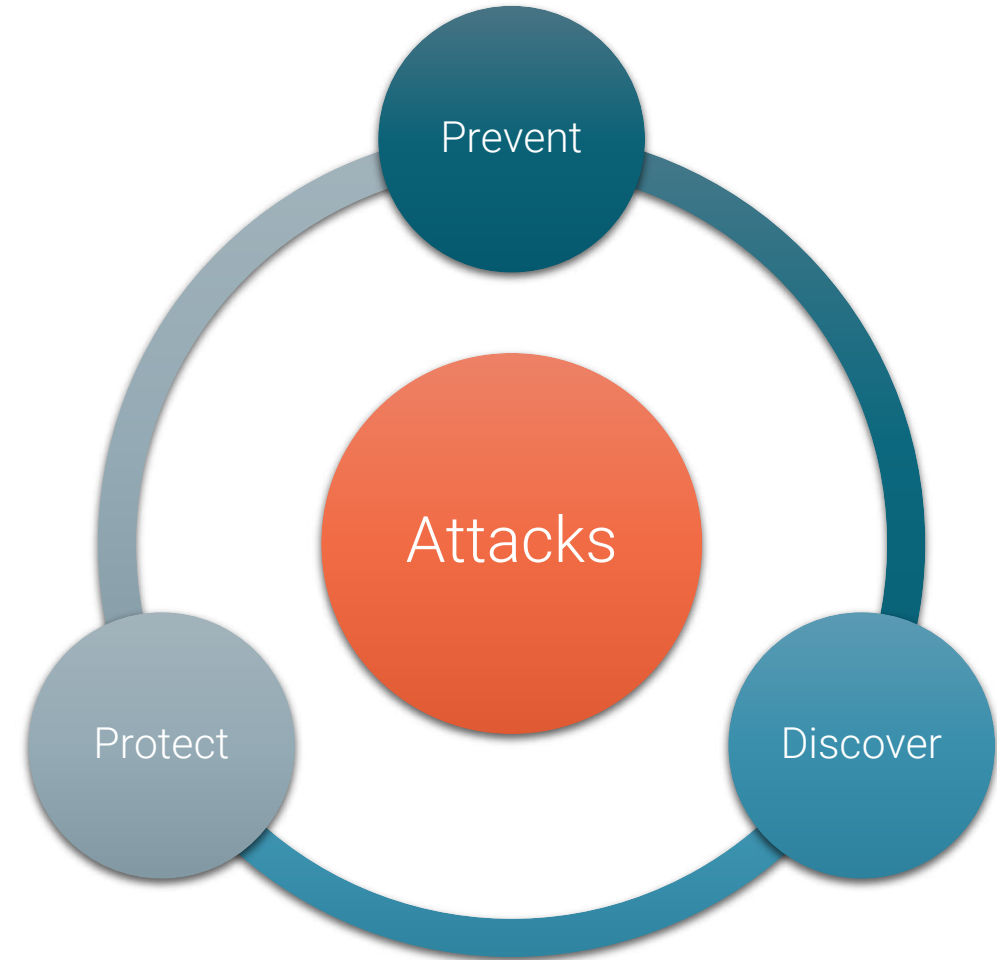
Data Security Requirements

- Stopping improper disclosure, modification and denial of access to information is very important
- Who wants an employee finding out boss's salary, changing his/her salary or stopping HR from printing paychecks
- Database Security Requirements includes:
 - Confidentiality
 - Integrity
 - Availability



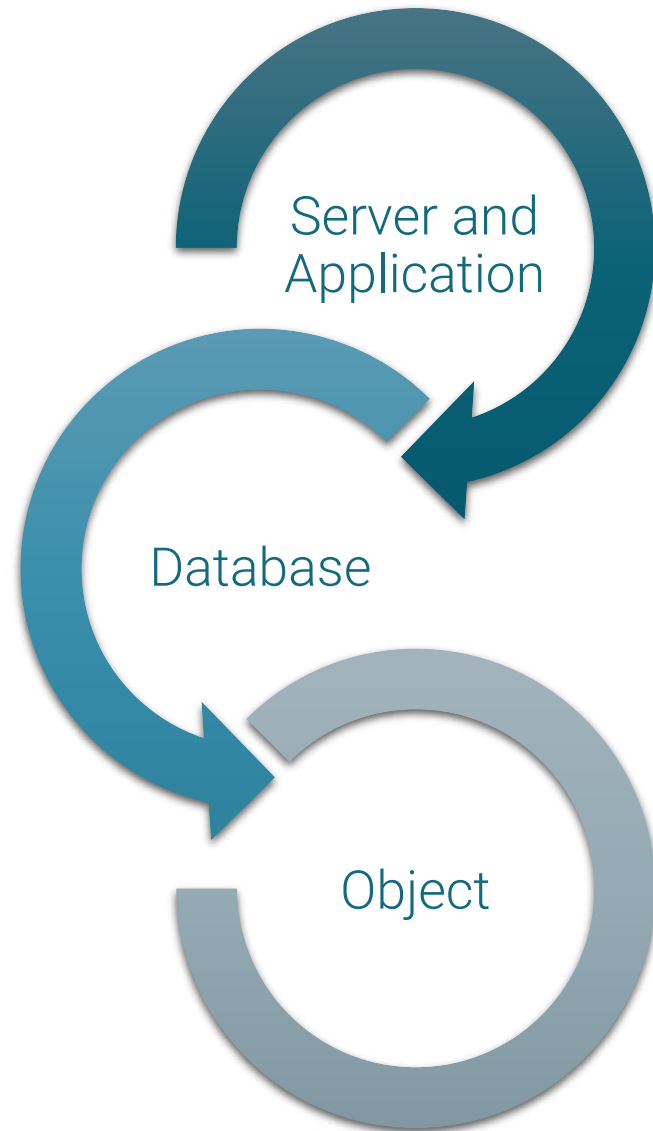
Protection Plan – We all need one

- Access Control
 - Authentication and Authorization
- Data Control
 - Views, Row Level Security, Encryptions
- Network Control
 - SSL Connections, Firewalls
- Auditing
 - Monitoring



Levels of Security

- User/Password
- Connect Privilege
- Schema Permissions

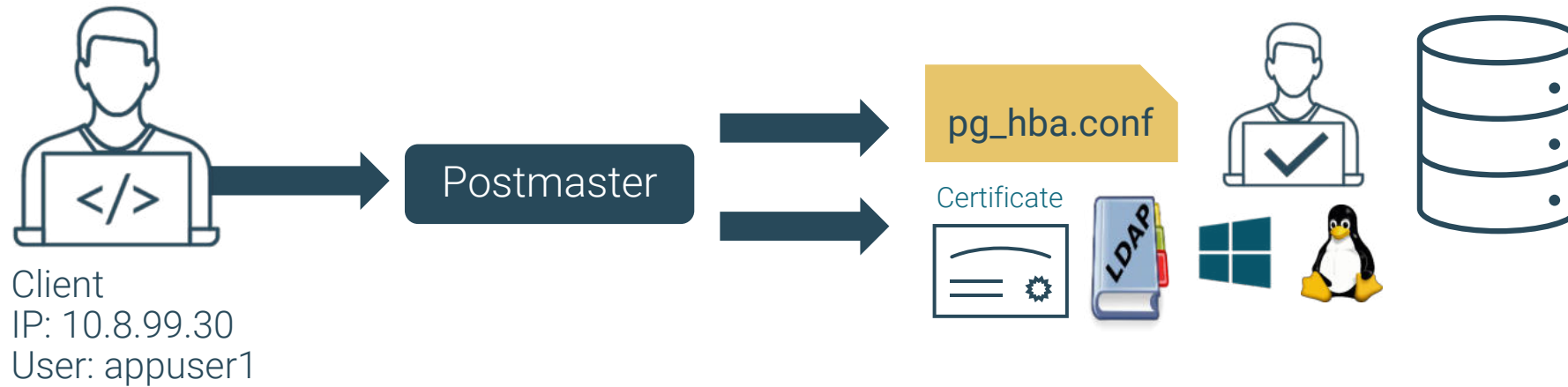


- Check Client IP
- pg_hba.conf

- Table Level Privileges
- Grant/Revoke



Host Based Access Control



- pg_hba.conf can be used to restrict the ability to connect to a database
- SSL can be forced for selected clients based on hostname or IP address
- Different authentication methods can be used
- Superuser access can be locked down to certain IPs using pg_hba.conf



pg_hba.conf - Access Control

- Host based access control file
- Located in the cluster data directory
- Read at startup, any change requires reload
- Contains a set of records, one per line
- Each record specifies a connection type, database name, user name, client IP and method of authentication
- Top to bottom read
- Hostnames, IPv6 and IPv4 supported
- Authentication methods - trust, reject, md5, password, gss, sspi, krb5, ident, peer, pam, ldap, radius, bsd, scram or cert



pg_hba.conf Example

#	TYPE	DATABASE	USER	ADDRESS	METHOD
# "local" is for Unix domain socket connections only					
local	all	all			peer
# IPv4 local connections:					
host	all	all		127.0.0.1/32	ident
# IPv6 local connections:					
host	all	all		::1/128	ident
# Allow replication connections from localhost, by a user with the					
# replication privilege.					
local	replication	all			peer
host	replication	all		127.0.0.1/32	ident
host	replication	all		::1/128	ident



Authentication Problems

```
FATAL:  no pg_hba.conf entry for host "192.168.10.23", user  
"edbstore", database "edbuser"
```

```
FATAL:  password authentication failed for user "edbuser"
```

```
FATAL:  user "edbuser" does not exist
```

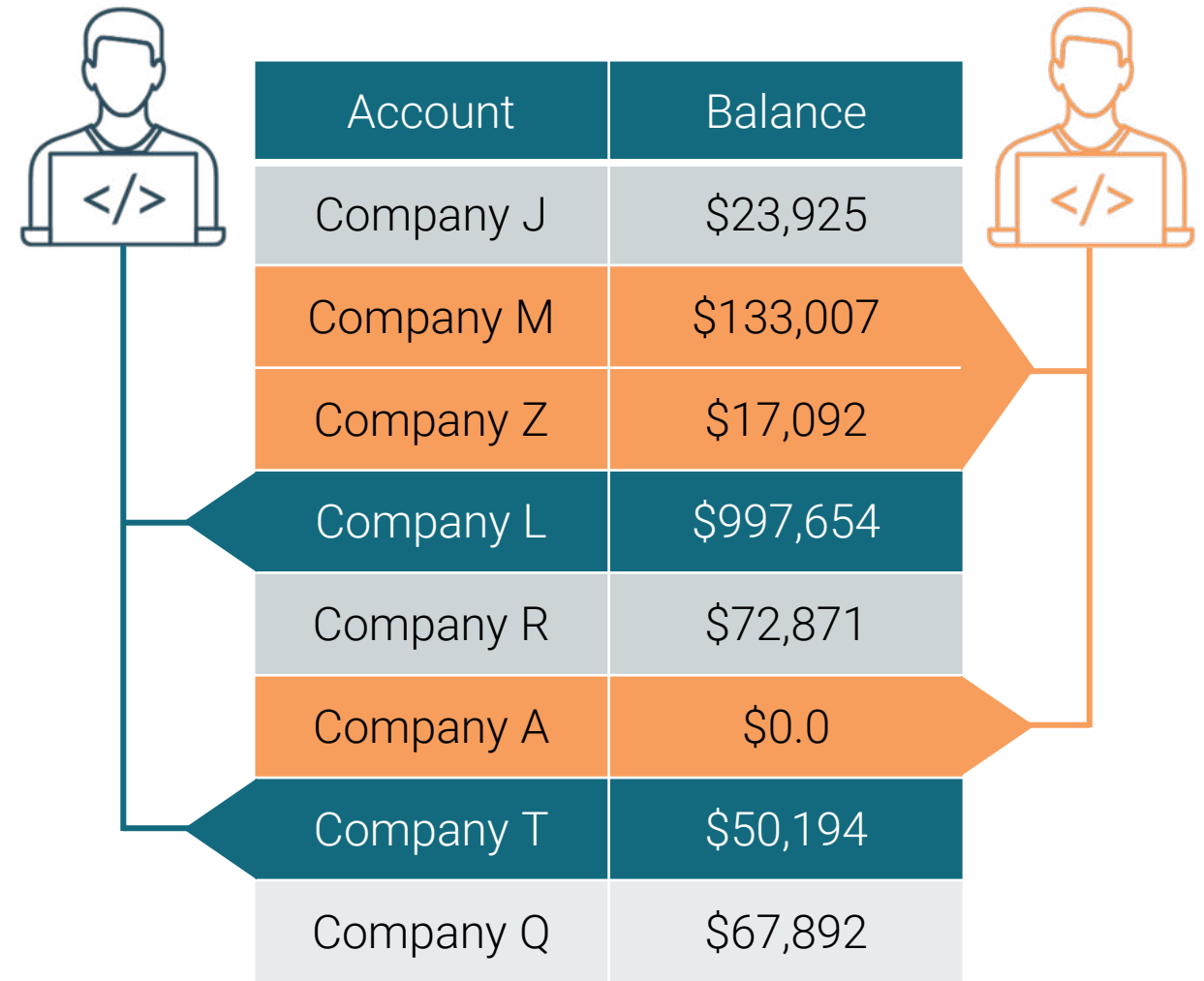
```
FATAL:  database "edbstore" does not exist
```

- Self-explanatory message is displayed
- Verify database name, username and Client IP in [pg_hba.conf](#)
- Reload Cluster after changing [pg_hba.conf](#)
- Check server log for more information



Row Level Security (RLS)

- GRANT and REVOKE can be used at table level
- PostgreSQL supports security policies for limiting access at row level
- By default, all rows of a table are visible
- Once RLS is enabled on a table, all queries must go through the security policy
- Security policies are controlled by DBA rather than application
- RLS offers stronger security as it is enforced by the database



The diagram shows two users, represented by icons with code symbols (</>), interacting with a table. The table has two columns: 'Account' and 'Balance'. The rows are color-coded: blue for 'Company J', 'Company L', and 'Company T'; orange for 'Company M', 'Company Z', and 'Company A'; and grey for 'Company R' and 'Company Q'. Arrows indicate that the blue user can access all rows, while the orange user is restricted to only the orange rows.

Account	Balance
Company J	\$23,925
Company M	\$133,007
Company Z	\$17,092
Company L	\$997,654
Company R	\$72,871
Company A	\$0.0
Company T	\$50,194
Company Q	\$67,892



Example - Row Level Security

- For example, to enable row level security for the table accounts :

- Create the table first

```
edb=# CREATE TABLE accounts (manager text, company text, contact_email text);
```

- Then alter the table

```
edb=# ALTER TABLE accounts ENABLE ROW LEVEL SECURITY;
```

- Syntax:

```
CREATE POLICY name ON table_name
[ AS { PERMISSIVE | RESTRICTIVE } ]
[ FOR { ALL | SELECT | INSERT | UPDATE | DELETE } ]
[ TO{ role_name | PUBLIC | CURRENT_USER | SESSION_USER}[,...] ]
[ USING ( using_expression ) ]
[ WITH CHECK ( check_expression ) ]
```



Example - Row Level Security (continued)

- To create a policy on the accounts table to allow the managers role to view the rows of their accounts, the CREATE POLICY command can be used:

```
edb=# CREATE POLICY account_managers ON accounts TO managers USING  
(manager = current_user);
```

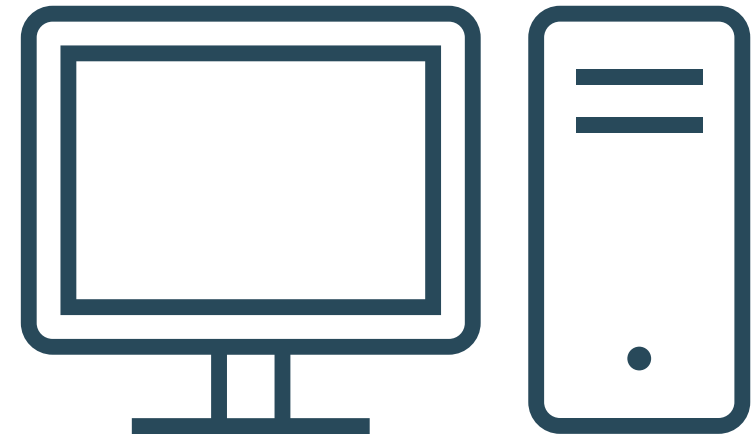
- To allow all users to view their own row in a user table, a simple policy can be used:

```
edb=# CREATE POLICY user_policy ON users USING (user = current_user);
```



Additional Security in Enterprise Postgres

- Enterprise Postgres supports all the security options available in PostgreSQL, plus:
 - Transparent Data Encryption
 - Database Level Encryption
 - User Profiles and Password Policy Manager
 - SQL/Protect - Protection against SQL Injections
 - DBMS_CRYPT, DBMS_RLS, DBMS_REDACT
 - Data redaction
 - EDB*Wrap - Obfuscate Source Code
 - Enhanced Auditing using edb_audit



Transparent Data Encryption

- Transparent Data Encryption was introduced in Enterprise Postgres version 15
- Helps securing user data by encrypting data files, write-ahead logs and temporary files
- TDE is transparent to user and doesn't require any application-level change
- With TDE, Database Server and backup storage files are unintelligible for unauthorized users
- Does not encrypt transaction status metadata, data directory structure, foreign tables, logs and configuration files
- You need to create a new TDE-enabled instance / cluster at "initdb" time or migrate your database to an existing TDE-enabled environment



Database Level Encryption

- Encrypting everything does not make data secure
- Resources are consumed when you query encrypted data
- `pgcrypto` provides mechanism for encrypting selected columns
- `pgcrypto` supports one-way and two-way data encryption
- Install `pgcrypto` using `CREATE EXTENSION` command

```
CREATE EXTENSION pgcrypto;
```

Emp_ID	EmpName	Salary_encrypt
1	Jim	0x003839E4B8C07047AAA64145FA4967E010000003396414...
2	Cary	0x003839E4B8C07047AAA64145FA4967E010000006910052...
3	Tom	0x003839E4B8C07047AAA64145FA4967E010000006F82168...
4	Crouse	0x003839E4B8C07047AAA64145FA4967E0100000008C83E...
5	Gary	0x003839E4B8C07047AAA64145FA4967E01000000F51EF75...
8	Matts	0x003839E4B8C07047AAA64145FA4967E010000009641E58...



SQL/PROTECT – Track, Warn and Protect

SQL firewall
installed directly in
the database server



Centrally managed
layer of security



Screens incoming
queries for common
attack profiles



Attacker



Threat
Deleted

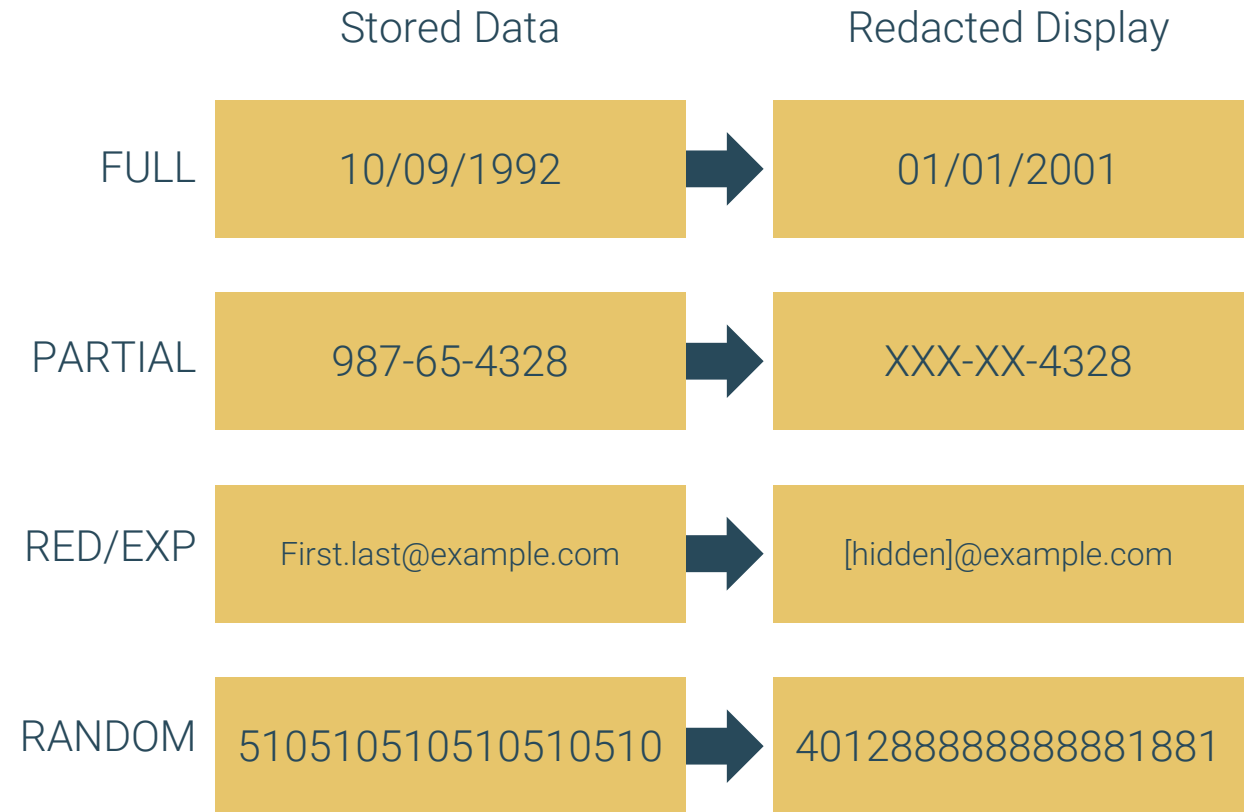


Injection
Attack
Report



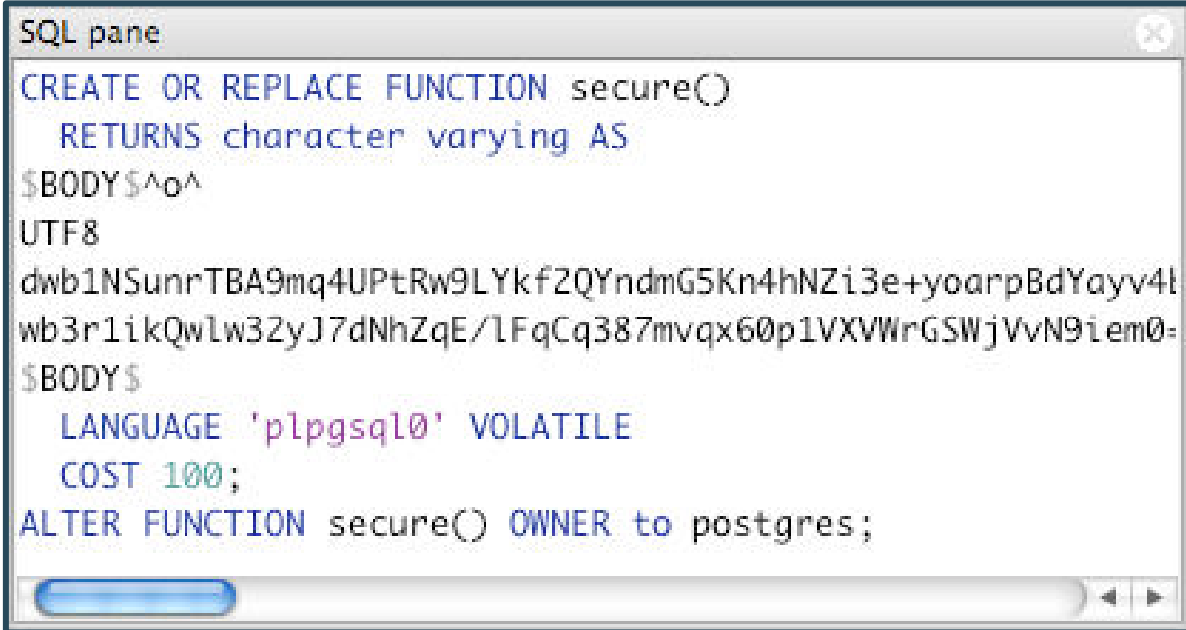
Data Redaction

- Data Redaction can be used to conceal data values from selected users
- The redaction function is incorporated into redaction policy using `CREATE REDACTION POLICY`
- Data Redaction is controlled by `edb_data_redaction` configuration parameter
- Useful for compliance to GDPR, PCI and HIPAA standards



EDB*Wrap – Protects Your Code

- Safeguards sensitive code from prying eyes inside your firewall
- Protects critical algorithms, processes, seed values and more
- Restricts access to intellectual property on customer sites
- Additional layer of security beyond standard user ACLs



```
SQL pane
CREATE OR REPLACE FUNCTION secure()
  RETURNS character varying AS
$BODY$^o^
UTF8
dwb1NSunrTBA9mq4UPtRw9LYkf2QYndmG5Kn4hNZi3e+yoarpBdYayv4l
wb3r1ikQwlw32yJ7dNhZqE/lFqCq387mvqx60p1VXVWrGSWjVvN9iem0=
$BODY$
  LANGUAGE 'plpgsql' VOLATILE
  COST 100;
ALTER FUNCTION secure() OWNER to postgres;
```



General Security Recommendations



General Recommendations - Database Server

- Always keep your system patched to the latest version
- Don't put a postmaster port on the Internet
 - Firewall this port appropriately
 - If that's not possible, make a read-only Replica database available on the port, not a R/W master
- Isolate the database port from other network traffic
- Don't rely solely on your front-end application to prevent unauthorized access to your database
- Avoid using trust authentication in [pg_hba.conf](#)



General Recommendations - Database Users

- Provide each user with their own login
 - Shared credentials make auditing more complicated and violate HIPAA, PCI, etc.
- Allow users the minimum access to do their jobs
- Use Roles and classes of privileges
- Use Views and View Security Barriers
- Use Row Level Security



General Recommendations - Connection Pooling

- When not practical to provide each user with their own login (i.e. connection pooling is in use):
 - Have one or more logins related to the application
 - Limit access to the database by the specific IP addresses where the application is certified to run
 - Ensure the login(s) have minimum rights needed to do their work (e.g. `SELECT` rights and only to specified tables)



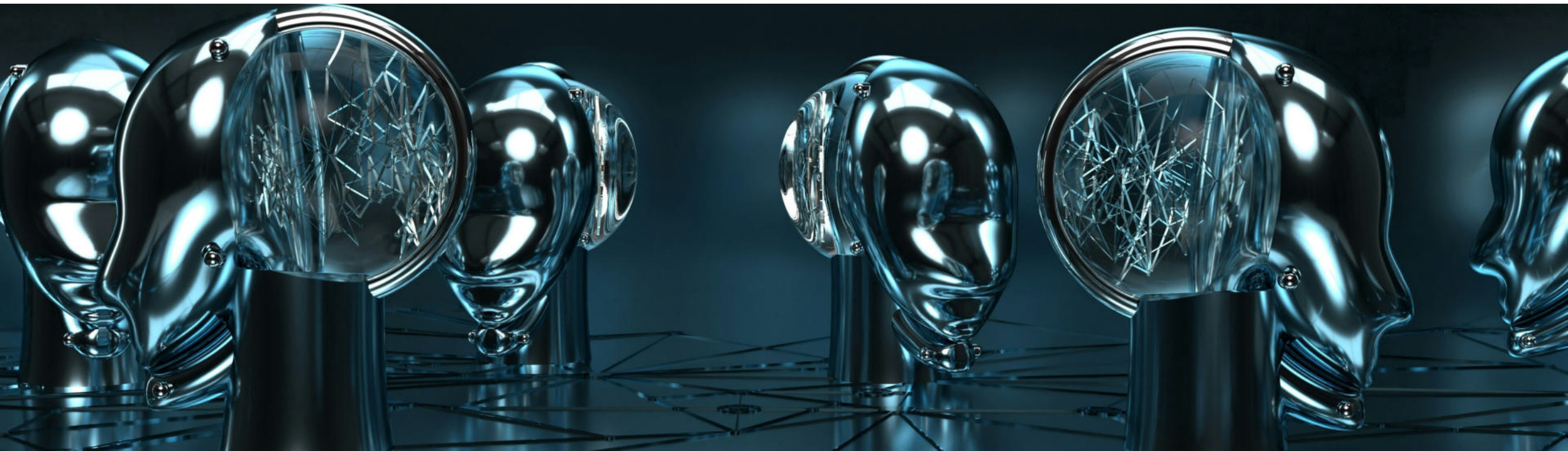
General Recommendations - Database Superuser

- Only allow the database superuser to log in from the server machine itself, with local or localhost connection
- Reserve use of superuser accounts for tasks or roles where it is absolutely required
- Make as few objects owned by the superuser as necessary
- Restrict access to configuration files ([postgresql.conf](#) and [pg_hba.conf](#)) and error log files to administrators
- Disallow host system login by database superuser roles ('[postgres](#)')



General Recommendations - Database Superuser (continued)

- Do not allow superuser to log into database server OS. Use personal OS login and then “sudo” to create an audit trail
- Use a separate database login to own each database and own everything in it



General Recommendations - Database Backups

- Keep backups and have a tested recovery plan. No matter how well you secure things, it's still possible an intruder could get in and delete or modify your data
- Have scripts perform backups and immediately test them and alert DBA on any failures
- Keep backups physically separate from the database server. A disaster can strike and take out an entire location, whether that's environmental (e.g. earthquake), malicious (e.g. hacker, insider), or human error



General Recommendations - Think AAA



Authenticate

Verify the user is
who she claims
to be



Authorize

Verify the user is
allowed access



Audit

Record which
user did what and
when they did it



Module Summary

- Database Security Requirements and Protection Plan
- Levels of Security in Postgres
- Access Control using pg_hba.conf
- Introduction to Row Level Security
- Data Encryption
- Advanced Features - Data Redaction, SQL/Protect and EDB*Wrap
- General Security Recommendations



Lab Exercise - 1

- You are working as an Enterprise Postgres DBA. Your server box has 2 network cards with ip addresses 192.168.30.10 and 10.4.2.10. 192.168.30.10 is used for the internal LAN and 10.4.2.10 is used by the web server to connect users from an external network. Your server should accept TCP/IP connections both from internal and external users.
- Configure your server to accept connections from external and internal networks.



Lab Exercise - 2

1. You are working as an Enterprise Postgres DBA. A developer showed you the following error:

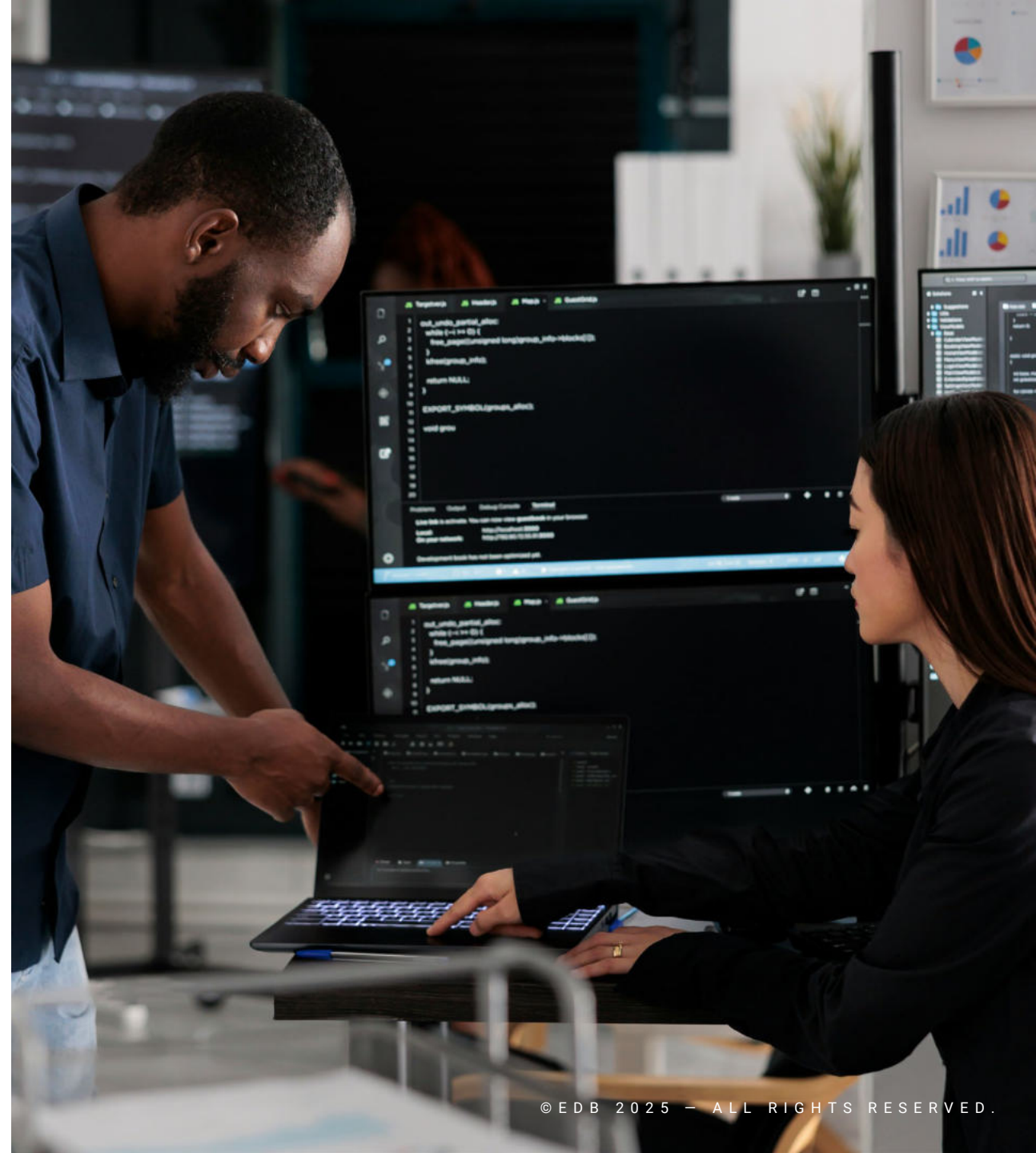
```
psql: could not connect to server: Connection refused  
(0x0000274D/10061)
```

- Is the server running on host 192.168.30.22 and accepting TCP/IP connections on port 5444?
2. Diagnose the problem and suggest the solution



Lab Exercise - 3

- A new developer has joined the team with ID number 89
- Create a new user by name `dev89` and password `password89`
- Then assign the necessary privileges to `dev89` so they can connect to the `edbstore` database and view all tables

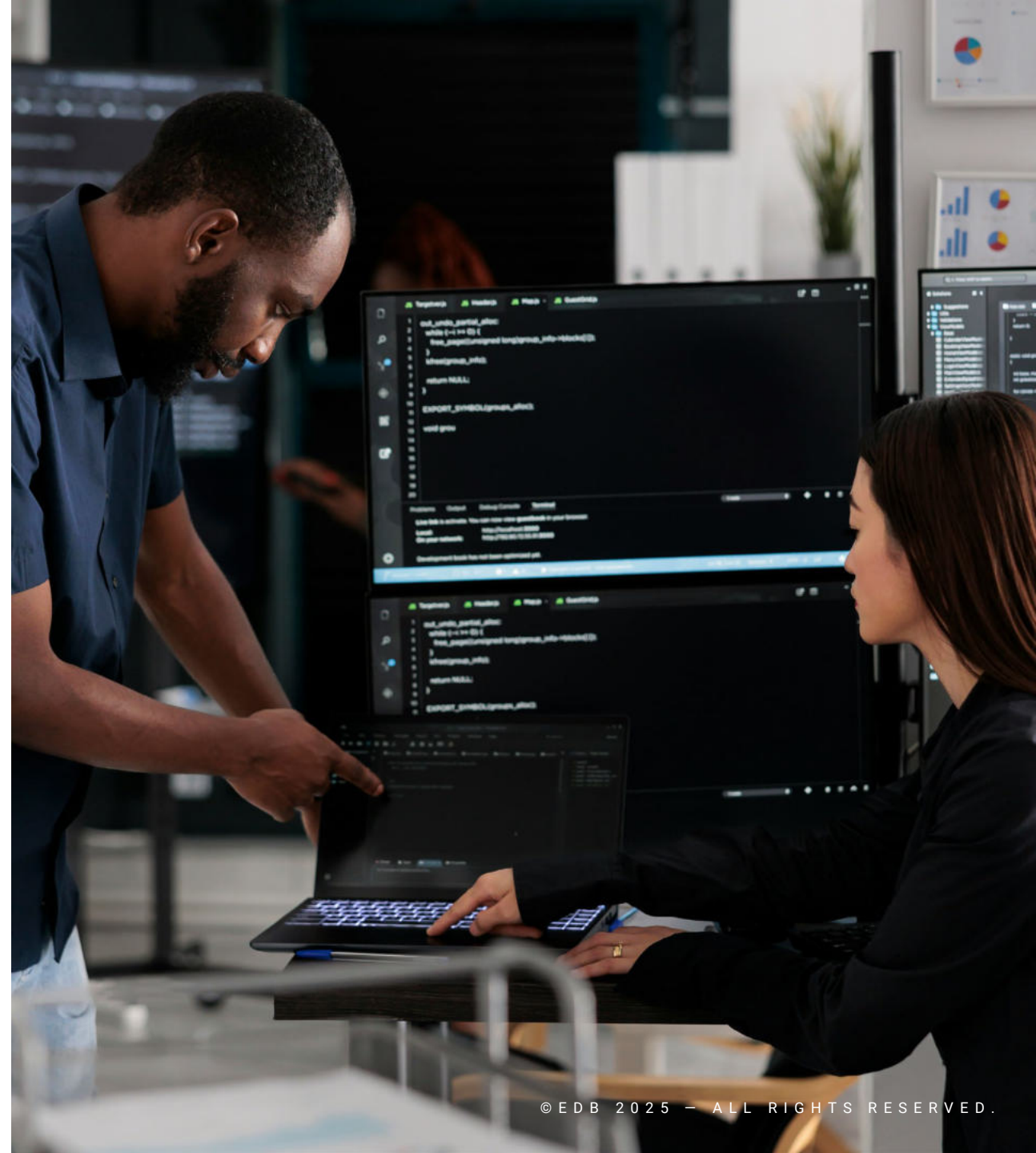


Lab Exercise - 4

- A new developer joins e-music corp. They have an ip address 192.168.30.89. They are not able to connect from their machine to the Enterprise Postgres and gets the following error on the server:

```
FATAL:  no pg_hba.conf entry  
for host "192.168.30.89", user  
"dev89", database "edbstore",  
SSL off
```

- Configure your server so that the new developer can connect from their machine





Monitoring and Admin Tools



Module Objectives

- Overview and Features of Postgres Enterprise Manager
- Access Postgres Enterprise Manager Client
- Register and Connect to a Database Server
- General Database Administration
- Object Browser - View Data, Query Tool, Server Status
- Overview of pgAdmin



Postgres Enterprise Manager

Manage, monitor, and tune Postgres at scale



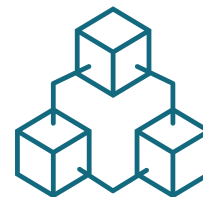
Manage from one interface

One place to visualize and manage everything



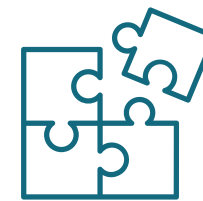
Optimize database performance

In-depth diagnostics for database reports and tuning



Monitor system health

Built-in dashboards and customizable alert thresholds



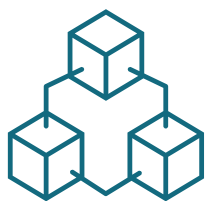
Integrate with other tools

APIs and webhooks to fetch data, send alerts, and manage servers



PEM - Features

Manage, Monitor and Tune PostgreSQL and Enterprise Postgres running on multiple Platforms



Management

- Integrated SQL IDE
- Built-in query debugger
- User/group access management
- Schema Diff
- Session profiling
- Job scheduling
- Backup and failover management



Monitoring

- Customizable charts and dashboards
- Predefined and custom alerts via email or SNMP
- User-defined metrics log analysis
- Database and OS level monitoring
- Web hooks and REST API for integrations

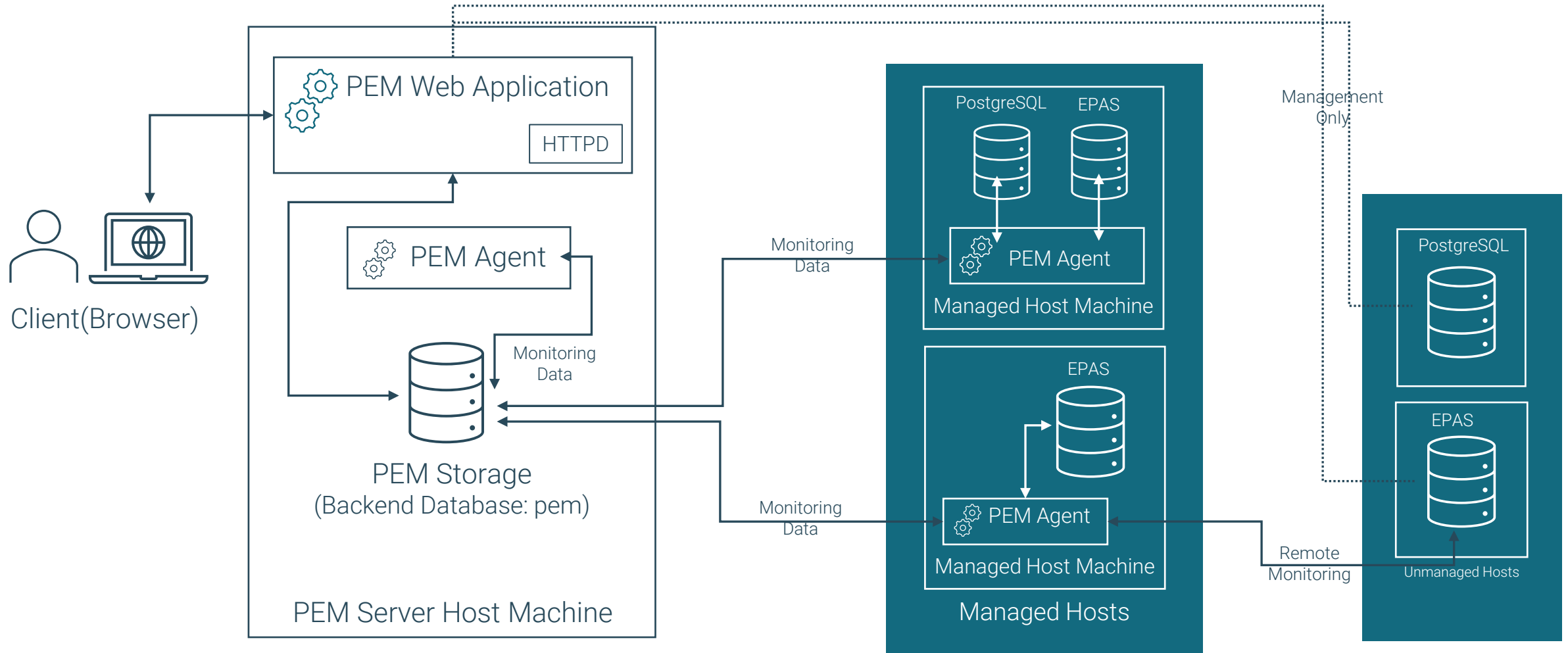


Tuning

- Detailed performance diagnostics
- SQL profiler
- Capacity management
- Log manager
- Expert wizards for configuration setup



PEM Architecture



Install and Configure PEM

- PEM Server can be installed using RPM or yum package manager :

```
dnf install edb-pem
```

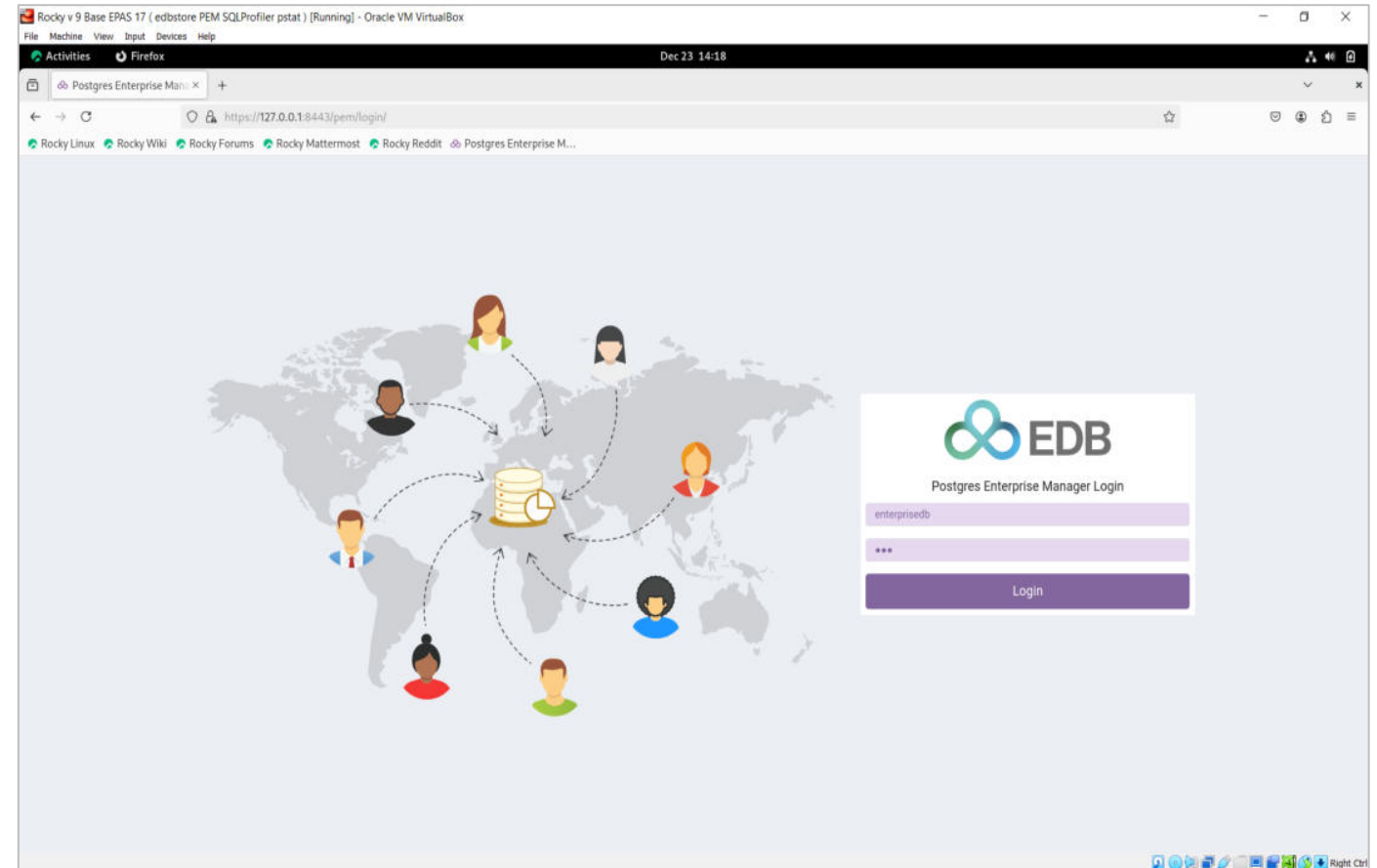
- After installation, PEM Server can be configured using [configure-pem-server.sh](#) script file

```
/usr/edb/pem/bin/configure-pem-server.sh
```



Open Postgres Enterprise Manager Client

- Postgres Enterprise Manager (PEM) Client can be run using a supported web browser on your OS



PEM Client First Look



File ▾Object ▾Management ▾Dashboards ▾Tools ▾Help ▾

enterisedb ▾

Browser

> BART Servers

> Barman Servers

> PEM Agents

> PEM Server Directory

DashboardPropertiesSQLStatisticsDependenciesDependentsMonitoring

Global Overview ▾

Object Type
System

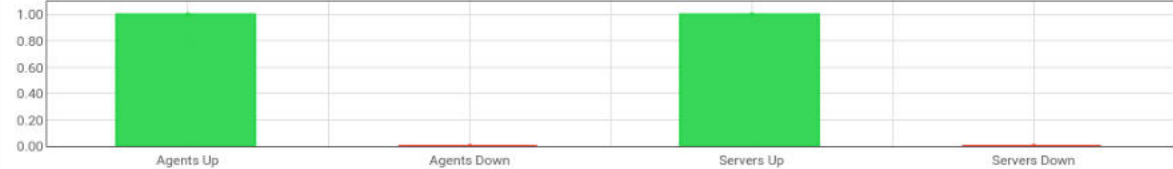
Status
N/A

Generated On
12/23/2024, 2:52:24 PM

No. of alerts
4 (Acknowledged : 0)

Enterprise Dashboard ▾

Status



Agents Up	Agents Down	Servers Up	Servers Down
1.00	0.00	1.00	0.00

Agent Status

Blackout	Status	Name	Alerts	Version	Processes	Threads	CPU Utilization (%)	Memory Utilization (%)	Swap Utilization (%)	Disk Utilization
☐	 Up	Postgres Enterprise Manager Host	1	9.8.0	236	931	16.15	28.21	0.00	32.86

Server Status

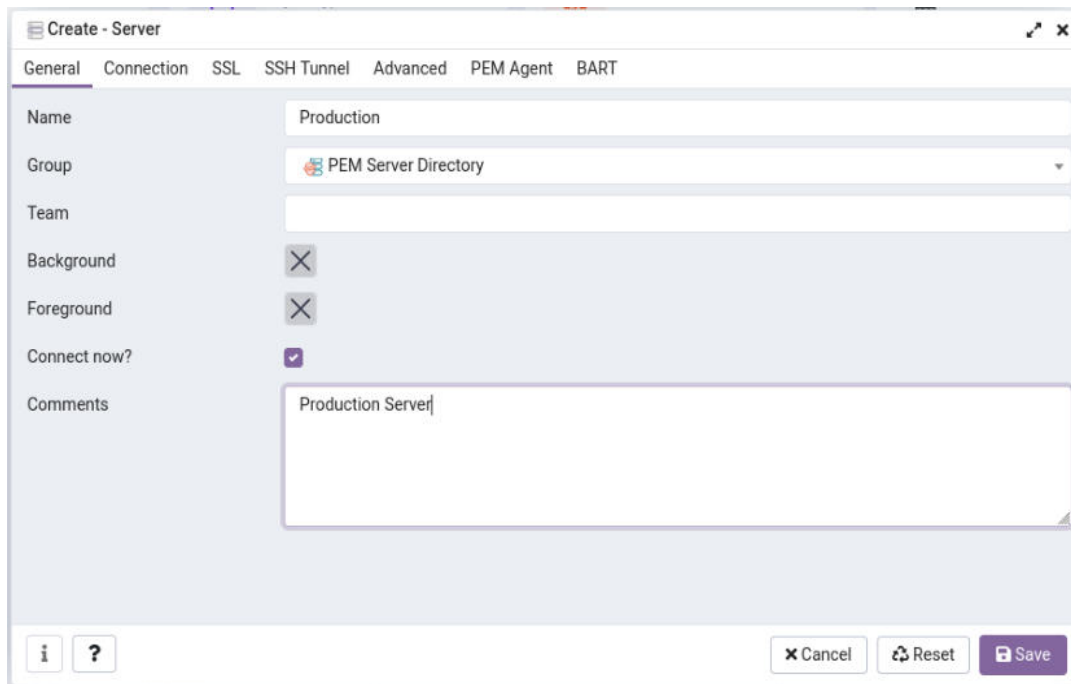
Blackout	Status	Name	Connections	Alerts	Version	Remotely Monitored?
☐	 Up	Postgres Enterprise Manager Server	7	3	PostgreSQL 17.2 (EnterpriseDB Advanced Server 17.2.0) on x86_64-pc-linux-gnu, compiled by gcc (GCC) 11.5.0 20240719 (Red Hat 11.5.0-2), 64-bit	No



© EDB 2025 - ALL RIGHTS RESERVED.

Register a Database Cluster

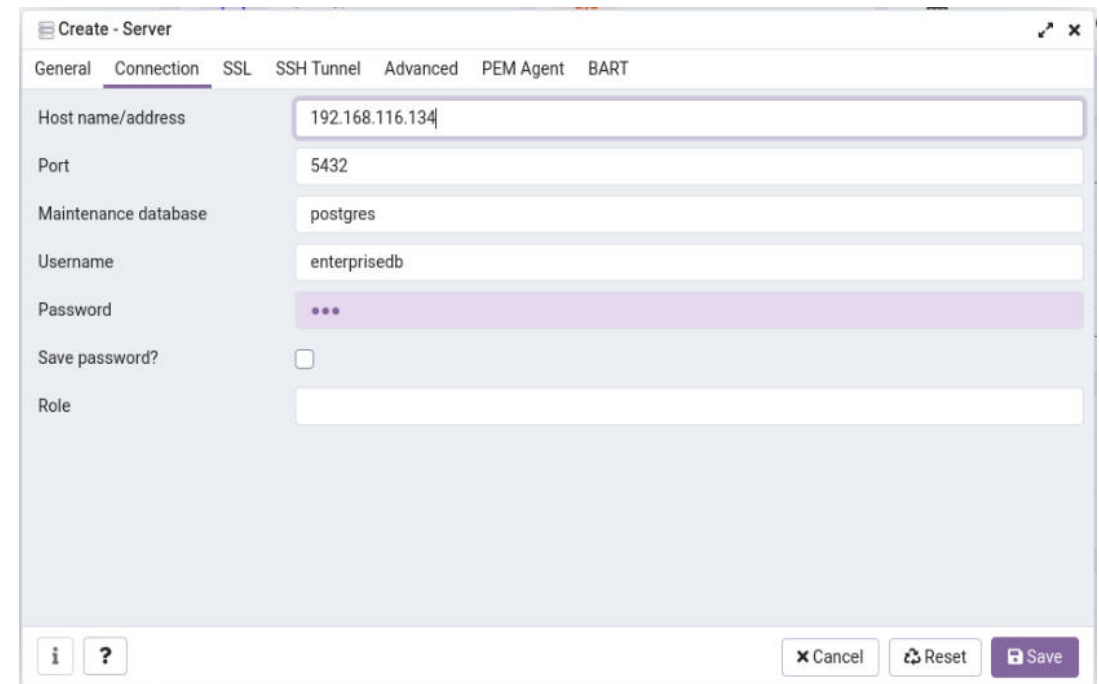
- Go to PEM Server Directory
- Right Click on PEM Enterprise Manager Server and select Create Server



The 'Create - Server' dialog box is shown with the 'General' tab selected. The fields are as follows:

Field	Value
Name	Production
Group	PEM Server Directory
Team	
Background	☐
Foreground	☐
Connect now?	☒
Comments	Production Server

At the bottom, there are buttons for 'Cancel', 'Reset', and 'Save'.



The 'Create - Server' dialog box is shown with the 'Connection' tab selected. The fields are as follows:

Field	Value
Host name/address	192.168.116.134
Port	5432
Maintenance database	postgres
Username	enterprisedb
Password	...
Save password?	☐
Role	

At the bottom, there are buttons for 'Cancel', 'Reset', and 'Save'.



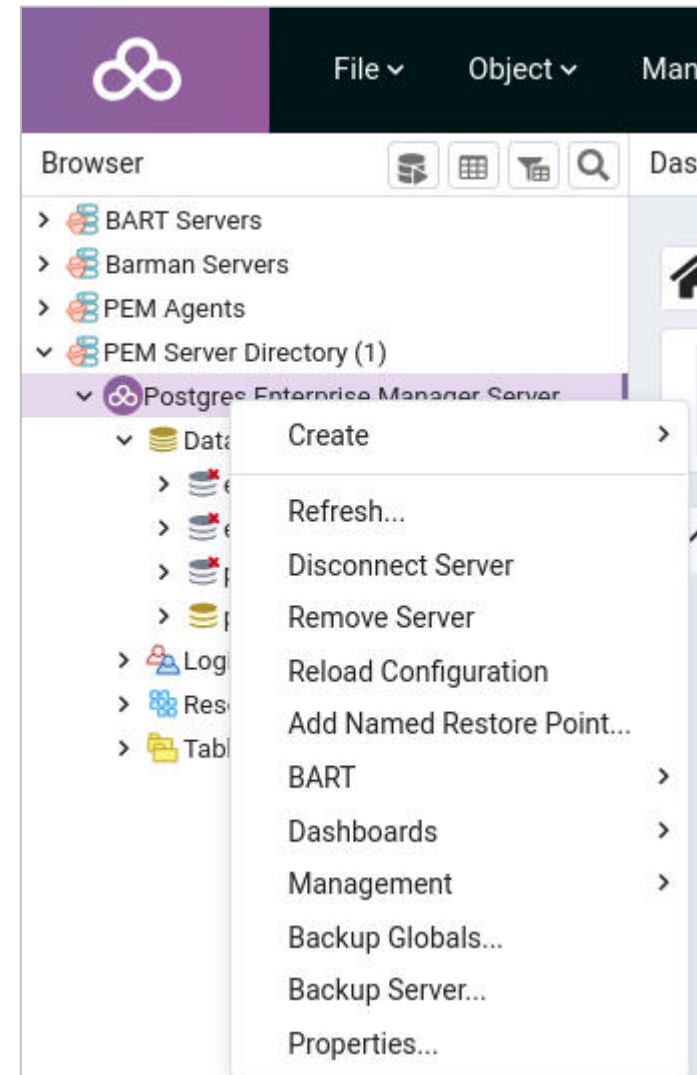
Common Connection Problems

- There are 2 common error messages encountered when connecting to an Enterprise Postgres database:
 - Could not connect to Server - Connection refused
 - This error occurs when either the database server isn't running OR the server isn't configured to accept external TCP/IP connections
 - httpd service not running
 - FATAL - no [pg_hba.conf](#) entry
 - This means your server can be contacted over the network but is not configured to accept the connection. Your client is not detected as a legal user for the database. You will have to add an entry for each of your clients to the [pg_hba.conf](#) file



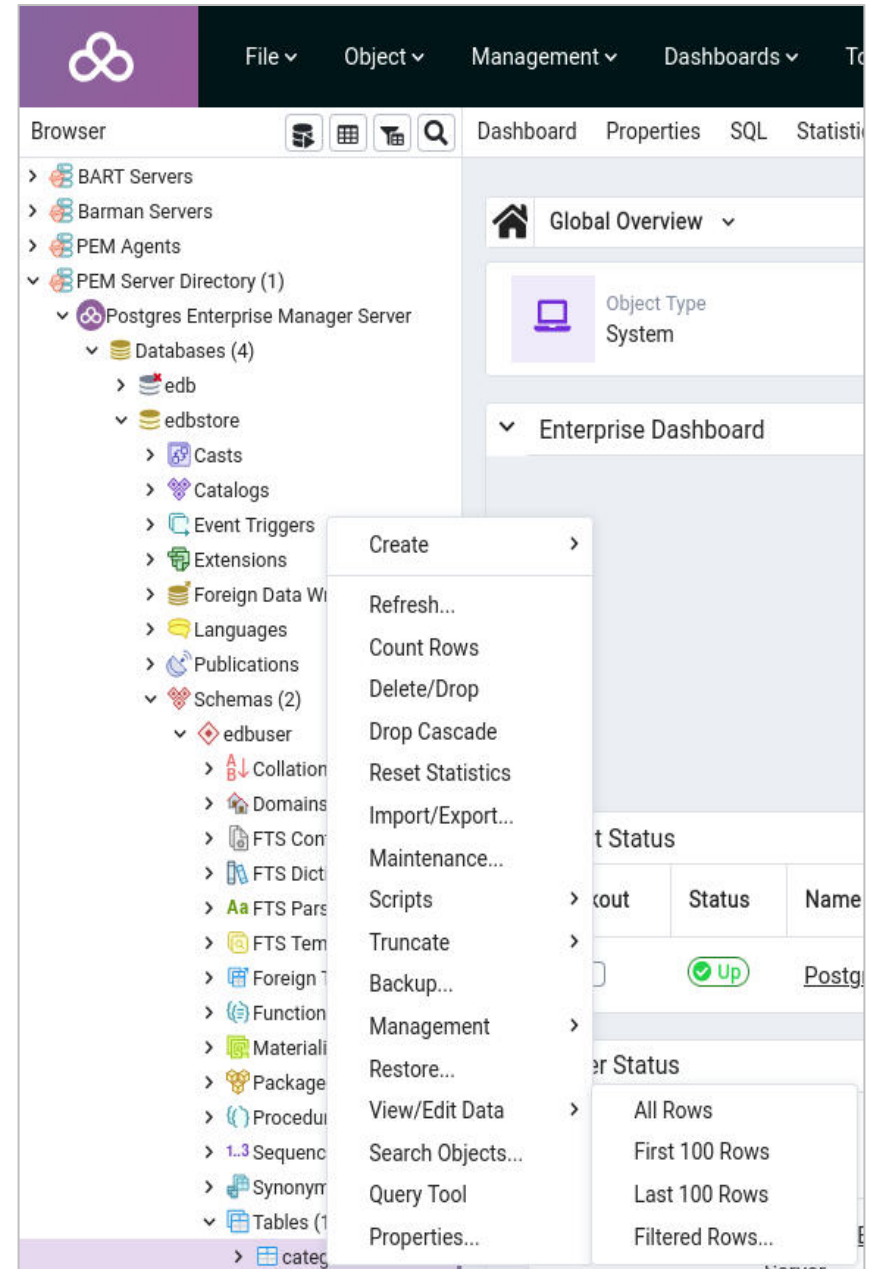
Changing a Server's Registration

- Right-click on a server entry to modify its properties
- Click on the [Remove Server](#) to remove a server's entry

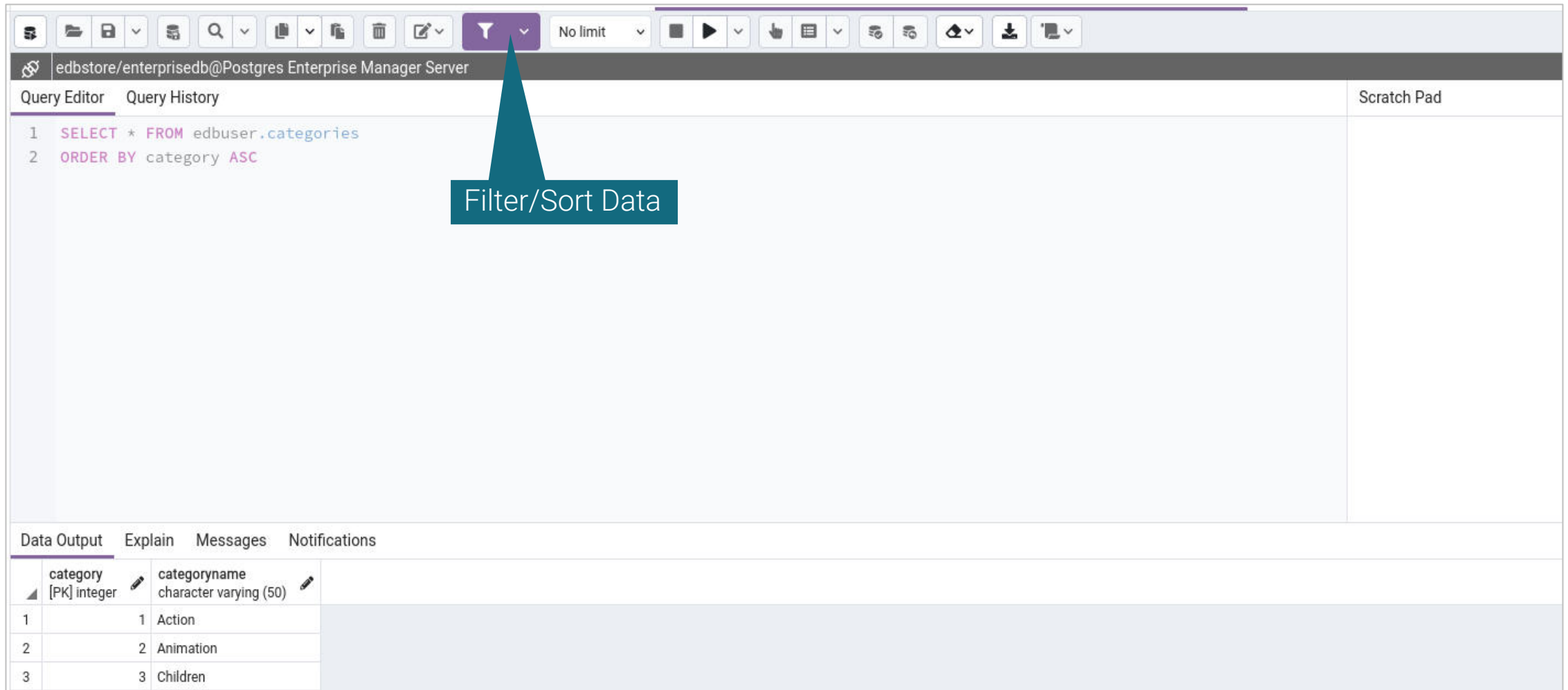


Viewing Data

- Expand Databases → Schemas → Tables
- Right-click on a table
- Select View Data



Filtering and Sorting Data



The screenshot displays the PostgreSQL Enterprise Manager interface. At the top, a toolbar contains various icons, including a funnel icon for filtering and sorting. A callout box labeled "Filter/Sort Data" points to this icon. Below the toolbar, the connection name "edbstore/enterisedb@Postgres Enterprise Manager Server" is shown. The main area is divided into two tabs: "Query Editor" and "Query History". The "Query Editor" tab is active, showing a SQL query:

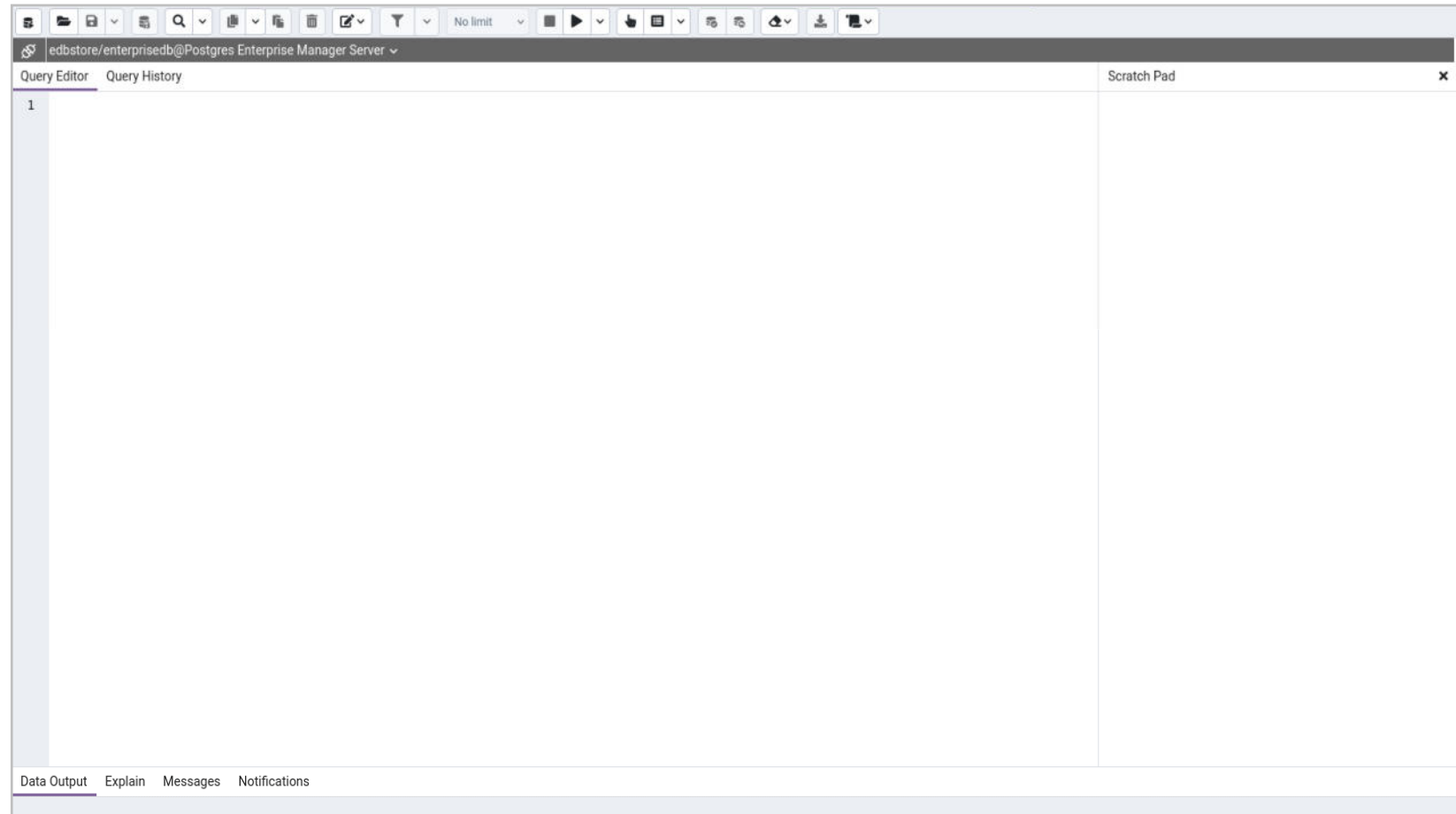
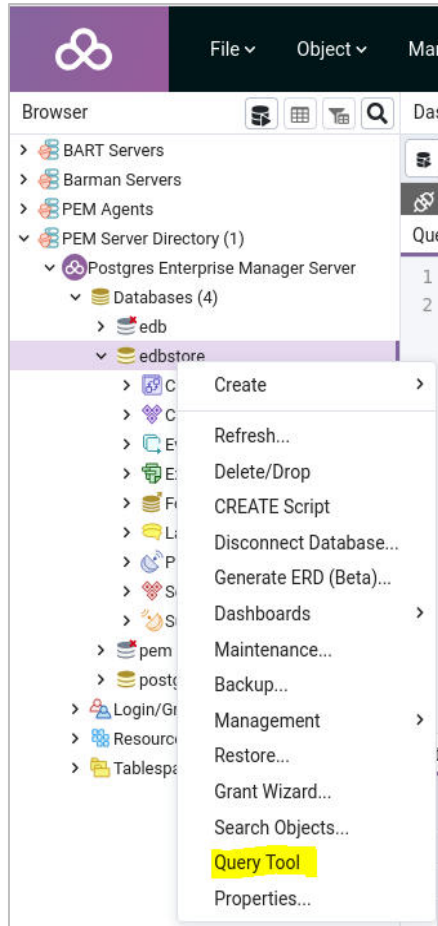
```
1 SELECT * FROM edbuser.categories
2 ORDER BY category ASC
```

To the right of the query editor is a "Scratch Pad" tab. At the bottom, there are four tabs: "Data Output", "Explain", "Messages", and "Notifications". The "Data Output" tab is active, displaying a table with the following data:

	category [PK] integer	categoryname character varying (50)
1	1	Action
2	2	Animation
3	3	Children

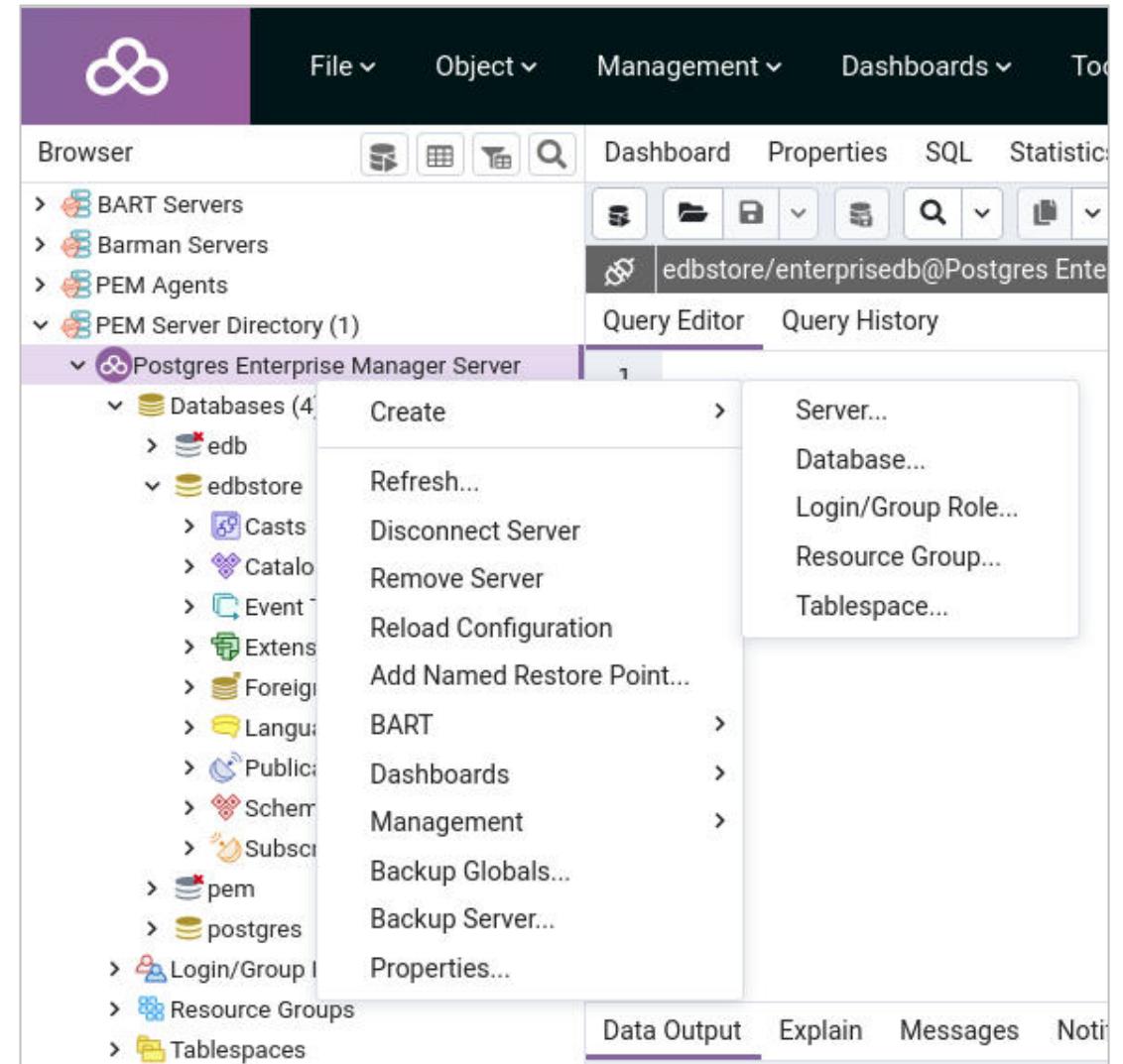


Query Tool

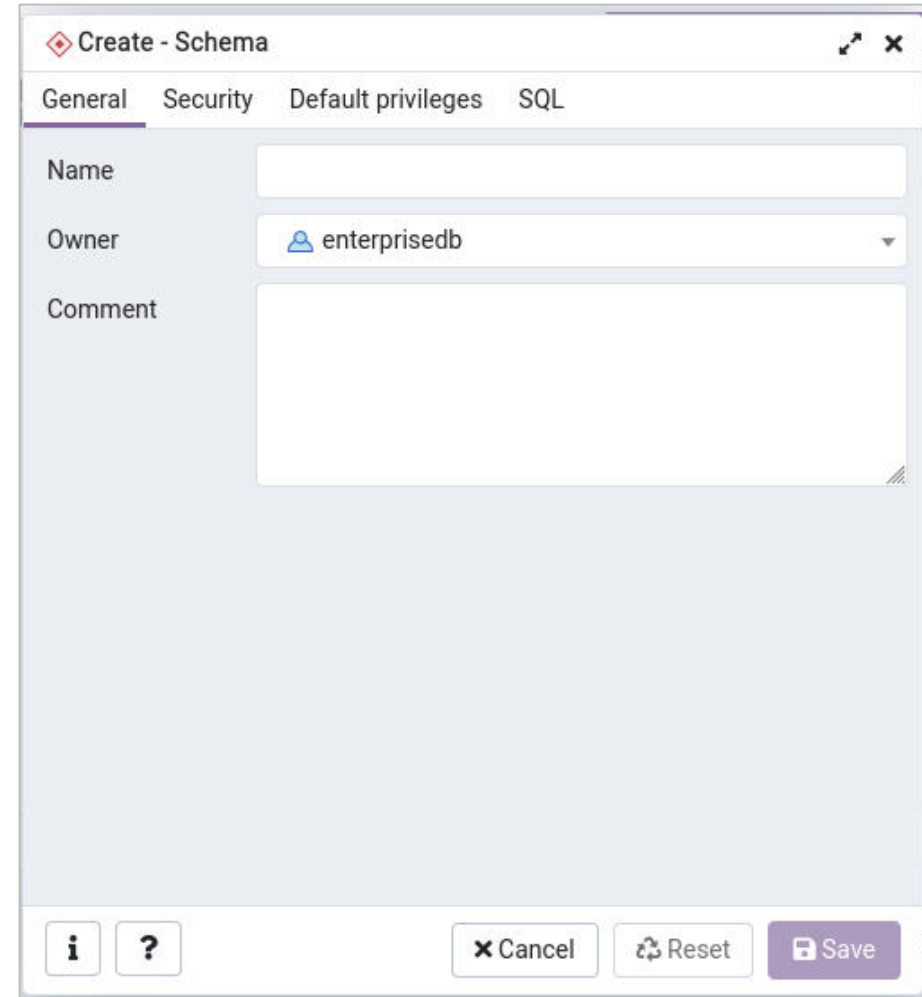
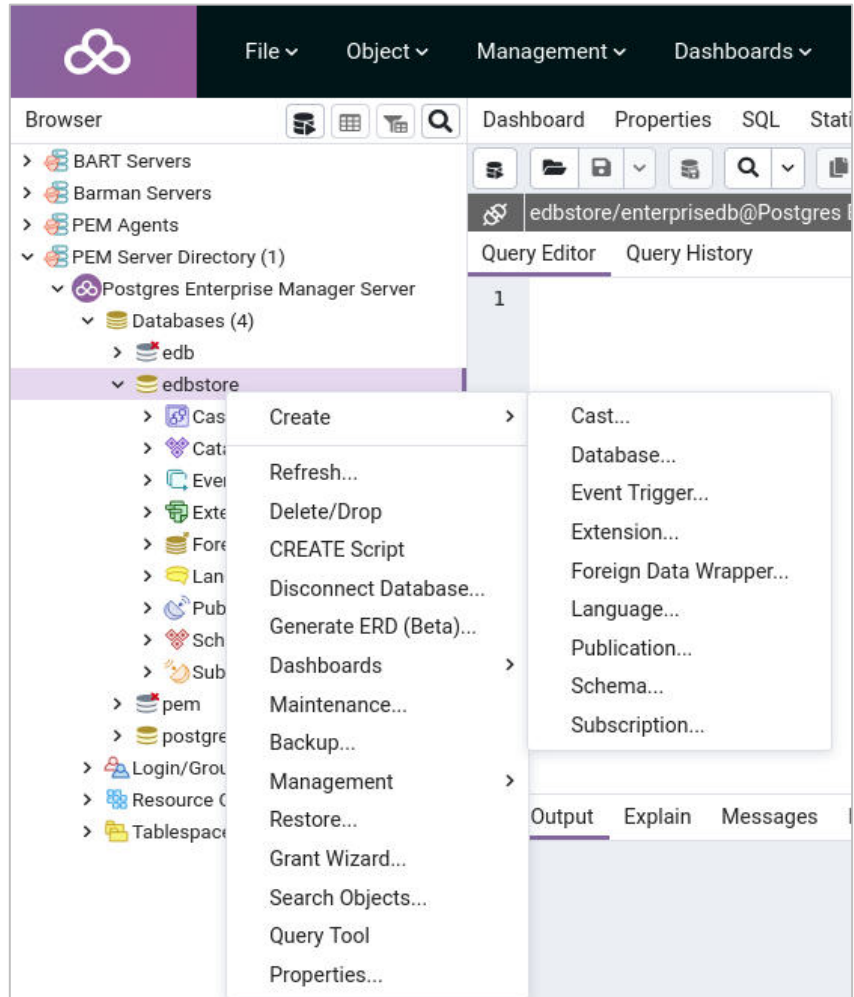


Databases

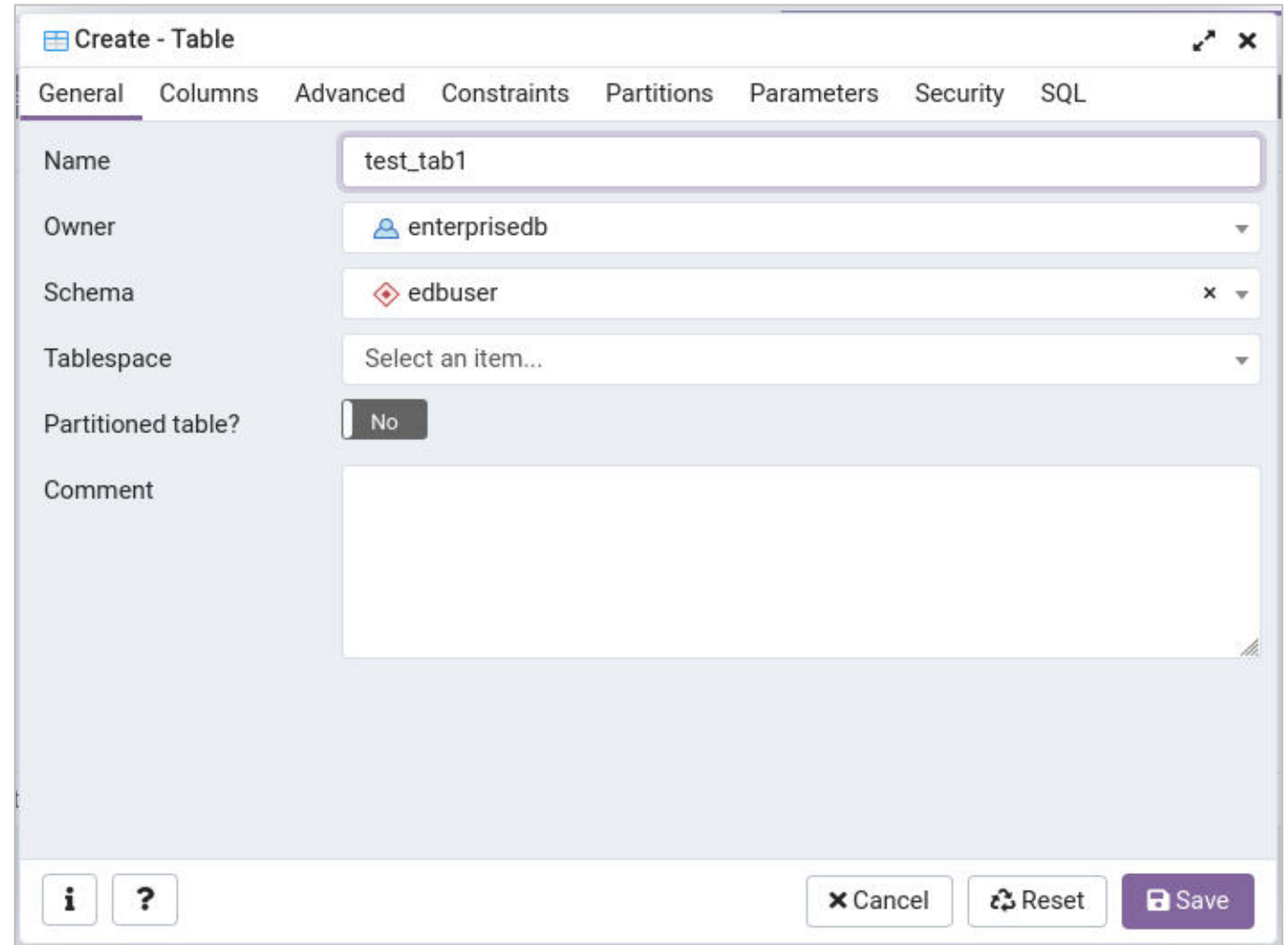
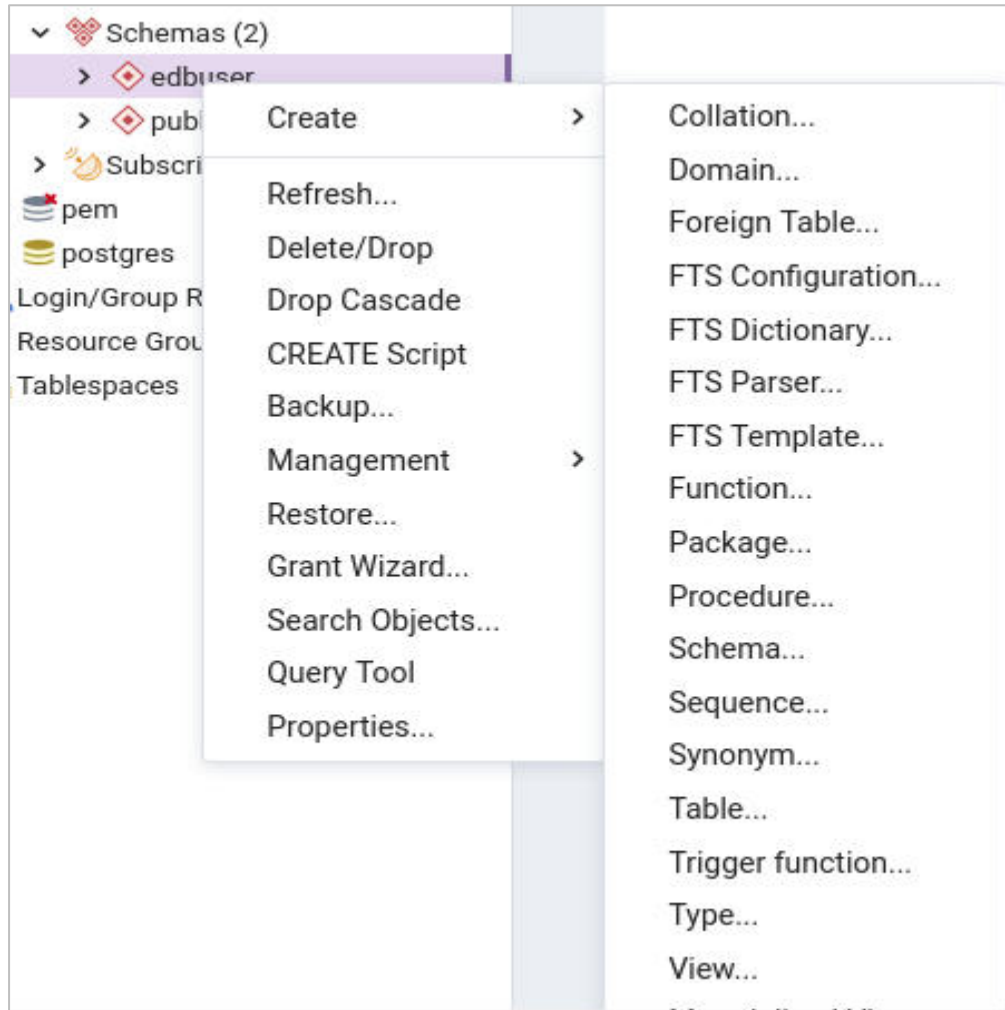
- Create a new database or run a report on the databases in a cluster with the databases menu
- Perform the following option with an individual database menu:
 - Create a new object in the database
 - Drop the database
 - Open the Query Tool with a script to re-create the database
 - Run reports
 - Perform maintenance
 - Backup or Restore
 - Modify the databases properties



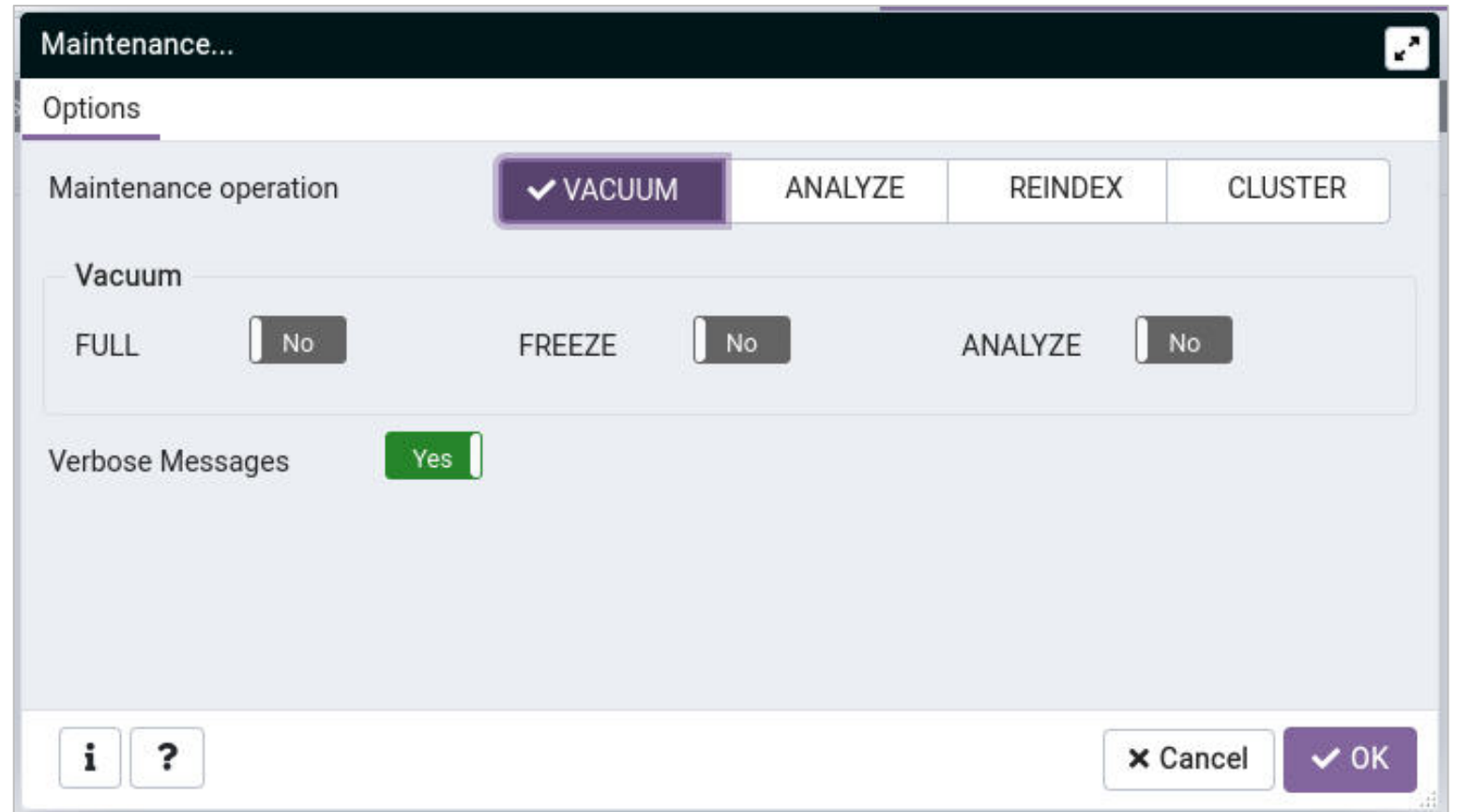
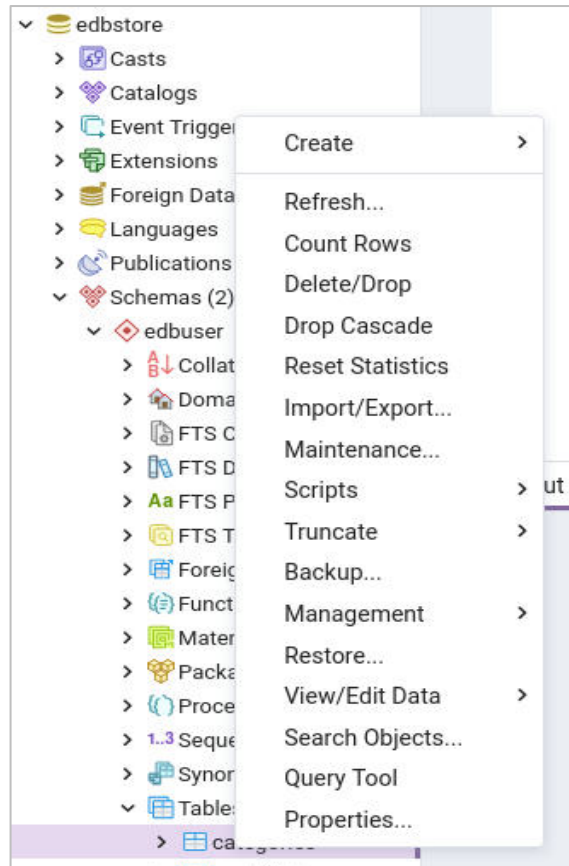
Schemas



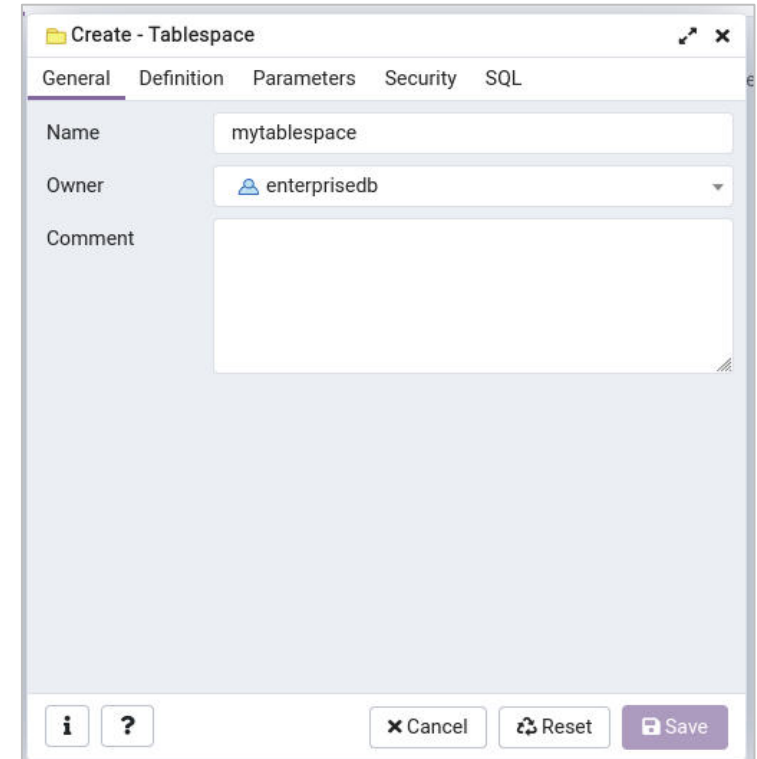
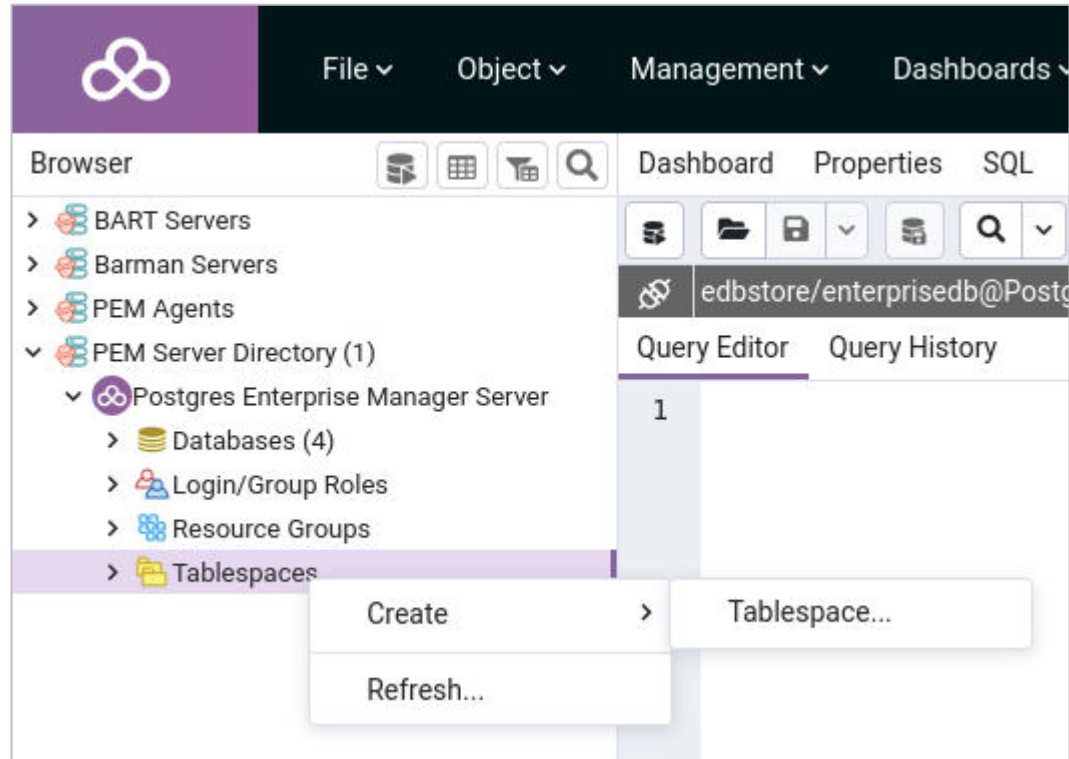
Tables



Tables - Maintenance



Tablespaces

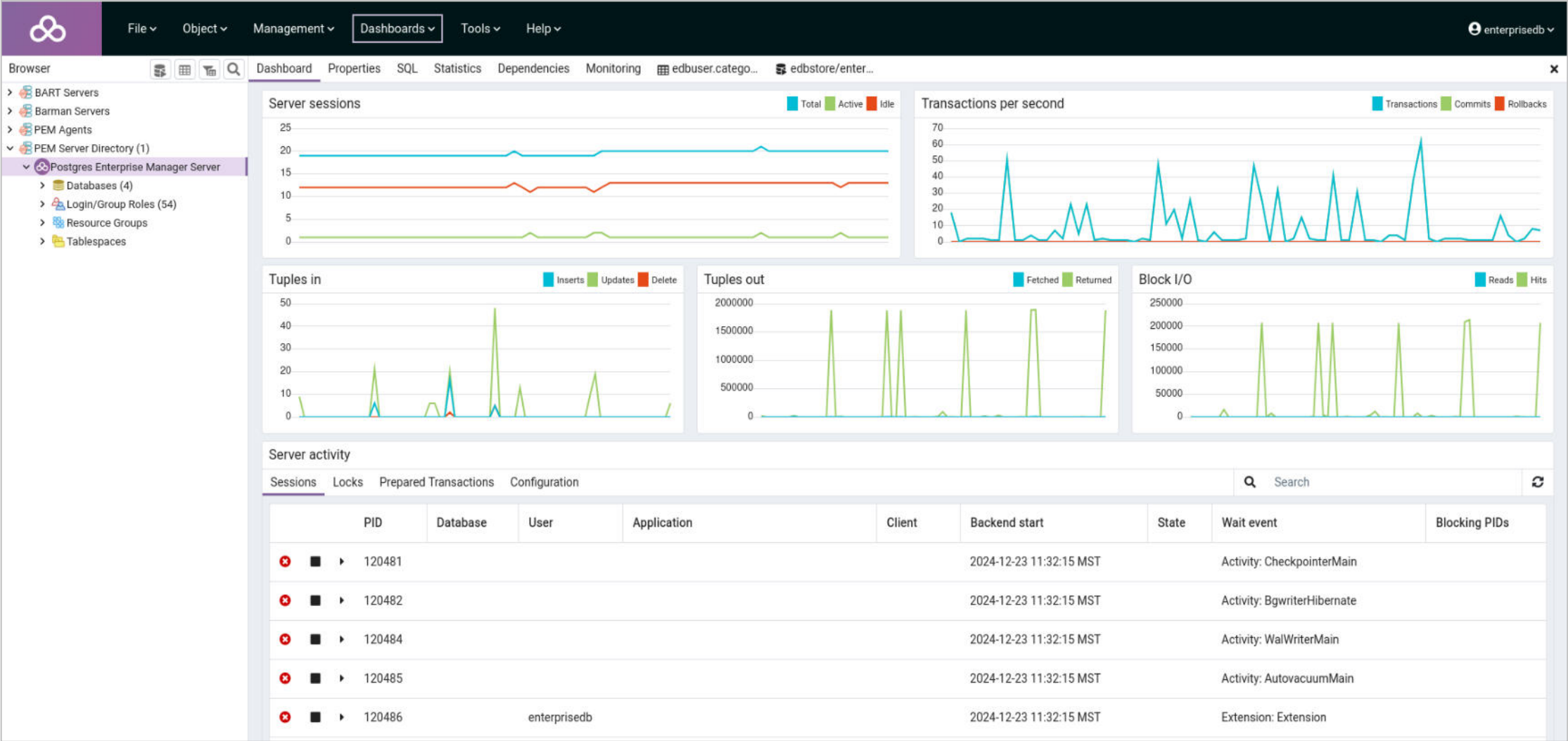


Roles

The screenshot displays the PostgreSQL Enterprise Manager (PEM) web interface. The top navigation bar includes a logo and menu items: File, Object, Management, Dashboards, Tools, and Help. Below this is a secondary navigation bar with tabs: Dashboard, Properties, SQL, Statistics, Dependencies, Monitoring, and two browser tabs labeled 'edbuser.catego...' and 'edbstore/enter...'. The left sidebar, titled 'Browser', shows a tree view of the database structure. Under 'PEM Server Directory (1)', the 'Postgres Enterprise Manager Server' is expanded, showing 'Databases (4)', 'Login/Group Roles (54)', 'Resource Groups', and 'Tablespaces'. The 'Login/Group Roles (54)' item is selected, and a context menu is open with options 'Create' and 'Refresh...'. The 'Create' option is highlighted, and a 'Login/Group Role...' dialog box is displayed. This dialog box has tabs for 'General', 'Definition', 'Privileges', 'Membership', 'Parameters', 'Security', and 'SQL'. The 'General' tab is active, showing a 'Name' field with the value 'enterprisedb' and a 'Comments' text area. At the bottom of the dialog box are buttons for 'i', '?', 'Cancel', 'Reset', and 'Save'.



Server Status



Overview of pgAdmin

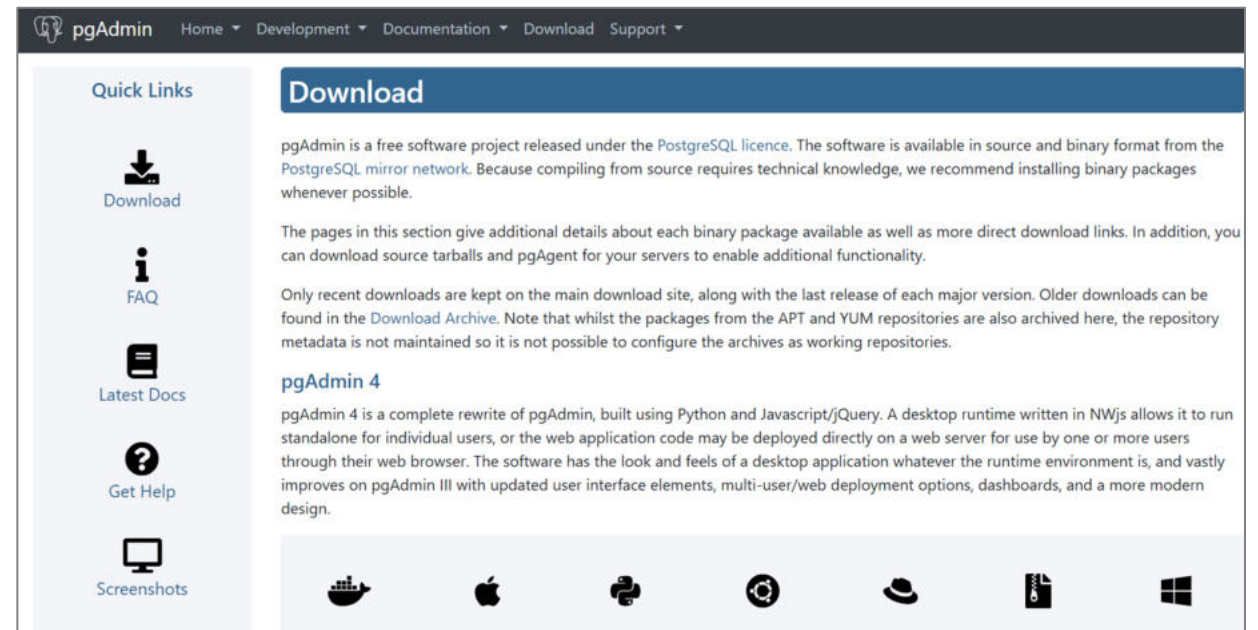


Introduction to pgAdmin

- Open-source graphical user interface for Postgres
- Create, manage and maintain database objects
- pgAdmin is web based and requires Apache HTTP server

Download and Install:

<https://www.pgadmin.org/download/>



pgAdmin Features

Multi-platform

Supports
PostgreSQL and
Enterprise Postgres

Multi-deployment
Mode – Desktop,
Server

Integrated SQL IDE

pl/pgsql and edb-spl
Debugger

Schema Diff Tool

ERD Tool

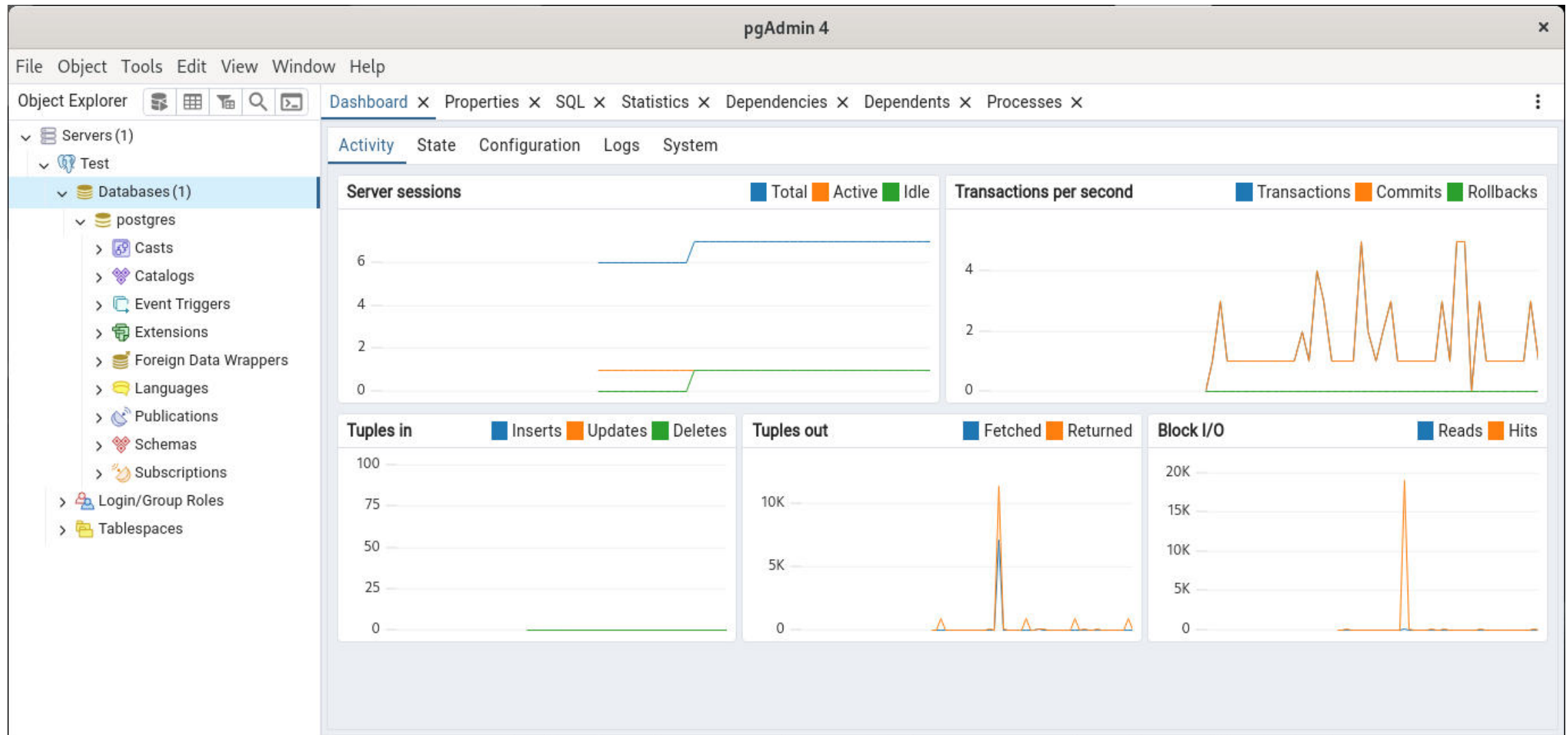
Perform
Maintenance Tasks-
Vacuum, Backups,
Restore etc.

Job Scheduler

Multibyte server-side
encoding support

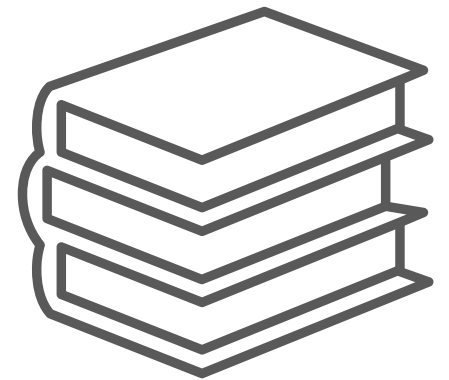


First Look - pgAdmin



Module Summary

- Overview and Features of Postgres Enterprise Manager
- Access Postgres Enterprise Manager Client
- Register and Connect to a Database Server
- General Database Administration
- Object Browser - View Data, Query Tool, Server Status
- Overview of pgAdmin





SQL Primer



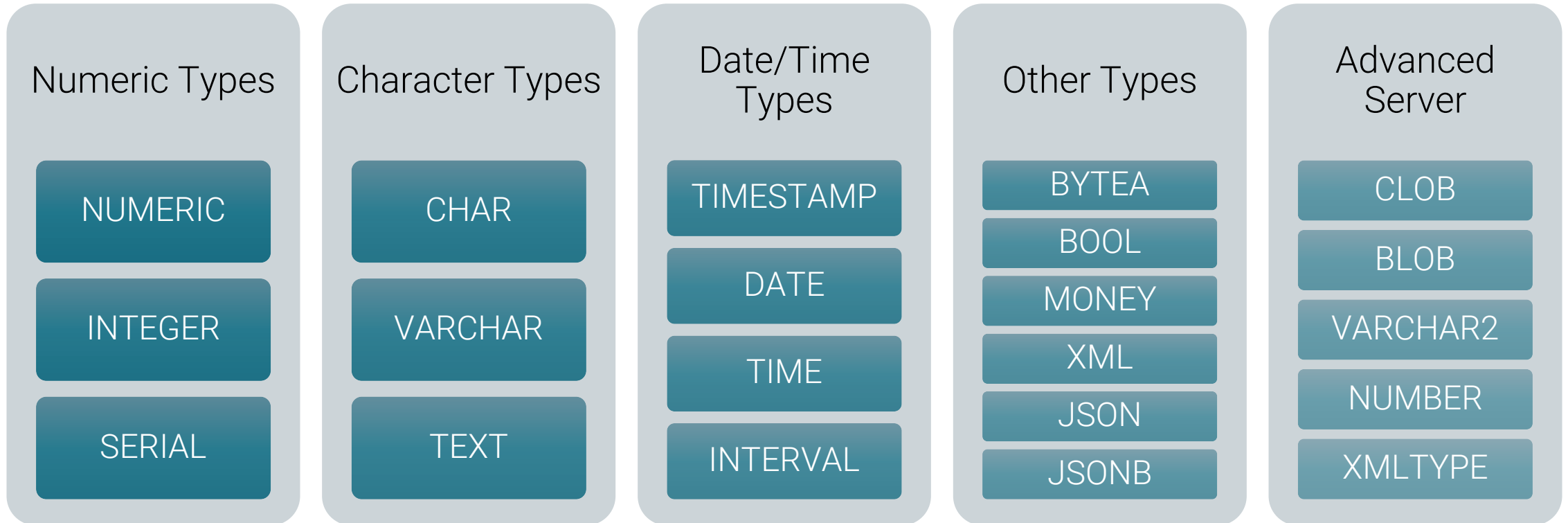
Module Objectives

- Data Types
- Structured Query Language (SQL)
- DDL, DML and DCL Statements
- Transaction Control Statements
- Tables and Constraints
- Views and Materialized Views
- Sequences
- Domains
- SQL Joins and Functions
- Explain Plans
- Quoting in PostgreSQL
- Indexes
- Oracle Compatibility and Tools



Data Types

Common Data Types:



Oracle Compatible Data Types

The built-in general-purpose data types:

BFILE	BLOB	BOOLEAN	CHAR	CLOB	DATE
DOUBLE	BINARY	VARBINARY	INTEGER	NUMBER	TIMESTAMP
VARCHAR2	NVARCHAR2	ROWID	INTERVAL	XML	



Structured Query Language

Data Definition Language

CREATE

ALTER

DROP

TRUNCATE

Data Manipulation Language

INSERT

UPDATE

DELETE

Data Control Language

GRANT

REVOKE

Transaction Control Language

COMMIT

ROLLBACK

SAVEPOINT

SET TRANSACTION



DDL Statements

Statement	Syntax
CREATE TABLE	<pre>CREATE [TEMPORARY][UNLOGGED] TABLE table_name ([column_name data_type [column_constraint]) [INHERITS (parent_table)] [TABLESPACE tablespace_name] [USING INDEX TABLESPACE tablespace_name] [PARTITION BY { RANGE LIST HASH } (column_name (expression)]</pre>
ALTER TABLE	<pre>ALTER TABLE [IF EXISTS] [ONLY] name [*] action [...]</pre>
DROP TABLE	<pre>DROP TABLE [IF EXISTS] name [, ...] [CASCADE RESTRICT]</pre>
TRUNCATE TABLE	<pre>TRUNCATE [TABLE] [ONLY] name [*] [,]</pre>



DML Statements

Statement	Syntax
INSERT	INSERT INTO table_name [(column_name [, ...])] { DEFAULT VALUES VALUES ({ expression DEFAULT } [, ...]) [, ...] query }
UPDATE	UPDATE [ONLY] table_name SET column_name = { expression DEFAULT } [WHERE condition]
DELETE	DELETE FROM [ONLY] table_name [WHERE condition]
SELECT	SELECT [ALL DISTINCT] [* expression] [FROM table [, ..]



DCL Statements

Statement	Syntax
GRANT	<pre>GRANT { { SELECT INSERT UPDATE } [, ...] ALL [PRIVILEGES] } ON { [TABLE] table_name [, ...] ALL TABLES IN SCHEMA schema_name [, ...] } TO role_specification [, ...] [WITH GRANT OPTION]</pre>
REVOKE	<pre>REVOKE [GRANT OPTION FOR] { { SELECT INSERT UPDATE } [, ...] ALL [PRIVILEGES] } ON { [TABLE] table_name [, ...] ALL TABLES IN SCHEMA schema_name [, ...] } FROM { [GROUP] role_name PUBLIC } [, ...]</pre>



Transaction Control Language

Statement	Syntax
COMMIT	COMMIT [WORK TRANSACTION]
ROLLBACK	ROLLBACK [WORK TRANSACTION]
SAVEPOINT	SAVEPOINT savepoint_name
SET TRANSACTION	SET TRANSACTION transaction_mode [, ...]



Database Objects

Object	Description
TABLE	Named collection of rows
VIEW	Virtual table, can be used to hide complex queries
SEQUENCE	Used to automatically generate integer values that follow a pattern
INDEX	A common way to enhance query performance
DOMAIN	A data type with optional constraints



Tables

- A table is a named collection of rows
- Each table row has same set of columns
- Each column has a data type
- Tables can be created using the CREATE TABLE statement
- Syntax:

```
CREATE [ [ GLOBAL | LOCAL ] { TEMPORARY | TEMP } | UNLOGGED ] TABLE [ IF NOT EXISTS ] table_name ( [  
    { column_name data_type [ COLLATE collation ] [ column_constraint [ ... ] ]  
    | table_constraint  
    | LIKE source_table [ like_option ... ] }  
    [, ... ]  
)  
[ INHERITS ( parent_table [, ... ] ) ]  
[ PARTITION BY { RANGE | LIST } ( { column_name | ( expression ) } [ COLLATE collation ] [ opclass ] [, ... ] ) ]  
[ WITH ( storage_parameter [= value] [, ... ] ) | WITH OIDS | WITHOUT OIDS ]  
[ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]  
[ TABLESPACE tablespace_name ]
```



Types of Constraints

- Constraints are used to enforce data integrity
- Enterprise Postgres supports different types of constraints:
 - `NOT NULL`
 - `CHECK`
 - `UNIQUE`
 - `PRIMARY KEY`
 - `FOREIGN KEY`
- Constraints can be defined at the column level or table level
- Constraints can be added to an existing table using the `ALTER TABLE` statement
- Constraints can be declared `DEFERRABLE` or `NOT DEFERRABLE`
- Constraints prevent the deletion of a table if there are dependencies



Views

- A View is a Virtual Table and can be used to hide complex queries
- Can also be used to represent a selected view of data
- Simple views are updatable and allow non-updatable columns
- Views can be created using the `CREATE VIEW` statement
- Syntax:

```
=> CREATE [ OR REPLACE ] VIEW name [ ( column_name [, ...] ) ]  
    [ WITH ( view_option_name [= view_option_value] [, ...] ) ]  
    AS query
```



Sequences

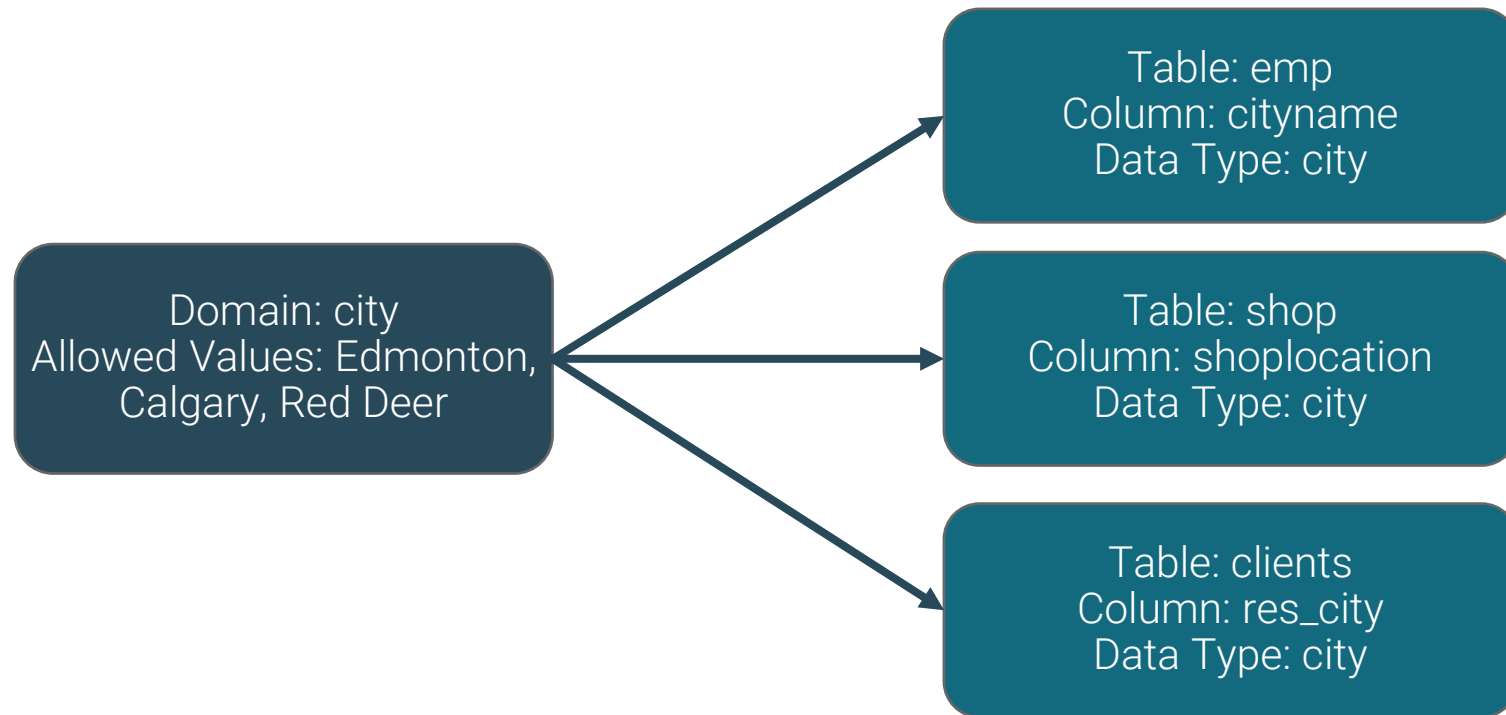
- A sequence is used to automatically generate integer values that follow a pattern
- A sequence has a name, start point and an end point
- Sequence values can be cached for performance
- Sequence can be used using CURRVAL and NEXTVAL functions
- Syntax:

```
=> CREATE SEQUENCE name [ INCREMENT [ BY ] increment ]  
    [ MINVALUE minvalue ] [ MAXVALUE maxvalue ]  
    [ START [ WITH ] start ] [ CACHE cache ] [ [ NO ] CYCLE ]  
    [ OWNED BY { table_name.column_name | NONE } ]
```



Domains

- A domain is a data type with optional constraints
- Domains can be used to create a data type which allows a selected list of values



Types of JOINS

Type	Description
INNER JOIN	Returns all matching rows from both tables
LEFT OUTER JOIN	Returns all matching rows and rows from left-hand table even if there is no corresponding row in the joined table
RIGHT OUTER JOIN	Returns all matching rows and rows from right-hand table even if there is no corresponding row in the joined table
FULL OUTER JOIN	Returns all matching as well as not matching rows from both tables
CROSS JOIN	Returns all rows of both tables with Cartesian product on number of rows



Using SQL Functions

- Can be used in `SELECT` statements and `WHERE` clauses
- Includes
 - String Functions
 - Format Functions
 - Date and Time Functions
 - Aggregate Functions
- Example:

```
=> SELECT lower(name) FROM departments;
```

```
=> SELECT * FROM departments  
    WHERE lower(name) = 'development';
```



SQL Format Functions

Function	Return Type	Description	Example
<code>to_char(timestamp, text)</code>	text	convert time stamp to string	<code>to_char(current_timestamp, 'HH12:MI:SS')</code>
<code>to_char(interval, text)</code>	text	convert interval to string	<code>to_char(interval '15h 2m 12s', 'HH24:MI:SS')</code>
<code>to_char(int, text)</code>	text	convert integer to string	<code>to_char(125, '999')</code>
<code>to_char(double precision, text)</code>	text	real/double precision to string	<code>to_char(125.8::real, '999D9')</code>
<code>to_char(numeric, text)</code>	text	convert numeric to string	<code>to_char(-125.8, '999D99S')</code>
<code>to_date(text, text)</code>	date	convert string to date	<code>to_date('05 Dec 2000', 'DD Mon YYYY')</code>
<code>to_number(text, text)</code>	numeric	convert string to numeric	<code>to_number('12,454.8-', '99G999D9S')</code>
<code>to_timestamp(text, text)</code>	timestamp with time zone	convert string to time stamp	<code>to_timestamp('05 Dec 2000', 'DD Mon YYYY')</code>
<code>to_timestamp(double precision)</code>	timestamp with time zone	convert Unix epoch to time stamp	<code>to_timestamp(1284352323)</code>



Execution Plan

- An execution plan shows the detailed steps necessary to execute a SQL statement
- Planner is responsible for generating the execution plan
- The Optimizer determines the most efficient execution plan
- Optimization is cost-based, cost is estimated resource usage for a plan
- Cost estimates rely on accurate table statistics, gathered with `ANALYZE`
- Costs also rely on `seq_page_cost`, `random_page_cost`, and others
- The `EXPLAIN` command is used to view a query plan
- `EXPLAIN ANALYZE` is used to run the query to get actual runtime stats



Execution Plan Components

- Execution Plan Components:
 - Cardinality - Row Estimates
 - Access Method - Sequential or Index
 - Join Method - Hash, Nested Loop etc.
 - Join Type, Join Order
 - Sort and Aggregates

- Syntax:

```
EXPLAIN [ ( option [, ...] ) ] statement
```

where option can be one of:

```
ANALYZE [ boolean ]
```

```
VERBOSE [ boolean ]
```

```
COSTS [ boolean ]
```

```
SETTINGS [ boolean ]
```

```
GENERIC_PLAN [ boolean ]
```

```
BUFFERS [ boolean ]
```

```
SERIALIZE [ { NONE | TEXT | BINARY } ]
```

```
WAL [ boolean ]
```

```
TIMING [ boolean ]
```

```
SUMMARY [ boolean ]
```

```
MEMORY [ boolean ]
```

```
FORMAT { TEXT | XML | JSON | YAML }
```



Explain Example

- Example

```
postgres=# EXPLAIN SELECT * FROM emp;
```

```
          QUERY PLAN
```

```
-----  
Seq Scan on emp  (cost=0.00..1.14 rows=14 width=145)
```

- The numbers that are quoted by `EXPLAIN` are:
 - Estimated start-up cost
 - Estimated total cost
 - Estimated number of rows output by this plan node
 - Estimated average width (in bytes) of rows output by this plan node



PEM - Query Tool's Visual Explain

edbstore/postgres@Postgres

Query Editor Query History Scratch Pad

```
1 select firstname,lastname, city, orderid from edbuser.customers join edbuser.cust_hist using (customerid);
```

Data Output Explain Messages Notifications

Graphical Analysis Statistics

edbuser.cust_hist

Hash Inner Join

edbuser.customers

Hash

Actual Startup Time	12.501
Actual Total Time	36.167
Actual Rows	60350
Actual Loops	1
Output	customers.firstname,customers.lastname,customers.city,cust_hist.orderid
Inner Unique	true
Hash Cond	(cust_hist.customerid = customers.customerid)
Shared Hit Blocks	3
Shared Read Blocks	833
Shared Dirtied Blocks	0
Shared Written Blocks	0
Local Hit Blocks	0
Local Read Blocks	0
Local Dirtied Blocks	0
Local Written Blocks	0
Temp Read Blocks	0
Temp Written Blocks	0
_serial	1



Quoting

- Single quotes and dollar quotes are used to specify non-numeric values

- Example:

```
'hello world'
```

```
'2011-07-04 13:36:24'
```

```
'{1,4,5}'
```

```
$$A string "with" various 'quotes' in.$$
```

```
$foo$A string with $$ quotes in $foo$
```

- Double quotes are used for names of database objects which either clash with keywords, contain mixed case letters, or contain characters other than a-z, 0-9 or underscore

- Example:

```
SELECT * FROM "select"
```

```
CREATE TABLE "HelloWorld" ...
```

```
SELECT * FROM "Hi everyone and everything"
```



Indexes

- Indexes are a common way to enhance performance
- Postgres supports several index types:

B-tree
(default)

Hash

Block Range
Index (BRIN)

GIN

GIST

SP-GiST
Indexes

Index on
Expressions



Example Index

- Syntax:

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ [ IF NOT EXISTS ] name ] ON [ ONLY ] table_name [ USING method ]
value [ ( { column_name | ( expression ) | constant } [ COLLATE collation ] [ opclass [ ( opclass_parameter =
... ) ] ] ] [ ASC | DESC ] [ NULLS { FIRST | LAST } ] [ ... ] )
[ INCLUDE ( column_name [ , ... ] ) ]
[ NULLS [ NOT ] DISTINCT ]
[ WITH ( storage_parameter [= value] [ , ... ] ) ]
[ LOGGING | NOLOGGING ]
[ LOCAL ]
[ TABLESPACE tablespace_name ]
[ PARALLEL [ integer | ( degree { integer | DEFAULT } ) ] | NOPARALLEL ]
[ WHERE predicate ]
```

- Example:

```
edbstore=> CREATE INDEX idx_ename ON emp(ename);
CREATE INDEX
edbstore=> █
```



Oracle Compatibility and Migration



Oracle Compatibility

- Oracle compatibility helps an application running in an Oracle environment to run in an Enterprise Postgres environment with minimal or no changes
- Oracle Compatibility in Enterprise Postgres offers:

Oracle Compatible
Tools

Data types

SQL statements

Oracle Compatible
Catalog Views

Stored Procedure
Language (SPL)

Built-in Packages

Triggers

Table Partitioning

Optimizer Hints

Open Client Library
(OCL) for Oracle
Call Interface (OCI)



Oracle Compatible Tools

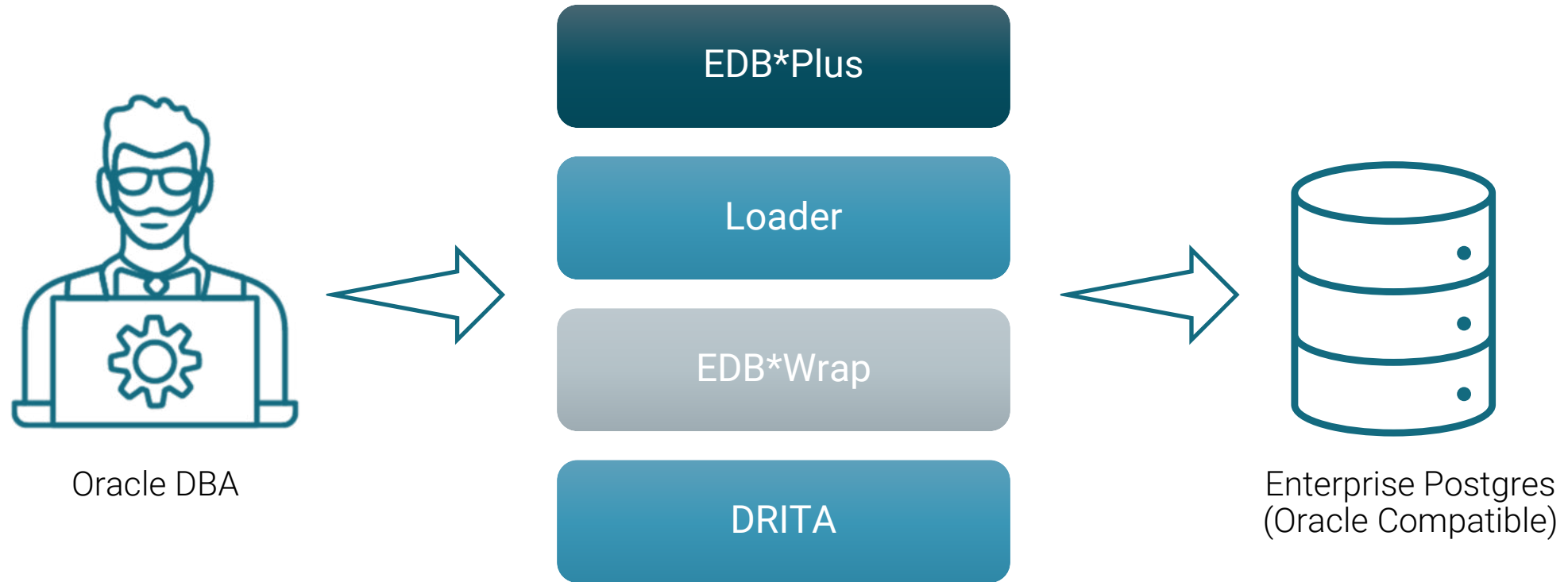


Table Partitioning

- Table partitioning can be used to break one logically large table into smaller physical pieces
- Enterprise Postgres supports Oracle compatible syntax for table partitioning with support for:
 - List Partitioning
 - Range Partitioning
 - Hash Partitioning
 - Sub Partitioning
 - Interval Partitioning



Stored Procedure Language

- SPL is the procedural language extension to SQL
- SPL is compatible with Oracle PL/SQL
- Can be used to create functions, procedures and packages
- Can be used to create sub-programs like sub-procedures and sub-functions inside standalone SPL Programs
- Provides procedural constructs such as:
 - Variables, constants, and types
 - Conditional statements
 - Dynamic SQL
 - Cursors
 - Triggers



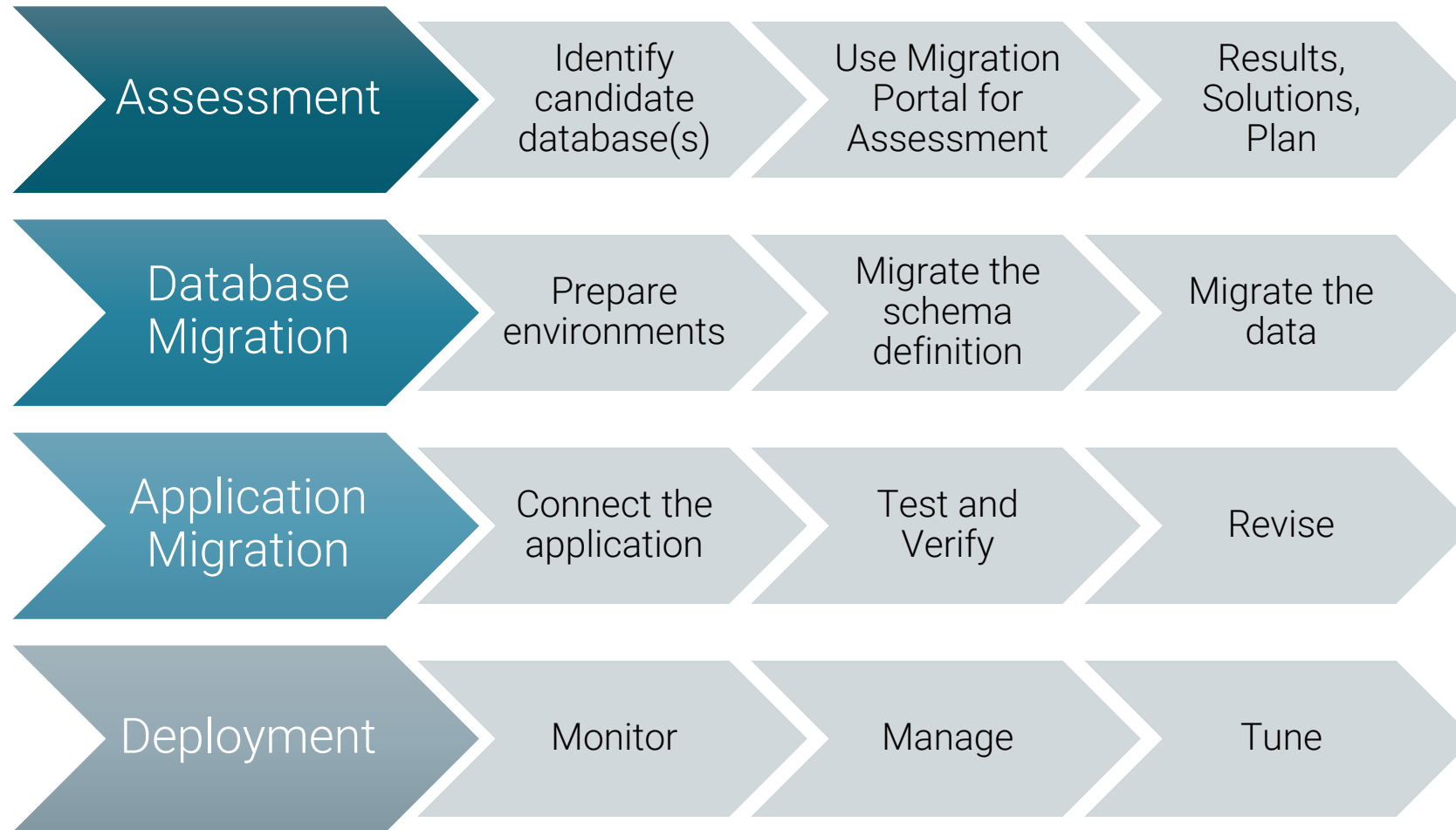
Built-in Packages

- Enterprise Postgres provides built-in packages compatible with Oracle
- Over 24 most commonly used Oracle built-in packages are available in Enterprise Postgres
- These built-in packages provide administration and maintenance utilities

DBMS_ALERT	DBMS_AQ	DBMS_AQADM	DBMS_CRYPTO	DBMS_JOB
DBMS_LOB	DBMS_LOCK	DBMS_MVIEW	DBMS_OUTPUT	DBMS_PIPE
DBMS_PROFILER	DBMS_RANDOM	DBMS_RLS	DBMS_SESSION	DBMS_SCHEDULER
DBMS_SQL	DBMS_UTILITY	DBMS_REDACT	UTL_ENCODE	UTL_FILE
UTL_HTTP	UTL_MAIL	UTL_SMTP	UTL_URL	UTL_RAW
DBMS_ASSERT	DBMS_PRIVILEGE_CAPTURE	DBMS_XLMDOM	HTP and HTF	



Migration Process



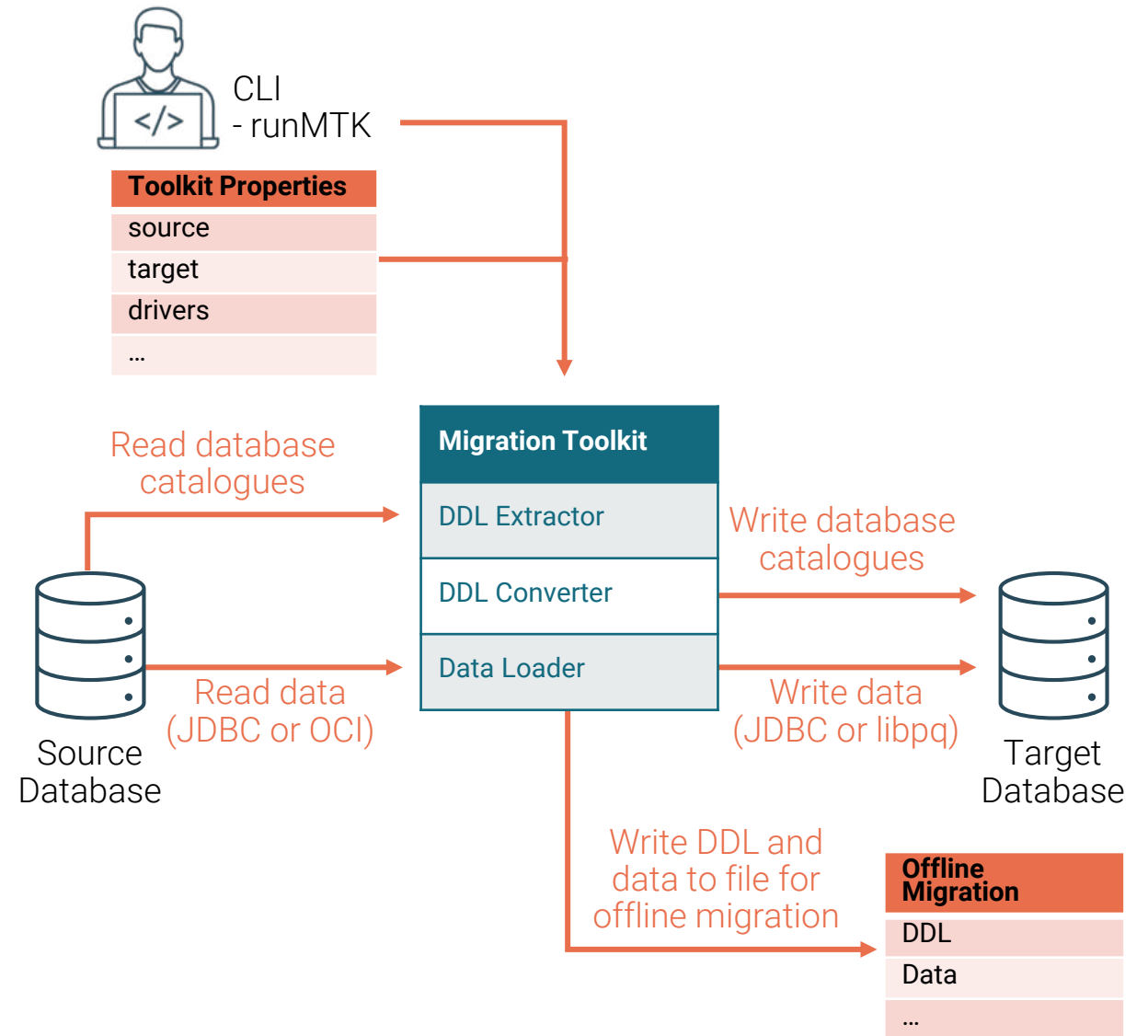
Migration Portal

- Migration Portal Combines:
 - Native Oracle Compatibility
 - Schemas
 - DB Code
 - Application interfaces
- Rich knowledge base from 10+ years of migrations
- Cloud-based machine learning of new code translations
- Supports assessment and migration from Oracle 11g to 19c to EDB Advanced Server 13 to 17



Database Migration Toolkit

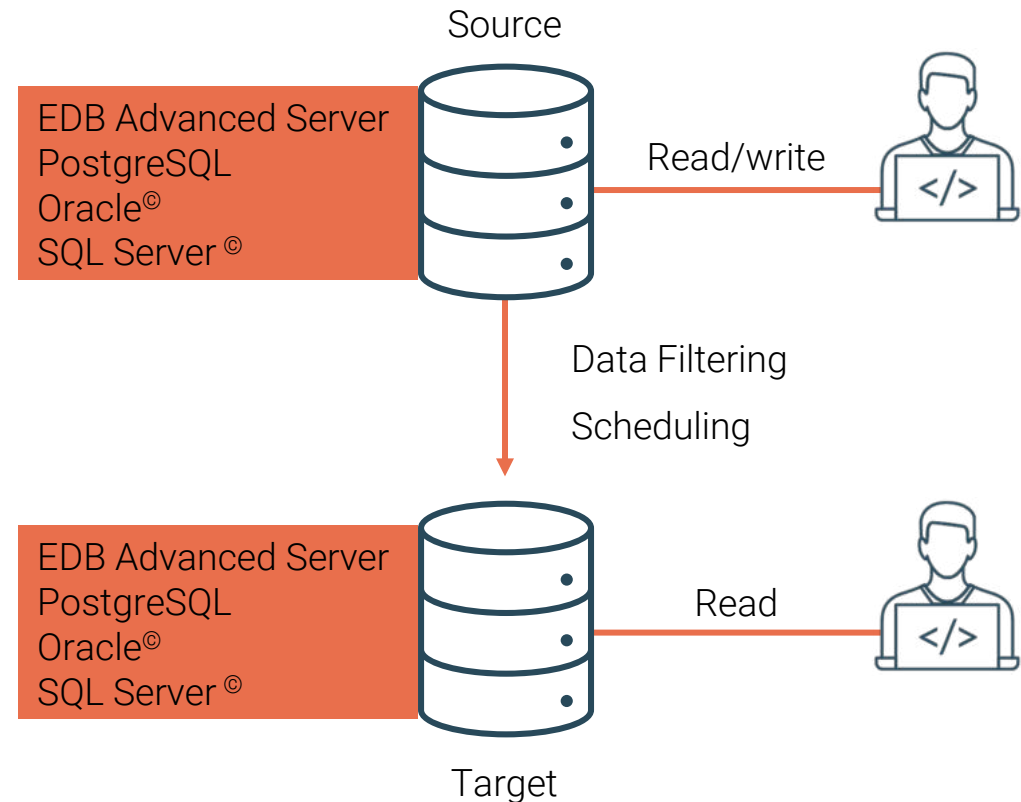
- MTK is a powerful command line tool
- It simplifies the process of migrating from other databases to Enterprise Postgres
- The Migration Toolkit can be used to migrate the entire schema including triggers and stored procedures



Database Migration using Replication Server

- Replicate Oracle or SQL Server data to Enterprise Postgres
- Distributed multi-Publication/Subscription Architecture
- Synchronize data across geographies
- Replicate selected tables and sequences

Single Master Replication (SMR) between Oracle and Enterprise Postgres for Migration



Module Summary

- Data Types
- Structured Query Language (SQL)
- DDL, DML and DCL Statements
- Transaction Control Statements
- Tables and Constraints
- Views and Materialized Views
- Sequences
- Domains
- SQL Joins and Functions
- Explain Plans
- Quoting in PostgreSQL
- Indexes
- Oracle Compatibility and Tools



Lab Exercise - 1

- Test your knowledge:

1. Initiate a psql session

2. edb-psql commands access the database	True/False
--	------------

3. The following SELECT statement executes successfully:	True/False
--	------------

=> `SELECT ename, job, sal AS Salary FROM emp;`

4. The following SELECT statement executes successfully:	True/False
--	------------

=> `SELECT * FROM emp;`

5. There are coding errors in the following statement. Can you identify them?

=> `SELECT empno, ename, sal * 12 annual salary FROM emp;`



Lab Exercise - 2

- The staff in the HR department wants to hide some of the data in the `EMP` table. They want a view called `EMPVU` based on the employee numbers, employee names, and department numbers from the `EMP` table. They want the heading for the employee name to be `EMPLOYEE`.
- Confirm that the view works. Display the contents of the `EMPVU` view.
- Using your `EMPVU` view, write a query for the `SALES` department to display all employee names and department numbers.



Lab Exercise - 3

- You need a sequence that can be used with the primary key column of the `dept` table. The sequence should start at 60 and have a maximum value of 200. Have your sequence increment by 10. Name the sequence `dept_id_seq`.
- To test your sequence, write a script to insert two rows in the `dept` table.





Backup, Recovery and PITR



Module Objectives

- Backup Types
- Database SQL Dumps
- Restoring SQL Dumps
- Offline Physical Backups
- Continuous Archiving
- Online Physical Backups Using pg_basebackup
- Point-in-time Recovery
- Recovery Settings
- Backup Tools – Barman and pgBackRest



Types of Backup

- As with any database, PostgreSQL databases should be backed up regularly

Logical Backups

- Database SQL Dumps using `pg_dump`
- Database Cluster SQL Dump using `pg_dumpall`

Physical Backups

- Offline File System Level Backups using OS commands
- Online File System Level Backups using `pg_basebackup`
- Backup Tool – Barman and `pgBackRest`



Database SQL Dump

- Generate a text file with SQL commands
- Enterprise Postgres provides the utility program `pg_dump` for this purpose
- `pg_dump` does not block readers or writers
- `pg_dump` does not operate with special permissions
- Dumps created by `pg_dump` are internally consistent, that is, the dump represents a snapshot of the database as of the time `pg_dump` begins running
- Syntax:

```
$ pg_dump [options] [dbname]
```



pg_dump Options

-a	- Data only. Do not dump the data definitions (schema)
-s	- Data definitions (schema) only. Do not dump the data
-n <schema>	- Dump from the specified schema only
-t <table>	- Dump specified table only
-f <file name>	- Send dump to specified file. Filename can be specified using absolute or relative location
-Fp	- Dump in plain-text SQL script (default)
-Ft	- Dump in tar format
-Fc	- Dump in compressed, custom format
-Fd	- Dump in directory format
-j njobs	- dump in parallel by dumping n jobs tables simultaneously. Only supported with -Fd
-B, --no-blobs	- Excludes large objects in dump
-v	- Verbose option



SQL Dump - Large Databases

- If the operating system has maximum file size limits, it can cause problems when creating large `pg_dump` output files
- Standard Unix tools can be used to work around this potential problem
 - Use a compression program, for example gzip:

```
$ pg_dump dbname | gzip > filename.gz
```

- The `split` command allows you to split the output into smaller files:

```
$ pg_dump dbname | split -b 1m - filename
```



Restore – SQL Dump

psql client

- Backups taken using pg_dump with plain text format(Fp)
- Backups taken using pg_dumpall

pg_restore utility

- Backup taken using pg_dump with custom(Fc), tar(Ft) or director(Fd) formats
- Supports parallel jobs for during restore
- Selected objects can be restored



pg_restore Options

-l	- Display TOC of the archive file
-F [c d t]	- Backup file format
-d <database name>	- Connect to the specified database. Also restores to this database if -C option is omitted
-C	- Create the database named in the dump file and restore directly into it
-a	- Restore the data only, not the data definitions (schema)
-s	- Restore the data definitions (schema) only, not the data
-n <schema>	- Restore only objects from specified schema
-N <schema>	- do not restore objects in this schema
-t <table>	- Restore only specified table
-v	- Verbose option



Entire Cluster - SQL Dump

- `pg_dumpall` is used to dump an entire database cluster in plain-text SQL format
- Dumps global objects - users, groups, and associated permissions
- Use `psql` to restore
- Syntax:

```
$ pg_dumpall [options...] > filename.backup
```



pg_dumpall Options

-a	- Data only. Do not dump schema
-s	- Data definitions (schema) only
-g	- Dump global objects only - not databases
-r	- Dump only roles
-c	- Clean (drop) databases before recreating
-O	- Skip restoration of object ownership
-x	- do not dump privileges (grant/revoke)
-v	- Verbose option
--disable-triggers	- disable triggers during data-only restore
--no-role-passwords	- do not dump passwords for roles. This allows use of pg_dumpall by non-superusers
--exclude-database	-exclude database whose name match with given pattern



Physical Backup - File system level backup

- An alternative backup strategy is to directly copy the files that Postgres uses to store the data in the database
- You can use whatever method you prefer for doing usual file system backups, for example:

```
$ tar -cf backup.tar /usr/local/edb/data
```

- The database server must be shut down or in backup mode in order to get a usable backup
- File system backups only work for complete backup and restoration of an entire database cluster
- Two types of File system backup
 - Offline backups
 - Online backups



File System Backups

Offline Backups

- Taken using OS Copy command
- Database Server must be shutdown
- Cluster Level Backup and Restore

Online Backups

- Continuous archiving must be enabled
- Database server start/end backup mode
- Cluster Level Backup and Restore with PITR
- Methods - pg_basebackup, Barman, pgBackRest



Continuous Archiving

- Postgres maintains WAL files for all transactions in [pg_wal](#) directory
- Postgres automatically maintains the WAL logs which are full and switched
- Continuous archiving can be setup to keep a copy of switched WAL Logs which can be later used for recovery
- It also enables online file system backup of a database cluster
- Requirements:
 - `wal_level` must be set to `replica`
 - `archive_mode` must be set to `on` (can be set to `always`)
 - `archive_command` must be set in [postgresql.conf](#) which archives WAL logs and supports PITR



Continuous Archiving Methods

Archiver Process

- Parameters in postgresql.conf file
 - wal_level = replica
 - archive_mode = on
 - archive_command = 'cp -i %p /edb/archive/%f'
- Restart the database server
- Archive files are generated after every log switch

Streaming WAL

- Parameters in postgresql.conf file
 - wal_level = replica
 - archive_mode = on
 - max_wal_senders = 3
- Restart the database server
- pg_receivewal -h localhost -D /edb/archive
- Transactions are streamed and written to archive files



Base Backup Using pg_basebackup Tool

- `pg_basebackup` can take an online base backup of a database cluster
- This backup can be used for PITR or Streaming Replication
- `pg_basebackup` makes a binary copy of the database cluster files
- System is automatically put in and out of backup mode



pg_basebackup - Online Backup

- Steps require to take Base Backup:

- Modify [pg_hba.conf](#)

```
host replication enterprisedb [Ipv4 address of client]/32 md5
```

- Modify [postgresql.conf](#)

```
wal_level = replica
```

```
archive_command = 'cp -i %p /users/enterprisedb/archive/%f'
```

```
archive_mode = on
```

```
max_wal_senders = 3
```

```
wal_keep_size = 512
```

- Backup Command:

```
$ pg_basebackup [options] ..
```



Options for pg_basebackup command

-D <directory name>	- Location of backup
-F <p or t>	- Backup files format. Plain(p) or tar(t)
-i, --incremental=OLDMANIFEST	-Take an incremental backup
-R	- write standby.signal and append postgresql.auto.conf
-T OLDDIR=NEWDIR	- relocate tablespace in OLDDIR to NEWDIR
--waldir	- Write ahead logs location
-z	- Enable compression(tar) for files
-Z	- Compress backup based on setting set to none, client or server
-P	- Progress Reporting
-h host	- host on which cluster is running
-p port	- cluster port

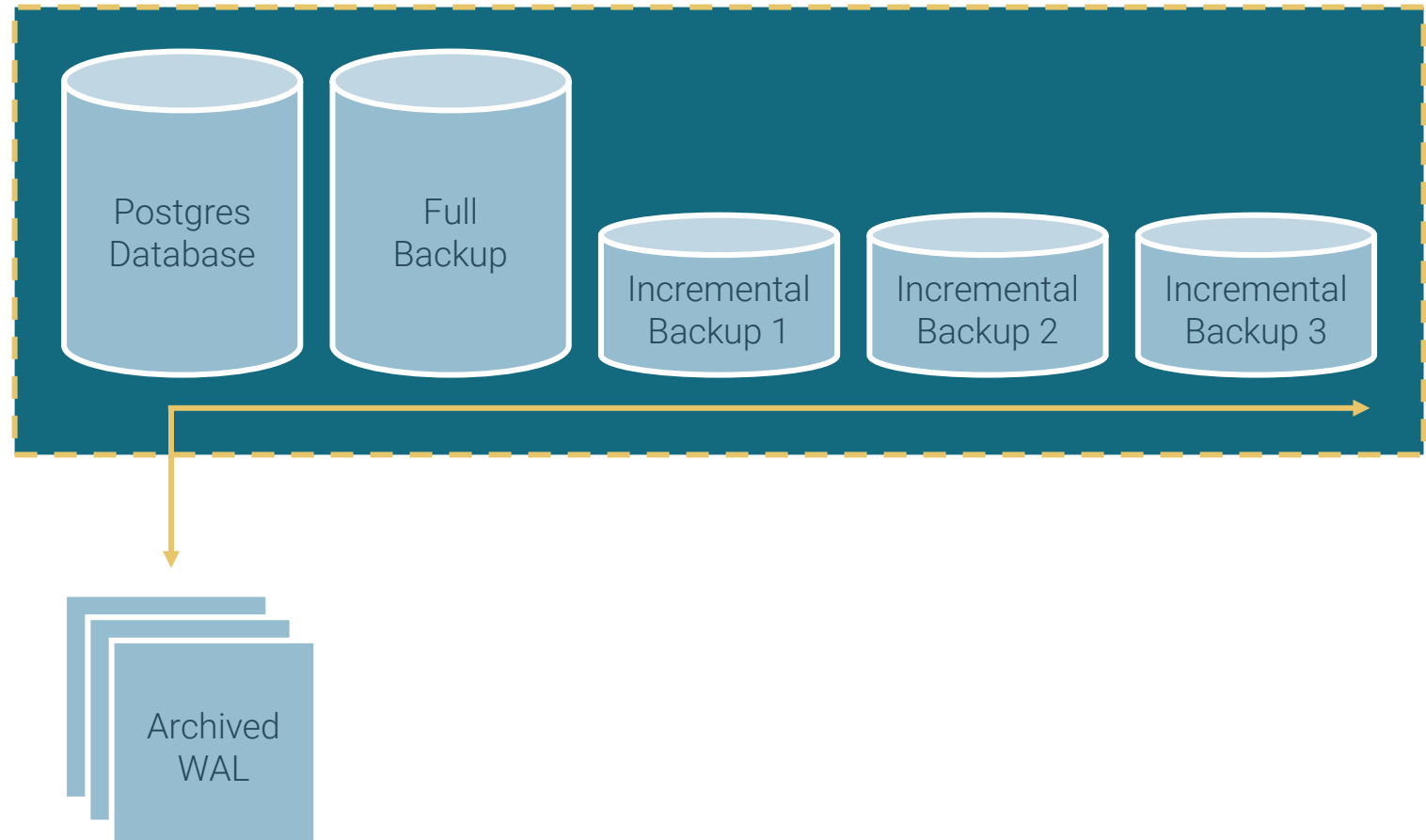
- To create a full backup of the server at localhost and store it in the local directory /usr/local/edb/backup

```
$ pg_basebackup -h localhost -D /usr/local/edb/backup
```



Incremental Backups

- pg_basebackup utility now supports incremental backups
- Track changes made since the last backup using WAL summaries
- Minimizes the time and storage needed for backups
- Reduced system load allowing frequent backups with minimal performance impact.



WAL Summary Files

- Write-ahead logs (WAL) capture all modifications made to database blocks.
- The WAL summarizer process:
 - Reads WAL as it is generated.
 - Produces concise summary files with necessary information for incremental backups.
- WAL Summary files:
 - Contain block numbers of changed data only.
 - Extremely small and inexpensive to create.
- Use the `pg_walsummary` tool to extract and analyze information from WAL summary files.



Backup Manifest File

- Purpose of the Backup Manifest:
 - Indicates the write-ahead log position (LSN) of the previous backup.
 - Identifies the required WAL summary files.
- Backup Process:
 - Read all files starting from the LSN of the previous backup up to the current time.
 - Determine what has changed based on the WAL summary files.



Incremental Backup Setup

- `pg_basebackup --incremental`
- Pre-requisites:
 - `summarize_wal` parameter must be set to ON
 - A full base backup taken using `pg_basebackup`
- Backup Steps:

- Verify WAL setting

```
psql -c "show summarize_wal;"
```

- Take a full base backup using `pg_basebackup`

```
pg_basebackup -D /home/  
enterprisedb/data_full_day1
```

```
edb=# show summarize_wal ;  
summarize_wal  
-----  
on  
  
[enterprisedb@ip-172-30-21-21 ~]$ ls  
/var/lib/edb/as17/data/pg_wal/summaries/  
00000001000000000100002800000000010B1D38.summary  
0000000100000000014E61A000000000014E62A0.summary
```

```
[enterprisedb@ip-172-30-21-21 ~]$ pg_basebackup -D  
/home/enterprisedb/data_full_day1 -p 5444 -U enterprisedb -h  
localhost  
[enterprisedb@ip-172-31-20-21 ~]$ cat  
data_full_day1/backup_manifest  
{ "PostgreSQL-Backup-Manifest-Version": 2,  
  "System-Identifier": 7462123271923429087,  
  "Files": [  
    { "Path": "backup_label", "Size": 225, "Last-Modified": "2025-01-  
20 22:13:24 GMT", "Checksum-Algorithm": "CRC32C", "Checksum":  
"00db1195" },  
    ....  
  ],  
  "WAL-Ranges": [  
    { "Timeline": 1, "Start-LSN": "0/3000028", "End-LSN": "0/3000120"  
    }  
  ],  
  "Manifest-Checksum":  
  "2fb4a73765484ab570ac90d0946553559623b12202fb62956f16036d4510aebb"  
}
```



Taking an Incremental Backup

- `pg_basebackup --incremental`
- Take an incremental backup using `pg_basebackup` with `--incremental` option

```
pg_basebackup -D /home/enterprisedb/data_inc_day2  
--incremental = /home/ enterprisedb/data_full_day1 /backup_manifest
```

```
[enterprisedb@ip-172-30-21-21 ~]$ pg_basebackup -D  
/home/enterprisedb/data_inc_day2 --  
incremental=/home/enterprisedb/data_full_day1/backup_manifest -p 5444 -U  
enterprisedb -h localhost -v  
pg_basebackup: initiating base backup, waiting for checkpoint to complete  
pg_basebackup: checkpoint completed  
pg_basebackup: write-ahead log start point: 0/7000028 on timeline 1  
pg_basebackup: starting background WAL receiver  
pg_basebackup: created temporary replication slot "pg_basebackup_16295"  
pg_basebackup: write-ahead log end point: 0/7000120  
pg_basebackup: waiting for background process to finish streaming ...  
pg_basebackup: syncing data to disk ...  
pg_basebackup: renaming backup_manifest.tmp to backup_manifest  
pg_basebackup: base backup completed
```



Verify Base Backups

- Verify backup taken by `pg_basebackup` using `pg_verifybackup` utility
- Backup is verified against a `backup_manifest` generated by the server at the time of the backup
- Only plain format backups can be verified

```
[postgres@pgsrv1 ~]$ pg_verifybackup --help
pg_verifybackup verifies a backup against the backup manifest.

Usage:
  pg_verifybackup [OPTION]... BACKUPDIR

Options:
  -e, --exit-on-error          exit immediately on error
  -i, --ignore=RELATIVE_PATH  ignore indicated path
  -m, --manifest-path=PATH    use specified path for manifest
  -n, --no-parse-wal          do not try to parse WAL files
  -q, --quiet                  do not print any output, except for errors
  -s, --skip-checksums        skip checksum verification
  -w, --wal-directory=PATH    use specified path for WAL files
  -V, --version                output version information, then exit
  -?, --help                  show this help, then exit
```



Point-in-time Recovery

- Point-in-time recovery (PITR) is the ability to restore a database cluster up to the present or to a specified point of time in the past
- Uses a full database cluster backup and the write-ahead logs found in the `/pg_wal` subdirectory
- Must be configured before it is needed (write-ahead log archiving must be enabled)



Performing Point-in-Time Recovery



Prepare

Stop the server

Take a file system level backup if possible

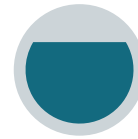
Clean the data directory



Restore

Copy data cluster files and folders from backup location to the data directory

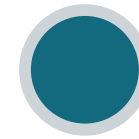
Use `cp -rp` to preserve privileges



Configure

Configure recovery settings in [postgresql.conf](#) file

Create [recovery.signal](#) file in the data directory



Recover

Start the server using service or `pg_ctl` utility

Check error log for any issue

[recovery.signal](#) file is removed automatically after recovery



Point-in-Time Recovery Settings

- Restoring archived WAL using restore_command parameter:

- Unix:

```
restore_command = 'cp /mnt/server/archivedir/%f "%p"'
```

- Windows:

```
restore_command = 'copy c:\\mnt\\server\\archivedir\\"%f" "%p"'
```

- Recovery target settings:

```
recovery_target_name
```

```
recovery_target_time
```

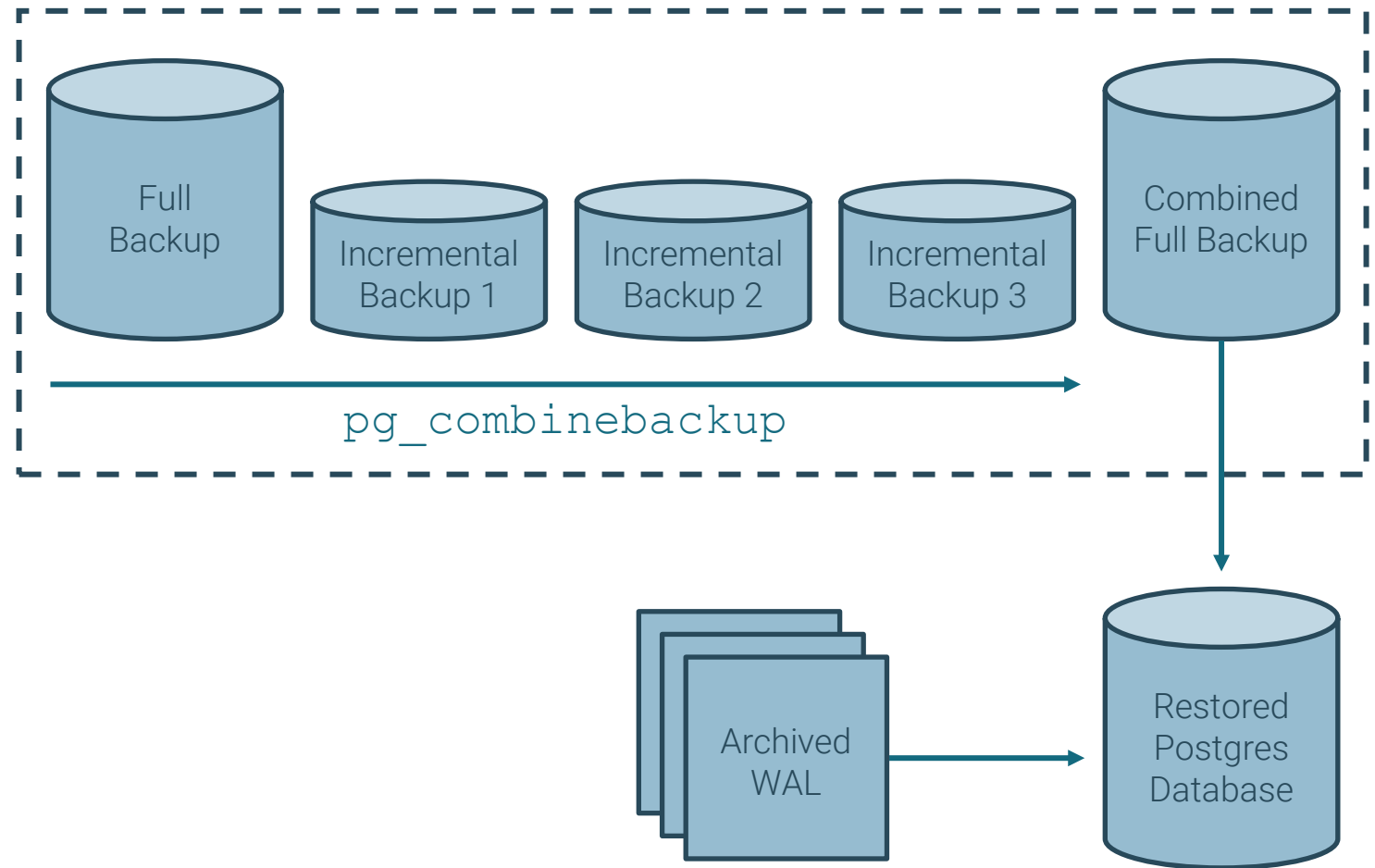
```
recovery_target_xid
```

```
recovery_target_action
```



Restoring From Incremental Backups

- Full backup, and all the incremental backups must be combined into a synthetic full backup during restore
- This can be achieved using `pg_combinebackup` utility
- Backups integrity can be checked using `pg_verifybackup` prior to the reconstruction
- Backups can be combined on same or separate server



Restoring Incremental Backup

- Check available Full and Incremental Backups

```
enterprisedb  
data_full_day1  data_inc_day2  data_inc_day3  data_inc_day4
```

- Combine a full backup with incremental backups ensuring a legal backup chain

```
[enterprisedb@ip-172-30-21-21 ~]$ pg_combinebackup --  
output=/home/enterprisedb/data_day1_4 /home/enterprisedb/data_full_day1  
/home/enterprisedb/data_inc_day2 /home/enterprisedb/data_inc_day3  
/home/enterprisedb/data_inc_day4
```

- Restore combined backup to the data directory and follow normal archive recovery or startup steps.

```
[enterprisedb@ip-172-30-21-21 ~]$ cp -rp /home/enterprisedb/data_day1_4/*  
/var/lib/edb/as17/data/
```



Overview: Backup and Recovery Tools



Backup And Recovery Manager (Barman)

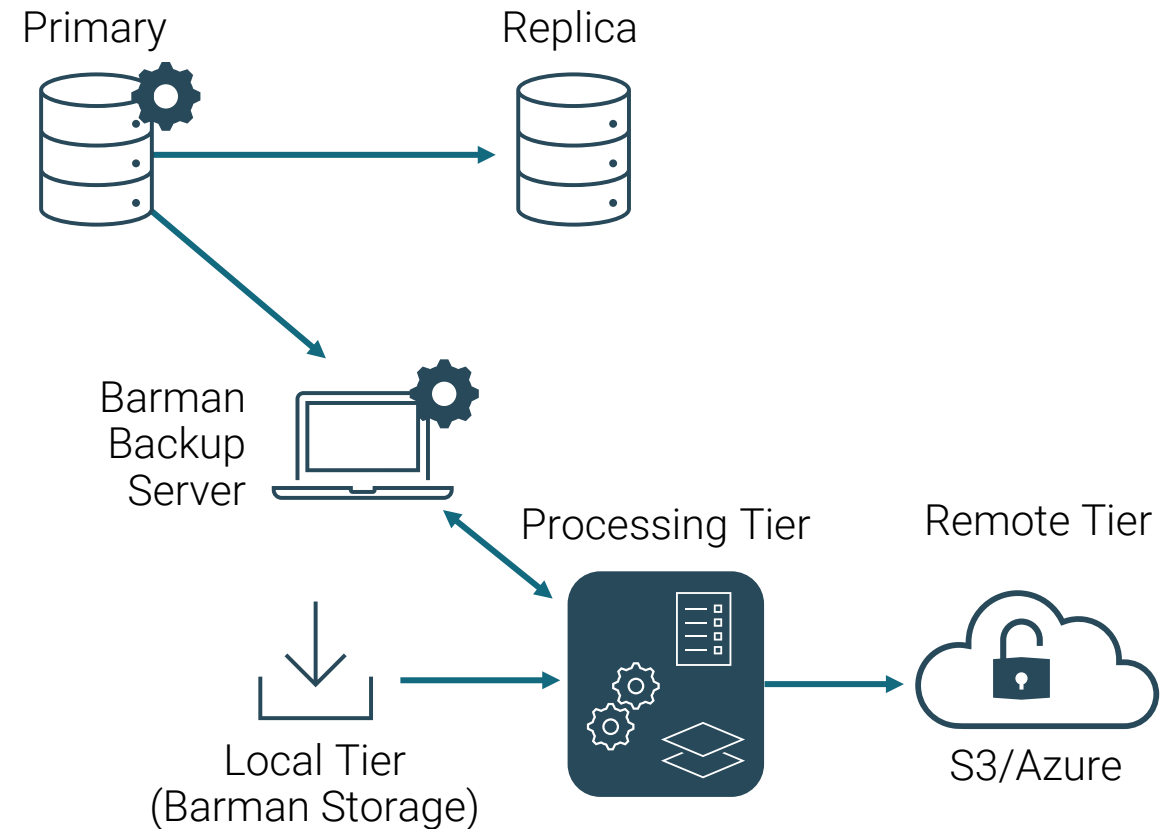
- Open-source administration tool for remote backups and disaster recovery
- Manage backups and the recovery phase of multiple servers from one location
- Distributed under GNU GPL 3 and maintained by EDB



Barman Architecture

<http://docs.pgbarman.org/>

- One Barman for multiple Postgres servers
- Standard connection to Postgres for management, coordination and monitoring
- Standard replication connection for running `pg_basebackup` and `pg_receivewal`
- Supports `rsync`/`SSH`



Barman - Features

<https://www.pgbarman.org/about/>

- Remote backup and restore with rsync and the PostgreSQL protocol
- Support for file level incremental backups with rsync
- Retention policy support
- WAL Archive Compression with gzip, bzip2, or pigz
- Backup data verification
- Backup with RPO=0 using a synchronous physical streaming replication connection
- Rate limiting



Postgres Backup And Restore

pgBackRest



Solves common bottleneck problems with parallel processing for backup, compression, restoring and archiving



Support capabilities like symmetric encryption, and partial restore



Fully supported Open Source troubleshooting support



Feature Comparison

Capability	Added value	Barman	pgBackRest	pg_basebackup
SSH protocol support		Yes	Yes	-
PostgreSQL protocol	Works without passwordless ssh.	Yes	-	Yes
Incremental backups		Yes	Yes	Yes
RPO=0	Restore up to the last commit	Yes	-	-
Rate limiting	Preserve IO for Postgres	Yes	-	Yes
Retention and List backups		Yes	Yes	-
Backup compression	Less backup space required	-	Yes	Yes
Symmetric encryption	Lower security footprint for the backup data	-	Yes	-
Partial restore (only selected databases)	Restore required data for analysis purposes	-	Yes	-
S3 and Azure Blob Support	Use flexible Cloud Storage for backup storage	Yes	Yes	-
Nagios integration	Monitor your backups with Nagios	Yes	Yes	-



Module Summary

- Backup Types
- Database SQL Dumps
- Restoring SQL Dumps
- Offline Physical Backups
- Continuous Archiving
- Online Physical Backups Using pg_basebackup
- Point-in-time Recovery
- Recovery Settings
- Backup Tools – Barman and pgBackRest



Lab Exercise - 1

1. The edbstore website database is all setup and as a DBA you need to plan a proper backup strategy and implement it
 - As the root user, create a folder /pgbackup and assign ownership to the Postgres user using the chown utility or the Windows security tab in folder properties
 - Take a full database dump of the edbstore database with the pg_dump utility. The dump should be in plain text format
 - Name the dump file as edbstore_full.sql and store it in the /pgbackup directory



Lab Exercise - 2

1. Perform a dump of the `edbuser` schema from the `edbstore` database and name the file as `edbstore_schema.sql`
2. Perform a data-only dump of the `edbstore` database, disable all triggers for a faster restore, use the `INSERT` command instead of `COPY`, and name the file as `edbstore_data.sql`
3. Perform a full dump of `customers` table and name the file as `edbstore_customers.sql`



Lab Exercise - 3

1. Perform a full database dump of `edbstore` in compressed format using the `pg_dump` utility, name the file as `edbstore_full_fc.dmp`
2. Perform a full database cluster dump using `pg_dumpall`. Remember `pg_dumpall` supports only plain text format; name the file `edbdata.sql`



Lab Exercise - 4

In this exercise you will demonstrate your ability to restore a database.

1. Drop database `edbstore`.
2. Create database `edbstore` with owner `edbuser`.
3. Restore the full dump from [edbstore_full.sql](#) and verify all the objects and their ownership.
4. Drop database `edbstore`.
5. Create database `edbstore` with `edbuser` owner.
6. Restore the full dump from the compressed file [edbstore_full_fc.dmp](#) and verify all the objects and their ownership.



Lab Exercise - 5

1. Create a directory `/opt/arch` or `c:\arch` and give ownership to the Postgres user.
2. Configure your cluster to run in archive mode with the ability to take incremental backups and set the archive log location to be `/opt/arch` or `c:\arch`.
3. Take a full online base backup of your cluster in the `/pgbackup` directory using the [pg_basebackup](#) utility.
4. Connect to the edbstore database and update a table. Then take an incremental backup. Then combine them for a potential restore





Routine Maintenance Tasks



Module Objectives

- Updating Optimizer Statistics
- Handling Data Fragmentation using Routine Vacuuming
- Preventing Transaction ID Wraparound Failures
- Automatic Maintenance using Autovacuum
- Re-indexing in Postgres



Database Maintenance

- Data files become fragmented as data is modified and deleted
- Database maintenance helps reconstruct the data files
- If done on time nobody notices but when not done everyone knows
- Must be done before you need it
- Improves performance of the database
- Saves database from transaction ID wraparound failures



Maintenance Tools

- Maintenance thresholds can be configured using the PEM Client
- Postgres maintenance thresholds can be configured in [postgresql.conf](#)
- Manual scripts can be written watch stat tables like `pg_stat_user_tables`
- Maintenance commands:
 - `ANALYZE`
 - `VACUUM`
 - `CLUSTER`
- Maintenance command `vacuumdb` can be run from OS prompt
- Autovacuum can help in automatic database maintenance



Optimizer Statistics

- Optimizer statistics play a vital role in query planning
- Not updated in real time
- Collects information for relations including size, row counts, average row size and row sampling
- Stored permanently in catalog tables
- The maintenance command `ANALYZE` updates the statistics
- Thresholds can be set using PEM Client to alert you when statistics are not collected on time



Example - Updating Statistics

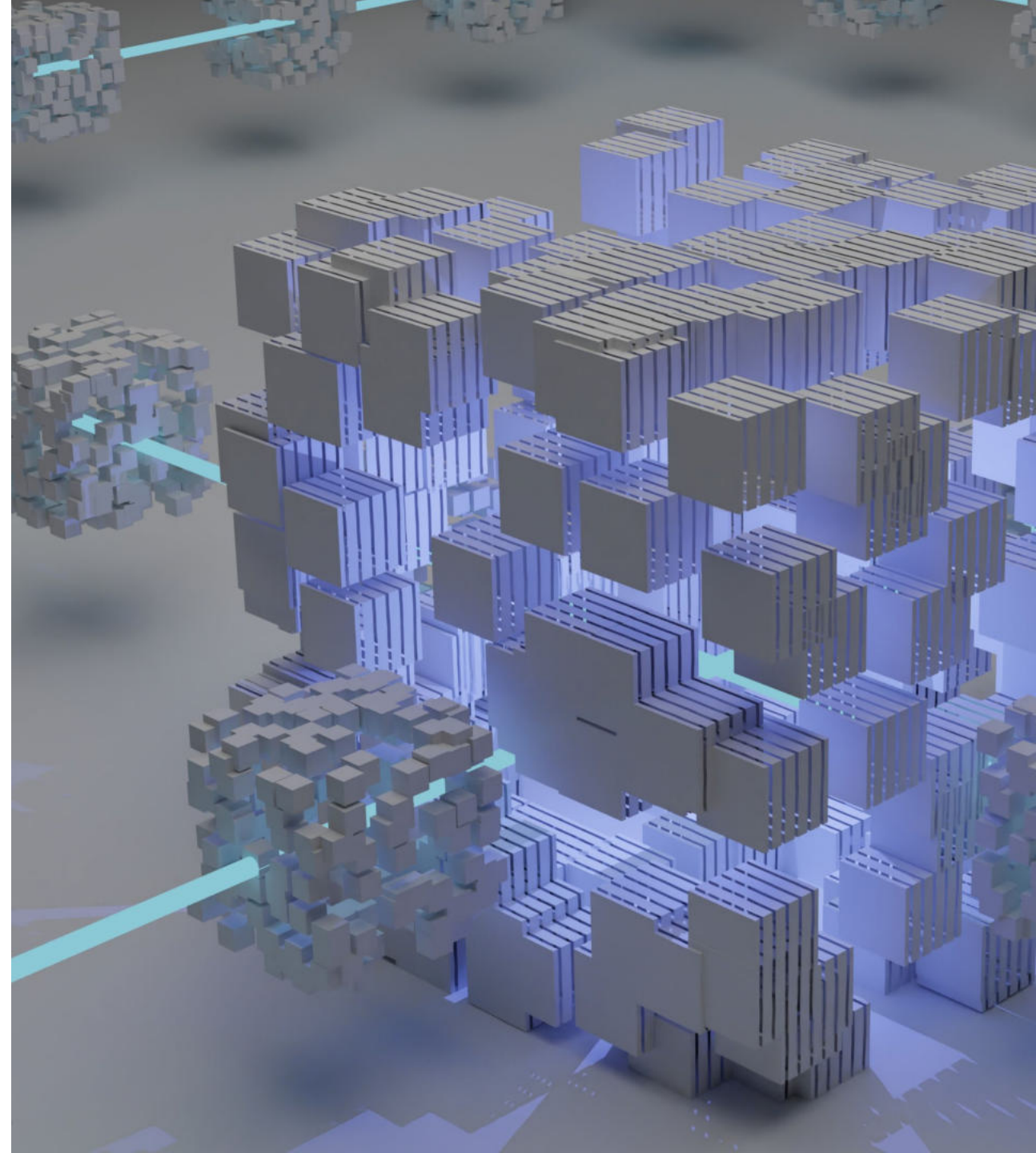
```
postgres=# CREATE TABLE testanalyze(id integer, name varchar);
CREATE TABLE
postgres=# INSERT INTO testanalyze VALUES(generate_series(1,10000), 'Sample');
INSERT 0 10000
postgres=# SELECT relname, reltuples FROM pg_class WHERE relname='testanalyze';
   relname   | reltuples 
-----+-----
testanalyze |         0
(1 row)

postgres=# ANALYZE testanalyze;
ANALYZE
postgres=# SELECT relname, reltuples FROM pg_class WHERE relname='testanalyze';
   relname   | reltuples 
-----+-----
testanalyze |       10000
(1 row)
```



Data Fragmentation and Bloat

- Data is stored in data file pages
- An update or delete of a row does not immediately remove the row from the disk page
- Eventually this row space becomes obsolete and causes fragmentation and bloating
- Set PEM Alert for notifications



Routine Vacuuming

- Obsoleted rows can be removed or reused using vacuuming
- Helps in shrinking data file size when required
- Vacuuming can be automated using autovacuum
- The `VACUUM` command locks tables in access exclusive mode
- Long running transactions may block vacuuming, thus it should be done during low usage times



Vacuuming Commands

- When executed, the `VACUUM` command:
 - Can recover or reuse disk space occupied by obsolete rows
 - Updates data statistics
 - Updates the visibility map, which speeds up index-only scans
 - Protects against loss of very old data due to transaction ID wraparound
- The `VACUUM` command can be run in two modes:
 - `VACUUM`
 - `VACUUM FULL`



Vacuum and Vacuum Full

- VACUUM
 - Removes dead rows and marks the space available for future reuse
 - Does not return the space to the operating system
 - Space is reclaimed if obsolete rows are at the end of a table
- VACUUM FULL
 - More aggressive algorithm compared to VACUUM
 - Compacts tables by writing a completely new version of the table file with no dead space
 - Takes more time
 - Requires extra disk space for the new copy of the table, until the operation completes



VACUUM Syntax

VACUUM [(option [, ...])] [table_and_columns [, ...]] where option can be one of: FULL [boolean]

FREEZE [boolean]

VERBOSE [boolean]

ANALYZE [boolean]

DISABLE_PAGE_SKIPPING [boolean]

SKIP_LOCKED [boolean]

INDEX_CLEANUP { AUTO | ON | OFF }

PROCESS_MAIN [boolean]

PROCESS_TOAST [boolean]

TRUNCATE [boolean]

PARALLEL integer

SKIP_DATABASE_STATS [boolean]

ONLY_DATABASE_STATS [boolean]

BUFFER_USAGE_LIMIT size and table_and_columns is: table_name [(column_name [, ...])]



Example - Vacuuming

```
edb=# CREATE TABLE testvac (id numeric, name varchar2);
CREATE TABLE
edb=# INSERT INTO testvac VALUES(generate_series(1,10000),'Sample');
INSERT 0 10000
edb=# SELECT pg_size_pretty(pg_relation_size('testvac'));
 pg_size_pretty
-----
440 kB
(1 row)

edb=# UPDATE testvac SET name='Sample';
UPDATE 10000
edb=# SELECT pg_size_pretty(pg_relation_size('testvac'));
 pg_size_pretty
-----
872 kB
(1 row)

edb=# █
```



Example – Vacuuming (continued)

```
edb=# VACUUM testvac;  
VACUUM  
edb=# UPDATE testvac SET name='Sample';  
UPDATE 10000  
edb=# SELECT pg_size_pretty(pg_relation_size('testvac'));  
pg_size_pretty  
-----  
872 kB  
(1 row)  
  
edb=# VACUUM FULL testvac;  
VACUUM  
edb=# SELECT pg_size_pretty(pg_relation_size('testvac'));  
pg_size_pretty  
-----  
440 kB  
(1 row)  
  
edb=# █
```



Preventing Transaction ID Wraparound Failures

- MVCC depends on transaction ID numbers
- Transaction IDs have limited size (32 bits at this writing)
- A cluster that runs for a long time (more than 4 billion transactions) would suffer transaction ID wraparound
- This causes a catastrophic data loss
- To avoid this problem, every table in the database must be vacuumed at least once for every two billion transactions



Vacuum Freeze

- `VACUUM FREEZE` will mark rows as frozen
- Postgres reserves a special XID, `FrozenTransactionId`
- `FrozenTransactionId` is always considered older than every normal XID
- `VACUUM FREEZE` replaces transaction IDs with `FrozenTransactionId`, thus rows will appear to be “in the past”
- `vacuum_freeze_min_age` controls when a row will be frozen
- `VACUUM` normally skips pages without dead row versions, but some rows may need `FREEZE`
- `vacuum_freeze_table_age` controls when a whole table must be scanned



The Visibility Map

- Each heap relation has a Visibility Map which keeps track of which pages contain only tuples
- Stored at `<relfilenode>_vm`
- Helps vacuum to determine whether pages contain dead rows
- Can also be used by index-only scans to answer queries
- `VACUUM` command updates the visibility map
- The visibility map is vastly smaller, so can be cached easily



vacuumdb Utility

- The VACUUM command has a command-line executable wrapper called vacuumdb
- vacuumdb can VACUUM all databases using a single command
- Syntax:
 - vacuumdb [OPTION] ... [DBNAME]
- Available options can be listed using:
 - vacuumdb --help



Autovacuuming

- Highly recommended feature of Postgres
- It automates the execution of `VACUUM`, `FREEZE` and `ANALYZE` commands
- Autovacuum consists of a launcher and many worker processes
- A maximum of `autovacuum_max_workers` worker processes are allowed
- Launcher will start one worker within each database every `autovacuum_naptime` seconds
- Workers check for inserts, updates and deletes and execute `VACUUM` and/or `ANALYZE` as needed
- `track_counts` must be set to `TRUE` as autovacuum depends on statistics
- Temporary tables cannot be accessed by autovacuum



Autovacuuming Parameters

Autovacuum Launcher Process

- `autovacuum`

Autovacuum Worker Processes

- `autovacuum_max_workers`
- `autovacuum_naptime`

Vacuuming Thresholds

- `autovacuum_vacuum_scale_factor`
- `autovacuum_vacuum_threshold`
- `autovacuum_analyze_scale_factor`
- `autovacuum_analyze_threshold`
- `autovacuum_vacuum_insert_scale_threshold`
- `autovacuum_vacuum_insert_threshold`
- `autovacuum_freeze_max_age`



Per-Table Thresholds

- Autovacuum workers are resource intensive
- Table-by-table autovacuum parameters can be configured for large tables
- Configure the following parameters using ALTER TABLE or CREATE TABLE:
 - `autovacuum_enabled`
 - `autovacuum_vacuum_threshold`
 - `autovacuum_vacuum_scale_factor`
 - `autovacuum_analyze_threshold`
 - `autovacuum_analyze_scale_factor`
 - `autovacuum_vacuum_insert_scale_threshold`
 - `autovacuum_vacuum_insert_threshold`
 - `autovacuum_freeze_max_age`



Maintenance Commands Buffer Access Strategy

Buffer Usage Limit

- Fine-grained control over shared buffer size during vacuum and analyze operations
- Larger values optimize vacuum speed but may impact concurrent query performance
- Buffer usage can be controlled using various options:
 - Configuration parameter `vacuum_buffer_usage_limit`
 - `VACUUM` and `ANALYZE` with `BUFFER_USAGE_LIMIT` option
 - `vacuumdb` command with option `--buffer-usage-limit`



Improvements in vacuuming operations v17 and above

- Vacuuming operations were limited to a 1 GB limit even when `maintenance_work_mem` or `autovacuum_work_mem` were set higher
- Postgres is no longer is limited to 1GB memory usage
 - New adaptive radix tree implementation for internal memory structure of vacuum
 - Reduces memory consumption
 - Vacuuming operations on large tables with multiples indexes perform better
- Reduction in WAL content generated by Vacuum
 - Single WAL record for freezing and pruning
 - Vacuuming of relations with no indexes is better optimized



Routine Re-indexing

- Indexes are used for faster data access
- `UPDATE` and `DELETE` on a table modify underlying index entries
- Indexes are stored on data pages and become fragmented over time
- `REINDEX` rebuilds an index using the data stored in the index's table
- Time required depends on:
 - Number of indexes
 - Size of indexes
 - Load on server when running command



When to Reindex

- There are several reasons to use REINDEX:
 - An index has become "bloated", meaning it contains many empty or nearly-empty pages
 - You have altered a storage parameter (such as fillfactor) for an index
 - An index built with the CONCURRENTLY option failed, leaving an "invalid" index
- Syntax:

```
=> REINDEX [ ( VERBOSE ) ] { INDEX | TABLE | SCHEMA | DATABASE |  
SYSTEM } [ CONCURRENTLY ] name
```



Module Summary

- Updating Optimizer Statistics
- Handling Data Fragmentation using Routine Vacuuming
- Preventing Transaction ID Wraparound Failures
- Automatic Maintenance using Autovacuum
- Re-indexing in Postgres



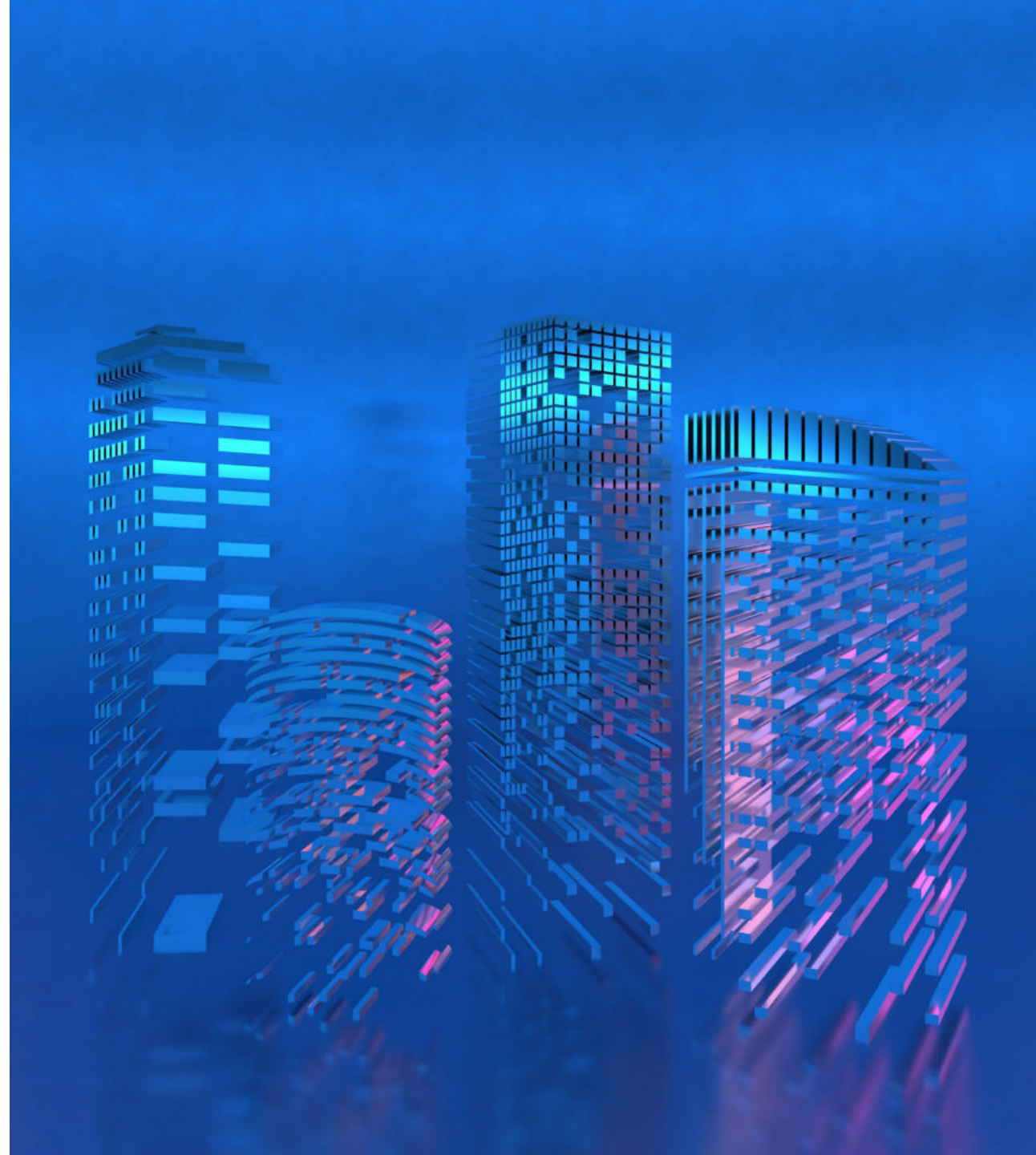
Lab Exercise - 1

1. While monitoring table statistics on the `edbstore` database, you found that some tables are not automatically maintained by `autovacuum`. You decided to perform manual maintenance on these tables. Write a SQL script to perform the following maintenance:
 - Reclaim obsolete row space from the `customers` table.
 - Update statistics for `emp` and `dept` tables.
 - Mark all the obsolete rows in the `orders` table for reuse.
2. Execute the newly created maintenance script on `edbstore` database.



Lab Exercise - 2

1. The composite index named `ix_orderlines_orderid` on `(orderid, orderlineid)` columns of the `orderlines` table is performing very slowly. Write a statement to reindex this index for better performance.





Moving Data Using COPY Command



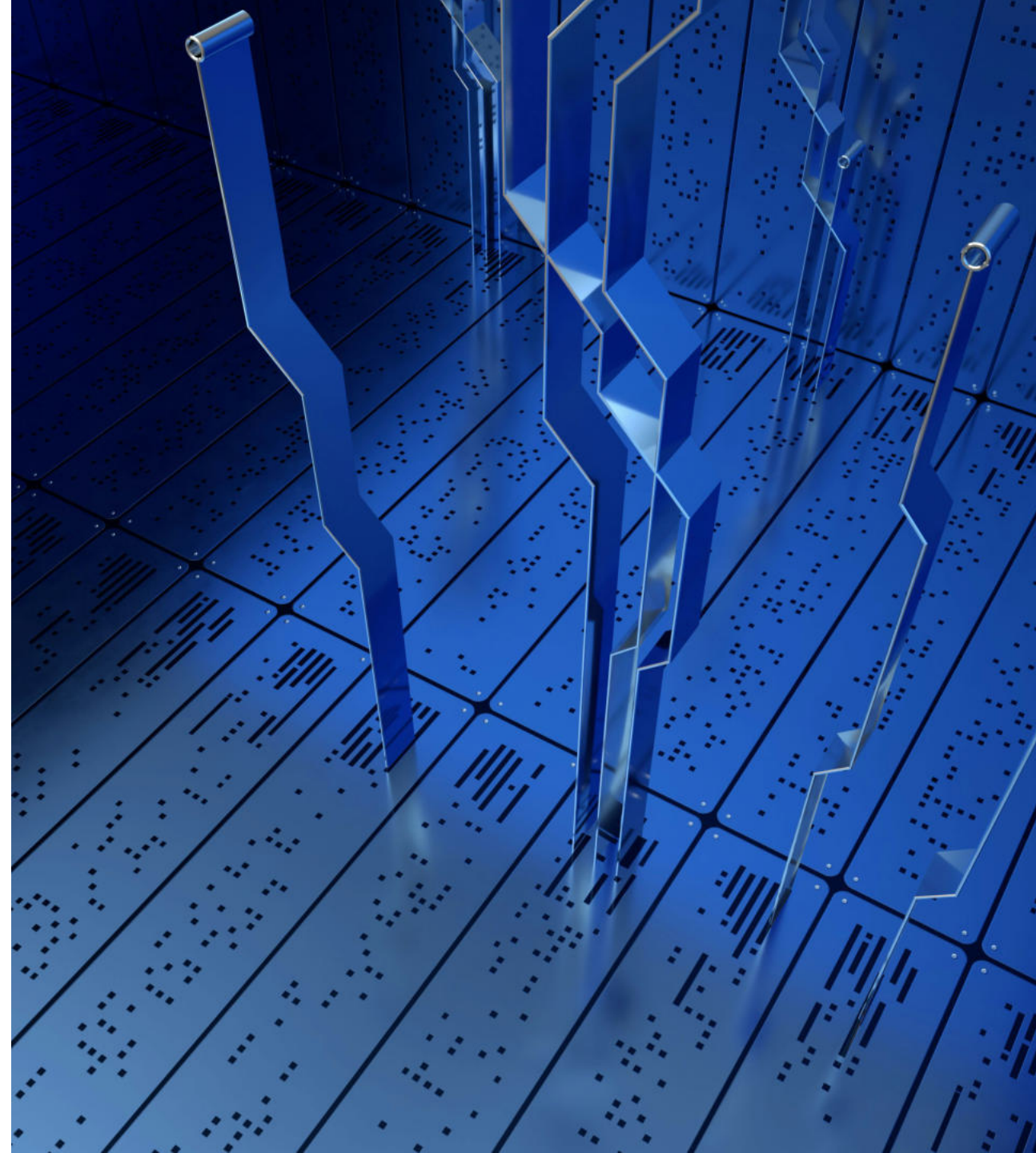
Module Objectives

- Loading flat files
- Import and export data using COPY
- Examples of COPY Command
- Using COPY FREEZE for performance
- Loader
- Data Loading Methods
- Invoking Loader and Control File
- Loader Exit Codes



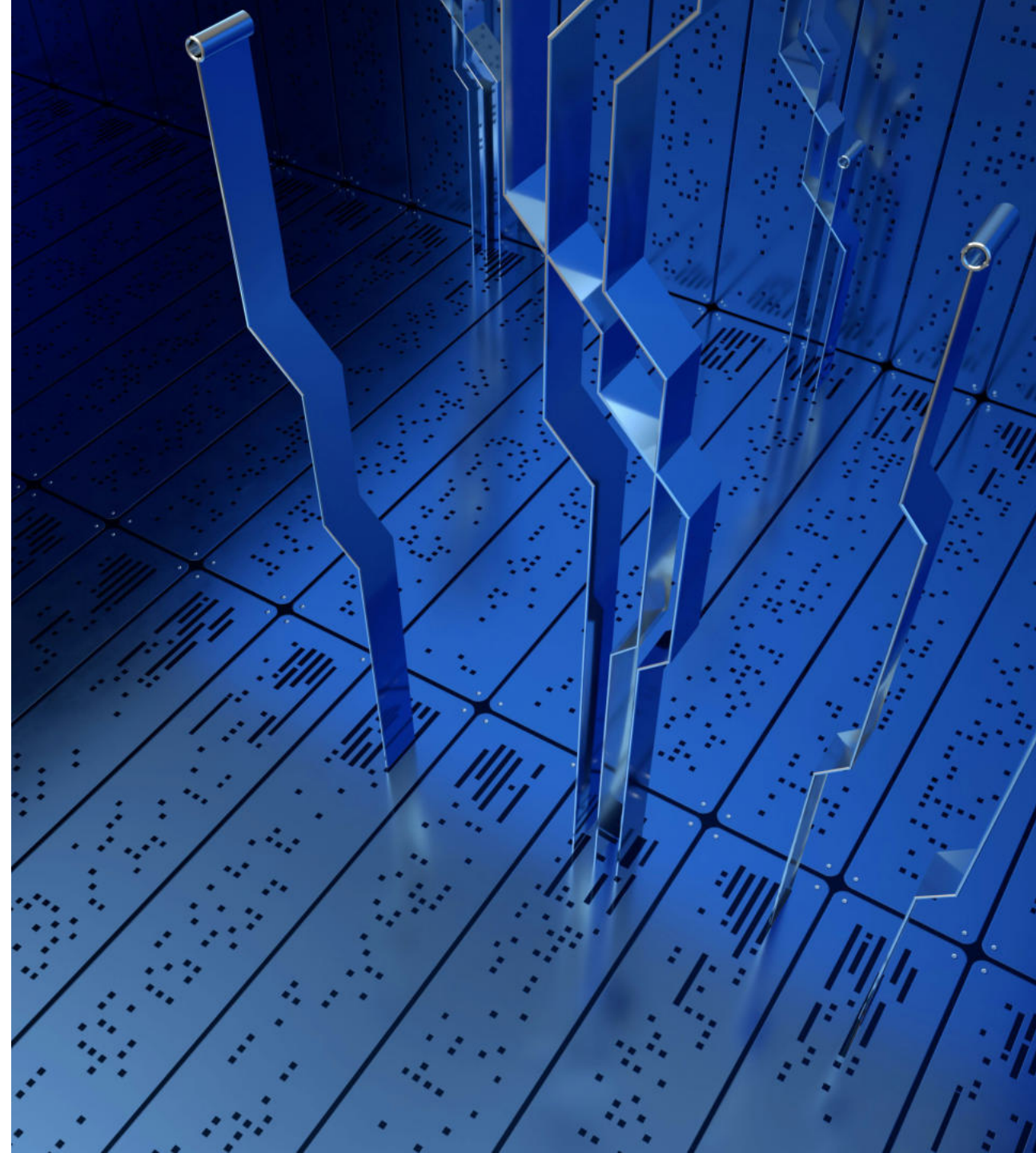
Loading Flat Files into Database Tables

- A "flat file" is a plain text or mixed text file which usually contains one record per line
- Enterprise Postgres offers two options to load flat files into a database table:
 - Loader
 - COPY Command



The COPY Command

- COPY moves data between Enterprise Postgres tables and standard file-system files
- COPY TO copies the contents of a table or a query to a file
- COPY FROM copies data from a file to a table
- The file must be accessible to the server



COPY Command Syntax

Copy From:

```
COPY table_name [(column list)] FROM 'filename'|PROGRAM 'command'|STDIN [options][WHERE cond.]
```

Copy To:

```
COPY table_name[(column list)]|(query) TO 'filename'|PROGRAM 'command'|STDOUT [options]
```

Copy Command Options

```
FORMAT format_name  
OIDS [ boolean ]  
FREEZE [ boolean ]  
DELIMITER 'delimiter_character'  
NULL 'null_string'  
HEADER [ boolean ]  
QUOTE 'quote_character'  
ESCAPE 'escape_character'  
FORCE_QUOTE { ( column_name [, ...] ) | * }  
FORCE_NOT_NULL ( column_name [, ...] )  
FORCE_NULL ( column_name [, ...] )  
ON_ERROR error_action  
ENCODING 'encoding_name'  
LOG_VERBOSITY verbosity
```



Example Export to File

```
=> COPY emp (empno,ename,job,sal,comm,hiredate) TO '/tmp/emp.csv' CSV HEADER;  
COPY
```

```
=> \! cat /tmp/emp.csv
```

```
empno,ename,job,sal,comm,hiredate
```

```
7369,SMITH,CLERK,800.00,,17-DEC-80 00:00:00
```

```
7499,ALLEN,SALESMAN,1600.00,300.00,20-FEB-81 00:00:00
```

```
7521,WARD,SALESMAN,1250.00,500.00,22-FEB-81 00:00:00
```

```
7566,JONES,MANAGER,2975.00,,02-APR-81 00:00:00
```

```
7654,MARTIN,SALESMAN,1250.00,1400.00,28-SEP-81 00:00:00
```

```
7698,BLAKE,MANAGER,2850.00,,01-MAY-81 00:00:00
```

```
7782,CLARK,MANAGER,2450.00,,09-JUN-81 00:00:00
```

```
7788,SCOTT,ANALYST,3000.00,,19-APR-87 00:00:00
```

```
7839,KING,PRESIDENT,5000.00,,17-NOV-81 00:00:00
```

```
7844,TURNER,SALESMAN,1500.00,0.00,08-SEP-81 00:00:00
```

```
7876,ADAMS,CLERK,1100.00,,23-MAY-87 00:00:00
```

```
7900,JAMES,CLERK,950.00,,03-DEC-81 00:00:00
```

```
7902,FORD,ANALYST,3000.00,,03-DEC-81 00:00:00
```

```
7934,MILLER,CLERK,1300.00,,23-JAN-82 00:00:00
```



Example Import from File

```
edb=# CREATE TEMP TABLE empcsv (LIKE emp);
CREATE TABLE
edb=# COPY empcsv (empno, ename, job, sal, comm, hiredate)
edb=# FROM '/tmp/emp.csv' CSV HEADER;
COPY
edb=# SELECT * FROM empcsv;
```

empno	ename	job	mgr	hiredate	sal	comm	deptno
7369	SMITH	CLERK		17-DEC-80 00:00:00	800.00		
7499	ALLEN	SALESMAN		20-FEB-81 00:00:00	1600.00	300.00	
7521	WARD	SALESMAN		22-FEB-81 00:00:00	1250.00	500.00	
7566	JONES	MANAGER		02-APR-81 00:00:00	2975.00		
7654	MARTIN	SALESMAN		28-SEP-81 00:00:00	1250.00	1400.00	
7698	BLAKE	MANAGER		01-MAY-81 00:00:00	2850.00		
7782	CLARK	MANAGER		09-JUN-81 00:00:00	2450.00		
7788	SCOTT	ANALYST		19-APR-87 00:00:00	3000.00		
7839	KING	PRESIDENT		17-NOV-81 00:00:00	5000.00		
7844	TURNER	SALESMAN		08-SEP-81 00:00:00	1500.00	0.00	
7876	ADAMS	CLERK		23-MAY-87 00:00:00	1100.00		
7900	JAMES	CLERK		03-DEC-81 00:00:00	950.00		
7902	FORD	ANALYST		03-DEC-81 00:00:00	3000.00		
7934	MILLER	CLERK		23-JAN-82 00:00:00	1300.00		

(14 rows)



Example - COPY Command on Remote Host

- COPY command on remote host using psql

```
$ cat emp.csv | ssh 192.168.192.83  
"psql -U edbstore edbstore -c  
'copy emp from stdin;'"
```



COPY FREEZE

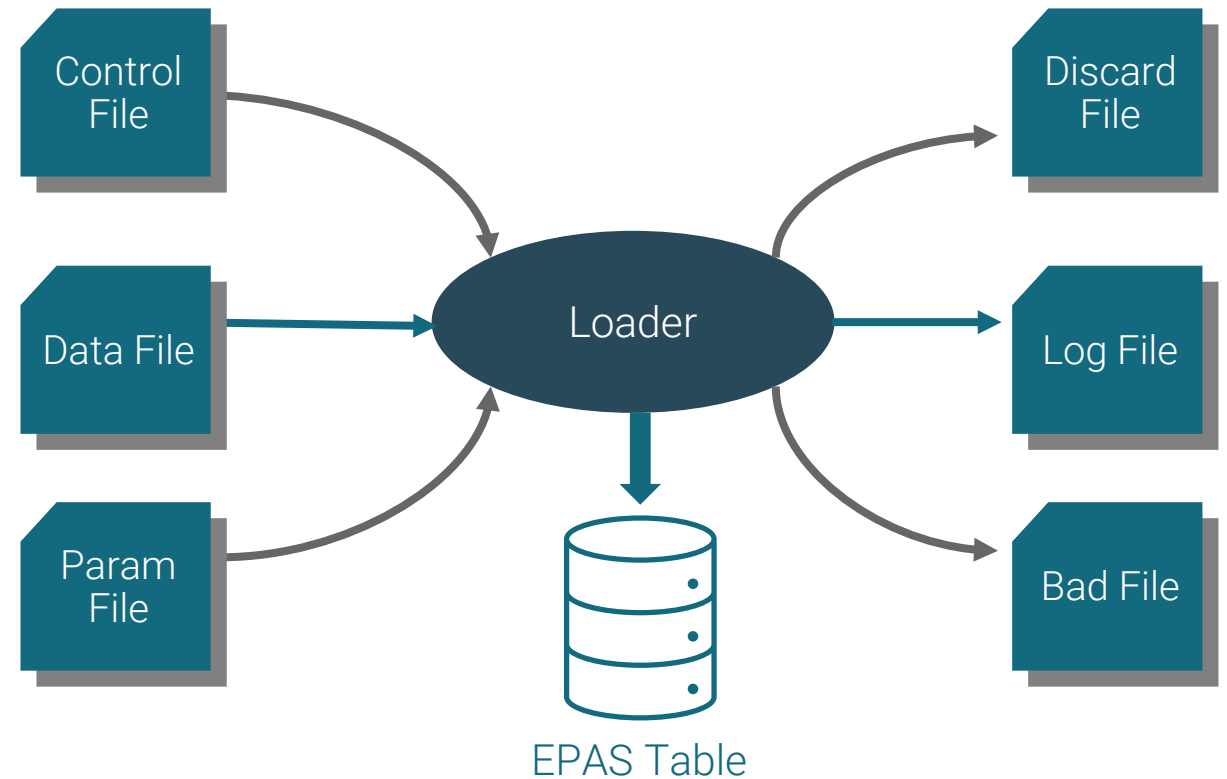
- FREEZE is a new option in the COPY statement
 - Add rows to a newly created table and freezes them
 - Table must be created or truncated in current subtransaction
 - Improves performance of initial bulk load
 - Does violate normal rules of MVCC
- Usage:

```
=> COPY tablename FROM filename  
FREEZE;
```



Loader

- Loader is a high-performance bulk data loader
- Supports Oracle SQL*Loader data loading methods:
 - Conventional
 - Direct
 - Parallel



Data Loading Methods

- Conventional path loading uses basic insert processing and is used to add rows to the table
- Constraints, indexes and triggers are enforced during conventional path data loading
- Direct path loading is faster than conventional path loading, but is non-recoverable
- Direct path loading also requires removal of constraints and triggers from the table
- Conventional path data loading is slower than direct path loading, but is fully recoverable
- A parallel direct path load provides even greater performance



Invoking Loader

- Use the following command to invoke Loader from the command line:

```
edblldr [-d DBNAME] [-p PORT] [-c "CONNECTION_STRING"]  
  
userid={dbuser[/dbpass]||/} direct={true|false} parallel={true|false}  
  
control=control_file_name log=log_file_name  
  
errors=num_errors  
  
skip_index_maintenance={true|false}  
  
skip=num_skips bad=bad_file_name parfile=par_file_name  
  
freeze={true|false}  
  
handle_conflicts={true|false}
```



The Loader Control File

- The Oracle SQL*Loader has compatible syntax for control file directives
- The control file includes the instructions that Loader uses to build the table (or tables) from the input file. It includes information such as:
 - The fully qualified name of the input file
 - The name of the table or tables
 - The name of the columns within the table or tables
 - The delimiters or other selection criteria used to choose the column content
 - The fully qualified names of the bad and discarded files



Control File Syntax

- The syntax for the Loader control file is as follows:

```
[ OPTIONS (param=value [, param=value ] ...) ]  
LOAD DATA  
    [ CHARACTERSET charset ]  
    [ INFILE '{ data_file | stdin }' ] [ BADFILE 'bad_file' ] [ DISCARDFILE  
'discard_file' ]  
    [ { DISCARDMAX | DISCARDS } max_discard_recs ]  
[ INSERT | APPEND | REPLACE | TRUNCATE ] [ PRESERVE BLANKS ]  
{ INTO TABLE target_table  
    [ WHEN field_condition [ AND field_condition ] ...] [ FIELDS TERMINATED BY  
'termstring'  
    [ OPTIONALLY ENCLOSED BY 'enclstring' ] ] [ RECORDS DELIMITED BY 'delimstring'  
] [ TRAILING NULLCOLS ]  
    (field_def [, field_def ] ...)  
} ...
```



Loader Example

- This example loads data from a file named `/tmp/mydata.csv` into a table named `emp`
- The data within the input file is delimited by a comma
- Create the control file:

```
LOAD DATA INFILE '/tmp/mydata.csv'
    BADFILE '/tmp/mydata.bad'
    DISCARDFILE '/tmp/mydata.dsc'
    INSERT INTO TABLE emp
    FIELDS TERMINATED BY ","
    OPTIONALLY ENCLOSED BY '"' (empno, empname, sal, deptno)
```

- Run the `edblldr` command:

```
$ edblldr -d edb CONTROL=empctl BAD=/tmp/emp.bad LOG=/tmp/emp.log SKIP=1 ERRORS=10
```



Loader Exit Codes

- Loader will return one of the following exit codes:

0

Indicates that all rows loaded successfully

1

Loader encountered syntax errors or aborted due to an unrecoverable error

2

Load completed with some rejected or discarded rows

3

Indicates EDB*Loader stopped due to an OS error



Module Summary

- Loading flat files
- Import and export data using COPY
- Examples of COPY Command
- Using COPY FREEZE for performance
- Loader
- Data Loading Methods
- Invoking Loader and Control File
- Loader Exit Codes



Lab Exercise - 1

- In this lab exercise you will demonstrate your ability to copy data:
 1. Unload the `emp` table from the `edbuser` schema to a csv file, with column headers
 2. Create a `copyemp` table with the same structure as the `emp` table
 3. Load the csv file (from step 1) into the `copyemp` table





Replication and High Availability Tools



Module Objectives

- Data Replication
- Data Replication in Postgres
- Streaming Replication and Architecture
- Synchronous, Asynchronous and Cascaded Replication
- Setup Streaming Replication
- Logical Replication Architecture
- Replication Manager (repmgr)
- High Availability Overview
- Replication Server (xdb)



Data Replication

- Replication is the process of copying data and changes to a secondary location for data safety and availability
- Data loss can occur due to several reasons
- Replication is aimed towards availability of the data when a primary source goes offline
- Data can be recovered from backup but downtimes are costly
- Replication aims towards lowering downtime
- Failovers can be configured to such a level where application may not notice the primary source is offline



Data Replication in Postgres

- Data replication options:
 - Log-Shipping Standby Servers
 - Streaming Replication
 - Logical Replication
 - Postgres-BDR
 - EDB Replication Server
- Cluster management tools:
 - High Availability Routing for Postgres(HARP)
 - EDB Failover Manager
 - Replication manager(repmgr)

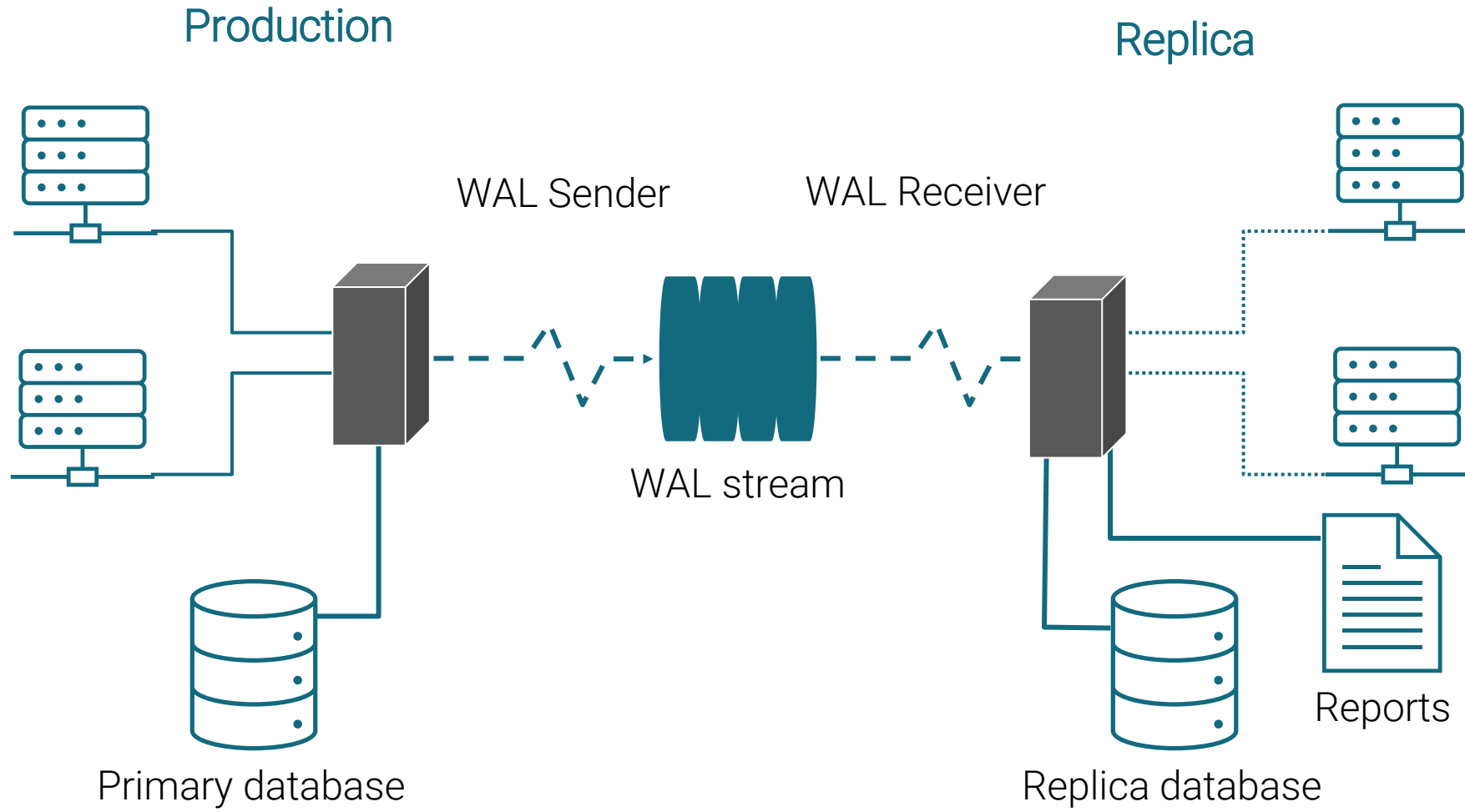


Streaming Replication

- Streaming Replication (Hot Standby) is a major feature of Postgres
- Replica connects to the primary node using `REPLICATION` protocol
- WAL segments are streamed to replica server
- No log shipping delays, stream WAL content across to replica immediately
- Synchronous/Asynchronous options available
- Supports cascading replication

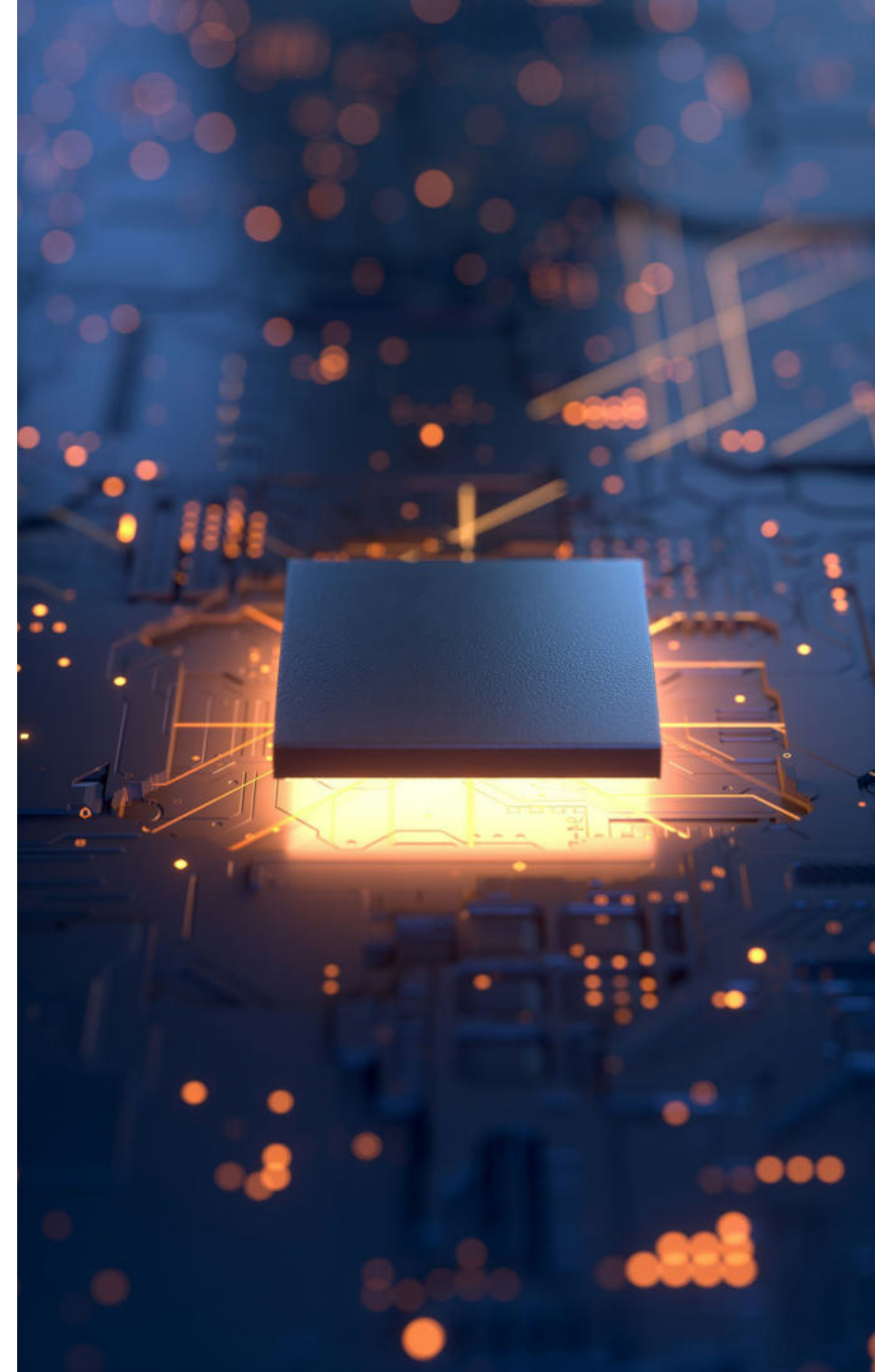


Hot Streaming Architecture



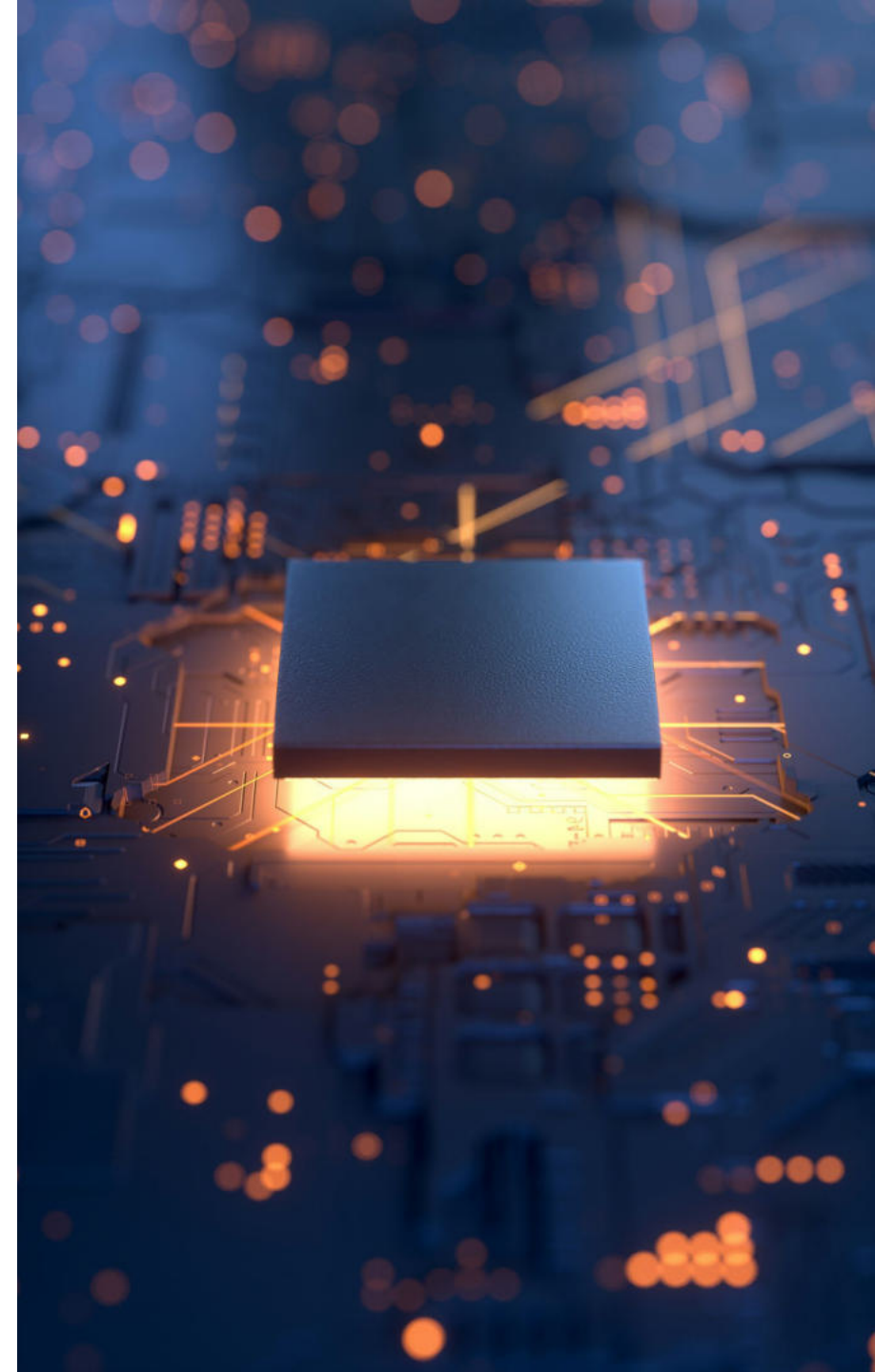
Asynchronous Replication

- Streaming replication is asynchronous by default but can be configured as synchronous
- Asynchronous
 - Disconnected architecture
 - Transaction is committed on primary and flushed to WAL segment
 - Later transaction is transmitted to replica server(s) using stream
 - Some data loss is possible
 - Replication using WAL Archive method is always asynchronous



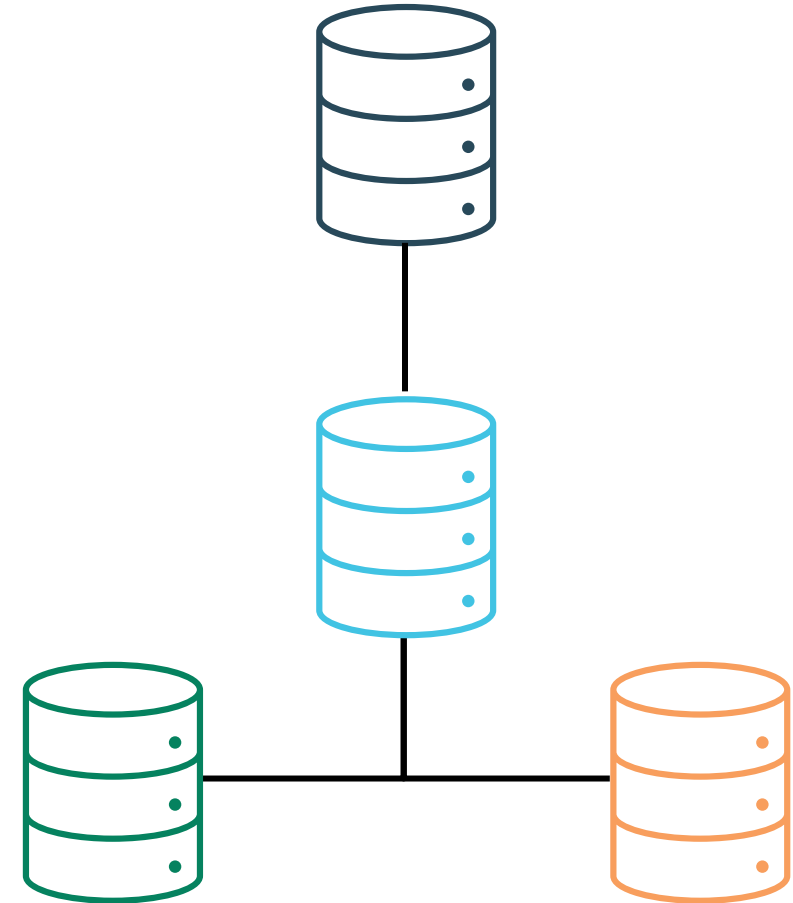
Synchronous Replication

- Synchronous Replication
 - A 2-safe replication method offering zero data loss
 - Transaction must apply changes to primary and synchronously replicated replicas using two-phase commit actions
 - User gets a commit message after confirmation from both primary and replica
 - This will introduce a delay in committing transactions



Cascading Replication

- Streaming replication supports single master node
- Cascade replication can be used to share the replication overhead of primary with other replicas
- Replica can stream changes to other Replicas
- Helps minimize inter-site bandwidth overheads on primary node
- Asynchronous only



Setup Streaming Replication



Primary Server Configuration

- For Physical Streaming Replication:
 - Change WAL Content parameter:
 - `wal_level = replica` #Default is replica
 - Two options to allow streaming connection:
 - `max_wal_senders`
 - `max_replication_slots`
 - Set only the minimum number of segments retained in `pg_wal`
 - `wal_keep_size = 1024`



Synchronous Streaming Replication Configuration

- Default level of Streaming Replication is Asynchronous
- Synchronous level can also be configured using additional parameters:

```
synchronous_commit=on
```

```
synchronous_standby_names
```

- If the synchronous replica stops responding, then COMMITs will be blocked forever until someone manually intervenes
- Transactions can be configured not to wait for replication by setting the `synchronous_commit` parameter to `local` or `off`



Configure Authentication

- Authentication setting on the primary server must allow replication connections from the replica(s)
- Provide a suitable entry or entries in `pg_hba.conf` with the database field set to replication
- Open `pg_hba.conf` of primary server:

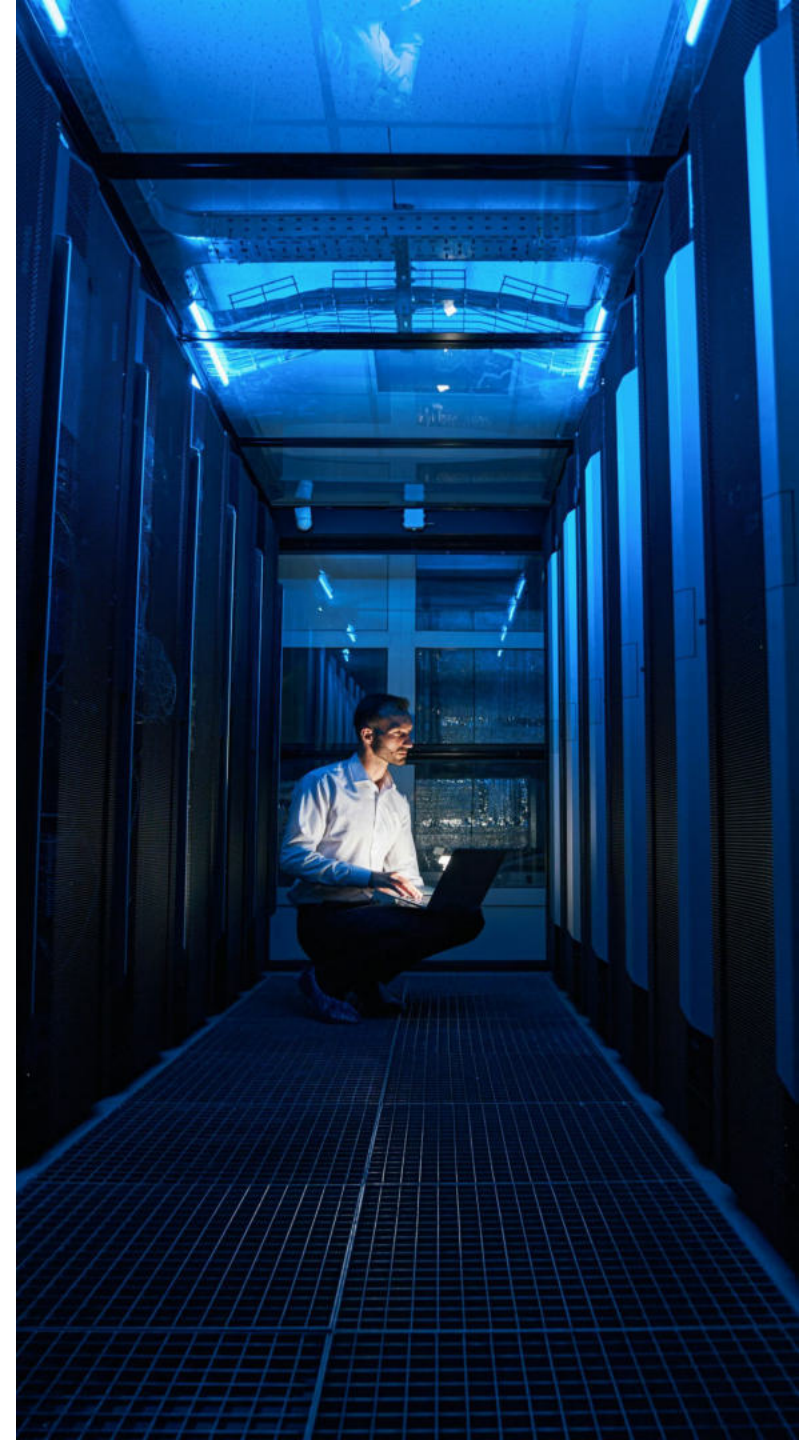
```
host    replication          all          192.168.56.2/32 md5
```

Note - You will need to reload the primary server



Take a Full Backup of the Primary Server

- Backup the Primary Server using `pg_basebackup`:
 - `pg_basebackup -h localhost -U postgres -p 5444 -D /backup/data1 -R`
 - `-R` option
 - Creates a default copy of `standby.signal` file
 - Add primary server connection info to `postgresql.auto.conf`



Replica Configuration

hot_standby

Set this parameter to "ON" for read-only replica

primary_conninfo

Set connection string to connect with primary or cascaded replica

primary_slot_name

Specify replication slot name to be used for connection

max_standby_streaming_delay

Duration for which replica has to wait during query conflicts

wal_receiver_create_temp_slot

Authorize WAL receiver process to be able to create a temporary replication slot

recovery_min_apply_delay

Parameter used for delayed replication



Replica Recovery Settings

- Replica configuration settings must be set in `postgresql.conf` or `postgresql.auto.conf`
- Create a file name `standby.signal` in the data directory
- `standby.signal` indicates the server should start as a replica
- Last step - start the replica using system services or `pg_ctl`



Logical Replication



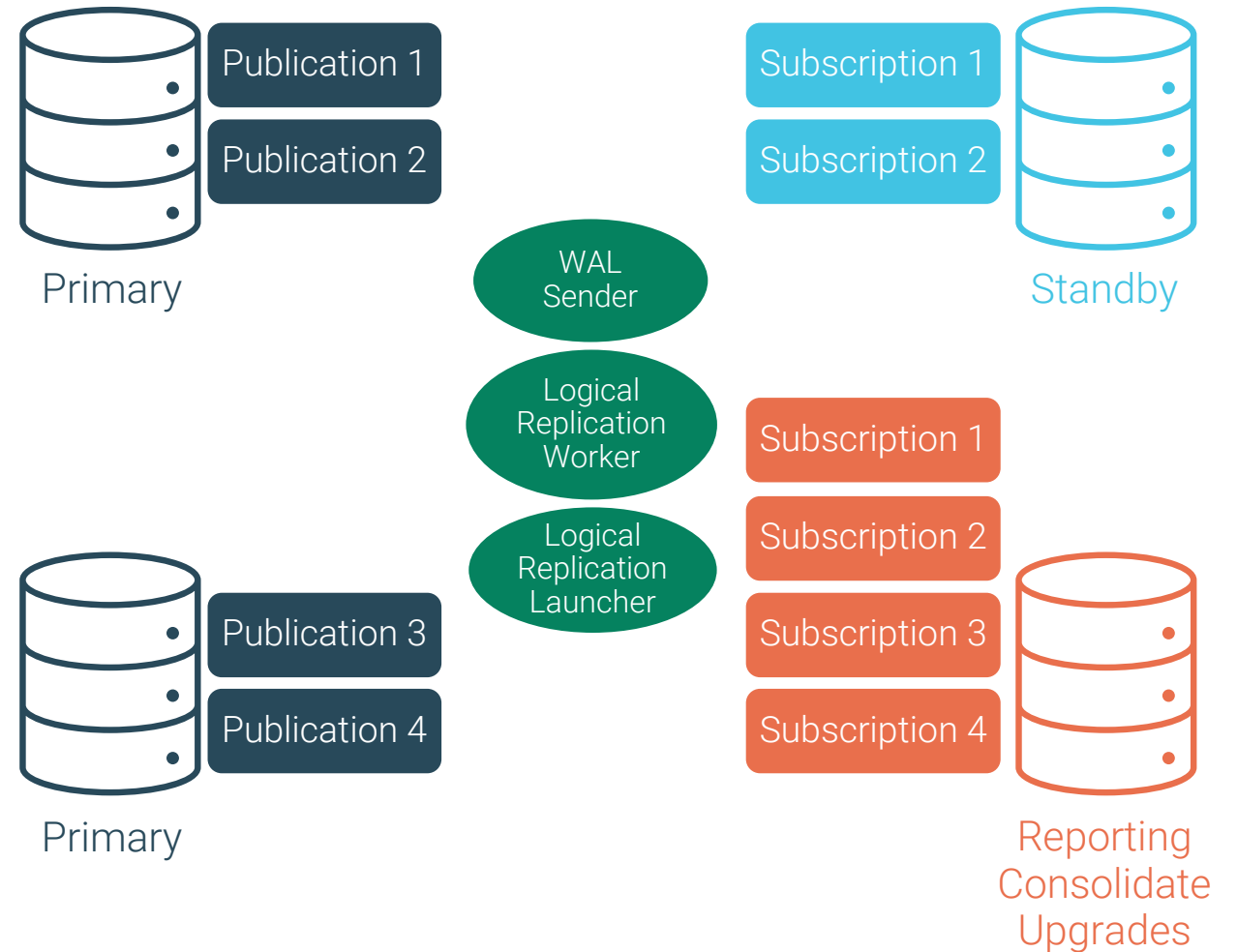
Logical Replication Use Cases

- Logical Replication can be used and configured several ways
 - Publisher / Subscriber (Typical)
 - Tables replicated from one database to another
 - Logical Replication Failover
 - HA Environment: Primary fails the Secondary will become the Publisher
 - Logical Replication from a Replica
 - HA Environment: Logically replicate from secondary
 - Initial Data Sync for Logical Replication
 - HA Environment: Convert from streaming to logical replication



Logical Replication: Publisher / Subscriber

- Logical replication is a method of replicating selected data objects and their changes
- Based on publications and subscriptions
- Can be used to consolidate data
- Portable across hardware and software version
- Tables on standby server which are part of a subscription must be treated as read only to avoid conflicts



When to Use Logical Replication

- Sending incremental changes in a single database or a subset of a database to subscribers
- Consolidating multiple databases into a single one
- Replicating between different major versions of Postgres
- Giving access to replicated data to different groups of users
- Sharing a subset of the database between multiple databases



Setting Up Logical Replication

Change `wal_level` to `logical` in `postgresql.conf`

Add `pg_hba.conf` entry in each server to allow connection

Connect to database in publication instance

Create a publication using `CREATE PUBLICATION` statement

A published table must have a “replica identity” configured in order to be able to replicate `UPDATE` and `DELETE` operations

Connect to database in subscription instance and create a subscription using `CREATE SUBSCRIPTION` statement



Example – Logical Replication Setup

- Initialize sample publication(primarypub) and subscription(primarysub) instance

```
[postgres@localhost ~]$ initdb --version
```

```
[postgres@localhost ~]$ initdb -D primarypub -U pubdba
```

```
[postgres@localhost ~]$ initdb -D primarysub -U subdba
```

- Edit postgresql.conf parameter for both instances

```
[postgres@localhost ~]$ vi primarypub/postgresql.conf
```

```
port=5420
```

```
wal_level=logical
```

```
[postgres@localhost ~]$ vi primarysub/postgresql.conf
```

```
port=5421
```

```
wal_level=logical
```



Example – HBA Entries and Starting Instances

- Add pg_hba.conf entries for connections

```
[enterprisedb@localhost ~]$ vi primarysub/pg_hba.conf
```

```
host all pubdba 192.168.56.101/32 md5
```

```
[enterprisedb@localhost ~]$ vi primarypub/pg_hba.conf
```

```
host all subdba 192.168.56.101/32 md5
```

- Start both instances

```
[enterprisedb@localhost ~]$ pg_ctl -D primarypub/ start
```

```
[enterprisedb@localhost ~]$ pg_ctl -D primarysub/ start
```



Example – Create Tables and Publication

- Connect to default database in publication instance

```
[postgres@localhost ~]$ psql -p 5420 -U pubdba postgres
```

- Create a sample table and publication

```
=# CREATE TABLE pubexample(id INT PRIMARY KEY,  
                             name VARCHAR(30));
```

```
=# INSERT INTO pubexample  
VALUES(generate_series(1,5000),'Test1');
```

```
=# SELECT count(*) FROM pubexample;
```

```
=# CREATE PUBLICATION testpub FOR TABLE pubexample;
```



Example – Create Tables and Subscription

- Connect to default database in subscription instance

```
[postgres@localhost ~]$ psql -p 5421 -U subdba postgres
```

- Create a sample table and subscription

```
=# CREATE TABLE pubexample(id INT PRIMARY KEY,  
                             name VARCHAR(30));
```

```
=# CREATE SUBSCRIPTION testsub CONNECTION
```

```
  'host=localhost port=5420 user=pubdba dbname=postgres'
```

```
  PUBLICATION testpub;
```

```
=# SELECT count(*) FROM pubexample;
```



Example – Test Logical Replication

- Add data to publication

```
[postgres@localhost ~]$ psql -p 5420 -U pubdba postgres
postgres=# INSERT INTO pubexample
           VALUES (generate_series(5001,10000),'Test1');
postgres=# \q
```

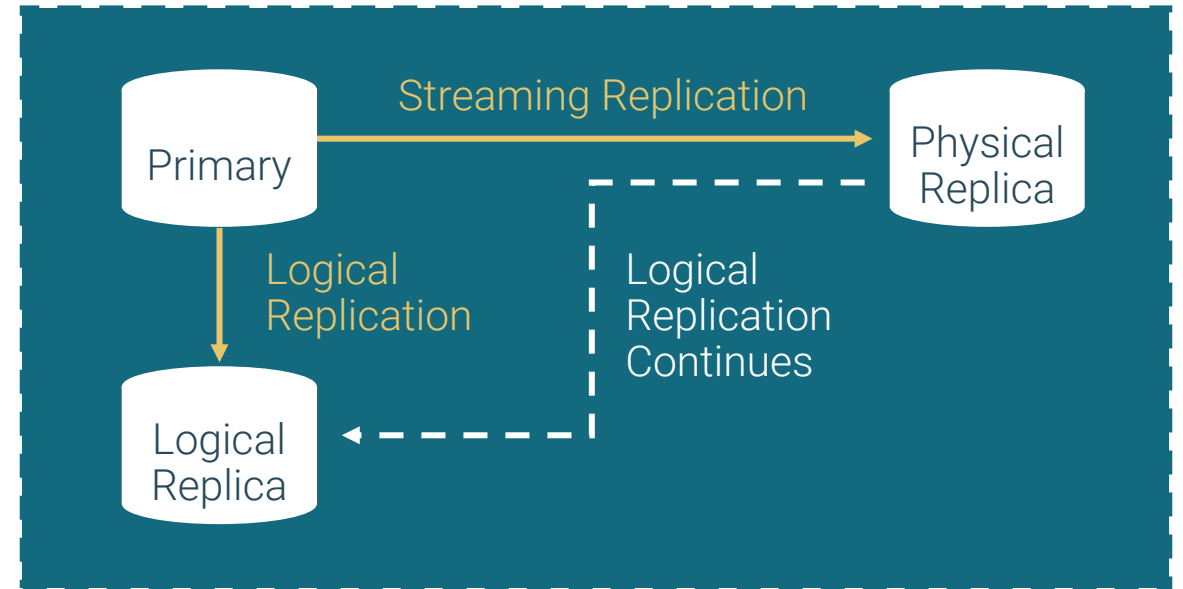
- Check changes on Subscription

```
[postgres@localhost ~]$ psql -p 5421 -U subdba postgres
postgres=# SELECT count(*) FROM pubexample;
```



Logical Replication Failover

- Subscriber nodes seamlessly continue replicating by pulling data from the new publisher node after failover
- Configure using parameters and system functions during replication setup



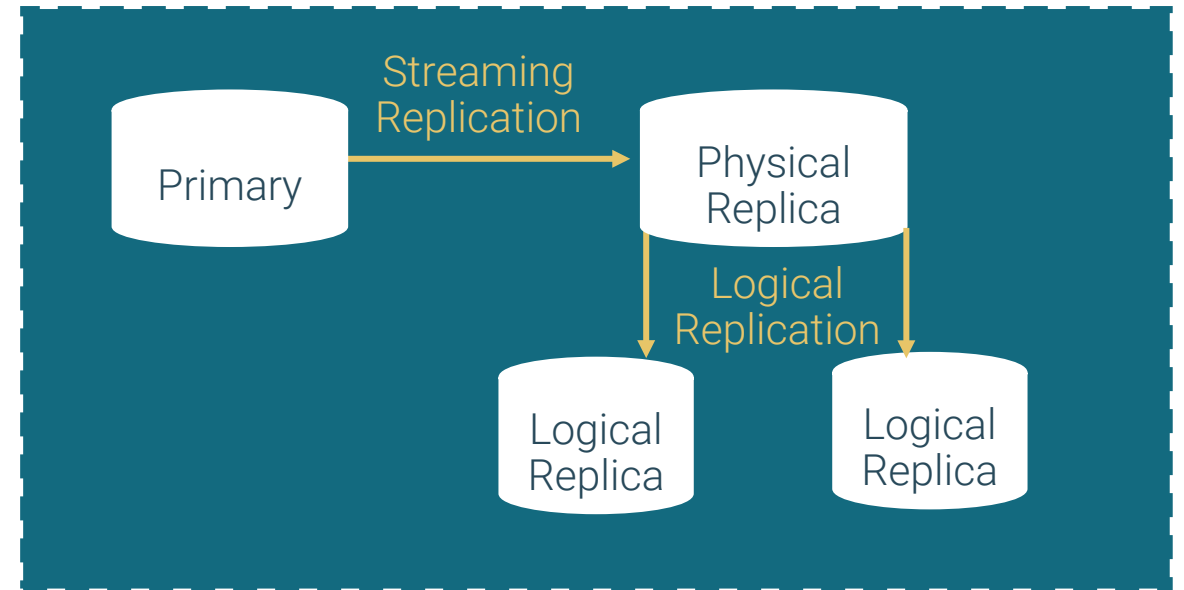
Setup Failover Control

1. Enable the `failover` property on subscriber node:
 - Set the failover property for slots linked to subscriptions using `CREATE SUBSCRIPTION` or `pg_create_logical_replication_slot()` API
2. Standby node must be configured for slot sync and be always ahead of the subscribers:
 - Enable `sync_replication_slots` parameter on standby to sync failover slots periodically.
 - Specify a valid database name in `primary_conninfo` string on standby node
 - On primary node, configure `synchronized_standby_slots` to include the replication slot used for physical streaming
3. Streaming between Primary and Physical Replica must be setup using Replication Slot:
 - Set `primary_slot_name` and enable `hot_standby_feedback` on physical replica
4. Verify slot sync using `pg_replication_slots` on primary node:
 - `SELECT slot_name, synced FROM pg_replication_slots;`



Logical Replication from a Replica

- Postgres uses logical decoding for performing logical replication
- In earlier versions, logical decoding was confined to the primary server
- Logical decoding can now be performed on Read-Only Standby
- Significant reduction of the workload on the primary server
- Seamless transition during primary failovers to standby



Setup Logical Replication from a Replica

1.Primary Server (Read-Write)

- wal_level set to logical
- WAL Sender sending all changes to Replica

2.Replica (Read Only)

- WAL Receiver replays changes from Primary
- wal_level set to logical
- Create a publication for logical replication
- Supports pg_create_logical_replication_slot

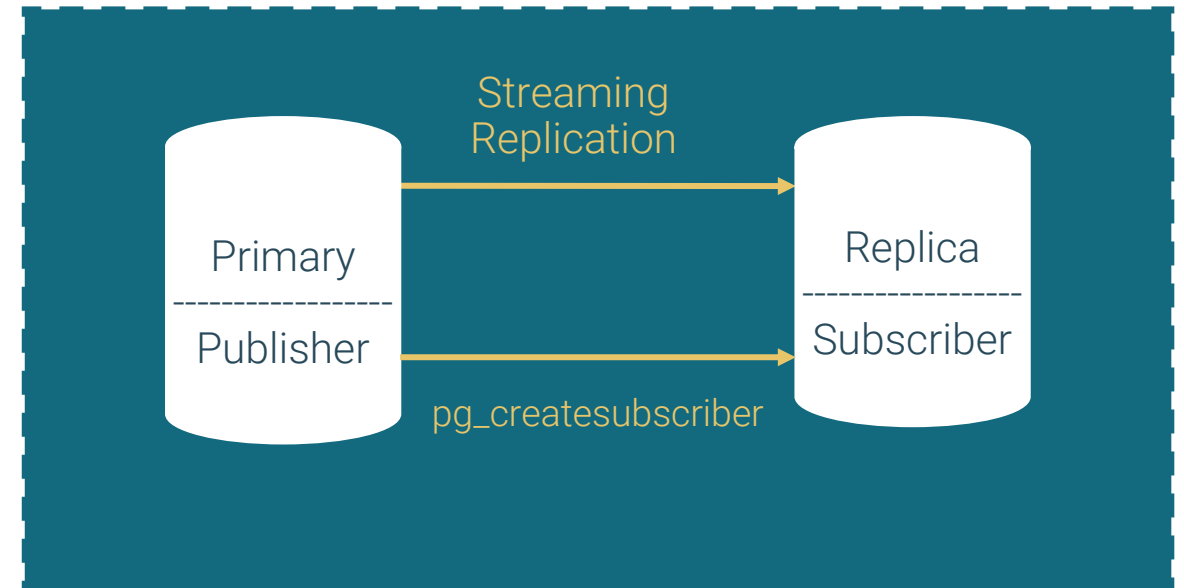
3.Subscriber Node

- Create a subscription for the publication on Replica Node
- Replicated tables must be read-only



Initial Data Sync for Logical Replication

- Logical Replication provides option for replicating selected objects using streaming
- Logical replication setup requires an initial data sync which can be resource intensive and time consuming
- Postgres now offers conversion of an existing Primary-Replica physical streaming setup to Logical Streaming using `pg_createsubscriber`



Convert Physical Replica to Logical Replication

- `pg_createsubscriber` utility:
 - Run on target server to convert existing physical streaming setup to logical replication
 - Creates a publisher node and subscriber node for each specified database including all the tables.
 - Does not copy the initial table data
 - Targets large database systems where initial sync can be a problem



Monitoring Basics



Monitoring Replication

- `pg_stat_replication`
 - Show connected replicas and their status on the primary
- `pg_stat_subscription`
 - Shows the status of subscription when using logical replication
- `pg_stat_wal_receiver`
 - Shows the WAL receiver process status on Replica
- Recovery information functions:
 - `pg_is_in_recovery()`
 - `pg_current_wal_lsn`
 - `pg_last_wal_receive_lsn`
 - `pg_last_xact_replay_timestamp()`



Example - Monitoring Replication

- Execute:

```
=# SELECT * FROM pg_stat_replication;
```

- Find lag (bytes):

```
=# SELECT pg_wal_lsn_diff(sent_lsn, replay_lsn) FROM  
pg_stat_replication;
```

- Find lag (seconds):

```
=# SELECT CASE WHEN pg_last_wal_receive_lsn() =  
pg_last_wal_replay_lsn()  
THEN 0 ELSE  
EXTRACT (EPOCH FROM now() -pg_last_xact_replay_timestamp())  
END AS stream_delay;
```



Recovery Control Functions

Name	Return Type	Description
<code>pg_is_wal_replay_paused()</code>	<code>bool</code>	True if recovery is paused.
<code>pg_wal_replay_pause()</code>	<code>void</code>	Pauses recovery immediately.
<code>pg_wal_replay_resume()</code>	<code>void</code>	Restarts recovery if it was paused.



EDB Postgres AI High Availability Overview



EDB Postgres AI High Availability

The most advanced replication solution for Postgres



Maintain extreme
high availability

Postgres clusters deployed with EDB Postgres AI High Availability keep top tier enterprise applications running



Upgrade with
near zero downtime

Rolling upgrades of application and database software eliminate the largest source of downtime



Choose the level
of consistency

Robust capabilities provide flexibility to meet application data loss requirements



Always ON

Top-tier enterprise applications are critical to an organization's success in all regions where business is conducted, whether a single region or globally



The application represents a promise to its customers



The application must perform well for a good user experience



The availability of the application directly ties to revenue generation



The application data must always be current and available, or the user loses trust



BDR is more than Bi-directional Replication

MULTI-MASTER REPLICATION ENABLING
HIGHLY AVAILABLE AND GEOGRAPHICALLY
DISTRIBUTED POSTGRES CLUSTERS

- Logical replication of data and schema enabled via standard Postgres extension
- Data consistency options that span from immediate to eventual consistency
- Robust tooling to manage conflicts, monitor performance, and validate consistency
- Deploy natively to cloud, virtual, or bare metal environments
- Geo-fencing, allowing selectively replicate data for security compliance and jurisdiction control.



High Availability Features

Multi-Master
Synchronous or
Asynchronous
Replication for
Postgres

Flexible Always-ON
Deployment
Architectures

DDL Replication

Row Level
Consistency

DDL and Row Filters

Parallel Apply

Database Rolling
Upgrades

Auto Partitioning

Configurable
Column-level
Conflict Resolution

Conflict-free
Replicated Data
Types (CRDTs)

Transactions State
Tracking Across
Failovers

Subscriber-only
Nodes

PGD CLI

Next Generation
Connection Routing
using PGD-Proxy

Open Telemetry
Integration

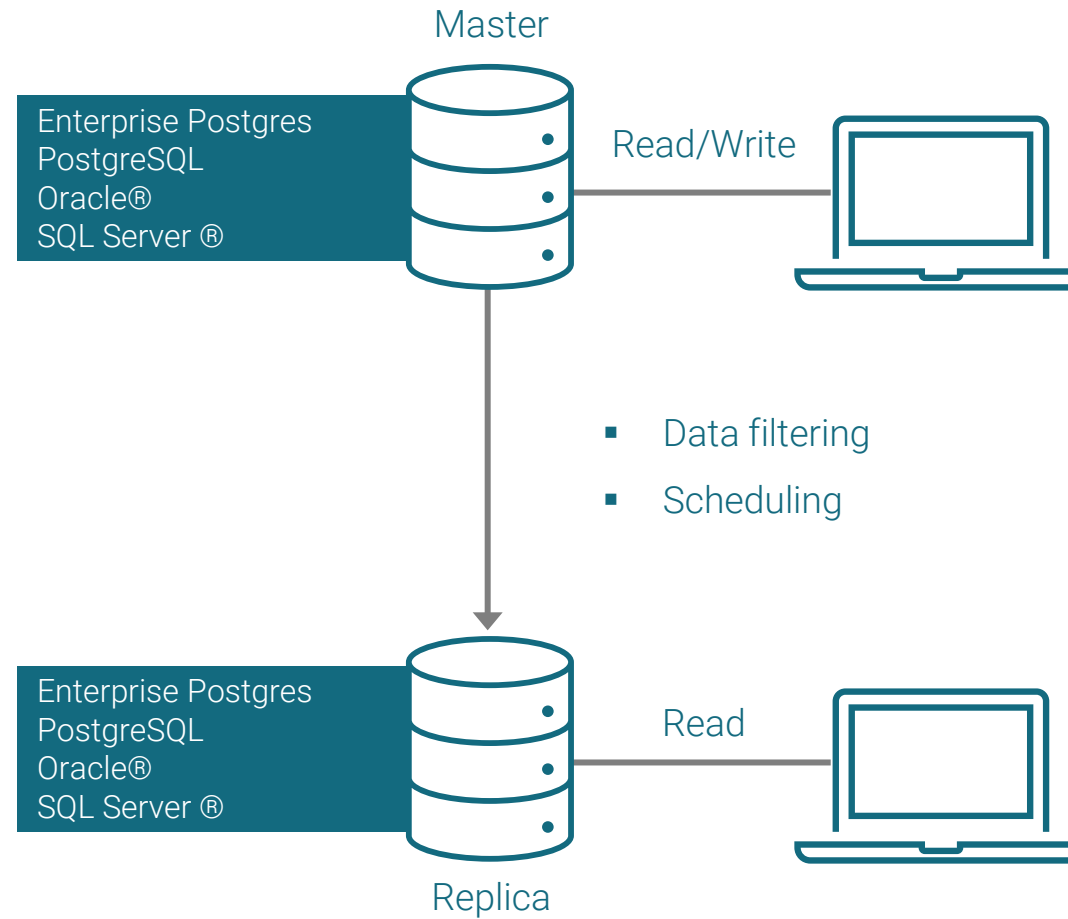


Replication Server Overview



Postgres Replication Server

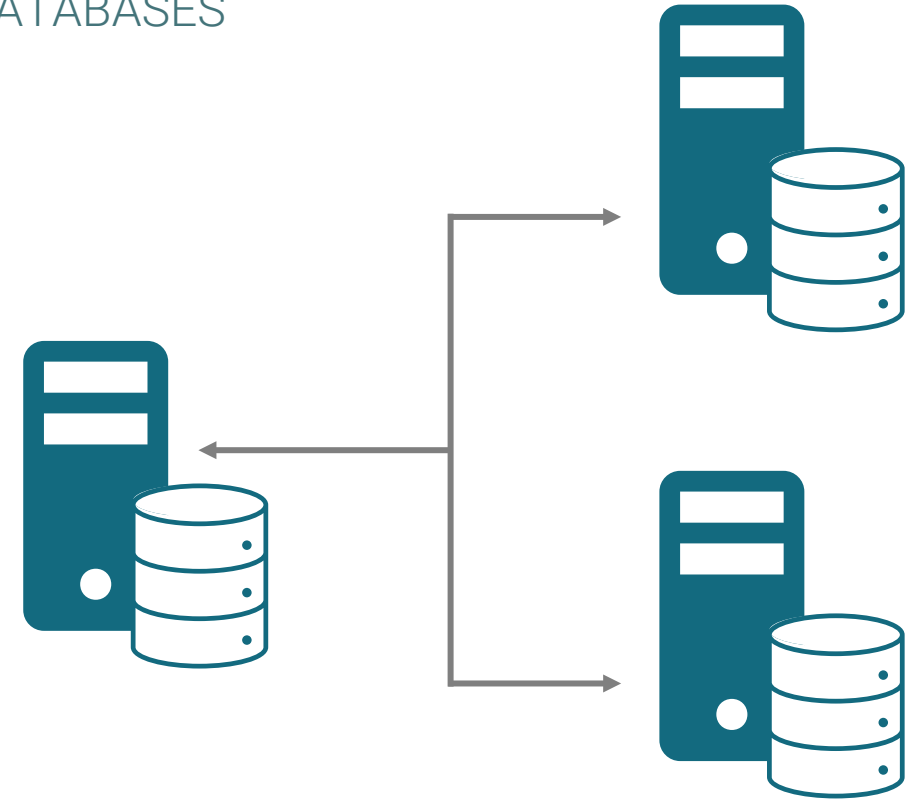
Single Master Replication (SMR) for Reporting or Migration



Replication Server

REPLICATES BETWEEN POSTGRES AND NON-POSTGRES DATABASES

- Integrate with Oracle or SQL Server databases to offload reporting or to feed data to legacy applications
- Flexibility to replicate a subset of data from the source database
- Graphical user interface provides easy configuration and management
- Includes utility to validate data consistency between the source and target databases



Replication from SQL Server
or Oracle to Postgres



Replication Server Features

- Replicate Oracle or SQL Server data to Enterprise Postgres (Oracle Compatible)
- Distributed multi-Publication/Subscription Architecture
- Synchronize data across geographies
- Replicate tables and sequences
- Controlled switchover and failover
- Supports cascading replication
- Trigger and Log-based replication
- Snapshot and continuous modes
- Define and apply row filters
- Flexible replication scheduler
- Replication History Viewer
- Graphical Replication Console and CLI



Replication Manager Overview



Replication Manager(repmgr)

Cluster Management tool for Postgres



Maintain high
availability

Automatic Failover to
Replica in a Streaming
Replication Environment



Perform upgrades
using switchovers

Add/remove replicas
and switchover of
primary instance



Open Source

Open Source from
EnterpriseDB and
licensed under GPL

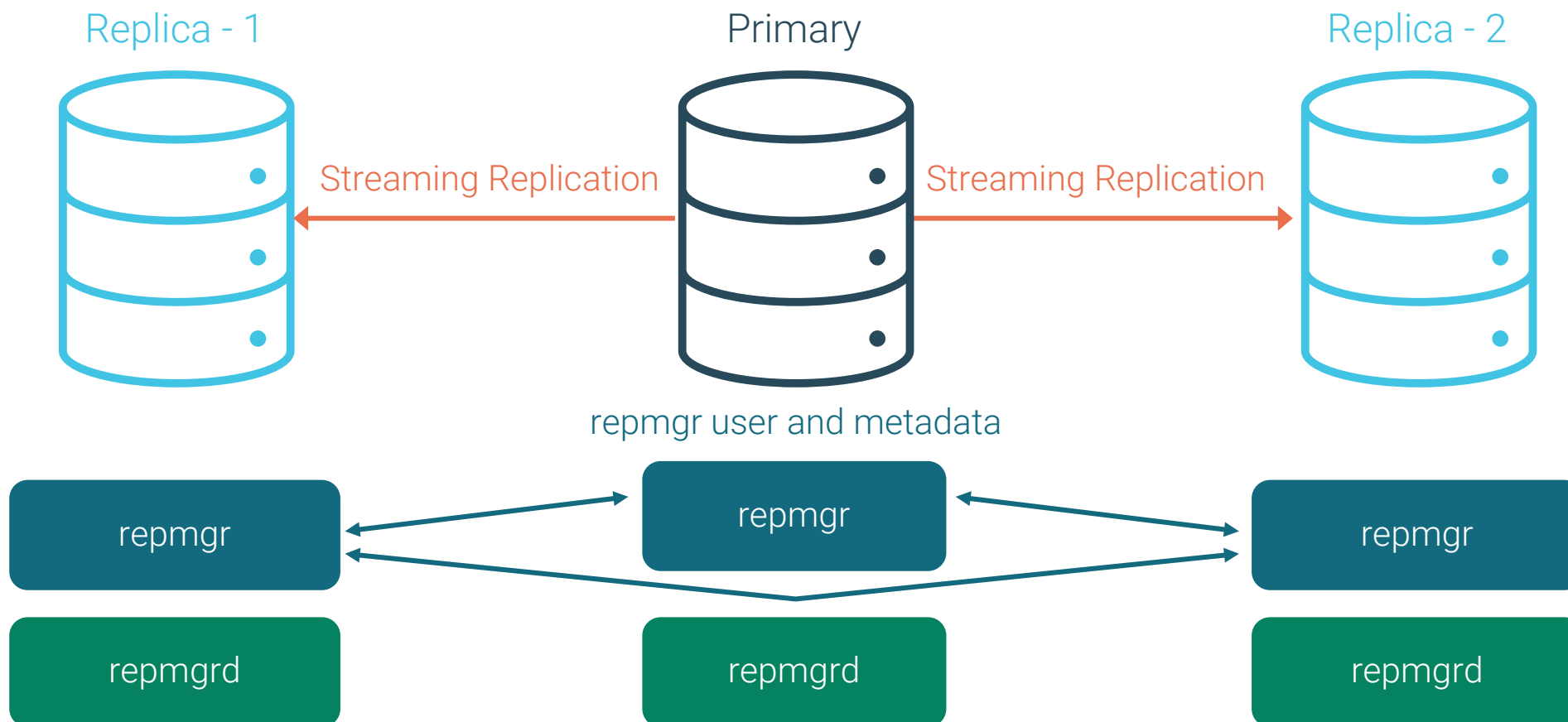


repmgr Features

- Open-source tool for managing replication and failover
- Supports Postgres Streaming Replication
- repmgr tool for setup:
 - Add/remove replicas
 - Perform switchovers
 - Promote a replica
- repmgrd tool:
 - Monitor replication
 - Automatic failover detection with witness protection
 - Email notification



repmgr Architecture



Module Summary

- Data Replication
- Data Replication in Postgres
- Streaming Replication and Architecture
- Synchronous, Asynchronous and Cascaded Replication
- Setup Streaming Replication
- Logical Replication Architecture
- Replication Manager (repmgr)
- High Availability Overview
- Replication Server (xdb)



Course Summary

- Introduction and Architectural Overview
- System Architecture
- Enterprise Postgres Installation
- User Tools - Command Line Interfaces
- Database Clusters
- Database Configuration
- Data Dictionary
- Creating and Managing Database Objects
- Database Security
- Monitoring and Admin Tools Overview
- SQL Primer
- Backup and Recovery
- Routine Maintenance Tasks
- Data Loading
- Data Replication and High Availability



Next Steps

- Certify your Postgres skills with [EDB Certifications for Postgres](#)
- Continue your skills development with the following classes:
 - Advanced EDB Postgres
 - EDB Postgres Distributed
 - EDB Postgres AI Platform Introduction
 - See the [Training Portal](#) for the full library of Postgres training classes
- For any questions related to EDB Postgres Trainings and Certifications, or for additional information, write to:
 - training@enterprisedb.com





Thank you!

Please visit our Training Portal for
more courses and workshops!